

Data Privacy Office (DPO) Request Management System

Source Code Compilation for Copyright Deposit

A web-based system for the management, review, electronic signing, and archival of Data Privacy Office requests, including Non-Disclosure Agreements (NDA) and Data Sharing Agreements.

Submission Date: April 29, 2026

Total Source Files: 65

Backend (Node.js / Express / MongoDB): 16 files

Frontend (React / Vite): 49 files

Total Lines of Code: 13,502

Primary Languages: JavaScript (Node.js), JSX (React)

This document contains only the original, manually authored source code authored by the development team. Auto-generated files, build artifacts, third-party libraries, configuration files, environment variables, and static assets have been excluded.

Table of Contents

Backend

1. server/server.js
2. server/config/db.js
3. server/models/User.js
4. server/models/Request.js
5. server/models/AuditLog.js
6. server/middleware/authMiddleware.js
7. server/middleware/piiRedactMiddleware.js
8. server/controllers/authController.js
9. server/controllers/requestController.js
10. server/controllers/auditController.js
11. server/routes/authRoutes.js
12. server/routes/requestRoutes.js
13. server/routes/auditRoutes.js
14. server/utils/emailService.js
15. server/scripts/runArchive.js
16. server/scripts/restoreArchive.js

Frontend

17. client/src/main.jsx
18. client/src/App.jsx
19. client/src/firebase.js
20. client/src/context/AuthContext.jsx
21. client/src/guards/RequireRole.jsx
22. client/src/services/authService.js
23. client/src/services/requestService.js
24. client/src/services/auditService.js
25. client/src/services/firebaseStorageService.js
26. client/src/services/qrService.js
27. client/src/utils/passwordPolicy.js
28. client/src/utils/inPageFeedback.js
29. client/src/utils/requestStatusCharts.js
30. client/src/config/fieldsFileSlotsConfig.js
31. client/src/config/documentTemplates/index.jsx
32. client/src/config/documentTemplates/styles.jsx
33. client/src/config/documentTemplates/AgreementDoc.jsx
34. client/src/config/documentTemplates/NDAResearchDoc.jsx
35. client/src/config/documentTemplates/NDASchoolOrgActivitiesDoc.jsx
36. client/src/components/Navbar.jsx
37. client/src/components/Headbar.jsx
38. client/src/components/FilterSelect.jsx
39. client/src/components/PasswordChecklist.jsx

- 40. client/src/components/SessionWarningModal.jsx
- 41. client/src/components/SignaturePad.jsx
- 42. client/src/components/RequestStepper.jsx
- 43. client/src/components/InPageFeedbackHost.jsx
- 44. client/src/pages/Landing.jsx
- 45. client/src/pages/ActivateAccountPage.jsx
- 46. client/src/pages/VerifyEmailPage.jsx
- 47. client/src/pages/VerifyDocument.jsx
- 48. client/src/pages/public/RepSigningPage.jsx
- 49. client/src/pages/student/StudentDashboard.jsx
- 50. client/src/pages/student/StudentProfile.jsx
- 51. client/src/pages/student/StudentNewRequest.jsx
- 52. client/src/pages/student/StudentAgreementRequest.jsx
- 53. client/src/pages/student/StudentAgreementRequirements.jsx
- 54. client/src/pages/student/StudentNDATypeChooser.jsx
- 55. client/src/pages/student/StudentNDAResponse.jsx
- 56. client/src/pages/student/StudentRequestReview.jsx
- 57. client/src/pages/student/StudentResubmitRequest.jsx
- 58. client/src/pages/admin/AdminDashboard.jsx
- 59. client/src/pages/admin/AdminProfile.jsx
- 60. client/src/pages/admin/AdminUsers.jsx
- 61. client/src/pages/admin/AdminRequests.jsx
- 62. client/src/pages/admin/AdminRequestReview.jsx
- 63. client/src/pages/admin/AdminReports.jsx
- 64. client/src/pages/admin/AdminAuditLog.jsx
- 65. client/src/pages/admin/AdminArchives.jsx

Backend Source Code

1. server/server.js (134 lines)

```
1  const express = require("express");
2  const cors = require("cors");
3  const helmet = require("helmet");
4  const compression = require("compression");
5  require("dotenv").config();
6
7  const mongoose = require("mongoose");
8  const dns = require("node:dns");
9  const cron = require("node-cron");
10
11 // Explicitly set DNS servers before connecting
12 dns.setServers(["8.8.8.8", "8.8.4.4"]);
13
14 const connectDB = require("./config/db");
15
16 const authRoutes = require("./routes/authRoutes");
17 const requestRoutes = require("./routes/requestRoutes");
18 const auditRoutes = require("./routes/auditRoutes");
19
20 const app = express();
21 const PORT = process.env.PORT || 5000;
22
23 // Middleware
24 const allowedOriginsFromEnv = process.env.ALLOWED_ORIGINS
25   ? process.env.ALLOWED_ORIGINS.split(",").map((o) => o.trim().replace(/\//+$/, ""))
26   : [];
27
28 const defaultLocalOrigins = [
29   process.env.CLIENT_URL,
30   "http://localhost:5173",
31   "http://localhost:5174",
32   "http://127.0.0.1:5173",
33   "http://127.0.0.1:5174",
34 ]
35   .filter(Boolean)
36   .map((o) => String(o).trim().replace(/\//+$/, ""));
37
38 const allowedOrigins = [...new Set([...allowedOriginsFromEnv, ...defaultLocalOrigins])];
39
40 const corsOptions = {
41   origin: (origin, callback) => {
42     // Allow requests with no origin (mobile apps, server-to-server, curl)
43     if (!origin) return callback(null, true);
44     const normalizedOrigin = origin.replace(/\//+$/, "");
45     if (allowedOrigins.length === 0 || allowedOrigins.includes(normalizedOrigin)) {
46       return callback(null, true);
47     }
48     // Return a CORS deny without throwing a generic 500.
49     return callback(null, false);
50   },
51   credentials: true,
52   methods: ["GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"],
53   allowedHeaders: ["Content-Type", "Authorization"],
54 };
55
56 // Handle preflight for all routes explicitly (required by some edge runtimes)
57 app.options(/.*/, cors(corsOptions));
58 app.use(cors(corsOptions));
59 app.use(helmet());
60 app.use(compression());
61 app.use(express.json());
62
63 // Express 5 compatibility: sanitize request payloads without reassigning req.query.
64 const sanitizeForNoSql = (value) => {
65   if (Array.isArray(value)) return value.map(sanitizeForNoSql);
66   if (!value || typeof value !== "object") return value;
67
68   const cleaned = {};
69   for (const [key, val] of Object.entries(value)) {
```

[illegible]

2. server/config/db.js (13 lines)

```
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6      console.log("MongoDB Connected:", mongoose.connection.name);
7    } catch (error) {
8      console.error("DB Connection Error:", error.message);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
```

3. server/models/User.js (76 lines)

```
1  const mongoose = require("mongoose");
2
3  const PASSWORD_REGEX = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\A-Za-z\d])[A-Za-z\d\W]{8}$/;
4
5  const trustedDeviceSchema = new mongoose.Schema(
6    {
7      ip: { type: String, required: true },
8      userAgent: { type: String, default: "" },
9      label: { type: String, default: "" },
10     lastOtpVerifiedAt: { type: Date, default: null },
11   },
12   { _id: false }
13 );
14
15 const userSchema = new mongoose.Schema({
16   // Structured name fields (preferred)
17   firstName: { type: String, trim: true, default: "" },
18   middleInitial: { type: String, trim: true, default: "" },
19   lastName: { type: String, trim: true, default: "" },
20   // Legacy single-name field kept for backward compatibility
21   name: {
22     type: String,
23     required: true,
24     trim: true,
25   },
26   email: {
27     type: String,
28     required: true,
29     unique: true,
30     lowercase: true,
31     trim: true,
32   },
33   password: {
34     type: String,
35     required: true,
36   },
37   role: {
38     type: String,
39     enum: ["student", "admin", "staff"],
40     default: "student",
41   },
42   isActive: {
43     type: Boolean,
44     default: true,
45   },
46   // Email verification
47   isVerified: { type: Boolean, default: false },
48   verificationToken: { type: String, default: null },
49   verificationExpires: { type: Date, default: null },
50   // OTP for login verification
51   otp: { type: String, default: null },
52   otpExpiry: { type: Date, default: null },
53   otpLastSentAt: { type: Date, default: null },
54   // Password reset
55   resetToken: { type: String, default: null },
56   resetTokenExpiry: { type: Date, default: null },
57   // Program/course
58   program: { type: String, trim: true, default: "" },
59   // Trusted devices for context-aware OTP
60   trustedDevices: { type: [trustedDeviceSchema], default: [] },
61   }, { timestamps: true });
62
63 userSchema.statics.isStrongPassword = (password) => PASSWORD_REGEX.test(String(password || ""));
64
65 userSchema.path("password").validate(function validatePasswordStrength(value) {
66   const v = String(value || "");
67   // Bcrypt hashes are already processed and should pass schema validation.
68   if (v.startsWith("$2a$") || v.startsWith("$2b$") || v.startsWith("$2y$")) return true;
69   return PASSWORD_REGEX.test(v);
70 }, "Password does not meet complexity policy");
71
72 // Query indexes for admin user management and role-based reporting views.
73 userSchema.index({ role: 1, isActive: 1, createdAt: -1 });
```

```
74 userSchema.index({ isVerified: 1, createdAt: -1 });  
75  
76 module.exports = mongoose.model("User", userSchema);
```


4. server/models/Request.js (158 lines)

```

1  const mongoose = require("mongoose");
2
3  const predocSchema = new mongoose.Schema(
4    {
5      origName:      { type: String },
6      url:           { type: String },
7      path:          { type: String },
8      contentType:   { type: String },
9      uploadedAt:    { type: String },
10     requirementLabel: { type: String, default: "" },
11     requestFolder:  { type: String, default: "" },
12     isResubmission: { type: Boolean, default: false },
13     resubmissionRound: { type: Number, default: 0 },
14   },
15   { _id: false }
16 );
17
18 const requestSchema = new mongoose.Schema(
19   {
20     userId: {
21       type: mongoose.Schema.Types.ObjectId,
22       ref: "User",
23       required: true,
24     },
25
26     type: {
27       type: String,
28       enum: ["nda", "agreement"],
29       required: true,
30     },
31
32     status: {
33       type: String,
34       enum: [
35         // NDA workflow
36         "nda_pending", "nda_approved",
37         // Shared NDA & Agreement revision
38         "stud_revision_requested",
39         // Agreement workflow
40         "agr_pending_1", "agr_awaiting_rep_signature", "agr_pending_2", "agr_approved",
41         // Agreement exception statuses
42         "agr_declined", "agr_rep_revision_requested",
43       ],
44       default: "nda_pending",
45     },
46
47     proxyRequestee: {
48       isProxy: { type: Boolean, default: false },
49       staffUserId: {
50         type: mongoose.Schema.Types.ObjectId,
51         ref: "User",
52         default: null,
53       },
54       // Structured name fields for proxy requestee
55       firstName: { type: String, default: "" },
56       middleInitial: { type: String, default: "" },
57       lastName: { type: String, default: "" },
58       // Legacy single fullName field kept for backward compatibility
59       fullName: { type: String, default: "" },
60       email: { type: String, default: "" },
61       idNumber: { type: String, default: "" },
62       departmentOrOrganization: { type: String, default: "" },
63     },
64
65     formData: {
66       type: Object,
67       required: true,
68     },
69
70     predocs: {
71       type: [predocSchema],
72       default: [],
73     },

```

```
74
75     postdocs: {
76         url: { type: String, default: "" },
77         path: { type: String, default: "" },
78         issuedAt: { type: String, default: "" },
79         verificationUrl: { type: String, default: "" },
80     },
81
82     resubmissionCount: {
83         type: Number,
84         default: 0,
85     },
86
87     remarks: {
88         type: String,
89         default: "",
90     },
91
92     serialNo: {
93         type: String,
94         default: "",
95         unique: true,
96         sparse: true,
97     },
98
99     // Agreement e-signature fields
100     signingToken: {
101         type: String,
102         default: "",
103     },
104
105     signingTokenUsed: {
106         type: Boolean,
107         default: false,
108     },
109
110     signingTokenExpiresAt: {
111         type: Date,
112         default: null,
113     },
114
115     // Representative info (submitted on signing page)
116     repInfo: {
117         name: { type: String, default: "" },
118         govIdDoc: {
119             origName: { type: String, default: "" },
120             url: { type: String, default: "" },
121             path: { type: String, default: "" },
122             contentType: { type: String, default: "" },
123             uploadedAt: { type: String, default: "" },
124         },
125     },
126
127     // Ephemeral signature storage (deleted after final PDF generation)
128     authorizerSigUrl: { type: String, default: "" },
129     authorizerSigPath: { type: String, default: "" },
130     repSigUrl: { type: String, default: "" },
131     repSigPath: { type: String, default: "" },
132
133     // NDA e-signature fields
134     studentSigUrl: { type: String, default: "" },
135     studentSigPath: { type: String, default: "" },
136     adminSigUrl: { type: String, default: "" },
137     adminSigPath: { type: String, default: "" },
138
139     // Signature timestamps
140     studentSignedAt: { type: Date, default: null },
141     adminSignedAt: { type: Date, default: null },
142     repSignedAt: { type: Date, default: null },
143
144     // 5-year retention / archiving
145     isArchived: { type: Boolean, default: false },
146     archivedAt: { type: Date, default: null },
147 },
148 { timestamps: true }
149 );
```

```
150
151 // Query-path indexes used by dashboard/report and owner-specific views.
152 requestSchema.index({ userId: 1, createdAt: -1 });
153 requestSchema.index({ status: 1, createdAt: -1 });
154 requestSchema.index({ isArchived: 1, createdAt: -1 });
155 requestSchema.index({ isArchived: 1, status: 1, createdAt: -1 });
156 requestSchema.index({ type: 1, status: 1, createdAt: -1 });
157
158 module.exports = mongoose.model("Request", requestSchema);
```

5. server/models/AuditLog.js (53 lines)

```
1  const mongoose = require("mongoose");
2
3  /**
4   * AuditLog Model
5   * Tracks system activity for the System Auditor & Analytics Agent (Agent 3).
6   * Stores user actions, document routing history, and login events.
7   */
8  const auditLogSchema = new mongoose.Schema(
9    {
10     userId: {
11       type: mongoose.Schema.Types.ObjectId,
12       ref: "User",
13       default: null,
14     },
15     action: {
16       type: String,
17       required: true,
18       // e.g. "login", "login_failed", "request_created", "request_approved",
19       // "request_revision_required", "document_generated"
20     },
21     resourceType: {
22       type: String,
23       default: "",
24       // e.g. "request", "user", "template"
25     },
26     resourceId: {
27       type: String,
28       default: "",
29     },
30     details: {
31       type: Object,
32       default: {},
33     },
34     ipAddress: {
35       type: String,
36       default: "",
37     },
38   },
39   { timestamps: true }
40 );
41
42 // Index for efficient querying by user and time window (used by Auditor Agent)
43 auditLogSchema.index({ userId: 1, createdAt: -1 });
44 auditLogSchema.index({ action: 1, createdAt: -1 });
45 auditLogSchema.index({ createdAt: -1 });
46 auditLogSchema.index({ resourceType: 1, createdAt: -1 });
47
48 module.exports = mongoose.model("AuditLog", auditLogSchema);
```

6. server/middleware/authMiddleware.js (39 lines)

```
1  const jwt = require("jsonwebtoken");
2
3  exports.protect = (req, res, next) => {
4    const header = req.headers.authorization;
5
6    if (!header) {
7      return res.status(401).json({ message: "Not authorized" });
8    }
9
10   const match = header.match(/^Bearer\s+(.+)$/i);
11   if (!match) {
12     return res.status(401).json({ message: "Not authorized" });
13   }
14
15   try {
16     const decoded = jwt.verify(match[1], process.env.JWT_SECRET);
17     req.user = decoded;
18     next();
19   } catch (error) {
20     if (error.name === "TokenExpiredError") {
21       return res.status(403).json({ message: "Token expired" });
22     }
23     res.status(401).json({ message: "Token invalid" });
24   }
25 };
26
27 exports.authorizeAdmin = (req, res, next) => {
28   if (req.user.role !== "admin") {
29     return res.status(403).json({ message: "Admin access only" });
30   }
31   next();
32 };
33
34 exports.authorizeRoles = (...allowedRoles) => (req, res, next) => {
35   if (!req.user || !allowedRoles.includes(req.user.role)) {
36     return res.status(403).json({ message: "Forbidden" });
37   }
38   next();
39 };
```

7. server/middleware/piiRedactMiddleware.js (69 lines)

```

1  /**
2   * PII Redaction Middleware
3   * Strips Personally Identifiable Information (PII) before passing data to AI agents.
4   * Per Phase 1 security requirements: names, emails, and specific student IDs must
5   * be removed before any data is sent to an LLM or AI processing layer.
6   */
7
8  const EMAIL_REGEX = /[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/g;
9  const PHONE_REGEX = /(\+?\d{1,3}[\s\-\.]?)?(?(\d{3})?[\s\-\.]?\d{3}[\s\-\.]?\d{4})/g;
10 const STUDENT_ID_REGEX = /\b\d{4}[-\s]?d{4,6}\b/g;
11
12 // Field names that are considered PII and should be redacted
13 const PII_FIELD_NAMES = [
14   "name", "email", "studentid", "studentno", "phone", "mobile",
15   "address", "fullname", "firstname", "lastname", "contactno",
16 ];
17
18 /**
19  * Checks if a field key is a PII field based on common naming conventions.
20  */
21 const isPiiFieldName = (key) => {
22   const lower = key.toLowerCase();
23   return PII_FIELD_NAMES.some((pii) => lower.includes(pii));
24 };
25
26 /**
27  * Recursively redact PII from a plain object or string.
28  * - Known PII field names are replaced with "[REDACTED]"
29  * - Email and phone patterns within string values are also replaced
30  */
31 const redactPII = (data) => {
32   if (data === null || data === undefined) return data;
33
34   if (typeof data === "string") {
35     return data
36       .replace(EMAIL_REGEX, "[EMAIL REDACTED]")
37       .replace(PHONE_REGEX, "[PHONE REDACTED]")
38       .replace(STUDENT_ID_REGEX, "[ID REDACTED]");
39   }
40
41   if (Array.isArray(data)) {
42     return data.map((item) => redactPII(item));
43   }
44
45   if (typeof data === "object") {
46     const result = {};
47     for (const [key, value] of Object.entries(data)) {
48       if (isPiiFieldName(key)) {
49         result[key] = "[REDACTED]";
50       } else {
51         result[key] = redactPII(value);
52       }
53     }
54     return result;
55   }
56
57   return data;
58 };
59
60 /**
61  * Express middleware that redacts PII from req.body before AI processing.
62  * Attaches the redacted copy to req.redactedBody so controllers can use it.
63  */
64 const piiRedactMiddleware = (req, res, next) => {
65   req.redactedBody = redactPII(req.body);
66   next();
67 };
68
69 module.exports = { piiRedactMiddleware, redactPII };

```

8. server/controllers/authController.js (920 lines)

```
1  const crypto = require("crypto");
2  const User = require("../models/User");
3  const bcrypt = require("bcrypt");
4  const jwt = require("jsonwebtoken");
5  const AuditLog = require("../models/AuditLog");
6  const {
7    sendOtpEmail,
8    sendWelcomeEmail,
9    sendLoginOtpEmail,
10   sendVerificationOtpEmail,
11   sendPasswordChangedAlertEmail,
12 } = require("../utils/emailService");
13
14 // Helper to safely log audit events without blocking the response
15 const logAudit = (data) => {
16   AuditLog.create(data).catch(() => {});
17 };
18
19 const generateOtp = () => String(Math.floor(100000 + Math.random() * 900000));
20
21 const getClientIp = (req) => {
22   const forwarded = req.headers["x-forwarded-for"];
23   if (forwarded) return forwarded.split(",")[0].trim();
24   return req.socket?.remoteAddress || req.ip || "";
25 };
26
27 const OTP_TRUST_WINDOW_MS = 72 * 60 * 60 * 1000; // 72 hours
28 const OTP_RESEND_COOLDOWN_MS = 60 * 1000;
29 const EMAIL_ERROR_MESSAGE = "Failed to send verification email. Please try again later.";
30 const PASSWORD_POLICY_MESSAGE = "Password must be at least 8 characters and include uppercase, lowercase, number, and special";
31
32 const isStrongPassword = (password) => User.isStrongPassword(password);
33
34 const buildErrorMessage = (error) => {
35   if (error?.isEmailError) {
36     return error.publicMessage || EMAIL_ERROR_MESSAGE;
37   }
38   return error?.message || "Internal server error";
39 };
40
41 const issueAuthPayload = (user) => {
42   const token = jwt.sign(
43     { id: user._id, role: user.role },
44     process.env.JWT_SECRET,
45     { expiresIn: "8h" }
46   );
47
48   return {
49     token,
50     user: { id: user._id, name: user.name, email: user.email, role: user.role },
51   };
52 };
53
54 const getOtpCooldownState = (user) => {
55   const last = user?.otpLastSentAt ? new Date(user.otpLastSentAt).getTime() : 0;
56   const elapsed = Date.now() - last;
57   const retryAfterMs = OTP_RESEND_COOLDOWN_MS - elapsed;
58   return {
59     isCoolingDown: last > 0 && retryAfterMs > 0,
60     retryAfterSeconds: Math.max(1, Math.ceil(retryAfterMs / 1000)),
61   };
62 };
63
64 const sendVerificationOtpToUser = async (user, { enforceCooldown = false } = {}) => {
65   if (enforceCooldown) {
66     const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
67     if (isCoolingDown) {
68       const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
69       err.statusCode = 429;
70       err.retryAfterSeconds = retryAfterSeconds;
71       throw err;
72     }
73   }
74 }
```

```
74
75   const otp = generateOtp();
76   user.otp = await bcrypt.hash(otp, 10);
77   user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
78   user.otpLastSentAt = new Date();
79   await user.save();
80   await sendVerificationOtpEmail(user.email, user.name, otp);
81 };
82
83 const sendLoginOtpToUser = async (user, { enforceCooldown = false } = {}) => {
84   if (enforceCooldown) {
85     const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
86     if (isCoolingDown) {
87       const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
88       err.statusCode = 429;
89       err.retryAfterSeconds = retryAfterSeconds;
90       throw err;
91     }
92   }
93
94   const otp = generateOtp();
95   user.otp = await bcrypt.hash(otp, 10);
96   user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
97   user.otpLastSentAt = new Date();
98   await user.save();
99   await sendLoginOtpEmail(user.email, otp);
100 };
101
102 const sendResetOtpToUser = async (user, { enforceCooldown = false } = {}) => {
103   if (enforceCooldown) {
104     const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
105     if (isCoolingDown) {
106       const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
107       err.statusCode = 429;
108       err.retryAfterSeconds = retryAfterSeconds;
109       throw err;
110     }
111   }
112
113   const otp = generateOtp();
114   user.otp = await bcrypt.hash(otp, 10);
115   user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000); // 10 minutes
116   user.otpLastSentAt = new Date();
117   user.resetToken = null;
118   user.resetTokenExpiry = null;
119   await user.save();
120   await sendOtpEmail(user.email, otp, "reset");
121 };
122
123 // REGISTER
124 exports.register = async (req, res) => {
125   try {
126     const { firstName, middleInitial, lastName, name, email, password, program } = req.body;
127
128     // Support both structured names and legacy single-name field
129     const resolvedFirstName = String(firstName || "").trim();
130     const resolvedLastName = String(lastName || "").trim();
131     const resolvedMiddleInitial = String(middleInitial || "").trim();
132
133     // Build composite name: prefer structured fields, fall back to legacy name
134     // Middle name is stored in full but displayed as initial (e.g. "Santos" → "S.")
135     const mi = resolvedMiddleInitial ? resolvedMiddleInitial.charAt(0).toUpperCase() + "." : "";
136     const compositeName = resolvedFirstName && resolvedLastName
137       ? `${resolvedFirstName}${mi} " " + mi : ""` : resolvedLastName;
138     : String(name || "").trim();
139
140     if (!compositeName || compositeName.length < 2) {
141       return res.status(400).json({ message: "First name and last name are required" });
142     }
143
144     if (!email || !password) {
145       return res.status(400).json({ message: "Email and password are required" });
146     }
147
148     if (!isStrongPassword(password)) {
149       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
150     }
151   } catch (err) {
152     return res.status(500).json({ message: "Internal server error" });
153   }
154 }
```



```
150     }
151
152     const existingUser = await User.findOne({ email: email.toLowerCase() });
153     if (existingUser)
154         return res.status(400).json({ message: "User already exists" });
155
156     const hashedPassword = await bcrypt.hash(password, 12);
157
158     const otp = generateOtp();
159     const hashedOtp = await bcrypt.hash(otp, 10);
160
161     const user = await User.create({
162         name: compositeName,
163         firstName: resolvedFirstName,
164         middleInitial: resolvedMiddleInitial,
165         lastName: resolvedLastName,
166         email: email.toLowerCase(),
167         password: hashedPassword,
168         program: String(program || "").trim(),
169         isVerified: false,
170         otp: hashedOtp,
171         otpExpiry: new Date(Date.now() + 10 * 60 * 1000), // 10 min
172     });
173
174     await sendVerificationOtpEmail(user.email, user.name, otp);
175
176     logAudit({ userId: user._id, action: "account_created", details: { role: user.role, method: "self_register" } });
177
178     res.status(201).json({ requireVerification: true, email: user.email, message: "Account created! Please check your email f
179 } catch (error) {
180     res.status(500).json({ message: buildErrorMessage(error) });
181 }
182 };
183
184 // VERIFY EMAIL (OTP-based)
185 exports.verifyEmail = async (req, res) => {
186     try {
187         const { email, otp, token } = req.body;
188
189         // Support legacy token-based verification for existing links
190         if (token) {
191             const user = await User.findOne({
192                 verificationToken: token,
193                 verificationExpires: { $gt: new Date() },
194             });
195             if (!user) {
196                 return res.status(400).json({ message: "Invalid or expired verification link. Please register again or contact an adm
197             }
198             user.isVerified = true;
199             user.verificationToken = null;
200             user.verificationExpires = null;
201             await user.save();
202             logAudit({ userId: user._id, action: "email_verified", details: { email: user.email } });
203             return res.json({
204                 message: "Email verified successfully!",
205                 ...issueAuthPayload(user),
206             });
207         }
208
209         // OTP-based verification
210         if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });
211
212         const user = await User.findOne({ email: email.toLowerCase() });
213         if (!user || !user.otp || !user.otpExpiry) {
214             return res.status(400).json({ message: "Invalid or expired OTP. Please request a new one." });
215         }
216
217         if (user.isVerified) {
218             return res.json({
219                 message: "Email is already verified.",
220                 ...issueAuthPayload(user),
221             });
222         }
223
224         if (user.otpExpiry < new Date()) {
225             return res.status(400).json({ message: "OTP has expired. Please request a new one." });
```

```
226     }
227
228     const otpMatch = await bcrypt.compare(String(otp), user.otp);
229     if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });
230
231     user.isVerified = true;
232     user.otp = null;
233     user.otpExpiry = null;
234     await user.save();
235
236     logAudit({ userId: user._id, action: "email_verified", details: { email: user.email } });
237
238     res.json({
239       message: "Email verified successfully!",
240       ...issueAuthPayload(user),
241     });
242   } catch (error) {
243     res.status(500).json({ message: buildErrorMessage(error) });
244   }
245 };
246
247 // RESEND VERIFICATION OTP
248 exports.resendVerificationOtp = async (req, res) => {
249   try {
250     const { email } = req.body;
251     if (!email) return res.status(400).json({ message: "Email is required" });
252
253     const user = await User.findOne({ email: email.toLowerCase() });
254     if (!user) {
255       // Don't reveal whether account exists
256       return res.json({ message: "If that account exists, a new OTP has been sent." });
257     }
258
259     if (user.isVerified) {
260       return res.status(400).json({ message: "This account is already verified. You can log in." });
261     }
262
263     await sendVerificationOtpToUser(user, { enforceCooldown: true });
264
265     res.json({ message: "A new verification OTP has been sent to your email." });
266   } catch (error) {
267     if (error?.statusCode === 429) {
268       return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
269     }
270     res.status(500).json({ message: buildErrorMessage(error) });
271   }
272 };
273
274 exports.resendLoginOtp = async (req, res) => {
275   try {
276     const { email } = req.body;
277     const clientIp = getClientIp(req);
278
279     if (!email) return res.status(400).json({ message: "Email is required" });
280
281     const user = await User.findOne({ email: email.toLowerCase() });
282     if (!user) {
283       return res.status(200).json({ message: "If that account exists, a new OTP has been sent." });
284     }
285
286     if (!user.isActive) {
287       return res.status(403).json({ message: "Account is deactivated. Contact an administrator." });
288     }
289
290     if (!user.isVerified) {
291       return res.status(403).json({ message: "Please verify your email first." });
292     }
293
294     await sendLoginOtpToUser(user, { enforceCooldown: true });
295
296     logAudit({ userId: user._id, action: "login_otp_sent", details: { ip: clientIp, reason: "resend" }, ipAddress: clientIp });
297     return res.status(200).json({ message: "A new login OTP has been sent." });
298   } catch (error) {
299     if (error?.statusCode === 429) {
300       return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
301     }
302   }
303 }
```

```
302     return res.status(500).json({ message: buildErrorMessage(error) });
303   }
304 };
305
306 // VERIFY EMAIL & SET PASSWORD (for admin-created users without a password)
307 exports.activateAccount = async (req, res) => {
308   try {
309     const { token, password } = req.body;
310     if (!token || !password) {
311       return res.status(400).json({ message: "Token and new password are required" });
312     }
313     if (!isStrongPassword(password)) {
314       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
315     }
316
317     const user = await User.findOne({
318       verificationToken: token,
319       verificationExpires: { $gt: new Date() },
320     });
321
322     if (!user) {
323       return res.status(400).json({ message: "Invalid or expired activation link." });
324     }
325
326     user.password = await bcrypt.hash(password, 12);
327     user.isVerified = true;
328     user.verificationToken = null;
329     user.verificationExpires = null;
330     await user.save();
331
332     logAudit({ userId: user._id, action: "account_activated", details: { email: user.email } });
333
334     res.json({ message: "Account activated! You can now log in." });
335   } catch (error) {
336     res.status(500).json({ message: buildErrorMessage(error) });
337   }
338 };
339
340 // LOGIN – Step 1: credentials check + OTP challenge if needed
341 exports.login = async (req, res) => {
342   try {
343     const { email, password } = req.body;
344     const clientIp = getClientIp(req);
345     const userAgent = req.headers["user-agent"] || "";
346
347     if (!email || !password) {
348       return res.status(400).json({ message: "Email and password are required" });
349     }
350
351     const user = await User.findOne({ email: email.toLowerCase() });
352     if (!user) {
353       logAudit({ action: "login_failed", details: { email: email.toLowerCase(), reason: "user_not_found" }, ipAddress: clientIp });
354       return res.status(401).json({ message: "Invalid email or password" });
355     }
356
357     if (!user.isActive) {
358       return res.status(403).json({ message: "Account is deactivated. Contact an administrator." });
359     }
360
361     if (!user.isVerified) {
362       await sendVerificationOtpToUser(user, { enforceCooldown: true });
363       logAudit({ userId: user._id, action: "verification_otp_sent", details: { reason: "login_intercept" }, ipAddress: clientIp });
364       return res.status(403).json({
365         requireVerification: true,
366         email: user.email,
367         message: "Your email is not verified yet. We sent a verification OTP to your email.",
368       });
369     }
370
371     const isMatch = await bcrypt.compare(password, user.password);
372     if (!isMatch) {
373       logAudit({ userId: user._id, action: "login_failed", details: { reason: "wrong_password" }, ipAddress: clientIp });
374       return res.status(401).json({ message: "Invalid email or password" });
375     }
376
377     // Check if this device/IP is trusted and within 72h window
```

```
378     const trustedDevice = user.trustedDevices.find((d) => d.ip === clientIp);
379     const isWithinWindow = trustedDevice?.lastOtpVerifiedAt &&
380       (Date.now() - trustedDevice.lastOtpVerifiedAt.getTime()) < OTP_TRUST_WINDOW_MS;
381
382     if (trustedDevice && isWithinWindow) {
383       // Device is trusted → issue token directly
384       const token = jwt.sign(
385         { id: user._id, role: user.role },
386         process.env.JWT_SECRET,
387         { expiresIn: "8h" }
388       );
389
390       logAudit({ userId: user._id, action: "login", details: { role: user.role, ip: clientIp }, ipAddress: clientIp });
391
392       return res.json({
393         token,
394         user: { id: user._id, name: user.name, email: user.email, role: user.role },
395       });
396     }
397
398     // Device not trusted or expired → send OTP
399     await sendLoginOtpToUser(user, { enforceCooldown: true });
400
401     logAudit({ userId: user._id, action: "login_otp_sent", details: { ip: clientIp }, ipAddress: clientIp });
402
403     return res.json({
404       requireOtp: true,
405       message: "A verification code has been sent to your email.",
406     });
407   } catch (error) {
408     if (error?.statusCode === 429) {
409       return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
410     }
411     res.status(500).json({ message: buildErrorMessage(error) });
412   }
413 };
414
415 // LOGIN – Step 2: verify login OTP
416 exports.verifyLoginOtp = async (req, res) => {
417   try {
418     const { email, otp } = req.body;
419     const clientIp = getClientIp(req);
420     const userAgent = req.headers["user-agent"] || "";
421
422     if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });
423
424     const user = await User.findOne({ email: email.toLowerCase() });
425     if (!user || !user.otp || !user.otpExpiry) {
426       return res.status(400).json({ message: "Invalid or expired OTP" });
427     }
428
429     if (user.otpExpiry < new Date()) {
430       return res.status(400).json({ message: "OTP has expired. Please log in again." });
431     }
432
433     const otpMatch = await bcrypt.compare(String(otp), user.otp);
434     if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });
435
436     // Clear OTP
437     user.otp = null;
438     user.otpExpiry = null;
439
440     // Trust this device
441     const existingIdx = user.trustedDevices.findIndex((d) => d.ip === clientIp);
442     const deviceEntry = {
443       ip: clientIp,
444       userAgent: userAgent.substring(0, 200),
445       label: userAgent.substring(0, 60),
446       lastOtpVerifiedAt: new Date(),
447     };
448
449     if (existingIdx >= 0) {
450       user.trustedDevices[existingIdx] = deviceEntry;
451     } else {
452       // Keep max 10 trusted devices
453       if (user.trustedDevices.length >= 10) {
```

```
454     user.trustedDevices.sort((a, b) => (a.lastOtpVerifiedAt || 0) - (b.lastOtpVerifiedAt || 0));
455     user.trustedDevices.shift();
456   }
457   user.trustedDevices.push(deviceEntry);
458 }
459
460 await user.save();
461
462 const token = jwt.sign(
463   { id: user._id, role: user.role },
464   process.env.JWT_SECRET,
465   { expiresIn: "8h" }
466 );
467
468 logAudit({ userId: user._id, action: "login", details: { role: user.role, ip: clientIp, method: "otp" }, ipAddress: clientIp });
469
470 return res.json({
471   token,
472   user: { id: user._id, name: user.name, email: user.email, role: user.role },
473 });
474 } catch (error) {
475   res.status(500).json({ message: buildErrorMessage(error) });
476 }
477 };
478
479 // REFRESH TOKEN (for session extension)
480 exports.refreshToken = async (req, res) => {
481   try {
482     const user = await User.findById(req.user.id).select("_id role name email");
483     if (!user) return res.status(404).json({ message: "User not found" });
484
485     const token = jwt.sign(
486       { id: user._id, role: user.role },
487       process.env.JWT_SECRET,
488       { expiresIn: "8h" }
489     );
490
491     res.json({ token, user: { id: user._id, name: user.name, email: user.email, role: user.role } });
492   } catch (error) {
493     res.status(500).json({ message: buildErrorMessage(error) });
494   }
495 };
496
497 // REQUEST OTP FOR PASSWORD CHANGE
498 exports.requestPasswordChangeOtp = async (req, res) => {
499   try {
500     const user = await User.findById(req.user.id);
501     if (!user) return res.status(404).json({ message: "User not found" });
502
503     const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
504     if (isCoolingDown) {
505       return res.status(429).json({ message: `Please wait ${retryAfterSeconds}s before requesting another OTP.`, retryAfterSeconds });
506     }
507
508     const otp = generateOtp();
509     user.otp = await bcrypt.hash(otp, 10);
510     user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
511     user.otpLastSentAt = new Date();
512     await user.save();
513     await sendOtpEmail(user.email, otp, "change");
514
515     res.json({ message: "An OTP has been sent to your email." });
516   } catch (error) {
517     res.status(500).json({ message: buildErrorMessage(error) });
518   }
519 };
520
521 // UPDATE PROFILE (name & password)
522 exports.updateProfile = async (req, res) => {
523   try {
524     const { firstName, middleInitial, lastName, name, currentPassword, newPassword, otp } = req.body;
525     const user = await User.findById(req.user.id);
526     if (!user) return res.status(404).json({ message: "User not found" });
527
528     let updated = false;
529
```

```
530 // Update name - support both structured and legacy fields
531 const resolvedFirstName = String(firstName || "").trim();
532 const resolvedLastName = String(lastName || "").trim();
533 const resolvedMiddleInitial = String(middleInitial || "").trim();
534
535 if (resolvedFirstName && resolvedLastName) {
536   user.firstName = resolvedFirstName;
537   user.middleInitial = resolvedMiddleInitial;
538   user.lastName = resolvedLastName;
539   const miUp = resolvedMiddleInitial ? resolvedMiddleInitial.charAt(0).toUpperCase() + "." : "";
540   user.name = `${resolvedFirstName}${miUp ? " " + miUp : ""} ${resolvedLastName}`;
541   updated = true;
542 } else if (name && String(name).trim().length >= 2) {
543   user.name = String(name).trim();
544   updated = true;
545 }
546
547 // Update password
548 if (newPassword) {
549   if (!currentPassword) {
550     return res.status(400).json({ message: "Current password is required to set a new password" });
551   }
552   if (!otp) {
553     return res.status(400).json({ message: "OTP verification is required to change password" });
554   }
555   if (!isStrongPassword(newPassword)) {
556     return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
557   }
558   const isMatch = await bcrypt.compare(currentPassword, user.password);
559   if (!isMatch) {
560     return res.status(400).json({ message: "Current password is incorrect" });
561   }
562   if (!user.otp || !user.otpExpiry) {
563     return res.status(400).json({ message: "No OTP found. Please request a new one." });
564   }
565   if (user.otpExpiry < new Date()) {
566     return res.status(400).json({ message: "OTP has expired. Please request a new one." });
567   }
568   const otpMatch = await bcrypt.compare(String(otp), user.otp);
569   if (!otpMatch) {
570     return res.status(400).json({ message: "Invalid OTP. Please try again." });
571   }
572   user.otp = null;
573   user.otpExpiry = null;
574   user.password = await bcrypt.hash(newPassword, 12);
575   updated = true;
576   logAudit({ userId: user._id, action: "password_changed", details: { method: "profile" } });
577   await sendPasswordChangedAlertEmail(user.email, user.name, "self-service change");
578 }
579
580 if (!updated) {
581   return res.status(400).json({ message: "No changes provided" });
582 }
583
584 await user.save();
585
586 // Return updated user info (so frontend can update context)
587 res.json({
588   message: "Profile updated successfully",
589   user: { id: user._id, name: user.name, email: user.email, role: user.role },
590 });
591 } catch (error) {
592   res.status(500).json({ message: buildErrorMessage(error) });
593 }
594 };
595
596 // FORGOT PASSWORD - sends OTP to email
597 exports.forgotPassword = async (req, res) => {
598   try {
599     const { email } = req.body;
600     if (!email) return res.status(400).json({ message: "Email is required" });
601
602     const user = await User.findOne({ email: email.toLowerCase() });
603     // Always respond 200 to prevent email enumeration
604     if (!user) return res.status(200).json({ message: "If that account exists, an OTP has been sent." });
605   }
```

```
606     await sendResetOtpToUser(user, { enforceCooldown: true });
607     logAudit({ userId: user._id, action: "password_reset_requested", details: { email: user.email } });
608
609     res.status(200).json({ message: "If that account exists, an OTP has been sent." });
610   } catch (error) {
611     if (error?.statusCode === 429) {
612       return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
613     }
614     res.status(500).json({ message: buildErrorMessage(error) });
615   }
616 };
617
618 exports.resendResetOtp = async (req, res) => {
619   try {
620     const { email } = req.body;
621     if (!email) return res.status(400).json({ message: "Email is required" });
622
623     const user = await User.findOne({ email: email.toLowerCase() });
624     if (!user) {
625       return res.status(200).json({ message: "If that account exists, an OTP has been sent." });
626     }
627
628     await sendResetOtpToUser(user, { enforceCooldown: true });
629     logAudit({ userId: user._id, action: "password_reset_requested", details: { email: user.email, reason: "resend" } });
630     return res.status(200).json({ message: "A new password reset OTP has been sent." });
631   } catch (error) {
632     if (error?.statusCode === 429) {
633       return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
634     }
635     return res.status(500).json({ message: buildErrorMessage(error) });
636   }
637 };
638
639 // VERIFY RESET OTP - returns a short-lived reset token
640 exports.verifyResetOtp = async (req, res) => {
641   try {
642     const { email, otp } = req.body;
643     if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });
644
645     const user = await User.findOne({ email: email.toLowerCase() });
646     if (!user || !user.otp || !user.otpExpiry)
647       return res.status(400).json({ message: "Invalid or expired OTP" });
648
649     if (user.otpExpiry < new Date())
650       return res.status(400).json({ message: "OTP has expired. Please request a new one." });
651
652     const otpMatch = await bcrypt.compare(String(otp), user.otp);
653     if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });
654
655     const resetToken = crypto.randomBytes(32).toString("hex");
656     user.resetToken = await bcrypt.hash(resetToken, 10);
657     user.resetTokenExpiry = new Date(Date.now() + 15 * 60 * 1000); // 15 min
658     user.otp = null;
659     user.otpExpiry = null;
660     await user.save();
661
662     res.json({ resetToken, message: "OTP verified" });
663   } catch (error) {
664     res.status(500).json({ message: buildErrorMessage(error) });
665   }
666 };
667
668 // RESET PASSWORD
669 exports.resetPassword = async (req, res) => {
670   try {
671     const { email, resetToken, newPassword } = req.body;
672     if (!email || !resetToken || !newPassword)
673       return res.status(400).json({ message: "Email, reset token, and new password are required" });
674
675     if (!isStrongPassword(newPassword)) {
676       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
677     }
678
679     const user = await User.findOne({ email: email.toLowerCase() });
680     if (!user || !user.resetToken || !user.resetTokenExpiry)
681       return res.status(400).json({ message: "Invalid or expired reset token" });
```

```
682
683     if (user.resetTokenExpiry < new Date())
684         return res.status(400).json({ message: "Reset token has expired" });
685
686     const tokenMatch = await bcrypt.compare(resetToken, user.resetToken);
687     if (!tokenMatch) return res.status(400).json({ message: "Invalid reset token" });
688
689     user.password = await bcrypt.hash(newPassword, 12);
690     user.resetToken = null;
691     user.resetTokenExpiry = null;
692     await user.save();
693
694     logAudit({ userId: user._id, action: "password_changed", details: { method: "reset" } });
695     await sendPasswordChangedAlertEmail(user.email, user.name, "account recovery reset");
696
697     res.json({ message: "Password reset successfully" });
698 } catch (error) {
699     res.status(500).json({ message: buildErrorMessage(error) });
700 }
701 };
702
703 // ADMIN: Get all users
704 exports.getAllUsers = async (req, res) => {
705     try {
706         const users = await User.find()
707             .select("-password -otp -otpExpiry -otpLastSentAt -resetToken -resetTokenExpiry -verificationToken -trustedDevices")
708             .sort({ createdAt: -1 })
709             .lean();
710         res.json(users);
711     } catch (error) {
712         res.status(500).json({ message: buildErrorMessage(error) });
713     }
714 };
715
716 // ADMIN: Create user (with verification email)
717 exports.adminCreateUser = async (req, res) => {
718     try {
719         const { name, email, role, password } = req.body;
720
721         if (!name || !email || !password) return res.status(400).json({ message: "Name, email, role, and temporary password are r
722         if (!["student", "admin", "staff"].includes(role))
723             return res.status(400).json({ message: "Invalid role" });
724         if (!isStrongPassword(password)) {
725             return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
726         }
727
728         const existing = await User.findOne({ email: email.toLowerCase() });
729         if (existing) return res.status(400).json({ message: "User with this email already exists" });
730
731         const hashedPassword = await bcrypt.hash(password, 12);
732
733         const user = await User.create({
734             name: String(name).trim(),
735             email: email.toLowerCase(),
736             password: hashedPassword,
737             role: role || "student",
738             isVerified: false,
739             verificationToken: null,
740             verificationExpires: null,
741         });
742
743         await sendVerificationOtpToUser(user);
744         await sendWelcomeEmail(user.email, user.name, password);
745
746         logAudit({
747             userId: req.user.id,
748             action: "account_created",
749             resourceType: "user",
750             resourceId: String(user._id),
751             details: { createdBy: "admin", targetEmail: user.email, role: user.role, tempPasswordSet: true },
752         });
753
754         res.status(201).json({
755             message: "User created. A temporary password and verification OTP were sent by email.",
756             user: { id: user._id, name: user.name, email: user.email, role: user.role },
757         });
758     } catch (error) {
759         res.status(500).json({ message: buildErrorMessage(error) });
760     }
761 }
```



```
758   } catch (error) {
759     res.status(500).json({ message: buildErrorMessage(error) });
760   }
761 };
762
763 // ADMIN: Toggle user active/inactive
764 exports.toggleUserActive = async (req, res) => {
765   try {
766     const user = await User.findById(req.params.id).select("-password -otp -resetToken");
767     if (!user) return res.status(404).json({ message: "User not found" });
768
769     if (String(user._id) === String(req.user.id))
770       return res.status(400).json({ message: "You cannot deactivate your own account" });
771
772     user.isActive = !user.isActive;
773     await user.save();
774
775     logAudit({
776       userId: req.user.id,
777       action: user.isActive ? "user_activated" : "user_deactivated",
778       resourceType: "user",
779       resourceId: String(user._id),
780       details: { targetEmail: user.email },
781     });
782
783     res.json({ message: `User ${user.isActive ? "activated" : "deactivated"}`, isActive: user.isActive });
784   } catch (error) {
785     res.status(500).json({ message: buildErrorMessage(error) });
786   }
787 };
788
789 // ADMIN: Edit user details and role
790 exports.adminUpdateUser = async (req, res) => {
791   try {
792     const { name, email, role, isActive } = req.body;
793     const user = await User.findById(req.params.id);
794     if (!user) return res.status(404).json({ message: "User not found" });
795
796     const updates = {};
797
798     if (name !== undefined) {
799       const trimmedName = String(name || "").trim();
800       if (trimmedName.length < 2) {
801         return res.status(400).json({ message: "Name must be at least 2 characters" });
802       }
803       updates.name = trimmedName;
804     }
805
806     if (email !== undefined) {
807       const normalizedEmail = String(email || "").trim().toLowerCase();
808       if (!normalizedEmail) return res.status(400).json({ message: "Email is required" });
809       const duplicate = await User.findOne({ email: normalizedEmail, _id: { $ne: user._id } });
810       if (duplicate) return res.status(400).json({ message: "Another user already uses this email" });
811       updates.email = normalizedEmail;
812     }
813
814     if (role !== undefined) {
815       if (![
816         "student",
817         "staff",
818         "admin",
819       ].includes(role)) {
820         return res.status(400).json({ message: "Invalid role" });
821       }
822       updates.role = role;
823     }
824
825     if (isActive !== undefined) {
826       if (String(user._id) === String(req.user.id) && isActive === false) {
827         return res.status(400).json({ message: "You cannot deactivate your own account" });
828       }
829       updates.isActive = Boolean(isActive);
830     }
831
832     Object.assign(user, updates);
833     await user.save();
```

```
834
835   logAudit({
836     userId: req.user.id,
837     action: "user_updated",
838     resourceType: "user",
839     resourceId: String(user._id),
840     details: { fields: Object.keys(updates), targetEmail: user.email },
841   });
842
843   return res.json({
844     message: "User updated successfully",
845     user: {
846       id: user._id,
847       name: user.name,
848       email: user.email,
849       role: user.role,
850       isActive: user.isActive,
851       isVerified: user.isVerified,
852     },
853   });
854 } catch (error) {
855   return res.status(500).json({ message: buildErrorMessage(error) });
856 }
857 };
858
859 // ADMIN: Permanently delete user
860 exports.adminDeleteUser = async (req, res) => {
861   try {
862     const user = await User.findById(req.params.id);
863     if (!user) return res.status(404).json({ message: "User not found" });
864     if (String(user._id) === String(req.user.id)) {
865       return res.status(400).json({ message: "You cannot delete your own account" });
866     }
867
868     await User.deleteOne({ _id: user._id });
869
870     logAudit({
871       userId: req.user.id,
872       action: "user_deleted",
873       resourceType: "user",
874       resourceId: String(user._id),
875       details: { targetEmail: user.email },
876     });
877
878     return res.json({ message: "User deleted successfully" });
879   } catch (error) {
880     return res.status(500).json({ message: buildErrorMessage(error) });
881   }
882 };
883
884 // ADMIN: Reset password with admin-defined temporary password
885 exports.adminResetUserPassword = async (req, res) => {
886   try {
887     const { temporaryPassword } = req.body;
888     if (!isStrongPassword(temporaryPassword)) {
889       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
890     }
891
892     const user = await User.findById(req.params.id);
893     if (!user) return res.status(404).json({ message: "User not found" });
894
895     user.password = await bcrypt.hash(temporaryPassword, 12);
896     user.isVerified = false;
897     user.otp = null;
898     user.otpExpiry = null;
899     user.resetToken = null;
900     user.resetTokenExpiry = null;
901     user.trustedDevices = [];
902     await user.save();
903
904     await sendVerificationOtpToUser(user);
905     await sendWelcomeEmail(user.email, user.name, temporaryPassword);
906     await sendPasswordChangedAlertEmail(user.email, user.name, "admin temporary reset");
907
908     logAudit({
909       userId: req.user.id,
```

```
910         action: "password_reset_triggered_by_admin",
911         resourceType: "user",
912         resourceId: String(user._id),
913         details: { targetEmail: user.email, forcedReverify: true },
914     });
915
916     return res.json({ message: `Temporary password set. Security alerts sent to ${user.email}.` });
917 } catch (error) {
918     return res.status(500).json({ message: buildErrorMessage(error) });
919 }
920 };
```

9. server/controllers/requestController.js (996 lines)

```

1  const crypto = require("crypto");
2  const mongoose = require("mongoose");
3  const Request = require("../models/Request");
4  const User = require("../models/User");
5  const AuditLog = require("../models/AuditLog");
6  const { sendStatusUpdateEmail, sendSigningLinkEmail, sendAgreementApprovedEmail } = require("../utils/emailService");
7
8  const logAudit = (data) => {
9    AuditLog.create(data).catch(() => {});
10 };
11
12 const STATUS_GROUPS = {
13   reviewal: ["nda_pending", "agr_pending_1", "agr_pending_2"],
14   approved: ["nda_approved", "agr_approved"],
15   stud_revision: ["stud_revision_requested"],
16   rep_revision: ["agr_rep_revision_requested"],
17   rep_declined: ["agr_declined"],
18   awaiting_rep: ["agr_awaiting_rep_signature"],
19 };
20
21 const resolveStatusFilter = (statusQuery) => {
22   if (!statusQuery) return [];
23
24   const tokens = String(statusQuery)
25     .split(",")
26     .map(v => v.trim())
27     .filter(Boolean);
28
29   const resolved = tokens.flatMap((token) => STATUS_GROUPS[token] || [token]);
30   return [...new Set(resolved)];
31 };
32
33 const escapeRegex = (value) => String(value).replace(/[\.\*\?\^\$\{\}|\[\]\$\&]/g, "\\$&");
34
35 const SERIAL_SUFFIX_CHARS = "ABCDEFGHGIJKLMNOPQRSTUVWXYZ0123456789";
36
37 const pad2 = (n) => String(n).padStart(2, "0");
38
39 const getSerialDatePart = (date = new Date()) => {
40   const year = date.getFullYear();
41   const month = pad2(date.getMonth() + 1);
42   const day = pad2(date.getDate());
43   return `${year}-${month}-${day}`;
44 };
45
46 const generateSerialSuffix = () => {
47   let suffix = "";
48   for (let i = 0; i < 4; i += 1) {
49     const idx = crypto.randomInt(0, SERIAL_SUFFIX_CHARS.length);
50     suffix += SERIAL_SUFFIX_CHARS[idx];
51   }
52   return suffix;
53 };
54
55 const buildSerialNo = (type) => {
56   const prefix = type === "nda" ? "NDA" : "DATA";
57   return `${prefix}-${getSerialDatePart()}-${generateSerialSuffix()}`;
58 };
59
60 const generateUniqueSerialNo = async (type) => {
61   const maxAttempts = 12;
62   for (let attempt = 0; attempt < maxAttempts; attempt += 1) {
63     const serialNo = buildSerialNo(type);
64     const exists = await Request.exists({ serialNo });
65     if (!exists) return serialNo;
66   }
67
68   throw new Error("Failed to generate a unique serial number");
69 };
70
71 const generateSigningTokenValue = () =>
72   crypto.randomBytes(20).toString("hex");
73

```



```
150     fullName: buildProxyFullName(proxyRequestee),
151     email: String(proxyRequestee.email || "").trim(),
152     idNumber: String(proxyRequestee.idNumber || "").trim(),
153     departmentOrOrganization: String(proxyRequestee.departmentOrOrganization || "").trim(),
154   }
155   : {
156     isProxy: false,
157     staffUserId: null,
158     firstName: "",
159     middleInitial: "",
160     lastName: "",
161     fullName: "",
162     email: "",
163     idNumber: "",
164     departmentOrOrganization: "",
165   },
166 };
167
168 if (type === "agreement") {
169   createPayload.authorizerSigUrl = authorizerSigUrl || "";
170   createPayload.authorizerSigPath = authorizerSigPath || "";
171   createPayload.studentSignedAt = new Date();
172 }
173
174 if (type === "nda") {
175   createPayload.studentSigUrl = studentSigUrl || "";
176   createPayload.studentSigPath = studentSigPath || "";
177   createPayload.studentSignedAt = new Date();
178 }
179
180 const created = await Request.create(createPayload);
181
182 logAudit({
183   userId: req.user.id,
184   action: "request_created",
185   resourceType: "request",
186   resourceId: String(created._id),
187   details: { type, status: initialStatus, isProxy: isStaffProxy },
188 });
189
190 return res.status(201).json({ message: "Request submitted", request: created });
191 } catch (err) {
192   return res.status(500).json({ message: err.message });
193 }
194 };
195
196 exports.getMyRequests = async (req, res) => {
197   try {
198     const list = await Request.find({ userId: req.user.id, isArchived: { $ne: true } })
199       .sort({ createdAt: -1 })
200       .select("_id type status formData createdAt proxyRequestee postdocs isArchived")
201       .lean();
202
203     return res.json(list);
204   } catch (err) {
205     return res.status(500).json({ message: err.message });
206   }
207 };
208
209 exports.resubmitRequest = async (req, res) => {
210   try {
211     const { formData, predocs, authorizerSigUrl, authorizerSigPath, studentSigUrl, studentSigPath } = req.body;
212
213     const r = await Request.findById(req.params.id);
214     if (!r) return res.status(404).json({ message: "Request not found" });
215
216     if (String(r.userId) !== String(req.user.id)) {
217       return res.status(403).json({ message: "Forbidden" });
218     }
219
220     if (r.status !== "stud_revision_requested") {
221       return res.status(400).json({ message: "Only revision-requested requests can be resubmitted" });
222     }
223
224     r.formData = formData && typeof formData === "object" ? formData : r.formData;
225     r.resubmissionCount = (r.resubmissionCount || 0) + 1;
```

[illegible]

```
302     return res.status(400).json({ message: "Invalid endDate" });
303   }
304   parsedEnd.setHours(23, 59, 59, 999);
305   query.createdAt.$lte = parsedEnd;
306 }
307 }
308
309 if (search && String(search).trim()) {
310   const term = String(search).trim();
311   const regex = new RegExp(escapeRegex(term), "i");
312
313   const userOr = [
314     { name: regex },
315     { email: regex },
316   ];
317
318   if (mongoose.Types.ObjectId.isValid(term)) {
319     userOr.push({ _id: term });
320   }
321
322   const matchingUsers = await User.find({ $or: userOr }).select("_id");
323   const matchedUserIds = matchingUsers.map((u) => u._id);
324
325   const searchableFormFields = [
326     "formData.purpose",
327     "formData.organizationName",
328     "formData.organization",
329     "formData.orgName",
330     "formData.tags",
331     "formData.title",
332     "formData.projectTitle",
333     "formData.ndaTypeLabel",
334     "proxyRequestee.fullName",
335     "proxyRequestee.email",
336     "proxyRequestee.idNumber",
337     "proxyRequestee.departmentOrOrganization",
338   ];
339
340   const formDataClauses = searchableFormFields.map((field) => ({ [field]: regex }));
341
342   const searchClauses = [
343     { serialNo: regex },
344     ...formDataClauses,
345   ];
346
347   if (matchedUserIds.length) {
348     searchClauses.push({ userId: { $in: matchedUserIds } });
349   }
350
351   if (mongoose.Types.ObjectId.isValid(term)) {
352     searchClauses.push({ _id: term });
353   }
354
355   // Partial Request ID match (last 7 chars of hex _id, case-insensitive)
356   searchClauses.push({
357     $expr: {
358       $regexMatch: {
359         input: { $toString: "$_id" },
360         regex: term,
361         options: "i",
362       },
363     },
364   });
365
366   query.$or = searchClauses;
367 }
368
369 const list = await Request.find(query)
370   .populate("userId", "name email role")
371   .sort({ createdAt: -1 })
372   .select("-__v")
373   .lean();
374
375 return res.json(list);
376 } catch (err) {
377   return res.status(500).json({ message: err.message });
378 }
```



```
378   }
379 };
380
381 exports.getRequestById = async (req, res) => {
382   try {
383     const r = await Request.findById(req.params.id)
384       .populate("userId", "name email role")
385       .select("-__v");
386
387     if (!r) return res.status(404).json({ message: "Request not found" });
388
389     const isPrivilegedViewer = req.user.role === "admin" || req.user.role === "staff";
390     const isAdmin = req.user.role === "admin";
391     const ownerId = r.userId?._id || r.userId;
392     const isOwner = String(ownerId) === String(req.user.id);
393
394     if (!isPrivilegedViewer && !isOwner) return res.status(403).json({ message: "Forbidden" });
395
396     // Strip signing token from non-privileged (student) responses for security
397     if (!isPrivilegedViewer) {
398       const obj = r.toObject();
399       delete obj.signingToken;
400       return res.json(obj);
401     }
402
403     return res.json(r);
404   } catch (err) {
405     return res.status(500).json({ message: err.message });
406   }
407 };
408
409 // Admin-only workflow updates for NDA and initial Agreement review phases.
410 exports.updateRequestStatus = async (req, res) => {
411   try {
412     const { id } = req.params;
413     const { status, remarks, adminSigUrl, adminSigPath } = req.body;
414
415     const allowedStatuses = [
416       "nda_approved",
417       "stud_revision_requested",
418     ];
419     if (!status || !allowedStatuses.includes(status)) {
420       return res.status(400).json({ message: "Invalid status" });
421     }
422
423     if (!mongoose.Types.ObjectId.isValid(id)) {
424       return res.status(400).json({ message: "Invalid request id" });
425     }
426
427     const existing = await Request.findById(id).select("type status serialNo");
428     if (!existing) return res.status(404).json({ message: "Request not found" });
429
430     const isNda = existing.type === "nda";
431     const isAgreement = existing.type === "agreement";
432
433     if (isNda) {
434       const allowedFrom = ["nda_pending"];
435       if (!allowedFrom.includes(existing.status)) {
436         return res.status(400).json({ message: "Only pending NDA requests can be updated here" });
437       }
438       const allowedTargets = ["nda_approved", "stud_revision_requested"];
439       if (!allowedTargets.includes(status)) {
440         return res.status(400).json({ message: "Invalid target status for NDA" });
441       }
442     }
443
444     if (isAgreement) {
445       const allowedFrom = ["agr_pending_1"];
446       if (!allowedFrom.includes(existing.status)) {
447         return res.status(400).json({ message: "Use agreement-specific endpoints for this stage" });
448       }
449       const allowedTargets = ["stud_revision_requested"];
450       if (!allowedTargets.includes(status)) {
451         return res.status(400).json({ message: "Invalid target status for Agreement initial review" });
452       }
453     }
454   }
455 }
```

```
454
455     const updatePayload = { status, remarks: remarks || "" };
456
457     if (status === "nda_approved" && !existing.serialNo) {
458       updatePayload.serialNo = await generateUniqueSerialNo("nda");
459     }
460
461     // Store admin signature for NDA approvals
462     if (status === "nda_approved" && existing.type === "nda" && adminSigUrl) {
463       updatePayload.adminSigUrl = adminSigUrl;
464       updatePayload.adminSigPath = adminSigPath || "";
465       updatePayload.adminSignedAt = new Date();
466     }
467
468     const updated = await Request.findByIdAndUpdate(id, updatePayload, { new: true }).select("-__v");
469     if (!updated) return res.status(404).json({ message: "Request not found" });
470
471     logAudit({
472       userId: req.user.id,
473       action: status === "nda_approved" ? "request_approved" : "request_revision_required",
474       resourceType: "request",
475       resourceId: String(id),
476       details: { newStatus: status, type: existing.type },
477     });
478
479     // Notify student by email
480     User.findById(updated.userId).then((owner) => {
481       if (owner?.email) {
482         sendStatusUpdateEmail(owner.email, owner.name, updated.type, status, remarks || "").catch((emailErr) => {
483           console.error("Request status notification email failed:", emailErr.message);
484         });
485       }
486     }).catch(() => {});
487
488     return res.json({ message: "Request updated", request: updated });
489   } catch (err) {
490     return res.status(500).json({ message: err.message });
491   }
492 };
493
494 exports.saveApprovedDocument = async (req, res) => {
495   try {
496     const { id } = req.params;
497     const { url, path, issuedAt, verificationUrl } = req.body;
498
499     if (!url || !path) return res.status(400).json({ message: "url and path are required" });
500     if (!mongoose.Types.ObjectId.isValid(id)) return res.status(400).json({ message: "Invalid request id" });
501
502     const r = await Request.findById(id);
503     if (!r) return res.status(404).json({ message: "Request not found" });
504
505     const validFinalStatuses = ["nda_approved", "agr_approved"];
506     if (!validFinalStatuses.includes(r.status)) {
507       return res.status(400).json({ message: "Request is not in a final approved state" });
508     }
509
510     r.postdocs = {
511       url,
512       path,
513       issuedAt: issuedAt || new Date().toISOString(),
514       verificationUrl: verificationUrl || "",
515     };
516
517     // Volatile signatures: immediately nullify signature data after PDF is saved
518     r.studentSigUrl = "";
519     r.studentSigPath = "";
520     r.adminSigUrl = "";
521     r.adminSigPath = "";
522     r.authorizerSigUrl = "";
523     r.authorizerSigPath = "";
524     r.repSigUrl = "";
525     r.repSigPath = "";
526
527     await r.save();
528
529     // Email representative with approved document link (agreement requests only)
```

[illegible]

```
606     signingToken: r.signingToken,
607     signingTokenExpiresAt: r.signingTokenExpiresAt,
608     request: r,
609   });
610 } catch (err) {
611   return res.status(500).json({ message: err.message });
612 }
613 };
614
615 // Public: get request data needed for the signing page
616 exports.getBySigningToken = async (req, res) => {
617   try {
618     const { token } = req.params;
619
620     if (!token) return res.status(400).json({ message: "Missing token" });
621
622     const r = await Request.findOne({ signingToken: token })
623       .populate("userId", "name email")
624       .select("_id formData status signingTokenUsed signingTokenExpiresAt authorizerSigUrl authorizerSigPath remarks repInfo")
625       .lean();
626
627     if (!r) return res.status(404).json({ message: "Invalid signing link" });
628
629     // Check for link expiry
630     if (r.signingTokenExpiresAt && new Date(r.signingTokenExpiresAt) < new Date()) {
631       return res.status(410).json({ message: "This signing link has expired. Please request a new one." });
632     }
633
634     return res.json(r);
635   } catch (err) {
636     return res.status(500).json({ message: err.message });
637   }
638 };
639
640 // Public: rep submits on the signing page
641 exports.repSubmit = async (req, res) => {
642   try {
643     const { token } = req.params;
644     const { repName, repGovIdDoc, repSigUrl, repSigPath } = req.body;
645
646     if (!token) return res.status(400).json({ message: "Missing token" });
647
648     const r = await Request.findOne({ signingToken: token });
649     if (!r) return res.status(404).json({ message: "Invalid signing link" });
650
651     if (r.signingTokenUsed) {
652       return res.status(400).json({ message: "This signing link has already been used" });
653     }
654
655     // Check for link expiry
656     if (r.signingTokenExpiresAt && new Date(r.signingTokenExpiresAt) < new Date()) {
657       return res.status(410).json({ message: "This signing link has expired. Please request a new one." });
658     }
659
660     const allowedStatuses = ["agr_awaiting_rep_signature", "agr_rep_revision_requested"];
661     if (!allowedStatuses.includes(r.status)) {
662       return res.status(400).json({ message: "This request is not awaiting representative signature" });
663     }
664
665     if (!repName || !repName.trim()) {
666       return res.status(400).json({ message: "Representative name is required" });
667     }
668     if (!repSigUrl) {
669       return res.status(400).json({ message: "Representative signature is required" });
670     }
671     if (!repGovIdDoc || !repGovIdDoc.url) {
672       return res.status(400).json({ message: "Government ID is required" });
673     }
674
675     r.repInfo = {
676       name: repName.trim(),
677       govIdDoc: repGovIdDoc,
678     };
679     r.repSigUrl = repSigUrl;
680     r.repSigPath = repSigPath || "";
681     r.repSignedAt = new Date();
```

```
682     r.signingTokenUsed = true;
683     r.status = "agr_pending_2";
684     r.remarks = "";
685
686     await r.save();
687
688     logAudit({
689       action: "rep_signature_submitted",
690       resourceType: "request",
691       resourceId: String(r._id),
692       details: { repName: repName.trim() },
693     });
694
695     return res.json({ message: "Submission received. Thank you." });
696   } catch (err) {
697     return res.status(500).json({ message: err.message });
698   }
699 };
700
701 // Public: rep rejects on the signing page
702 exports.repReject = async (req, res) => {
703   try {
704     const { token } = req.params;
705
706     if (!token) return res.status(400).json({ message: "Missing token" });
707
708     const r = await Request.findOne({ signingToken: token });
709     if (!r) return res.status(404).json({ message: "Invalid signing link" });
710
711     if (r.signingTokenUsed) {
712       return res.status(400).json({ message: "This signing link has already been used" });
713     }
714
715     const allowedStatuses = ["agr_awaiting_rep_signature", "agr_rep_revision_requested"];
716     if (!allowedStatuses.includes(r.status)) {
717       return res.status(400).json({ message: "This request is not awaiting representative signature" });
718     }
719
720     r.signingTokenUsed = true;
721     r.status = "agr_declined";
722
723     await r.save();
724
725     logAudit({
726       action: "rep_signature_declined",
727       resourceType: "request",
728       resourceId: String(r._id),
729     });
730
731     return res.json({ message: "You have declined the signing request." });
732   } catch (err) {
733     return res.status(500).json({ message: err.message });
734   }
735 };
736
737 // Admin: agreement_final_admin_reviewal -> agreement_approved or agreement_rep_revision_requested
738 exports.adminPhase3Action = async (req, res) => {
739   try {
740     const { id } = req.params;
741     const { action, remarks } = req.body;
742
743     if (!["approve", "rep_revision_requested"].includes(action)) {
744       return res.status(400).json({ message: "action must be 'approve' or 'rep_revision_requested'" });
745     }
746
747     if (!mongoose.Types.ObjectId.isValid(id)) {
748       return res.status(400).json({ message: "Invalid request id" });
749     }
750
751     const r = await Request.findById(id);
752     if (!r) return res.status(404).json({ message: "Request not found" });
753
754     if (r.type !== "agreement") {
755       return res.status(400).json({ message: "Only agreement requests use this endpoint" });
756     }
757     if (r.status !== "agr_pending_2") {
```

```

758     return res.status(400).json({ message: `Cannot perform phase3 action from status: ${r.status}` });
759   }
760 
761   if (action === "approve") {
762     r.status = "agr_approved";
763     if (!r.serialNo) r.serialNo = await generateUniqueSerialNo("agreement");
764     r.remarks = "";
765   } else {
766     // rep_revision_requested: generate new signing token
767     r.signingToken = generateSigningTokenValue();
768     r.signingTokenUsed = false;
769     r.signingTokenExpiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);
770     r.status = "agr_rep_revision_requested";
771     r.remarks = remarks || "";
772   }
773 
774   await r.save();
775 
776   logAudit({
777     userId: req.user.id,
778     action: action === "approve" ? "request_approved" : "request_revision_required",
779     resourceType: "request",
780     resourceId: String(id),
781     details: { newStatus: r.status, type: r.type },
782   });
783 
784   // Notify student
785   User.findById(r.userId).then((owner) => {
786     if (owner?.email) {
787       sendStatusUpdateEmail(owner.email, owner.name, r.type, r.status, r.remarks).catch((emailErr) => {
788         console.error("Agreement status notification email failed:", emailErr.message);
789       });
790     }
791   }).catch(() => {});
792 
793   // Send new signing link to representative on revision
794   if (action === "rep_revision_requested") {
795     const repEmail = r.formData?.repEmail;
796     const repName = [r.formData?.repFirstName, r.formData?.repLastName].filter(Boolean).join(" ") || "Representative";
797     const requestOwner = await User.findById(r.userId).select("name").lean();
798     const requestorName = r.proxyRequestee?.isProxy ? r.proxyRequestee.fullName : requestOwner?.name;
799     const origin = req.headers.origin || process.env.CLIENT_ORIGIN || "http://localhost:5173";
800     const signingLink = `${origin}/sign/${r.signingToken}`;
801     if (repEmail) {
802       sendSigningLinkEmail(repEmail, repName, requestorName, signingLink, r.signingTokenExpiresAt, true).catch((err) => {
803         console.error("Failed to send revision signing link email:", err.message);
804       });
805     }
806   }
807 
808   return res.json({
809     message: action === "approve" ? "Agreement approved" : "Rep revision requested",
810     signingToken: action === "rep_revision_requested" ? r.signingToken : undefined,
811     signingTokenExpiresAt: action === "rep_revision_requested" ? r.signingTokenExpiresAt : undefined,
812     request: r,
813   });
814 } catch (err) {
815   return res.status(500).json({ message: err.message });
816 }
817 };
818 
819 // ■■■ Admin: proxy Firebase signature images to avoid browser CORS ■■■■■■■■■■■■
820 
821 exports.getSignatureImages = async (req, res) => {
822   try {
823     const request = await Request.findById(req.params.id)
824       .select("authorizerSigUrl repSigUrl studentSigUrl adminSigUrl type");
825     if (!request) return res.status(404).json({ message: "Request not found" });
826 
827     const toDataUrl = async (url) => {
828       if (!url) return null;
829       const response = await fetch(url);
830       const buffer = await response.arrayBuffer();
831       const mimeType = response.headers.get("content-type") || "image/png";
832       const base64 = Buffer.from(buffer).toString("base64");
833       return `data:${mimeType};base64,${base64}`;

```

[illegible]

[illegible]


```
986         { $set: { isArchived: true, archivedAt: new Date() } }
987     );
988
989     res.json({
990         message: `Archived ${result.modifiedCount} request(s).`,
991         archivedCount: result.modifiedCount,
992     });
993 } catch (err) {
994     res.status(500).json({ message: err.message });
995 }
996 };
```

10. server/controllers/auditController.js (47 lines)

```
1  const AuditLog = require("../models/AuditLog");
2
3  // GET /api/audit - Admin only, paginated
4  exports.getAuditLogs = async (req, res) => {
5    try {
6      const limit = Math.min(parseInt(req.query.limit) || 50, 200);
7      const page = Math.max(parseInt(req.query.page) || 1, 1);
8      const skip = (page - 1) * limit;
9      const { action, userId, resourceId } = req.query;
10
11      const filter = {};
12      if (action) filter.action = action;
13      if (userId) filter.userId = userId;
14      if (resourceId) filter.resourceId = resourceId;
15
16      const [logs, total] = await Promise.all([
17        AuditLog.find(filter)
18          .populate("userId", "name email role")
19          .sort({ createdAt: -1 })
20          .skip(skip)
21          .limit(limit)
22          .select("-__v")
23          .lean(),
24        AuditLog.countDocuments(filter),
25      ]);
26
27      res.json({ logs, total, page, limit });
28    } catch (error) {
29      res.status(500).json({ message: error.message });
30    }
31  };
32
33  // GET /api/audit/recent - last N entries for dashboard widget
34  exports.getRecentAuditLogs = async (req, res) => {
35    try {
36      const count = Math.min(parseInt(req.query.count) || 5, 20);
37      const logs = await AuditLog.find()
38        .populate("userId", "name email role")
39        .sort({ createdAt: -1 })
40        .limit(count)
41        .select("-__v")
42        .lean();
43      res.json(logs);
44    } catch (error) {
45      res.status(500).json({ message: error.message });
46    }
47  };
```

11. server/routes/authRoutes.js (67 lines)

```
1  const express = require("express");
2  const router = express.Router();
3  const rateLimit = require("express-rate-limit");
4
5
6  const authLimiter = rateLimit({
7    windowMs: 15 * 60 * 1000,
8    max: 50,
9    standardHeaders: true,
10   legacyHeaders: false,
11   message: { message: "Too many authentication attempts. Please try again later." },
12 });
13
14 const {
15   register,
16   login,
17   verifyLoginOtp,
18   resendLoginOtp,
19   verifyEmail,
20   resendVerificationOtp,
21   activateAccount,
22   forgotPassword,
23   resendResetOtp,
24   verifyResetOtp,
25   resetPassword,
26   refreshToken,
27   updateProfile,
28   requestPasswordChangeOtp,
29   getAllUsers,
30   adminCreateUser,
31   toggleUserActive,
32   adminUpdateUser,
33   adminDeleteUser,
34   adminResetUserPassword,
35 } = require("../controllers/authController");
36 const { protect, authorizeAdmin } = require("../middleware/authMiddleware");
37
38 // Public (rate-limited)
39 router.post("/register", authLimiter, register);
40 router.post("/login", authLimiter, login);
41 router.post("/verify-login-otp", authLimiter, verifyLoginOtp);
42 router.post("/resend-login-otp", authLimiter, resendLoginOtp);
43 router.post("/verify-email", authLimiter, verifyEmail);
44 router.post("/resend-verification-otp", authLimiter, resendVerificationOtp);
45 router.post("/activate-account", authLimiter, activateAccount);
46 router.post("/forgot-password", authLimiter, forgotPassword);
47 router.post("/resend-reset-otp", authLimiter, resendResetOtp);
48 router.post("/verify-reset-otp", authLimiter, verifyResetOtp);
49 router.post("/reset-password", authLimiter, resetPassword);
50
51 // Authenticated
52 router.post("/refresh-token", protect, refreshToken);
53 router.post("/request-password-change-otp", protect, requestPasswordChangeOtp);
54 router.patch("/profile", protect, updateProfile);
55
56 // Admin-only
57 router.get("/admin", protect, authorizeAdmin, (req, res) => {
58   res.json({ message: "Admin access granted" });
59 });
60 router.get("/users", protect, authorizeAdmin, getAllUsers);
61 router.post("/users", protect, authorizeAdmin, adminCreateUser);
62 router.patch("/users/:id/toggle-active", protect, authorizeAdmin, toggleUserActive);
63 router.patch("/users/:id", protect, authorizeAdmin, adminUpdateUser);
64 router.delete("/users/:id", protect, authorizeAdmin, adminDeleteUser);
65 router.post("/users/:id/reset-password", protect, authorizeAdmin, adminResetUserPassword);
66
67 module.exports = router;
```

12. server/routes/requestRoutes.js (58 lines)

```

1 const express = require("express");
2 const router = express.Router();
3
4 const { protect, authorizeRoles } = require("../middleware/authMiddleware");
5 const {
6   createRequest,
7   getMyRequests,
8   resubmitRequest,
9   getAllRequests,
10  saveApprovedDocument,
11  updateRequestStatus,
12  getRequestById,
13  verifyRequestCode,
14  generateSigningLink,
15  getBySigningToken,
16  repSubmit,
17  repReject,
18  adminPhase3Action,
19  getSignatureImages,
20  getRequestStats,
21  getArchivedRequests,
22  getRetentionStats,
23  runArchiveNow,
24 } = require("../controllers/requestController");
25
26 // ■■ Public ████████████████████████████████████████████████████████████
27 router.get("/verify/:code", verifyRequestCode);
28
29 // Agreement signing page (public - no auth required)
30 router.get("/sign/:token", getBySigningToken);
31 router.post("/sign/:token/submit", repSubmit);
32 router.post("/sign/:token/reject", repReject);
33
34 // ■■ Student ████████████████████████████████████████████████████████████
35 router.post("/", protect, createRequest);
36 router.get("/my", protect, getMyRequests);
37 router.patch("/:id/resubmit", protect, resubmitRequest);
38
39 // ■■ Admin ████████████████████████████████████████████████████████████
40 router.get("/all", protect, authorizeRoles("admin", "staff"), getAllRequests);
41 router.get("/stats", protect, authorizeRoles("admin", "staff"), getRequestStats);
42 router.get("/archived", protect, authorizeRoles("admin", "staff"), getArchivedRequests);
43 router.get("/retention-stats", protect, authorizeRoles("admin"), getRetentionStats);
44 router.post("/run-archive", protect, authorizeRoles("admin"), runArchiveNow);
45 router.patch("/:id/approved-document", protect, authorizeRoles("admin", "staff"), saveApprovedDocument);
46
47 // Agreement-specific admin actions
48 router.patch("/:id/generate-signing-link", protect, authorizeRoles("admin", "staff"), generateSigningLink);
49 router.patch("/:id/admin-phase3", protect, authorizeRoles("admin"), adminPhase3Action);
50 router.get("/:id/sig-images", protect, authorizeRoles("admin", "staff"), getSignatureImages);
51
52 // NDA status update (pending → approved | revision_required)
53 router.patch("/:id", protect, authorizeRoles("admin"), updateRequestStatus);
54
55 router.get("/:id", protect, getRequestById);
56
57 module.exports = router;
58
```

13. server/routes/auditRoutes.js (9 lines)

```
1 const express = require("express");
2 const router = express.Router();
3 const { getAuditLogs, getRecentAuditLogs } = require("../controllers/auditController");
4 const { protect, authorizeRoles } = require("../middleware/authMiddleware");
5
6 router.get("/", protect, authorizeRoles("admin"), getAuditLogs);
7 router.get("/recent", protect, authorizeRoles("admin"), getRecentAuditLogs);
8
9 module.exports = router;
```

14. server/utils/emailService.js (346 lines)

```
1  const SibApiV3Sdk = require("sib-api-v3-sdk");
2
3  const EMAIL_FAILURE_MESSAGE = "Failed to send verification email. Please try again later.";
4
5  class EmailDeliveryError extends Error {
6    constructor(message, statusCode, details) {
7      super(message || EMAIL_FAILURE_MESSAGE);
8      this.name = "EmailDeliveryError";
9      this.isEmailError = true;
10     this.statusCode = statusCode || 500;
11     this.publicMessage = EMAIL_FAILURE_MESSAGE;
12     this.details = details || null;
13   }
14 }
15
16 const getBrevoClient = () => {
17   const apiKey = process.env.BREVO_API_KEY;
18   if (!apiKey) {
19     throw new EmailDeliveryError("Missing BREVO_API_KEY", 500);
20   }
21
22   const defaultClient = SibApiV3Sdk.ApiClient.instance;
23   defaultClient.authentications["api-key"].apiKey = apiKey;
24   return new SibApiV3Sdk.TransactionalEmailsApi();
25 };
26
27 const getSender = () => {
28   const email = process.env.BREVO_SENDER_EMAIL;
29   const name = process.env.BREVO_SENDER_NAME || "RTU DPO Portal";
30
31   if (!email) {
32     throw new EmailDeliveryError("Missing BREVO_SENDER_EMAIL", 500);
33   }
34
35   return { email, name };
36 };
37
38 const normalizeRecipients = (to) => {
39   if (!to) return [];
40
41   if (Array.isArray(to)) {
42     return to
43       .filter(Boolean)
44       .map((email) => ({ email: String(email).trim() }))
45       .filter((recipient) => recipient.email.length > 0);
46   }
47
48   return [{ email: String(to).trim() }].filter((recipient) => recipient.email.length > 0);
49 };
50
51 const sendBrevoEmail = async ({ to, subject, htmlContent }) => {
52   try {
53     const client = getBrevoClient();
54     const recipients = normalizeRecipients(to);
55     if (!recipients.length) {
56       throw new EmailDeliveryError("Recipient email is required", 400);
57     }
58
59     const payload = new SibApiV3Sdk.SendSmtEmail();
60     payload.sender = getSender();
61     payload.to = recipients;
62     payload.subject = subject;
63     payload.htmlContent = htmlContent;
64
65     return await client.sendTransacEmail(payload);
66   } catch (error) {
67     if (error?.isEmailError) {
68       throw error;
69     }
70
71     const statusCode = error?.status || error?.response?.status || error?.response?.statusCode || 500;
72     const details = error?.response?.body || error?.response?.data || error?.message || "Unknown Brevo error";
73     const brevoMessage = typeof details === "string" ? details : JSON.stringify(details);
```

```

74
75     console.error(`[Brevo] Email send failed (${statusCode}): ${brevoMessage}`);
76
77     throw new EmailDeliveryError(`Brevo API request failed (${statusCode})`, statusCode, details);
78 }
79 };
80
81 /**
82  * Send a 6-digit OTP for login verification or password reset.
83  */
84 exports.sendOtpEmail = (email, otp, purpose = "login") => {
85     const label = purpose === "reset" ? "Password Reset" : purpose === "change" ? "Password Change" : "Login Verification";
86     return sendBrevoEmail({
87         to: email,
88         subject: `RTU DPO Portal - Your ${label} OTP`,
89         htmlContent: `
90             <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
91             <div style="background:#0f2d6b;padding:24px 32px">
92                 <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
93             </div>
94             <div style="padding:32px">
95                 <p style="color:#0f172a;font-size:15px;margin-top:0">Your <strong>${label} OTP</strong> is:</p>
96                 <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
97                 <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. Do not share it with anyone.
98                 <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
99                 <p style="color:#94a3b8;font-size:12px;margin:0">If you did not request this, please ignore this email or contact your administrator.
100             </div>
101         `;
102     });
103 };
104
105 /**
106  * Send a password-reset link.
107  */
108 exports.sendPasswordResetEmail = (email, resetUrl) => {
109     sendBrevoEmail({
110         to: email,
111         subject: "RTU DPO Portal - Password Reset Request",
112         htmlContent: `
113             <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
114             <div style="background:#0f2d6b;padding:24px 32px">
115                 <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
116             </div>
117             <div style="padding:32px">
118                 <p style="color:#0f172a;font-size:15px;margin-top:0">We received a request to reset your password.</p>
119                 <a href="${resetUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:8px;
120                 <p style="color:#64748b;font-size:13px;margin-top:20px">This link expires in <strong>1 hour</strong>. If you did not request this, please ignore this email or contact your administrator.
121             </div>
122         `;
123     });
124
125 /**
126  * Notify a user that their request status has changed.
127  */
128 exports.sendStatusUpdateEmail = (email, name, requestType, newStatus, remarks) => {
129     const statusLabels = {
130         nda_pending: "Reviewal",
131         nda_approved: "Approved",
132         stud_revision_requested: "Student Revisions",
133         agr_pending_1: "Initial Reviewal",
134         agr_awaiting_rep_signature: "Recipient Reviewal",
135         agr_pending_2: "Final Reviewal",
136         agr_approved: "Approved",
137         agr_declined: "Recipient Declined",
138         agr_rep_revision_requested: "Recipient Revisions",
139     };
140     const label = statusLabels[newStatus] || newStatus;
141     const remarksBlock = remarks
142         ? `<div style="background:#f8fafc;border:1px solid #e2e8f0;border-radius:8px;padding:12px 16px;margin-top:16px"><strong>Remarks</strong>:
143         : "";
144
145     return sendBrevoEmail({
146         to: email,
147         subject: `RTU DPO Portal - Your ${requestType.toUpperCase()} Request has been updated`,
148         htmlContent: `
149             <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:

```

```
150     <div style="background:#0f2d6b;padding:24px 32px">
151       <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
152     </div>
153     <div style="padding:32px">
154       <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong></p>
155       <p style="color:#475569;font-size:14px">Your <strong>${requestType.toUpperCase()}</strong> request status has been
156       <div style="font-size:20px;font-weight:700;color:#0f2d6b;padding:8px 0">${label}</div>
157       ${remarksBlock}
158       <p style="color:#94a3b8;font-size:12px;margin-top:24px">Log in to the RTU DPO Portal for more details.</p>
159     </div>
160   </div>`,
161   });
162 };
163
164 /**
165  * Send a welcome / account-created email with a temporary password.
166  */
167 exports.sendWelcomeEmail = (email, name, tempPassword) =>
168   sendBrevoEmail({
169     to: email,
170     subject: "RTU DPO Portal - Your Account Has Been Created",
171     htmlContent: `
172       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
173       <div style="background:#0f2d6b;padding:24px 32px">
174         <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
175       </div>
176       <div style="padding:32px">
177         <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>, your account has been created.</p>
178         <p style="color:#475569;font-size:14px">Temporary password: <strong style="color:#0f2d6b">${tempPassword}</strong></p>
179         <p style="color:#64748b;font-size:13px">Please log in and change your password immediately.</p>
180       </div>
181     </div>`,
182   });
183
184 /**
185  * Send an email verification link.
186  */
187 exports.sendVerificationEmail = (email, name, verifyUrl) =>
188   sendBrevoEmail({
189     to: email,
190     subject: "RTU DPO Portal - Verify Your Email",
191     htmlContent: `
192       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
193       <div style="background:#0f2d6b;padding:24px 32px">
194         <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
195       </div>
196       <div style="padding:32px">
197         <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong></p>
198         <p style="color:#475569;font-size:14px">Please verify your email address to activate your account.</p>
199         <a href="${verifyUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:8px">
200         <p style="color:#64748b;font-size:13px">This link expires in <strong>24 hours</strong>. If you did not create this
201       </div>
202     </div>`,
203   });
204
205 /**
206  * Send a welcome email with a verification/activation link (admin-created user, no password set).
207  */
208 exports.sendWelcomeVerificationEmail = (email, name, activateUrl) =>
209   sendBrevoEmail({
210     to: email,
211     subject: "RTU DPO Portal - Welcome! Activate Your Account",
212     htmlContent: `
213       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
214       <div style="background:#0f2d6b;padding:24px 32px">
215         <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
216       </div>
217       <div style="padding:32px">
218         <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong></p>
219         <p style="color:#475569;font-size:14px">An administrator has created an account for you at the RTU DPO Portal. Clic
220         <a href="${activateUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:
221         <p style="color:#64748b;font-size:13px">This link expires in <strong>24 hours</strong>. If you did not expect this,
222       </div>
223     </div>`,
224   });
225
```



```
226 /**
227  * Send a verification OTP email for new account registration.
228  */
229 exports.sendVerificationOtpEmail = (email, name, otp) =>
230   sendBrevoEmail({
231     to: email,
232     subject: "RTU DPO Portal \u2013 Verify Your Email",
233     htmlContent: `
234       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
235         <div style="background:#0f2d6b;padding:24px 32px">
236           <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
237         </div>
238         <div style="padding:32px">
239           <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>,</p>
240           <p style="color:#475569;font-size:14px">Use the code below to verify your email address and activate your account.</p>
241           <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
242           <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. Do not share it with anyone.
243           <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
244           <p style="color:#94a3b8;font-size:12px;margin:0">If you did not create this account, please ignore this email.</p>
245         </div>
246       </div>`,
247   });
248
249 /**
250  * Send a login OTP email for device/IP verification.
251  */
252 exports.sendLoginOtpEmail = (email, otp) =>
253   sendBrevoEmail({
254     to: email,
255     subject: "RTU DPO Portal - Login Verification OTP",
256     htmlContent: `
257       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
258         <div style="background:#0f2d6b;padding:24px 32px">
259           <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
260         </div>
261         <div style="padding:32px">
262           <p style="color:#0f172a;font-size:15px;margin-top:0">A login attempt was made from an unrecognized device or location.
263           <p style="color:#475569;font-size:14px">Your verification code is:</p>
264           <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
265           <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. If you did not attempt to login,
266           </div>
267         </div>`,
268   });
269
270 exports.sendPasswordChangedAlertEmail = (email, name, reason = "account security event") =>
271   sendBrevoEmail({
272     to: email,
273     subject: "RTU DPO Portal - Security Alert: Password Changed",
274     htmlContent: `
275       <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
276         <div style="background:#7f1d1d;padding:24px 32px">
277           <h1 style="color:#fff;margin:0;font-size:18px">Security Alert</h1>
278         </div>
279         <div style="padding:32px">
280           <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name || "User"}</strong>,</p>
281           <p style="color:#475569;font-size:14px">Your account password was changed due to <strong>${reason}</strong>.</p>
282           <p style="color:#475569;font-size:14px">If this was not you, reset your password immediately and contact the DPO officer.
283           <p style="color:#94a3b8;font-size:12px;margin-top:18px">Timestamp: ${new Date().toISOString()}</p>
284         </div>
285       </div>`,
286   });
287
288 exports.sendSigningLinkEmail = (repEmail, repName, requestorName, signingLink, expiresAt, isRevision = false) => {
289   const expiryStr = expiresAt
290     ? new Date(expiresAt).toLocaleDateString("en-PH", { year: "numeric", month: "long", day: "numeric" })
291     : "7 days";
292   const subject = isRevision
293     ? "RTU DPO Portal - Agreement Revision Required"
294     : "RTU DPO Portal - Agreement Signing Request";
295   const headerBg = isRevision ? "#92400e" : "#0f2d6b";
296   const bodyText = isRevision
297     ? `Your previously submitted agreement for the request by <strong>${requestorName || "an RTU student"}</strong> has been
298       ? `You have been designated as the authorized representative for an agreement request submitted by <strong>${requestorName}</strong>`
299   return sendBrevoEmail({
300     to: repEmail,
301     subject,
```

```
302     htmlContent: `
303     <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
304     <div style="background:${headerBg};padding:24px 32px">
305       <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
306     </div>
307     <div style="padding:32px">
308       <p style="color:#0f172a;font-size:15px;margin-top:0">Dear <strong>${repName || "Representative"}</strong>,</p>
309       <p style="color:#475569;font-size:14px">${bodyText}</p>
310       <div style="text-align:center;margin:28px 0">
311         <a href="${signingLink}" style="background:${headerBg};color:#fff;padding:14px 28px;border-radius:8px;text-decoration:underline none">Sign Agreement</a>
312       </div>
313       <p style="color:#64748b;font-size:13px">This link is valid until <strong>${expiryStr}</strong> and can only be used once.
314       <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
315       <p style="color:#94a3b8;font-size:12px;margin:0">If you were not expecting this email, please disregard it or contact us at <a href="${contactEmail}">${contactEmail}</a>
316     </div>
317     </div>`,
318   });
319 };
320
321 exports.sendAgreementApprovedEmail = (repEmail, repName, requestorName, documentUrl, verificationUrl) =>
322   sendBrevoEmail({
323     to: repEmail,
324     subject: "RTU DPO Portal - Agreement Approved",
325     htmlContent: `
326     <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overflow:
327     <div style="background:#065f46;padding:24px 32px">
328       <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
329     </div>
330     <div style="padding:32px">
331       <p style="color:#0f172a;font-size:15px;margin-top:0">Dear <strong>${repName || "Representative"}</strong>,</p>
332       <p style="color:#475569;font-size:14px">We are pleased to inform you that the agreement request submitted by <strong>${requestorName}</strong> has been approved.
333       <p style="color:#475569;font-size:14px">You may access and download the approved agreement document using the button below.
334       <div style="text-align:center;margin:28px 0">
335         <a href="${documentUrl}" style="background:#065f46;color:#fff;padding:14px 28px;border-radius:8px;text-decoration:underline none">Download Agreement</a>
336       </div>
337       <p style="color:#64748b;font-size:13px;text-align:center">You may also verify document authenticity using the link below.
338       <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
339       <p style="color:#94a3b8;font-size:12px;margin:0">This is an automated notification from the RTU Data Protection Office.
340     </div>
341     </div>`,
342   });
343
344 exports.sendBrevoEmail = sendBrevoEmail;
345 exports.EmailDeliveryError = EmailDeliveryError;
346
```

15. server/scripts/runArchive.js (48 lines)

```
1  require('dotenv').config();
2  const connectDB = require('../config/db');
3  const mongoose = require('mongoose');
4  const Request = require('../models/Request');
5
6  (async () => {
7    try {
8      await connectDB();
9      const archiveTimestamp = new Date();
10     const fiveYearsAgo = new Date();
11     fiveYearsAgo.setFullYear(fiveYearsAgo.getFullYear() - 5);
12
13     // Archive ALL requests older than 5 years regardless of status (universal retention policy)
14     const filter = {
15       $and: [
16         {
17           $or: [
18             { isArchived: false },
19             { isArchived: { $exists: false } },
20           ],
21         },
22         { createdAt: { $lte: fiveYearsAgo } },
23       ],
24     };
25
26     const result = await Request.updateMany(filter, {
27       $set: {
28         isArchived: true,
29         archivedAt: archiveTimestamp,
30       },
31     });
32
33     console.log(`[runArchive] Archive timestamp: ${archiveTimestamp.toISOString()}`);
34     console.log(`[runArchive] Matched ${result.matchedCount || 0} requests.`);
35     console.log(`[runArchive] Archived ${result.modifiedCount || 0} requests.`);
36     process.exitCode = 0;
37   } catch (err) {
38     console.error('[runArchive] Error:', err);
39     process.exitCode = 1;
40   } finally {
41     try {
42       await mongoose.connection.close();
43     } catch (e) {
44       // ignore
45     }
46     process.exit();
47   }
48 })();
```

16. server/scripts/restoreArchive.js (14 lines)

```
1  require('dotenv').config();
2  const connectDB = require('../config/db');
3  const mongoose = require('mongoose');
4  const Request = require('../models/Request');
5
6  (async () => {
7    await connectDB();
8    const result = await Request.updateMany(
9      { isArchived: true },
10     { $set: { isArchived: false }, $unset: { archivedAt: '' } }
11   );
12   console.log(`Restored: ${result.modifiedCount} request(s).`);
13   await mongoose.connection.close();
14 })();
```

Frontend Source Code

17. client/src/main.jsx (26 lines)

```
1 import { StrictMode } from "react";
2 import { createRoot } from "react-dom/client";
3 import { Buffer } from "buffer";
4 import "./index.css";
5 import App from "./App.jsx";
6 import { notify } from "./utils/inPageFeedback";
7
8 import { AuthProvider } from "./context/AuthContext.jsx";
9
10 window.Buffer = Buffer;
11
12 // Guard against duplicate patching during HMR updates.
13 if (typeof window !== "undefined" && !window.__dpoAlertPatched) {
14   window.__dpoAlertPatched = true;
15   window.alert = (message) => {
16     notify(String(message ?? ""), { type: "info" });
17   };
18 }
19
20 createRoot(document.getElementById("root")).render(
21   <StrictMode>
22     <AuthProvider>
23       <App />
24     </AuthProvider>
25   </StrictMode>
26 );
```

18. client/src/App.jsx (346 lines)

```

1 import { BrowserRouter as Router, Routes, Route, Navigate, Outlet, useLocation } from "react-router-dom";
2 import { Suspense, lazy, useContext, useMemo } from "react";
3 import { AnimatePresence, motion } from "framer-motion";
4 import { AuthContext } from "../context/AuthContext";
5
6 import Navbar from "../components/Navbar";
7 import Headbar from "../components/Headbar";
8 import SessionWarningModal from "../components/SessionWarningModal";
9 import InPageFeedbackHost from "../components/InPageFeedbackHost";
10 import RequireRole from "../guards/RequireRole";
11
12 import Landing from "../pages/Landing";
13 const VerifyDocument = lazy(() => import("../pages/VerifyDocument"));
14 const VerifyEmailPage = lazy(() => import("../pages/VerifyEmailPage"));
15 const ActivateAccountPage = lazy(() => import("../pages/ActivateAccountPage"));
16 const RepSigningPage = lazy(() => import("../pages/public/RepSigningPage"));
17
18 // Student pages
19 const StudentDashboard = lazy(() => import("../pages/student/StudentDashboard"));
20 const StudentNewRequest = lazy(() => import("../pages/student/StudentNewRequest"));
21 const StudentNDATypeChooser = lazy(() => import("../pages/student/StudentNDATypeChooser"));
22 const StudentNDAOrgActivities = lazy(() => import("../pages/student/StudentNDAOrgActivities"));
23 const StudentNDAREsearch = lazy(() => import("../pages/student/StudentNDAREsearch"));
24 const StudentAgreementRequirements = lazy(() => import("../pages/student/StudentAgreementRequirements"));
25 const StudentAgreementRequest = lazy(() => import("../pages/student/StudentAgreementRequest"));
26 const StudentRequestReview = lazy(() => import("../pages/student/StudentRequestReview"));
27 const StudentProfile = lazy(() => import("../pages/student/StudentProfile"));
28 const StudentResubmitRequest = lazy(() => import("../pages/student/StudentResubmitRequest"));
29
30 // Admin pages
31 const AdminDashboard = lazy(() => import("../pages/admin/AdminDashboard"));
32 const AdminRequests = lazy(() => import("../pages/admin/AdminRequests"));
33 const AdminRequestReview = lazy(() => import("../pages/admin/AdminRequestReview"));
34 const AdminReports = lazy(() => import("../pages/admin/AdminReports"));
35 const AdminProfile = lazy(() => import("../pages/admin/AdminProfile"));
36 const AdminUsers = lazy(() => import("../pages/admin/AdminUsers"));
37 const AdminArchives = lazy(() => import("../pages/admin/AdminArchives"));
38 const AdminAuditLog = lazy(() => import("../pages/admin/AdminAuditLog"));
39 const StaffProxyTypeChooser = lazy(() => import("../pages/admin/StaffProxyTypeChooser"));
40 const StaffProxyNDATypeChooser = lazy(() => import("../pages/admin/StaffProxyNDATypeChooser"));
41 const StaffProxyNDAOrgActivities = lazy(() => import("../pages/admin/StaffProxyNDAOrgActivities"));
42 const StaffProxyNDAREsearch = lazy(() => import("../pages/admin/StaffProxyNDAREsearch"));
43 const StaffProxyAgreementRequest = lazy(() => import("../pages/admin/StaffProxyAgreementRequest"));
44
45 function GlobalLoader() {
46   return (
47     <div
48       className="route-transition-shell"
49       style={{
50         minHeight: "160px",
51         display: "grid",
52         placeItems: "center",
53       }}
54       aria-live="polite"
55       aria-busy="true"
56     >
57       <div
58         style={{
59           width: "34px",
60           height: "34px",
61           borderRadius: "999px",
62           border: "3px solid rgba(15, 45, 107, 0.2)",
63           borderTopColor: "var(--primary)",
64           animation: "spin 0.8s linear infinite",
65         }}
66       />
67     </div>
68   );
69 }
70
71 const buildTitle = (pathname) => {
72   if (pathname === "/student" || pathname === "/admin") return "Dashboard";
73   if (pathname.startsWith("/student/new-request")) return "Create Request";

```

```

74   if (pathname.startsWith("/student/requests")) return "Request Details";
75   if (pathname.startsWith("/student/resubmit")) return "Resubmit Request";
76   if (pathname.startsWith("/student/profile")) return "Profile";
77   if (pathname === "/admin/requests") return "Request Management";
78   if (pathname.startsWith("/admin/requests/")) return "Request Review";
79   if (pathname.startsWith("/admin/reports")) return "Transaction Records";
80   if (pathname.startsWith("/admin/users")) return "User Management";
81   if (pathname.startsWith("/admin/archives")) return "Archives";
82   if (pathname.startsWith("/admin/audit")) return "Audit Logs";
83   if (pathname.startsWith("/admin/profile")) return "Profile";
84   if (pathname.startsWith("/admin/proxy-request")) return "Proxy Request";
85   return "Dashboard";
86 };
87
88 function AppLayout() {
89   const { user, showSessionWarning, extendSession } = useContext(AuthContext);
90   const location = useLocation();
91   const title = useMemo(() => buildTitle(location.pathname), [location.pathname]);
92
93   return (
94     <div className="app-container">
95       {user && <Headbar />}
96
97       <div className="app-body">
98         {user && <Navbar />}
99
100        <div className="page-content">
101          {user && <h1 className="page-title">{title}</h1>}
102          <AnimatePresence mode="wait">
103            <motion.div
104              key={location.pathname}
105              className="route-transition-shell"
106              initial={{ opacity: 0, y: 8 }}
107              animate={{ opacity: 1, y: 0 }}
108              exit={{ opacity: 0, y: -6 }}
109              transition={{ duration: 0.22, ease: "easeOut" }}
110            >
111              <Suspense fallback={<GlobalLoader />}>
112                <Outlet />
113              </Suspense>
114            </motion.div>
115          </AnimatePresence>
116        </div>
117      </div>
118
119      {showSessionWarning && <SessionWarningModal onExtend={extendSession} />}
120    </div>
121  );
122 }
123
124 function App() {
125   const { user } = useContext(AuthContext);
126
127   return (
128     <>
129       <Router>
130         <Suspense fallback={<GlobalLoader />}>
131           <Routes>
132             /* Public routes (no auth, no layout) */
133             <Route path="/verify/:code" element={<VerifyDocument />} />
134             <Route path="/sign/:token" element={<RepSigningPage />} />
135             <Route path="/verify-email" element={<VerifyEmailPage />} />
136             <Route path="/activate" element={<ActivateAccountPage />} />
137
138             <Route element={<AppLayout />}>
139               <Route
140                 path="/"
141                 element={
142                   user
143                     ? <Navigate to={user.role === "student" ? "/student" : "/admin/requests"} replace />
144                     : <Landing />
145                 }
146               />
147             <Route path="/login" element={<Navigate to="/" replace />} />
148             <Route path="/register" element={<Navigate to="/" replace />} />
149           </Routes>
150         </Suspense>
151       </Router>
152     </>
153   );
154 }

```

```
150     <Route
151       path="/student"
152       element={
153         <RequireRole allowedRoles={["student"]} >
154           <StudentDashboard />
155         </RequireRole>
156       }
157     />
158     <Route
159       path="/student/new-request"
160       element={
161         <RequireRole allowedRoles={["student"]} >
162           <StudentNewRequest />
163         </RequireRole>
164       }
165     />
166     <Route
167       path="/student/new-request/nda"
168       element={
169         <RequireRole allowedRoles={["student"]} >
170           <StudentNDATypeChooser />
171         </RequireRole>
172       }
173     />
174     <Route
175       path="/student/new-request/nda/orgactivities"
176       element={
177         <RequireRole allowedRoles={["student"]} >
178           <StudentNDAOrgActivities />
179         </RequireRole>
180       }
181     />
182     <Route
183       path="/student/new-request/nda/research"
184       element={
185         <RequireRole allowedRoles={["student"]} >
186           <StudentNDAResearch />
187         </RequireRole>
188       }
189     />
190     <Route
191       path="/student/new-request/agreement"
192       element={
193         <RequireRole allowedRoles={["student"]} >
194           <StudentAgreementRequirements />
195         </RequireRole>
196       }
197     />
198     <Route
199       path="/student/new-request/agreement/form"
200       element={
201         <RequireRole allowedRoles={["student"]} >
202           <StudentAgreementRequest />
203         </RequireRole>
204       }
205     />
206     <Route
207       path="/student/requests/:id"
208       element={
209         <RequireRole allowedRoles={["student"]} >
210           <StudentRequestReview />
211         </RequireRole>
212       }
213     />
214     <Route
215       path="/student/profile"
216       element={
217         <RequireRole allowedRoles={["student"]} >
218           <StudentProfile />
219         </RequireRole>
220       }
221     />
222     <Route
223       path="/student/resubmit/:id"
224       element={
225         <RequireRole allowedRoles={["student"]} >
```



```
226         <StudentResubmitRequest />
227     </RequireRole>
228 }
229 />
230
231 <Route
232     path="/admin"
233     element={
234         <RequireRole allowedRoles={["admin", "staff"]}>
235             <AdminDashboard />
236         </RequireRole>
237     }
238 />
239 <Route
240     path="/admin/requests"
241     element={
242         <RequireRole allowedRoles={["admin", "staff"]}>
243             <AdminRequests />
244         </RequireRole>
245     }
246 />
247 <Route
248     path="/admin/requests/:id"
249     element={
250         <RequireRole allowedRoles={["admin", "staff"]}>
251             <AdminRequestReview />
252         </RequireRole>
253     }
254 />
255 <Route
256     path="/admin/proxy-request"
257     element={
258         <RequireRole allowedRoles={["staff"]}>
259             <StaffProxyTypeChooser />
260         </RequireRole>
261     }
262 />
263 <Route
264     path="/admin/proxy-request/nda"
265     element={
266         <RequireRole allowedRoles={["staff"]}>
267             <StaffProxyNDATypeChooser />
268         </RequireRole>
269     }
270 />
271 <Route
272     path="/admin/proxy-request/nda/orgactivities"
273     element={
274         <RequireRole allowedRoles={["staff"]}>
275             <StaffProxyNDAOrgActivities />
276         </RequireRole>
277     }
278 />
279 <Route
280     path="/admin/proxy-request/nda/research"
281     element={
282         <RequireRole allowedRoles={["staff"]}>
283             <StaffProxyNDAResearch />
284         </RequireRole>
285     }
286 />
287 <Route
288     path="/admin/proxy-request/agreement"
289     element={
290         <RequireRole allowedRoles={["staff"]}>
291             <StaffProxyAgreementRequest />
292         </RequireRole>
293     }
294 />
295 <Route
296     path="/admin/reports"
297     element={
298         <RequireRole allowedRoles={["admin", "staff"]}>
299             <AdminReports />
300         </RequireRole>
301     }
```

```
302     />
303     <Route
304       path="/admin/profile"
305       element={
306         <RequireRole allowedRoles={["admin", "staff"]}>
307           <AdminProfile />
308         </RequireRole>
309       }
310     />
311     <Route
312       path="/admin/users"
313       element={
314         <RequireRole allowedRoles={["admin"]}>
315           <AdminUsers />
316         </RequireRole>
317       }
318     />
319     <Route
320       path="/admin/archives"
321       element={
322         <RequireRole allowedRoles={["admin", "staff"]}>
323           <AdminArchives />
324         </RequireRole>
325       }
326     />
327     <Route
328       path="/admin/audit"
329       element={
330         <RequireRole allowedRoles={["admin"]}>
331           <AdminAuditLog />
332         </RequireRole>
333       }
334     />
335
336     <Route path="*" element={<Navigate to="/" replace />} />
337   </Route>
338 </Routes>
339 </Suspense>
340 </Router>
341 <InPageFeedbackHost />
342 </>
343 );
344 }
345
346 export default App;
```

19. client/src/firebase.js (11 lines)

```
1 import { initializeApp } from "firebase/app";
2 import { getStorage } from "firebase/storage";
3
4 const firebaseConfig = {
5   apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
6   projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
7   storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
8 };
9
10 const app = initializeApp(firebaseConfig);
11 export const storage = getStorage(app);
```

20. client/src/context/AuthContext.jsx (138 lines)

```
1  /* eslint-disable react-refresh/only-export-components */
2  import { createContext, useCallack, useEffect, useRef, useState } from "react";
3  import { logout as logoutService, getCurrentUser, getToken, refreshToken } from "../services/authService";
4
5  export const AuthContext = createContext();
6
7  const decodeToken = (token) => {
8    try {
9      const payload = token.split(".")[1];
10     return JSON.parse(atob(payload));
11   } catch {
12     return null;
13   }
14 };
15
16 const INACTIVITY_TIMEOUT = 15 * 60 * 1000; // 15 minutes
17 const WARNING_BEFORE = 2 * 60 * 1000; // Show warning 2 min before timeout
18 const WARNING_AT = INACTIVITY_TIMEOUT - WARNING_BEFORE; // 13 minutes
19
20 export const AuthProvider = ({ children }) => {
21   const [user, setUser] = useState(getCurrentUser());
22   const [showSessionWarning, setShowSessionWarning] = useState(false);
23   const logoutTimerRef = useRef(null);
24   const warningTimerRef = useRef(null);
25   const inactivityTimerRef = useRef(null);
26   const inactivityWarningRef = useRef(null);
27
28   const clearAllTimers = useCallack(() => {
29     [logoutTimerRef, warningTimerRef, inactivityTimerRef, inactivityWarningRef].forEach((ref) => {
30       if (ref.current) {
31         clearTimeout(ref.current);
32         ref.current = null;
33       }
34     });
35   }, []);
36
37   const logout = useCallack(() => {
38     logoutService();
39     setUser(null);
40     setShowSessionWarning(false);
41     clearAllTimers();
42   }, [clearAllTimers]);
43
44   const scheduleAutoLogout = useCallack(() => {
45     if (logoutTimerRef.current) clearTimeout(logoutTimerRef.current);
46     const token = getToken();
47     if (!token) return;
48
49     const decoded = decodeToken(token);
50     if (!decoded?.exp) return;
51
52     const expiresAtMs = decoded.exp * 1000;
53     const delay = expiresAtMs - Date.now();
54
55     if (delay <= 0) {
56       logout();
57       return;
58     }
59
60     logoutTimerRef.current = setTimeout(() => {
61       logout();
62     }, delay);
63   }, [logout]);
64
65   // Inactivity timer management
66   const resetInactivityTimer = useCallack(() => {
67     if (!getToken()) return;
68
69     setShowSessionWarning(false);
70
71     if (inactivityWarningRef.current) clearTimeout(inactivityWarningRef.current);
72     if (inactivityTimerRef.current) clearTimeout(inactivityTimerRef.current);
73   });
```

```
74 // Show warning at 13 min
75 inactivityWarningRef.current = setTimeout(() => {
76   setShowSessionWarning(true);
77 }, WARNING_AT);
78
79 // Force logout at 15 min
80 inactivityTimerRef.current = setTimeout(() => {
81   setShowSessionWarning(false);
82   logout();
83 }, INACTIVITY_TIMEOUT);
84 }, [logout]);
85
86 const extendSession = useCallback(async () => {
87   try {
88     const data = await refreshToken();
89     if (data.user) {
90       setUser(data.user);
91     }
92     setShowSessionWarning(false);
93     scheduleAutoLogout();
94     resetInactivityTimer();
95   } catch {
96     logout();
97   }
98 }, [logout, scheduleAutoLogout, resetInactivityTimer]);
99
100 const login = useCallback((userData) => {
101   setUser(userData);
102   scheduleAutoLogout();
103   resetInactivityTimer();
104 }, [scheduleAutoLogout, resetInactivityTimer]);
105
106 const updateUser = useCallback((userData) => {
107   setUser(userData);
108   localStorage.setItem("user", JSON.stringify(userData));
109 }, []);
110
111 // Activity listeners
112 useEffect(() => {
113   if (!user) return;
114
115   const events = ["mousemove", "keydown", "scroll", "mousedown", "touchstart"];
116   const handler = () => resetInactivityTimer();
117
118   events.forEach((e) => window.addEventListener(e, handler, { passive: true }));
119   resetInactivityTimer();
120
121   return () => {
122     events.forEach((e) => window.removeEventListener(e, handler));
123   };
124 }, [user, resetInactivityTimer]);
125
126 useEffect(() => {
127   // on load or refresh, check token validity
128   scheduleAutoLogout();
129   return () => clearAllTimers();
130   // eslint-disable-next-line react-hooks/exhaustive-deps
131 }, []);
132
133 return (
134   <AuthContext.Provider value={{ user, login, logout, updateUser, showSessionWarning, extendSession }}>
135     {children}
136   </AuthContext.Provider>
137 );
138 };
```

21. client/src/guards/RequireRole.jsx (17 lines)

```
1 import { useContext } from "react";
2 import { Navigate } from "react-router-dom";
3 import { AuthContext } from "../context/AuthContext";
4
5 const RequireRole = ({ allowedRoles, children }) => {
6   const { user } = useContext(AuthContext);
7
8   if (!user) return <Navigate to="/" replace />;
9
10  if (!allowedRoles.includes(user.role)) {
11    return <Navigate to={user.role === "student" ? "/student" : "/admin"} replace />;
12  }
13
14  return children;
15 };
16
17 export default RequireRole;
```

22. client/src/services/authService.js (148 lines)

[illegible]

[illegible]

23. client/src/services/requestService.js (110 lines)

[illegible]

```
74 };  
75  
76 // ■ Agreement admin endpoints ████████████████████████████████████████████████████████  
77 export const generateSigningLink = async (id) => {  
78   const res = await axios.patch(`${API_URL}/${id}/generate-signing-link`, {}, { headers: authHeader() });  
79   return res.data;  
80 };  
81  
82 export const adminPhase3Action = async (id, payload) => {  
83   const res = await axios.patch(`${API_URL}/${id}/admin-phase3`, payload, { headers: authHeader() });  
84   return res.data;  
85 };  
86  
87 export const getSignatureImages = async (id) => {  
88   const res = await axios.get(`${API_URL}/${id}/sig-images`, { headers: authHeader() });  
89   return res.data;  
90 };  
91  
92 export const getRequestStats = async () => {  
93   const res = await axios.get(`${API_URL}/stats`, { headers: authHeader() });  
94   return res.data;  
95 };  
96  
97 export const getArchivedRequests = async () => {  
98   const res = await axios.get(`${API_URL}/archived`, { headers: authHeader() });  
99   return res.data;  
100 };  
101  
102 export const getRetentionStats = async () => {  
103   const res = await axios.get(`${API_URL}/retention-stats`, { headers: authHeader() });  
104   return res.data;  
105 };  
106  
107 export const runArchiveNow = async () => {  
108   const res = await axios.post(`${API_URL}/run-archive`, {}, { headers: authHeader() });  
109   return res.data;  
110 };
```

24. client/src/services/auditService.js (22 lines)

```
1 import axios from "axios";
2
3 const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/audit`;
4
5 const authHeader = () => {
6   const token = localStorage.getItem("token");
7   return token ? { Authorization: `Bearer ${token}` } : {};
8 };
9
10 export const getAuditLogs = async ({ page = 1, limit = 50, action = "", userId = "", resourceId = "" } = {}) => {
11   const params = new URLSearchParams({ page, limit });
12   if (action) params.set("action", action);
13   if (userId) params.set("userId", userId);
14   if (resourceId) params.set("resourceId", resourceId);
15   const res = await axios.get(`${API_URL}?${params}`, { headers: authHeader() });
16   return res.data;
17 };
18
19 export const getRecentAuditLogs = async (count = 5) => {
20   const res = await axios.get(`${API_URL}/recent?count=${count}`, { headers: authHeader() });
21   return res.data;
22 };
```

25. client/src/services/firebaseStorageService.js (134 lines)

```

1 import { ref, uploadBytes, getDownloadURL, deleteObject } from "firebase/storage";
2 import { storage } from "../firebase";
3
4 const slugify = (str = "") =>
5   String(str)
6     .trim()
7     .toLowerCase()
8     .replace(/\s+/g, "-")
9     .replace(/[^a-z0-9-]/g, "");
10
11 const safeFileName = (name = "") =>
12   String(name).replace(/[^a-zA-Z0-9._-]/g, "_");
13
14 // date_timestamp folder example: "2-16-2026_1139" or "2-16-2026_2329"
15 export const getDateRequestFolder = () => {
16   const d = new Date();
17   const date = d.toLocaleDateString("en-US").replaceAll("/", "-"); // M-D-YYYY
18   const hhmm = `${String(d.getHours()).padStart(2, "0")}${String(
19     d.getMinutes()
20   ).padStart(2, "0")}`;
21   return `${date}_${hhmm}`;
22 };
23
24 export async function uploadRequirements(files, type, studentName, requestFolder) {
25   const uploaded = [];
26   const studentSlug = slugify(studentName);
27   const folder = requestFolder || getDateRequestFolder();
28
29   for (const file of Array.from(files || [])) {
30     const name = safeFileName(file.name);
31
32     // <slugify_name_folder>/<requests_folder>/<request_type_folder>/<date_timestamp_folder>/<actual file>
33     const path = `${studentSlug}/requests/${type}/${folder}/${name}`;
34
35     const fileRef = ref(storage, path);
36     const snap = await uploadBytes(fileRef, file);
37     const url = await getDownloadURL(snap.ref);
38
39     uploaded.push({
40       origName: file.name,
41       url,
42       path,
43       contentType: file.type,
44       uploadedAt: new Date().toISOString(),
45       requestFolder: folder,
46     });
47   }
48
49   return uploaded;
50 }
51
52 export async function uploadApprovedForm(pdfBlob, type, studentName, requestFolder, fileName) {
53   const studentSlug = slugify(studentName);
54   const safeName = safeFileName(fileName);
55
56   const path = `${studentSlug}/requests/${type}/${requestFolder}/approved_form/${safeName}`;
57
58   const typedBlob = new Blob([pdfBlob], { type: "application/pdf" });
59   const meta = { contentType: "application/pdf" };
60
61   const fileRef = ref(storage, path);
62   const snap = await uploadBytes(fileRef, typedBlob, meta);
63   const url = await getDownloadURL(snap.ref);
64
65   return { url, path, issuedAt: new Date().toISOString() };
66 }
67
68 export async function uploadApprovedQrImage(qrDataUrl, type, studentName, requestFolder) {
69   const studentSlug = slugify(studentName);
70   const path = `${studentSlug}/requests/${type}/${requestFolder}/approved_form/qr_code.png`;
71
72   const qrBlob = await fetch(qrDataUrl).then((res) => res.blob());
73   const meta = { contentType: "image/png" };

```

```
74
75   const fileRef = ref(storage, path);
76   const snap = await uploadBytes(fileRef, qrBlob, meta);
77   const url = await getDownloadURL(snap.ref);
78
79   return { url, path };
80 }
81
82 /**
83  * Upload a base64 signature data URL as a PNG to Firebase Storage.
84  * Returns { url, path }.
85  */
86 export async function uploadSignatureImage(dataUrl, type, studentName, requestFolder, sigName) {
87   const studentSlug = slugify(studentName);
88   const path = `${studentSlug}/requests/${type}/${requestFolder}/sigs/${sigName}`;
89
90   const blob = await fetch(dataUrl).then((r) => r.blob());
91   const meta = { contentType: "image/png" };
92
93   const fileRef = ref(storage, path);
94   const snap = await uploadBytes(fileRef, blob, meta);
95   const url = await getDownloadURL(snap.ref);
96
97   return { url, path };
98 }
99
100 /**
101  * Upload a File object for the representative's government ID.
102  * Returns { origName, url, path, contentType, uploadedAt }.
103  */
104 export async function uploadRepGovId(file, type, studentName, requestFolder) {
105   const studentSlug = slugify(studentName);
106   const name = safeFileName(file.name);
107   const path = `${studentSlug}/requests/${type}/${requestFolder}/rep_docs/${name}`;
108
109   const fileRef = ref(storage, path);
110   const snap = await uploadBytes(fileRef, file);
111   const url = await getDownloadURL(snap.ref);
112
113   return {
114     origName: file.name,
115     url,
116     path,
117     contentType: file.type,
118     uploadedAt: new Date().toISOString(),
119   };
120 }
121
122 /**
123  * Delete a file from Firebase Storage by its storage path.
124  * Silently ignores not-found errors.
125  */
126 export async function deleteStorageFile(filePath) {
127   if (!filePath) return;
128   try {
129     const fileRef = ref(storage, filePath);
130     await deleteObject(fileRef);
131   } catch (err) {
132     console.warn("deleteStorageFile:", err.message);
133   }
134 }
```

26. client/src/services/qrService.js (8 lines)

```
1 import QRCode from "qrcode";
2
3 export const buildVerificationUrl = (serialNo) =>
4   `${window.location.origin}/verify/${serialNo}`;
5
6 export const generateQrDataUrl = async (url) => {
7   return QRCode.toDataURL(url, { margin: 1, width: 180 });
8 };
```

27. client/src/utis/passwordPolicy.js (24 lines)

```
1  export const PASSWORD_POLICY_MESSAGE =
2    "Minimum 8 chars, with uppercase, lowercase, number, and at least one special character";
3
4  export const getPasswordChecks = (value = "") => {
5    const p = String(value || "");
6    return [
7      { key: "length", label: "At least 8 characters", pass: p.length >= 8 },
8      { key: "upper", label: "At least 1 uppercase letter", pass: /[A-Z]/.test(p) },
9      { key: "lower", label: "At least 1 lowercase letter", pass: /[a-z]/.test(p) },
10     { key: "number", label: "At least 1 number", pass: /[0-9]/.test(p) },
11     { key: "symbol", label: "At least 1 special character", pass: /[^\A-Za-z0-9]/.test(p) },
12   ];
13 };
14
15 export const isStrongPassword = (value = "") => {
16   const p = String(value || "");
17   return (
18     p.length >= 8 &&
19     /[A-Z]/.test(p) &&
20     /[a-z]/.test(p) &&
21     /[0-9]/.test(p) &&
22     /[^\A-Za-z0-9]/.test(p)
23   );
24 };
```

28. client/src/utls/inPageFeedback.js (113 lines)

```

1  const NOTIFY_EVENT = "dpo:notify";
2  const CONFIRM_EVENT = "dpo:confirm";
3
4  const STATUS_LABELS = {
5    nda_pending: "Reviewal",
6    nda_approved: "Approved",
7    stud_revision_requested: "Student Revisions",
8    agr_pending_1: "Initial Reviewal",
9    agr_awaiting_rep_signature: "Recipient Reviewal",
10   agr_pending_2: "Final Reviewal",
11   agr_approved: "Approved",
12   agr_declined: "Recipient Declined",
13   agr_rep_revision_requested: "Recipient Revisions",
14 };
15
16 const toTitleCase = (value) => String(value || "")
17   .replace(/_/g, " ")
18   .toLowerCase()
19   .replace(/\b\w/g, (char) => char.toUpperCase());
20
21 const humanizeStatusCode = (status) => STATUS_LABELS[status] || toTitleCase(status);
22
23 export function normalizeFeedbackMessage(message = "") {
24   const raw = String(message || "").trim();
25   if (!raw) return "";
26
27   const updatedMatch = raw.match(/^Updated to\s+([A-Za-z0-9_]+)$/i);
28   if (updatedMatch) {
29     return `Request status updated to ${humanizeStatusCode(updatedMatch[1])}.`;
30   }
31
32   const shortStatusMatch = raw.match(/^Request moved to\s+(.+)\.$/i);
33   if (shortStatusMatch) {
34     return `Request moved to ${shortStatusMatch[1]}.`;
35   }
36
37   if (/^Failed\b/i.test(raw)) {
38     return raw.endsWith(".") ? raw : `${raw}.`;
39   }
40
41   if (/^Approved\b/i.test(raw) || /^Resubmitted\b/i.test(raw) || /^Copied!$/i.test(raw)) {
42     return raw.endsWith(".") ? raw : `${raw}.`;
43   }
44
45   return raw;
46 }
47
48 export function inferFeedbackType(message = "") {
49   const text = String(message).toLowerCase();
50   if (
51     text.includes("fail")
52     || text.includes("error")
53     || text.includes("invalid")
54     || text.includes("missing")
55     || text.includes("declined")
56   ) {
57     return "error";
58   }
59   if (text.includes("warn") || text.includes("expired") || text.includes("required")) {
60     return "warning";
61   }
62   if (
63     text.includes("success")
64     || text.includes("approved")
65     || text.includes("submitted")
66     || text.includes("copied")
67     || text.includes("updated")
68     || text.includes("generated")
69   ) {
70     return "success";
71   }
72   return "info";
73 }

```



```
74
75 export function notify(message, options = {}) {
76   if (typeof window === "undefined") return;
77   const normalizedMessage = normalizeFeedbackMessage(message);
78   window.dispatchEvent(
79     new CustomEvent(NOTIFY_EVENT, {
80       detail: {
81         message: normalizedMessage,
82         type: options.type || inferFeedbackType(normalizedMessage),
83         title: options.title,
84         duration: options.duration,
85       },
86     })
87   );
88 }
89
90 export function confirmInPage(options) {
91   if (typeof window === "undefined") return Promise.resolve(false);
92   const normalized = typeof options === "string" ? { message: options } : (options || {});
93
94   return new Promise((resolve) => {
95     window.dispatchEvent(
96       new CustomEvent(CONFIRM_EVENT, {
97         detail: {
98           title: normalized.title || "Please confirm",
99           message: normalized.message || "Are you sure you want to continue?",
100           confirmText: normalized.confirmText || "Confirm",
101           cancelText: normalized.cancelText || "Cancel",
102           tone: normalized.tone || "warning",
103           resolve,
104         },
105       })
106     );
107   });
108 }
109
110 export const FEEDBACK_EVENTS = {
111   NOTIFY_EVENT,
112   CONFIRM_EVENT,
113 };
```

29. client/src/utils/requestStatusCharts.js (43 lines)

```
1  export const APPROVED_STATUSES = ["nda_approved", "agr_approved"];
2
3  export const PENDING_STATUSES = [
4    "nda_pending",
5    "agr_pending_1",
6    "agr_pending_2",
7    "agr_awaiting_rep_signature",
8  ];
9
10 export const REVISION_STATUSES = [
11   "stud_revision_requested",
12   "agr_rep_revision_requested",
13 ];
14
15 export const CHART_LABELS = {
16   chart_reviewal: "Reviewal",
17   chart_approved: "Approved",
18   chart_stud_revision: "Student Revisions",
19   chart_rep_revision: "Recipient Revisions",
20   chart_rep_declined: "Recipient Declined",
21   chart_awaiting_rep: "Recipient Reviewal",
22 };
23
24 export const CHART_COLORS = {
25   chart_reviewal: "#ca8a04",
26   chart_approved: "#059669",
27   chart_stud_revision: "#ea580c",
28   chart_rep_revision: "#7c3aed",
29   chart_rep_declined: "#dc2626",
30   chart_awaiting_rep: "#2563eb",
31 };
32
33 export const normalizeStatusForChart = (status) => {
34   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(status)) return "chart_reviewal";
35   if (APPROVED_STATUSES.includes(status)) return "chart_approved";
36   if (status === "stud_revision_requested") return "chart_stud_revision";
37   if (status === "agr_rep_revision_requested") return "chart_rep_revision";
38   if (status === "agr_declined") return "chart_rep_declined";
39   if (status === "agr_awaiting_rep_signature") return "chart_awaiting_rep";
40
41   // Unknown statuses fallback
42   return "chart_reviewal";
43 };
```

30. client/src/config/fieldsFileSlotsConfig.js (45 lines)

```
1 export const FIELDS_FILE_SLOTS_CONFIG = {
2   nda: {
3     orgactivities: {
4       label: "Student Organization Activities",
5       fields: [
6         { name: "projectTitle", label: "Project Title", required: true },
7       ],
8       fileSlots: [
9         { label: "Letter of Intent", required: true },
10        { label: "School ID", required: false },
11        { label: "Registration Form", required: false },
12        { label: "Sample Forms or Collaterals (if applicable)", required: false },
13      ],
14    },
15    research: {
16      label: "Conduct of Research",
17      fields: [
18        { name: "researchTitle", label: "Research Title", required: true },
19      ],
20      fileSlots: [
21        { label: "Letter of Intent", required: true },
22        { label: "Questionnaire with Data Privacy Consent Form", required: false },
23        { label: "School Id", required: false },
24        { label: "Registration Form", required: false },
25      ],
26    },
27  },
28 },
29
30 agreement: {
31   label: "Agreement",
32   fields: [
33     { name: "repFirstName", label: "First Name", required: true },
34     { name: "repMiddleInitial", label: "Middle Name", required: false },
35     { name: "repLastName", label: "Last Name", required: true },
36     { name: "repEmail", label: "Email", required: true, type: "email" },
37   ],
38   // Note: Representative's Government Issued Valid ID is now collected directly
39   // from the representative on the signing page, not from the student.
40   fileSlots: [
41     { label: "Notarized Authorization Letter", required: true },
42     { label: "Requestor's Government Issued Valid ID", required: false },
43   ],
44 },
45 };
```

31. client/src/config/documentTemplates/index.jsx (48 lines)

```
1  /* eslint-disable react-refresh/only-export-components */
2  import React from "react";
3
4  const renderFallbackDoc = ({ request, renderer, styles }) => {
5    const { Document, Page } = renderer;
6    const { s, Header, BodyText, Footer } = styles;
7
8    return (
9      <Document>
10        <Page size="A4" style={s.page}>
11          <Header title="APPROVED DOCUMENT" />
12          <BodyText>Approved document for request {request?._id}</BodyText>
13          <BodyText>(No template found for this request type)</BodyText>
14          <Footer />
15        </Page>
16      </Document>
17    );
18  };
19
20  export const generateApprovedPDF = async (request) => {
21    const [renderer, styles] = await Promise.all([
22      import("@react-pdf/renderer"),
23      import("../styles"),
24    ]);
25
26    let DocComponent = null;
27
28    if (request.type === "agreement") {
29      const mod = await import("../AgreementDoc");
30      DocComponent = mod.default;
31    } else if (request.type === "nda") {
32      const t = request.formData?.ndaType;
33      if (t === "orgactivities") {
34        const mod = await import("../NDASStudentOrgActivitiesDoc");
35        DocComponent = mod.default;
36      } else if (t === "research") {
37        const mod = await import("../NDAResearchDoc");
38        DocComponent = mod.default;
39      }
40    }
41
42    const doc = DocComponent
43      ? <DocComponent request={request} />
44      : renderFallbackDoc({ request, renderer, styles });
45
46    const blob = await renderer.pdf(doc).toBlob();
47    return new Blob([blob], { type: "application/pdf" });
48  };
```

32. client/src/config/documentTemplates/styles.jsx (100 lines)

```

1  /* eslint-disable react-refresh/only-export-components */
2  import React from "react";
3  import { Text, View, StyleSheet, Image } from "@react-pdf/renderer";
4
5  export const s = StyleSheet.create({
6    page: { padding: 40, fontSize: 10, fontFamily: "Helvetica", color: "#222" },
7
8    headerTitle: { fontSize: 18, fontFamily: "Helvetica-Bold", textAlign: "center", marginBottom: 2 },
9    headerSubtitle: { fontSize: 12, fontFamily: "Helvetica-Bold", textAlign: "center", marginBottom: 2 },
10   headerOffice: { fontSize: 9, textAlign: "center", color: "#555", marginBottom: 12 },
11
12   hr: { borderBottomWidth: 1, borderBottomColor: "#bbb", marginBottom: 12 },
13
14   metaRow: { flexDirection: "row", marginBottom: 4 },
15   metaLabel: { width: 105, fontFamily: "Helvetica-Bold", fontSize: 10 },
16   metaValue: { flex: 1, fontSize: 10 },
17
18   sectionTitle: { fontSize: 11, fontFamily: "Helvetica-Bold", marginTop: 14, marginBottom: 6, color: "#111" },
19
20   fieldRow: { flexDirection: "row", marginBottom: 4, paddingLeft: 8 },
21   fieldLabel: { width: 130, fontFamily: "Helvetica-Bold", fontSize: 10 },
22   fieldValue: { flex: 1, fontSize: 10 },
23
24   bodyText: { fontSize: 10, lineHeight: 1.6, paddingLeft: 8, marginBottom: 6 },
25
26   footerWrap: { position: "absolute", bottom: 40, left: 40, right: 40 },
27   footerHr: { borderBottomWidth: 1, borderBottomColor: "#bbb", marginBottom: 8 },
28   footerRow: { flexDirection: "column", alignItems: "flex-start" },
29   footerLeft: { flex: 1, paddingRight: 10 },
30   footerTitle: { fontSize: 11, fontFamily: "Helvetica-Bold", textAlign: "left", marginBottom: 4 },
31   footerNote: { fontSize: 8, textAlign: "left", color: "#888" },
32   footerQrWrap: { alignItems: "flex-start", width: 90, marginTop: 8, paddingLeft: 8 },
33   footerQr: { width: 70, height: 70 },
34   footerQrLabel: { fontSize: 7, color: "#888", marginTop: 4, textAlign: "left" },
35 });
36
37 export const formatDate = (iso) => (iso ? new Date(iso).toLocaleDateString("en-US", { year: "numeric", month: "short", day: "numeric" }) : "");
38
39 export const getTemplateAssetUrl = (assetFileName) => {
40   const clean = String(assetFileName || "").replace(/^\//, "");
41   if (!clean) return "";
42
43   const basePath = typeof import.meta !== "undefined" && import.meta.env?.BASE_URL
44     ? import.meta.env.BASE_URL
45     : "/";
46   const normalizedBase = `${basePath}.endsWith("/") ? `${basePath}` : `${basePath}/`;
47   const relative = `${normalizedBase}${clean}`;
48
49   if (typeof window !== "undefined" && window.location?.origin) {
50     return new URL(relative, window.location.origin).toString();
51   }
52
53   return `${clean}`;
54 };
55
56 export const Header = ({ title, subtitle }) => (
57   <View>
58     <Text style={s.headerTitle}>{title}</Text>
59     {subtitle ? <Text style={s.headerSubtitle}>{subtitle}</Text> : null}
60     <Text style={s.headerOffice}>Data Protection Office</Text>
61   </View style={s.hr} />
62 </View>
63 );
64
65 export const MetaRow = ({ label, value }) => (
66   <View style={s.metaRow}>
67     <Text style={s.metaLabel}>{label}</Text>
68     <Text style={s.metaValue}>{String(value || "N/A")}</Text>
69   </View>
70 );
71
72 export const SectionTitle = ({ children }) => <Text style={s.sectionTitle}>{children}</Text>;
73

```

```
74 export const FieldRow = ({ label, value }) => (  
75   <View style={s.fieldRow}>  
76     <Text style={s.fieldLabel}>{label}</Text>  
77     <Text style={s.fieldValue}>{String(value || "N/A")}</Text>  
78   </View>  
79 );  
80  
81 export const BodyText = ({ children }) => <Text style={s.bodyText}>{children}</Text>;  
82  
83 export const Footer = ({ qrDataUrl, verificationUrl }) => (  
84   <View style={s.footerWrap} fixed>  
85     <View style={s.footerHr} />  
86     <View style={s.footerRow}>  
87       <View style={s.footerLeft}>  
88         <Text style={s.footerTitle}>APPROVED – Data Protection Office</Text>  
89         <Text style={s.footerNote}>(Placeholder – e-signature + QR code will go here)</Text>  
90         {verificationUrl ? <Text style={s.footerNote}>{verificationUrl}</Text> : null}  
91       </View>  
92       {qrDataUrl ? (  
93         <View style={s.footerQrWrap}>  
94           <Image style={s.footerQr} src={qrDataUrl} />  
95           <Text style={s.footerQrLabel}>Scan to verify</Text>  
96         </View>  
97       ) : null}  
98     </View>  
99   </View>  
100 );
```

33. client/src/config/documentTemplates/AgreementDoc.jsx (191 lines)

```

1 import React from "react";
2 import { Document, Page, Image, View, Text, StyleSheet } from "@react-pdf/renderer";
3 import { formatDate, getTemplateAssetUrl } from "../styles";
4
5 const styles = StyleSheet.create({
6   page: {
7     paddingTop: 18,
8     paddingBottom: 18,
9     paddingHorizontal: 20,
10    fontSize: 8.6,
11    fontFamily: "Times-Roman",
12    color: "#111",
13  },
14  copyCard: {
15    paddingTop: 8,
16    paddingBottom: 8,
17    paddingHorizontal: 10,
18    minHeight: 348,
19  },
20  cutLine: {
21    textAlign: "center",
22    fontSize: 7,
23    color: "#666",
24    letterSpacing: 1.5,
25    marginVertical: 5,
26  },
27  headerRow: { flexDirection: "row", justifyContent: "space-between", alignItems: "flex-start" },
28  headerBlock: { width: "48%" },
29  headerLeftTitle: { fontSize: 7.8, fontFamily: "Times-Bold" },
30  headerLeftCode: { fontSize: 7, marginTop: 1 },
31  headerRight: { alignItems: "flex-end" },
32  headerRightLine: { fontSize: 7 },
33  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 6, marginBottom: 6 },
34  logo: { width: 32, height: 32, objectFit: "contain", marginHorizontal: 6 },
35  logoDpo: { width: 28, height: 28, objectFit: "contain", marginHorizontal: 6 },
36  centerTextWrap: { alignItems: "center", marginHorizontal: 4 },
37  centerLine: { fontSize: 7.4, textAlign: "center" },
38  centerOffice: { fontSize: 7.4, textAlign: "center" },
39  centerAgreement: { fontSize: 8.6, fontFamily: "Times-Bold", textAlign: "center", marginTop: 1 },
40  paragraph: {
41    width: "92%",
42    alignSelf: "center",
43    textAlign: "justify",
44    textIndent: 12,
45    lineHeight: 1.12,
46    marginTop: 10,
47  },
48  paragraphFinal: {
49    width: "92%",
50    alignSelf: "center",
51    textAlign: "justify",
52    textIndent: 12,
53    lineHeight: 1.12,
54    marginTop: 10,
55  },
56  afterParagraph: {
57    marginTop: 8,
58    flexDirection: "row",
59    justifyContent: "space-between",
60    alignItems: "flex-start",
61    width: "92%",
62    alignSelf: "center",
63  },
64  rightColumn: { alignItems: "flex-end" },
65  sigBlock: { width: 158, alignItems: "flex-start", marginBottom: 6 },
66  sigImage: { width: 122, height: 28, objectFit: "contain", borderBottomWidth: 0.8, borderBottomColor: "#333" },
67  sigPlaceholder: { width: 122, height: 28, borderBottomWidth: 0.8, borderBottomColor: "#333" },
68  sigName: { fontSize: 7.4, fontFamily: "Times-Bold", marginTop: 2, textAlign: "left" },
69  sigLabel: { fontSize: 6.8, textAlign: "left" },
70  dateReceived: { fontSize: 7, alignSelf: "flex-start", marginBottom: 2, textAlign: "left" },
71  qrWrap: { alignItems: "flex-start", marginBottom: 4 },
72  qrImage: { width: 44, height: 44 },
73 });

```

```

74
75 const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
76   <View style={styles.sigBlock}>
77     {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
78     {imgUrl
79       ? <Image style={styles.sigImage} src={imgUrl} />
80       : <View style={styles.sigPlaceholder} />
81     }
82     <Text style={styles.sigName}>{name || ""}</Text>
83     <Text style={styles.sigLabel}>{label}</Text>
84   </View>
85 );
86
87 /**
88  * AgreementDoc renders progressively based on which signatures are present:
89  * - authorizerSigUrl → shown from phase2 preview onward
90  * - repSigUrl → shown from phase3 review onward
91  * - adminSigDataUrl → only in the final approved PDF
92  */
93 export default function AgreementDoc({ request }) {
94   const student = request.userId || {};
95   const proxy = request.proxyRequestee || {};
96   const repInfo = request.repInfo || {};
97   const { authorizerSigUrl, repSigUrl, adminSigDataUrl } = request;
98   const approverName = request.approverName || request.adminName || request.approvedByName || "";
99   const toUpper = (value) => (value ? String(value).toUpperCase() : "");
100   const repNameFromForm = [
101     request.formData?.repFirstName,
102     request.formData?.repMiddleInitial,
103     request.formData?.repLastName,
104   ]
105     .filter(Boolean)
106     .join(" ")
107     .replace(/\s+/g, " ")
108     .trim();
109   const recipientName = repInfo.name || repNameFromForm || request.formData?.repName;
110   const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
111   const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");
112
113   const hasAnySig = authorizerSigUrl || repSigUrl || adminSigDataUrl;
114
115   const AgreementCopy = () => (
116     <View style={styles.copyCard}>
117       <View style={styles.headerRow}>
118         <View style={styles.headerBlock}>
119           <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
120           <Text style={styles.headerLeftCode}>RTU-OP-UDPO-F002</Text>
121         </View>
122         <View style={[styles.headerBlock, styles.headerRight]}>
123           <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
124           <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
125         </View>
126       </View>
127
128       <View style={styles.logoRow}>
129         <Image style={styles.logo} src={rtuLogoSrc} />
130         <View style={styles.centerTextWrap}>
131           <Text style={styles.centerLine}>Rizal Technological University</Text>
132           <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
133           <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
134           <Text style={styles.centerAgreement}>AGREEMENT</Text>
135         </View>
136         <Image style={styles.logoDpo} src={dpoLogoSrc} />
137       </View>
138
139       <Text style={styles.paragraph}>
140         I declare that as the intended recipient, it is my responsibility to safeguard the integrity and
141         confidentiality of the enclosed information. As such, I ensure not to reproduce, share, or in any way
142         disclose any information to anyone without the consent of the data subject. Further, I attest that the
143         herein data shall be used for processing of school credentials.
144       </Text>
145
146       <Text style={styles.paragraphFinal}>
147         Finally, I understand that I may be held liable for any damage caused by unauthorized processing of
148         information.
149       </Text>
150     </View>
151   );

```



```

150     <View style={styles.afterParagraph}>
151       {request.qrDataUrl ? (
152         <View style={styles.qrWrap}>
153           <Image style={styles.qrImage} src={request.qrDataUrl} />
154         </View>
155       ) : null}
156     <View style={styles.rightColumn}>
157       {hasAnySig ? (
158         <>
159           <SigBlock
160             label="Authorizer's signature over printed name"
161             name={toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""))}
162             imgUrl={authorizerSigUrl || null}
163           />
164           <SigBlock
165             label="Recipient's signature over printed name"
166             name={toUpper(recipientName)}
167             imgUrl={repSigUrl || null}
168           />
169           <SigBlock
170             dateReceived={`Date Received: ${formatDate(request.updatedAt)}`}
171             label="Approving Officer's Signature over printed name"
172             name={toUpper(approverName)}
173             imgUrl={adminSigDataUrl || null}
174           />
175         </>
176       ) : null}
177     </View>
178   </View>
179 </View>
180 );
181
182 return (
183   <Document>
184     <Page size="A4" style={styles.page}>
185       <AgreementCopy />
186       <Text style={styles.cutLine}>{"- - - - -"}
187       <AgreementCopy />
188     </Page>
189   </Document>
190 );
191 }

```

34. client/src/config/documentTemplates/NDAResearchDoc.jsx (178 lines)

```

1 import React from "react";
2 import { Document, Page, View, Image, Text, StyleSheet } from "@react-pdf/renderer";
3 import { formatDate, getTemplateAssetUrl } from "../styles";
4
5 const styles = StyleSheet.create({
6   page: {
7     paddingTop: 36,
8     paddingBottom: 40,
9     paddingHorizontal: 36,
10    fontSize: 10,
11    fontFamily: "Times-Roman",
12    color: "#111",
13  },
14  headerRow: { flexDirection: "row", justifyContent: "space-between" },
15  headerBlock: { width: "48%" },
16  headerLeftTitle: { fontSize: 10, fontFamily: "Times-Bold" },
17  headerLeftCode: { fontSize: 9, marginTop: 2 },
18  headerRight: { alignItems: "flex-end" },
19  headerRightLine: { fontSize: 9 },
20  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 12, marginBottom: 10 },
21  logo: { width: 70, height: 70, objectFit: "contain", marginHorizontal: 8 },
22  logoDpo: { width: 60, height: 60, objectFit: "contain", marginHorizontal: 8 },
23  centerTextWrap: { alignItems: "center", marginHorizontal: 6 },
24  centerLine: { fontSize: 10, textAlign: "center" },
25  centerOffice: { fontSize: 10, textAlign: "center" },
26  centerAgreement: { fontSize: 11, fontFamily: "Times-Bold", textAlign: "center", marginTop: 2 },
27  paragraph: {
28    width: "85%",
29    alignSelf: "center",
30    textAlign: "justify",
31    textIndent: 24,
32    lineHeight: 1.25,
33    marginTop: 14,
34  },
35  paragraphNoIndent: {
36    width: "85%",
37    alignSelf: "center",
38    textAlign: "center",
39    marginTop: 8,
40    marginBottom: 4,
41  },
42  titleText: {
43    fontFamily: "Times-BoldItalic",
44  },
45  paragraphFinal: {
46    width: "85%",
47    alignSelf: "center",
48    textAlign: "justify",
49    textIndent: 24,
50    lineHeight: 1.25,
51    marginTop: 14,
52  },
53  afterParagraph: {
54    marginTop: 18,
55    flexDirection: "row",
56    justifyContent: "space-between",
57    alignItems: "flex-end",
58    width: "85%",
59    alignSelf: "center",
60  },
61  rightColumn: { alignItems: "flex-end" },
62  sigBlock: { width: 220, alignItems: "flex-start", marginBottom: 14 },
63  sigImage: { width: 150, height: 50, objectFit: "contain", borderBottomWidth: 1, borderBottomColor: "#333" },
64  sigPlaceholder: { width: 150, height: 50, borderBottomWidth: 1, borderBottomColor: "#333" },
65  sigName: { fontSize: 10, fontFamily: "Times-Bold", marginTop: 4, textAlign: "left" },
66  sigLabel: { fontSize: 8.5, textAlign: "left" },
67  dateReceived: { fontSize: 9, alignSelf: "flex-start", marginBottom: 4, textAlign: "left" },
68  qrWrap: { alignItems: "flex-start", marginBottom: 10 },
69  qrImage: { width: 70, height: 70 },
70 });
71
72 const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
73   <View style={styles.sigBlock}>

```

```

74     {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
75     {imgUrl
76     ? <Image style={styles.sigImage} src={imgUrl} />
77     : <View style={styles.sigPlaceholder} />
78     }
79     <Text style={styles.sigName}>{name || ""}</Text>
80     <Text style={styles.sigLabel}>{label}</Text>
81   </View>
82 );
83
84 export default function NDAResearchDoc({ request }) {
85   const fd = request.formData || {};
86   const student = request.userId || {};
87   const proxy = request.proxyRequestee || {};
88   const { studentSigDataUrl, adminSigDataUrl } = request;
89   const approverName = request.approverName || request.adminName || request.approvedByName || "";
90   const toUpper = (value) => (value ? String(value).toUpperCase() : "");
91   const studentNameUpper = toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""));
92   const hasAnySig = studentSigDataUrl || adminSigDataUrl;
93   const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
94   const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");
95
96   return (
97     <Document>
98       <Page size="A4" style={styles.page}>
99         <View style={styles.headerRow}>
100           <View style={styles.headerBlock}>
101             <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
102             <Text style={styles.headerLeftCode}>RTU-OEVP-UDPC-F005</Text>
103           </View>
104           <View style={[styles.headerBlock, styles.headerRight]}>
105             <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
106             <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
107           </View>
108         </View>
109
110         <View style={styles.logoRow}>
111           <Image style={styles.logo} src={rtuLogoSrc} />
112           <View style={styles.centerTextWrap}>
113             <Text style={styles.centerLine}>Rizal Technological University</Text>
114             <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
115             <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
116             <Text style={styles.centerAgreement}>NON-DISCLOSURE AGREEMENT</Text>
117           </View>
118           <Image style={styles.logoDpo} src={dpoLogoSrc} />
119         </View>
120
121         <Text style={styles.paragraph}>
122           I, {studentNameUpper || "_____"}, as the representative of the group, declare that it is our
123           responsibility to protect the integrity and confidentiality of the enclosed information. As the intended
124           recipient, we ensure that we will not reproduce, share, or disclose any information to anyone without the
125           consent of the data subject. Furthermore, we affirm that the data will only be used to conduct the research
126           titled:
127         </Text>
128         <Text style={styles.paragraphNoIndent}>
129           <Text style={styles.titleText}>"{fd.researchTitle || "_____"}"</Text>
130         </Text>
131         <Text style={styles.paragraph}>
132           We guarantee our commitment to strictly adhere to the Data Privacy Act of 2012, as well as any current and
133           applicable Implementing Rules and Regulations of the National Privacy Commission and other relevant
134           legislation, rules, and publications.
135         </Text>
136         <Text style={styles.paragraph}>
137           We acknowledge the necessity of implementing the required security measures to protect the data, which
138           includes establishing appropriate organizational, physical, and technical security actions. Any
139           unauthorized disclosure, risk, or suspicious regarding such matters must be reported immediately to the
140           Data Protection Office.
141         </Text>
142         <Text style={styles.paragraph}>
143           We categorically guarantee that we will not reveal any generally Confidential, highly sensitive data, or
144           private information obtained, either directly or indirectly, from the respondents to any third party.
145         </Text>
146         <Text style={styles.paragraphFinal}>
147           Finally, we understand that we may be held liable for any damages caused by the unauthorized processing of
148           this information.
149         </Text>

```

```
150
151     <View style={styles.afterParagraph}>
152       {request.qrDataUrl ? (
153         <View style={styles.qrWrap}>
154           <Image style={styles.qrImage} src={request.qrDataUrl} />
155         </View>
156       ) : null}
157     <View style={styles.rightColumn}>
158       {hasAnySig ? (
159         <>
160           <SigBlock
161             label="Researcher's signature over printed name"
162             name={studentNameUpper}
163             imgUrl={studentSigDataUrl || null}
164           />
165           <SigBlock
166             dateReceived={`Date Received: ${formatDate(request.updatedAt)} `}
167             label="Approving Officer's Signature over printed name"
168             name={toUpper(approverName)}
169             imgUrl={adminSigDataUrl || null}
170           />
171         </>
172       ) : null}
173     </View>
174   </View>
175 </Page>
176 </Document>
177 );
178 }
```

35. client/src/config/documentTemplates/NDASStudentOrgActivitiesDoc.jsx (178 lines)

```

1 import React from "react";
2 import { Document, Page, View, Image, Text, StyleSheet } from "@react-pdf/renderer";
3 import { formatDate, getTemplateAssetUrl } from "../styles";
4
5 const styles = StyleSheet.create({
6   page: {
7     paddingTop: 36,
8     paddingBottom: 40,
9     paddingHorizontal: 36,
10    fontSize: 10,
11    fontFamily: "Times-Roman",
12    color: "#111",
13  },
14  headerRow: { flexDirection: "row", justifyContent: "space-between" },
15  headerBlock: { width: "48%" },
16  headerLeftTitle: { fontSize: 10, fontFamily: "Times-Bold" },
17  headerLeftCode: { fontSize: 9, marginTop: 2 },
18  headerRight: { alignItems: "flex-end" },
19  headerRightLine: { fontSize: 9 },
20  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 12, marginBottom: 10 },
21  logo: { width: 70, height: 70, objectFit: "contain", marginHorizontal: 8 },
22  logoDpo: { width: 60, height: 60, objectFit: "contain", marginHorizontal: 8 },
23  centerTextWrap: { alignItems: "center", marginHorizontal: 6 },
24  centerLine: { fontSize: 10, textAlign: "center" },
25  centerOffice: { fontSize: 10, textAlign: "center" },
26  centerAgreement: { fontSize: 11, fontFamily: "Times-Bold", textAlign: "center", marginTop: 2 },
27  paragraph: {
28    width: "85%",
29    alignSelf: "center",
30    textAlign: "justify",
31    textIndent: 24,
32    lineHeight: 1.25,
33    marginTop: 14,
34  },
35  paragraphNoIndent: {
36    width: "85%",
37    alignSelf: "center",
38    textAlign: "center",
39    marginTop: 8,
40    marginBottom: 4,
41  },
42  titleText: {
43    fontFamily: "Times-BoldItalic",
44  },
45  paragraphFinal: {
46    width: "85%",
47    alignSelf: "center",
48    textAlign: "justify",
49    textIndent: 24,
50    lineHeight: 1.25,
51    marginTop: 14,
52  },
53  afterParagraph: {
54    marginTop: 18,
55    flexDirection: "row",
56    justifyContent: "space-between",
57    alignItems: "flex-end",
58    width: "85%",
59    alignSelf: "center",
60  },
61  rightColumn: { alignItems: "flex-end" },
62  sigBlock: { width: 220, alignItems: "flex-start", marginBottom: 14 },
63  sigImage: { width: 150, height: 50, objectFit: "contain", borderBottomWidth: 1, borderBottomColor: "#333" },
64  sigPlaceholder: { width: 150, height: 50, borderBottomWidth: 1, borderBottomColor: "#333" },
65  sigName: { fontSize: 10, fontFamily: "Times-Bold", marginTop: 4, textAlign: "left" },
66  sigLabel: { fontSize: 8.5, textAlign: "left" },
67  dateReceived: { fontSize: 9, alignSelf: "flex-start", marginBottom: 4, textAlign: "left" },
68  qrWrap: { alignItems: "flex-start", marginBottom: 10 },
69  qrImage: { width: 70, height: 70 },
70 });
71
72 const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
73   <View style={styles.sigBlock}>

```

```

74     {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
75     {imgUrl
76     ? <Image style={styles.sigImage} src={imgUrl} />
77     : <View style={styles.sigPlaceholder} />
78     }
79     <Text style={styles.sigName}>{name || ""}</Text>
80     <Text style={styles.sigLabel}>{label}</Text>
81   </View>
82 );
83
84 export default function NDASStudentOrgActivitiesDoc({ request }) {
85   const fd = request.formData || {};
86   const student = request.userId || {};
87   const proxy = request.proxyRequestee || {};
88   const { studentSigDataUrl, adminSigDataUrl } = request;
89   const approverName = request.approverName || request.adminName || request.approvedByName || "";
90   const toUpper = (value) => (value ? String(value).toUpperCase() : "");
91   const studentNameUpper = toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""));
92   const hasAnySig = studentSigDataUrl || adminSigDataUrl;
93   const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
94   const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");
95
96   return (
97     <Document>
98       <Page size="A4" style={styles.page}>
99         <View style={styles.headerRow}>
100           <View style={styles.headerBlock}>
101             <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
102             <Text style={styles.headerLeftCode}>RTU-OEVP-UDPC-F005</Text>
103           </View>
104           <View style={[styles.headerBlock, styles.headerRight]}>
105             <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
106             <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
107           </View>
108         </View>
109
110         <View style={styles.logoRow}>
111           <Image style={styles.logo} src={rtuLogoSrc} />
112           <View style={styles.centerTextWrap}>
113             <Text style={styles.centerLine}>Rizal Technological University</Text>
114             <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
115             <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
116             <Text style={styles.centerAgreement}>NON-DISCLOSURE AGREEMENT</Text>
117           </View>
118           <Image style={styles.logoDpo} src={dpoLogoSrc} />
119         </View>
120
121         <Text style={styles.paragraph}>
122           I, {studentNameUpper || "_____"}, as the representative of the group, declare that it is our
123           responsibility to protect the integrity and confidentiality of the enclosed information. As the intended
124           recipient, we ensure that we will not reproduce, share, or disclose any information to anyone without the
125           consent of the data subject. Furthermore, we affirm that the data will only be used to conduct the project
126           titled:
127         </Text>
128         <Text style={styles.paragraphNoIndent}>
129           <Text style={styles.titleText}>"{fd.projectTitle || "_____"}"</Text>
130         </Text>
131         <Text style={styles.paragraph}>
132           We guarantee our commitment to strictly adhere to the Data Privacy Act of 2012, as well as any current and
133           applicable Implementing Rules and Regulations of the National Privacy Commission and other relevant
134           legislation, rules, and publications.
135         </Text>
136         <Text style={styles.paragraph}>
137           We acknowledge the necessity of implementing the required security measures to protect the data, which
138           includes establishing appropriate organizational, physical, and technical security actions. Any
139           unauthorized disclosure, risk, or suspicious regarding such matters must be reported immediately to the
140           Data Protection Office.
141         </Text>
142         <Text style={styles.paragraph}>
143           We categorically guarantee that we will not reveal any generally Confidential, highly sensitive data, or
144           private information obtained, either directly or indirectly, from the respondents to any third party.
145         </Text>
146         <Text style={styles.paragraphFinal}>
147           Finally, we understand that we may be held liable for any damages caused by the unauthorized processing of
148           this information.
149         </Text>

```

```
150
151     <View style={styles.afterParagraph}>
152       {request.qrDataUrl ? (
153         <View style={styles.qrWrap}>
154           <Image style={styles.qrImage} src={request.qrDataUrl} />
155         </View>
156       ) : null}
157     <View style={styles.rightColumn}>
158       {hasAnySig ? (
159         <>
160           <SigBlock
161             label="Group Representative's signature over printed name"
162             name={studentNameUpper}
163             imgUrl={studentSigDataUrl || null}
164           />
165           <SigBlock
166             dateReceived={`Date Received: ${formatDate(request.updatedAt)} `}
167             label="Approving Officer's Signature over printed name"
168             name={toUpper(approverName)}
169             imgUrl={adminSigDataUrl || null}
170           />
171         </>
172       ) : null}
173     </View>
174   </View>
175 </Page>
176 </Document>
177 );
178 }
```

36. client/src/components/Navbar.jsx (134 lines)

```

1 import { NavLink } from "react-router-dom";
2 import { useContext, useState } from "react";
3 import { AuthContext } from "../context/AuthContext";
4 import {
5   FilePlus,
6   ClipboardList,
7   BarChart2,
8   History,
9   Users,
10  Archive,
11  Search,
12  UserCircle,
13  Logout,
14 } from "lucide-react";
15 import { motion, AnimatePresence } from "framer-motion";
16 import "../Navbar.css";
17
18 const Navbar = () => {
19   const { user, logout } = useContext(AuthContext);
20   const [showLogoutConfirm, setShowLogoutConfirm] = useState(false);
21
22   const linkClass = ({ isActive }) =>
23     `navbar-link${isActive ? " active" : ""}`;
24
25   const initials = user?.name
26     ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
27     : (user?.firstName ? user.firstName[0].toUpperCase() : "?");
28
29   return (
30     <>
31       <nav className="navbar">
32         /* Navigation links */
33         <div className="navbar-links">
34           {user?.role === "student" && (
35             <>
36               <NavLink to="/student" className={linkClass} end>
37                 <ClipboardList size={16} className="nav-icon" />
38                 Dashboard
39               </NavLink>
40               <NavLink to="/student/new-request" className={linkClass}>
41                 <FilePlus size={16} className="nav-icon" />
42                 Create Request
43               </NavLink>
44               <NavLink to="/student/profile" className={linkClass}>
45                 <UserCircle size={16} className="nav-icon" />
46                 Profile
47               </NavLink>
48             </>
49           )}
50
51           {(user?.role === "admin" || user?.role === "staff") && (
52             <>
53               <NavLink to="/admin" className={linkClass} end>
54                 <BarChart2 size={16} className="nav-icon" />
55                 Dashboard
56               </NavLink>
57               <NavLink to="/admin/requests" className={linkClass}>
58                 <ClipboardList size={16} className="nav-icon" />
59                 Request Management
60               </NavLink>
61               {user?.role === "staff" && (
62                 <NavLink to="/admin/proxy-request" className={linkClass}>
63                   <FilePlus size={16} className="nav-icon" />
64                   Proxy Request
65                 </NavLink>
66               )}
67               <NavLink to="/admin/reports" className={linkClass}>
68                 <History size={16} className="nav-icon" />
69                 Transaction Records
70               </NavLink>
71               {user?.role === "admin" && (
72                 <NavLink to="/admin/audit" className={linkClass}>
73                   <Search size={16} className="nav-icon" />

```



```

74         Audit Logs
75     </NavLink>
76 })
77 {user?.role === "admin" && (
78     <NavLink to="/admin/users" className={linkClass}>
79         <Users size={16} className="nav-icon" />
80         User Management
81     </NavLink>
82 })
83 <NavLink to="/admin/archives" className={linkClass}>
84     <Archive size={16} className="nav-icon" />
85     Archives
86 </NavLink>
87 <NavLink to="/admin/profile" className={linkClass}>
88     <UserCircle size={16} className="nav-icon" />
89     Profile
90 </NavLink>
91 </>
92 })
93 </div>
94
95 {/* User section */}
96 {user && (
97     <div className="navbar-user">
98         <div className="navbar-user-top">
99             <div className="navbar-user-avatar">
100                 {initials}
101             </div>
102             <div className="navbar-user-info">
103                 <span className="navbar-user-name">{user.name}</span>
104                 <span className="navbar-user-role">{user.role}</span>
105             </div>
106         </div>
107         <button className="navbar-logout" onClick={() => setShowLogoutConfirm(true)}>
108             <LogOut size={14} style={{ marginRight: 6 }} />
109             Log out
110         </button>
111     </div>
112 })
113 </nav>
114
115 {/* Logout Confirmation Modal */}
116 <AnimatePresence>
117     {showLogoutConfirm && (
118         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
119             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
120                 opacity: 0 }}>
121                 <h3 className="modal-title">Log Out</h3>
122                 <p className="modal-text">Are you sure you want to log out?</p>
123                 <div className="modal-actions">
124                     <button className="ui-btn ui-btn--secondary" onClick={() => setShowLogoutConfirm(false)}>Cancel</button>
125                     <button className="ui-btn ui-btn--primary" onClick={logout}>Log Out</button>
126                 </div>
127             </motion.div>
128         </motion.div>
129     )}
130 </AnimatePresence>
131 </>
132 );
133 };
134 export default Navbar;

```

37. client/src/components/Headbar.jsx (39 lines)

```
1 import { useContext } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { AuthContext } from "../context/AuthContext";
4 import "../Headbar.css";
5
6 const Headbar = () => {
7   const { user } = useContext(AuthContext);
8   const navigate = useNavigate();
9
10  if (!user) return null;
11
12  const dashboardPath = user.role === "student" ? "/student" : "/admin";
13
14  return (
15    <div className="headbar">
16      <div
17        className="headbar-brand"
18        onClick={() => navigate(dashboardPath)}
19        style={{ cursor: "pointer" }}
20      >
21        
30        <div className="headbar-brand-text">
31          <span className="headbar-brand-name">Data Protection Office</span>
32          <span className="headbar-brand-sub">Rizal Technological University</span>
33        </div>
34      </div>
35    </div>
36  );
37 };
38
39 export default Headbar;
```

38. client/src/components/FilterSelect.jsx (49 lines)

```
1 import { useEffect, useRef, useState } from "react";
2
3 const FilterSelect = ({ value, onChange, options, defaultValue = "all", className = "" }) => {
4   const [open, setOpen] = useState(false);
5   const ref = useRef(null);
6   const isActive = value !== defaultValue;
7   const selectedLabel = options.find((o) => o.value === value)?.label ?? options[0]?.label;
8
9   useEffect(() => {
10     const handler = (e) => {
11       if (ref.current && !ref.current.contains(e.target)) setOpen(false);
12     };
13     document.addEventListener("mousedown", handler);
14     return () => document.removeEventListener("mousedown", handler);
15   }, []);
16
17   return (
18     <div
19       className={`filter-select${isActive ? " filter-select--active" : ""}${className ? ` ${className}` : ""}`}
20       ref={ref}
21       onKeyDown={(e) => e.key === "Escape" && setOpen(false)}
22     >
23       <button
24         type="button"
25         className="filter-select__trigger"
26         data-open={open}
27         onClick={() => setOpen((p) => !p)}
28       >
29         <span className="filter-select__value">{selectedLabel}</span>
30       </button>
31       {open && (
32         <div className="filter-select__menu">
33           {options.map((o) => (
34             <button
35               key={o.value}
36               type="button"
37               className={`filter-select__option${o.value === value ? " filter-select__option--active" : ""}`}
38               onClick={() => { onChange(o.value); setOpen(false); }}
39             >
40               {o.label}
41             </button>
42           ))}
43         </div>
44       )}
45     </div>
46   );
47 };
48
49 export default FilterSelect;
```

39. client/src/components/PasswordChecklist.jsx (122 lines)

```

1  import { useEffect, useLayoutEffect, useState } from "react";
2  import { createPortal } from "react-dom";
3
4  const useIsoLayoutEffect = typeof window !== "undefined" ? useLayoutEffect : useEffect;
5
6  const REQUIREMENTS = [
7    "8 characters",
8    "1 uppercase",
9    "1 lowercase",
10   "1 number",
11   "1 special character",
12 ];
13
14 const TOOLTIP_WIDTH = 210;
15 const GAP = 10;
16
17 const BORDER = "#b91c1c";
18 const BG = "#fff1f2";
19 const TITLE_COLOR = "#7f1d1d";
20 const TEXT_COLOR = "#991b1b";
21
22 function useAutoDismiss(active, persistent = false, duration = 2000) {
23   const [visible, setVisible] = useState(false);
24   useEffect(() => {
25     if (persistent) return;
26     if (!active) { setVisible(false); return; }
27     setVisible(true);
28     const t = setTimeout(() => setVisible(false), duration);
29     return () => clearTimeout(t);
30   }, [active, persistent, duration]);
31   return persistent ? active : visible;
32 }
33
34 function useTooltipPos(active, anchorRef, side) {
35   const [pos, setPos] = useState(null);
36   const [actualSide, setActualSide] = useState(side);
37
38   useIsoLayoutEffect(() => {
39     if (!active || !anchorRef?.current) { setPos(null); return undefined; }
40
41     const recompute = () => {
42       const rect = anchorRef.current.getBoundingClientRect();
43       const isMobile = window.innerWidth <= 640;
44
45       if (isMobile) {
46         setActualSide("bottom");
47         setPos({ position: "fixed", top: rect.bottom + GAP, left: Math.max(8, rect.left), width: Math.min(TOOLTIP_WIDTH, window.innerWidth - rect.left - 2 * GAP) });
48         return;
49       }
50
51       if (side === "left" && rect.left - TOOLTIP_WIDTH - GAP >= 8) {
52         setActualSide("left");
53         setPos({ position: "fixed", top: rect.top + rect.height / 2, left: rect.left - TOOLTIP_WIDTH - GAP, width: TOOLTIP_WIDTH });
54         return;
55       }
56
57       setActualSide("right");
58       setPos({ position: "fixed", top: rect.top + rect.height / 2, left: Math.min(rect.right + GAP, window.innerWidth - TOOLTIP_WIDTH), width: TOOLTIP_WIDTH });
59     };
60
61     recompute();
62     window.addEventListener("resize", recompute);
63     window.addEventListener("scroll", recompute, true);
64     return () => {
65       window.removeEventListener("resize", recompute);
66       window.removeEventListener("scroll", recompute, true);
67     };
68   }, [active, anchorRef, side]);
69
70   return [pos, actualSide];
71 }
72
73 function TooltipShell({ pos, actualSide, children }) {

```

```
74   const arrowBase = { position: "absolute", width: 8, height: 8, background: BG, transform: "rotate(45deg)" };
75   const arrowStyle =
76     actualSide === "left" ? { ...arrowBase, right: -5, top: "50%", marginTop: -4, borderTop: `1px solid ${BORDER}`, borderRight: `1px solid ${BORDER}` }
77     : actualSide === "right" ? { ...arrowBase, left: -5, top: "50%", marginTop: -4, borderBottom: `1px solid ${BORDER}`, borderLeft: `1px solid ${BORDER}` }
78     : { ...arrowBase, top: -5, left: 16, borderTop: `1px solid ${BORDER}`, borderLeft: `1px solid ${BORDER}` };
79
80   const content = (
81     <div style={{ ...pos, zIndex: 9999, background: BG, border: `1px solid ${BORDER}`, borderRadius: 8, padding: "10px 13px",
82       <span style={{ position: "absolute", ...arrowStyle }} />
83       {children}
84     </div>
85   );
86
87   return typeof document !== "undefined" ? createPortal(content, document.body) : content;
88 }
89
90 /** Requirements tooltip – shown when the password field is focused */
91 export default function PasswordChecklist({ focused = false, anchorRef, side = "right", persistent = false }) {
92   const visible = useAutoDismiss(focused, persistent);
93   const [pos, actualSide] = useTooltipPos(visible, anchorRef, side);
94   if (!visible || !pos) return null;
95
96   return (
97     <TooltipShell pos={pos} actualSide={actualSide}>
98       <div style={{ fontSize: 10.5, fontWeight: 700, color: TITLE_COLOR, marginBottom: 7, textTransform: "uppercase", letterSpacing: 0.5 }}>
99         Minimum password requirements:
100       </div>
101       {REQUIREMENTS.map((req) => (
102         <div key={req} style={{ display: "flex", alignItems: "center", gap: 7, fontSize: 12.5, color: TEXT_COLOR, padding: "2px 10px" }}>
103           <span style={{ width: 4, height: 4, borderRadius: "50%", background: BORDER, flexShrink: 0 }} />
104           {req}
105         </div>
106       ))}
107     </TooltipShell>
108   );
109 }
110
111 /** Error tooltip – shown when show=true with a custom message */
112 export function ErrorTooltip({ show = false, message, anchorRef, side = "right", persistent = false }) {
113   const visible = useAutoDismiss(show, persistent);
114   const [pos, actualSide] = useTooltipPos(visible, anchorRef, side);
115   if (!visible || !pos) return null;
116
117   return (
118     <TooltipShell pos={pos} actualSide={actualSide}>
119       <div style={{ fontSize: 12.5, color: TEXT_COLOR, fontWeight: 600 }}>{message}</div>
120     </TooltipShell>
121   );
122 }
```

40. client/src/components/SessionWarningModal.jsx (60 lines)

```

1  import { useEffect, useState } from "react";
2  import { TimerReset } from "lucide-react";
3
4  export default function SessionWarningModal({ onExtend }) {
5    const [secondsLeft, setSecondsLeft] = useState(120);
6
7    useEffect(() => {
8      const interval = setInterval(() => {
9        setSecondsLeft((s) => {
10          if (s <= 1) {
11            clearInterval(interval);
12            return 0;
13          }
14          return s - 1;
15        });
16      }, 1000);
17      return () => clearInterval(interval);
18    }, []);
19
20    const minutes = Math.floor(secondsLeft / 60);
21    const seconds = secondsLeft % 60;
22
23    return (
24      <div style={{
25        position: "fixed", inset: 0, zIndex: 9999,
26        background: "rgba(0,0,0,0.45)",
27        display: "flex", alignItems: "center", justifyContent: "center",
28      }}>
29        <div style={{
30          background: "#fff", borderRadius: 14, padding: "32px 36px",
31          maxWidth: 400, width: "90%", textAlign: "center",
32          boxShadow: "0 8px 32px rgba(0,0,0,0.18)",
33        }}>
34          <div style={{ marginBottom: 12, display: "inline-flex", alignItems: "center", justifyContent: "center" }}>
35            <TimerReset size={36} color="#1d4ed8" strokeWidth={1.8} aria-hidden="true" />
36          </div>
37          <h2 style={{ margin: "0 0 8px", fontSize: 18, fontWeight: 700, color: "#0f172a" }}>
38            Session Expiring
39          </h2>
40          <p style={{ margin: "0 0 20px", fontSize: 14, color: "#475569", lineHeight: 1.5 }}>
41            Your session will expire in{" "}
42            <strong style={{ color: "#dc2626" }}>
43              {minutes}:{seconds.toString().padStart(2, "0")}
44            </strong>{" "}
45            due to inactivity.
46          </p>
47          <button
48            onClick={onExtend}
49            style={{
50              padding: "10px 28px", borderRadius: 10, border: "none",
51              background: "#0f2d6b", color: "#fff", fontSize: 14,
52              fontWeight: 600, cursor: "pointer", fontFamily: "inherit",
53            }}
54          >
55            Stay Logged In
56          </button>
57        </div>
58      </div>
59    );
60  }

```

41. client/src/components/SignaturePad.jsx (143 lines)

```
1 import { useRef, useEffect, forwardRef, useImperativeHandle } from "react";
2 import { Paintbrush2 } from "lucide-react";
3
4 /**
5  * Canvas-based signature pad.
6  * Expose ref methods: getDataUrl(), isEmpty(), clear()
7  */
8 const SignaturePad = forwardRef(function SignaturePad(
9   { height = 150, disabled = false, style = {} },
10  ref
11 ) {
12   const canvasRef = useRef(null);
13   const drawing = useRef(false);
14
15   useImperativeHandle(ref, () => ({
16     getDataUrl: () => canvasRef.current?.toDataURL("image/png") ?? null,
17     isEmpty: () => {
18       const canvas = canvasRef.current;
19       if (!canvas) return true;
20       const ctx = canvas.getContext("2d");
21       const data = ctx.getImageData(0, 0, canvas.width, canvas.height).data;
22       return !Array.from(data).some((v) => v !== 0);
23     },
24     clear: () => {
25       const canvas = canvasRef.current;
26       if (!canvas) return;
27       canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
28     },
29   }));
30
31   useEffect(() => {
32     const canvas = canvasRef.current;
33     if (!canvas) return;
34     const ctx = canvas.getContext("2d");
35     ctx.strokeStyle = "#000";
36     ctx.lineWidth = 2;
37     ctx.lineCap = "round";
38     ctx.lineJoin = "round";
39   }, []);
40
41   const getPos = (e, canvas) => {
42     const rect = canvas.getBoundingClientRect();
43     const scaleX = canvas.width / rect.width;
44     const scaleY = canvas.height / rect.height;
45     const clientX = e.touches ? e.touches[0].clientX : e.clientX;
46     const clientY = e.touches ? e.touches[0].clientY : e.clientY;
47     return {
48       x: (clientX - rect.left) * scaleX,
49       y: (clientY - rect.top) * scaleY,
50     };
51   };
52
53   const onStart = (e) => {
54     if (disabled) return;
55     e.preventDefault();
56     const canvas = canvasRef.current;
57     const ctx = canvas.getContext("2d");
58     const { x, y } = getPos(e, canvas);
59     drawing.current = true;
60     ctx.beginPath();
61     ctx.moveTo(x, y);
62   };
63
64   const onMove = (e) => {
65     if (!drawing.current || disabled) return;
66     e.preventDefault();
67     const canvas = canvasRef.current;
68     const ctx = canvas.getContext("2d");
69     const { x, y } = getPos(e, canvas);
70     ctx.lineTo(x, y);
71     ctx.stroke();
72   };
73
```

```

74   const onEnd = (e) => {
75     e.preventDefault();
76     drawing.current = false;
77   };
78
79   const handleClear = () => {
80     const canvas = canvasRef.current;
81     if (!canvas) return;
82     canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
83   };
84
85   return (
86     <div style={{ position: "relative", display: "inline-block", width: "100%", cursor: disabled ? "default" : "crosshair", u
87       <canvas
88         ref={canvasRef}
89         width={600}
90         height={height}
91         draggable={false}
92         className={disabled ? "sig-pad-canvas--disabled" : "sig-pad-canvas"}
93         style={{
94           border: "1px solid #ccc",
95           borderRadius: 4,
96           touchAction: "none",
97           userSelect: "none",
98           width: "100%",
99           display: "block",
100          background: "#fafafa",
101          ...style,
102        }}
103        onMouseDown={onStart}
104        onMouseMove={onMove}
105        onMouseUp={onEnd}
106        onMouseLeave={onEnd}
107        onTouchStart={onStart}
108        onTouchMove={onMove}
109        onTouchEnd={onEnd}
110      />
111      {!disabled && (
112        <button
113          type="button"
114          onClick={handleClear}
115          title="Clear signature"
116          style={{
117            position: "absolute",
118            top: 6,
119            right: 6,
120            width: 28,
121            height: 28,
122            display: "flex",
123            alignItems: "center",
124            justifyContent: "center",
125            background: "#fafafa",
126            border: "none",
127            borderRadius: 4,
128            cursor: "pointer",
129            color: "#64748b",
130            padding: 0,
131            transition: "background 0.15s ease, color 0.15s ease",
132          }}
133          onMouseEnter={(e) => { e.currentTarget.style.background = "#fee2e2"; e.currentTarget.style.color = "#dc2626"; }}
134          onMouseLeave={(e) => { e.currentTarget.style.background = "rgba(255,255,255,0.85)"; e.currentTarget.style.color = "
135        >
136        <Paintbrush2 size={14} strokeWidth={1.8} style={{ transform: "scaleY(-1)" }} />
137      </button>
138    )}
139  </div>
140  );
141  });
142
143  export default SignaturePad;

```


42. client/src/components/RequestStepper.jsx (123 lines)

```

1 import { Check, X } from "lucide-react";
2 import { motion } from "framer-motion";
3 import "../RequestStepper.css";
4
5 const NDA_STEPS = ["Submitted", "Reviewal", "Approved"];
6 const AGREEMENT_STEPS = [
7   "Submitted",
8   "Initial Reviewal",
9   "Recipient Reviewal",
10  "Final Reviewal",
11  "Approved",
12 ];
13
14 const STATUS_TO_INDEX = {
15   // NDA workflow
16   nda_pending: 1,
17   stud_revision_requested: 1,
18   nda_approved: 2,
19
20   // Agreement workflow
21   agr_pending_1: 1,
22   agr_awaiting_rep_signature: 2,
23   agr_rep_revision_requested: 3,
24   agr_pending_2: 3,
25   agr_approved: 4,
26   agr_declined: 4,
27 };
28
29 const REVISION_LABEL = {
30   stud_revision_requested: "Student Revisions",
31   agr_rep_revision_requested: "Recipient Revisions",
32 };
33
34 export default function RequestStepper({ status, type }) {
35   const steps = type === "agreement" ? AGREEMENT_STEPS : NDA_STEPS;
36   const isDeclined = status === "agr_declined";
37   const isRevision = status in REVISION_LABEL;
38   const finalStepIndex = steps.length - 1;
39   const declinedIndex = type === "agreement" ? 2 : 0;
40   const activeIndex = isDeclined ? declinedIndex : (STATUS_TO_INDEX[status] ?? 0);
41   const isFinalApproved = ["nda_approved", "agr_approved", "agreement_approved"].includes(status);
42
43   return (
44     <div className="request-stepper-wrap">
45       <div className="request-stepper-track">
46         {steps.map((step, index) => {
47           const isCompleted = index < activeIndex || (isFinalApproved && index <= finalStepIndex);
48           const isActive = index === activeIndex;
49           const isDeclinedNode = isDeclined && index === declinedIndex;
50
51           return (
52             <div key={`-${step}-${index}`} className="request-stepper-segment">
53               <div className="request-stepper-node-wrap">
54                 <div
55                   className={`request-stepper-node ${
56                     isDeclinedNode
57                       ? "is-declined"
58                       : isCompleted
59                       ? "is-completed"
60                       : isActive && isRevision
61                       ? "is-active-revision"
62                       : isActive
63                       ? "is-active"
64                       : "is-upcoming"
65                   }`}
66                 >
67                   {isDeclinedNode ? (
68                     <X size={13} strokeWidth={3} />
69                   ) : isCompleted ? (
70                     <Check size={14} strokeWidth={3} />
71                   ) : isActive ? (
72                     <motion.span
73                       className="request-stepper-node-core"

```

```
74         animate={{ scale: [1, 1.25, 1] }}
75         transition={{ duration: 1.6, repeat: Infinity, ease: "easeInOut" }}
76     />
77     ) : (
78     <span className="request-stepper-node-number">{index + 1}</span>
79     )}
80 </div>
81
82 <div
83     className={`request-stepper-label ${
84         isDeclinedNode
85         ? "is-declined"
86         : isCompleted
87         ? "is-completed"
88         : isActive && isRevision
89         ? "is-active-revision"
90         : isActive
91         ? "is-active"
92         : "is-upcoming"
93     }`}
94 >
95     {isDeclinedNode
96     ? "Declined"
97     : isActive && isRevision
98     ? REVISION_LABEL[status]
99     : step}
100 </div>
101 </div>
102
103 {index < steps.length - 1 && (
104     <div className="request-stepper-line-shell" aria-hidden="true">
105         <div className="request-stepper-line-base" />
106         <motion.div
107             className="request-stepper-line-fill"
108             initial={false}
109             animate={{
110                 width: index < activeIndex ? "100%" : "0%",
111             }}
112             transition={{ duration: 0.45, ease: "easeOut" }}
113         />
114     </div>
115     )}
116 </div>
117 );
118 }}}}
119 </div>
120 </div>
121 );
122 }
123
```

43. client/src/components/InPageFeedbackHost.jsx (154 lines)

```

1 import { useEffect, useMemo, useRef, useState } from "react";
2 import { CircleAlert, CircleCheck, CircleX, Info } from "lucide-react";
3 import { FEEDBACK_EVENTS, inferFeedbackType, notify } from "../utils/inPageFeedback";
4
5 const TYPE_META = {
6   success: {
7     icon: CircleCheck,
8     title: "Success",
9   },
10  error: {
11    icon: CircleX,
12    title: "Something went wrong",
13  },
14  warning: {
15    icon: CircleAlert,
16    title: "Action needed",
17  },
18  info: {
19    icon: Info,
20    title: "Notice",
21  },
22 };
23
24 export default function InPageFeedbackHost() {
25   const [toasts, setToasts] = useState([]);
26   const [confirmState, setConfirmState] = useState(null);
27   const nativeAlertRef = useRef(null);
28
29   useEffect(() => {
30     const onNotify = (event) => {
31       const detail = event.detail || {};
32       const id = `${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
33       const toast = {
34         id,
35         type: detail.type || inferFeedbackType(detail.message),
36         title: detail.title,
37         message: detail.message || "",
38         duration: typeof detail.duration === "number" ? detail.duration : 3800,
39       };
40
41       setToasts((prev) => {
42         const next = [...prev, toast];
43         return next.slice(-4);
44       });
45
46       if (toast.duration > 0) {
47         window.setTimeout(() => {
48           setToasts((prev) => prev.filter((item) => item.id !== id));
49         }, toast.duration);
50       }
51     };
52
53     const onConfirm = (event) => {
54       const detail = event.detail || {};
55       setConfirmState({
56         title: detail.title,
57         message: detail.message,
58         confirmText: detail.confirmText,
59         cancelText: detail.cancelText,
60         tone: detail.tone,
61         resolve: detail.resolve,
62       });
63     };
64
65     window.addEventListener(FEEDBACK_EVENTS.NOTIFY_EVENT, onNotify);
66     window.addEventListener(FEEDBACK_EVENTS.CONFIRM_EVENT, onConfirm);
67
68     nativeAlertRef.current = window.alert;
69     window.alert = (message) => {
70       notify(String(message ?? ""));
71     };
72
73     return () => {

```

```

74     window.removeEventListener(FEEDBACK_EVENTS.NOTIFY_EVENT, onNotify);
75     window.removeEventListener(FEEDBACK_EVENTS.CONFIRM_EVENT, onConfirm);
76     if (nativeAlertRef.current) {
77         window.alert = nativeAlertRef.current;
78     }
79 };
80 }, []);
81
82 const confirmMeta = useMemo(() => {
83     const tone = confirmState?.tone || "warning";
84     return TYPE_META[tone] || TYPE_META.warning;
85 }, [confirmState]);
86 const ConfirmIcon = confirmMeta.icon;
87
88 const closeConfirm = (accepted) => {
89     if (confirmState?.resolve) confirmState.resolve(Boolean(accepted));
90     setConfirmState(null);
91 };
92
93 useEffect(() => {
94     if (!confirmState) return undefined;
95     const onKeyDown = (event) => {
96         if (event.key === "Escape") {
97             event.preventDefault();
98             closeConfirm(false);
99         }
100     };
101     window.addEventListener("keydown", onKeyDown);
102     return () => window.removeEventListener("keydown", onKeyDown);
103 }, [confirmState]);
104
105 return (
106     <>
107         <div className="inpage-feedback-stack" role="status" aria-live="polite" aria-atomic="false">
108             {toasts.map((toast) => {
109                 const meta = TYPE_META[toast.type] || TYPE_META.info;
110                 const Icon = meta.icon;
111                 return (
112                     <div key={toast.id} className={`inpage-toast inpage-toast--${toast.type}`}>
113                         <div className="inpage-toast-icon-wrap" aria-hidden="true">
114                             <Icon size={18} strokeWidth={2} />
115                         </div>
116                         <div className="inpage-toast-content">
117                             <div className="inpage-toast-title">{toast.title || meta.title}</div>
118                             <div className="inpage-toast-message">{toast.message}</div>
119                         </div>
120                     </div>
121                 );
122             })}
123         </div>
124
125         {confirmState && (
126             <div className="inpage-confirm-backdrop" role="presentation" onClick={() => closeConfirm(false)}>
127                 <div
128                     className="inpage-confirm"
129                     role="alertdialog"
130                     aria-modal="true"
131                     aria-labelledby="inpage-confirm-title"
132                     onClick={(event) => event.stopPropagation()}
133                 >
134                     <div className={`inpage-confirm-icon inpage-confirm-icon--${confirmState.tone || "warning"}`} aria-hidden="true">
135                         <ConfirmIcon size={22} strokeWidth={2} />
136                     </div>
137                     <h3 id="inpage-confirm-title" className="inpage-confirm-title">
138                         {confirmState.title || "Please confirm"}
139                     </h3>
140                     <p className="inpage-confirm-message">{confirmState.message}</p>
141                     <div className="inpage-confirm-actions">
142                         <button type="button" className="inpage-btn inpage-btn--ghost" onClick={() => closeConfirm(false)}>
143                             {confirmState.cancelText || "Cancel"}
144                         </button>
145                         <button type="button" className="inpage-btn inpage-btn--primary" onClick={() => closeConfirm(true)}>
146                             {confirmState.confirmText || "Confirm"}
147                         </button>
148                     </div>
149                 </div>

```

```
150         </div>
151     )}
152 </>
153 );
154 }
```

44. client/src/pages/Landing.jsx (587 lines)

```

1 import { useState, useContext, useCallback, useEffect, useRef } from "react";
2 import { useNavigate } from "react-router-dom";
3 import axios from "axios";
4 import { motion, AnimatePresence } from "framer-motion";
5 import { Eye, EyeOff } from "lucide-react";
6 import { AuthContext } from "../../context/AuthContext";
7 import {
8   login as loginService,
9   verifyLoginOtp,
10  resendLoginOtp,
11  forgotPassword,
12  resendResetOtp,
13  verifyResetOtp,
14  resetPassword,
15 } from "../../services/authService";
16 import PasswordChecklist from "../../components/PasswordChecklist";
17 import { isStrongPassword, PASSWORD_POLICY_MESSAGE } from "../../utils/passwordPolicy";
18 import { notify } from "../../utils/inPageFeedback";
19 import "../../components/Landing.css";
20 import "../../components/FloatingLabel.css";
21
22 const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/auth`;
23
24 // ■■ Forgot-password steps: "email" → "otp" → "newpass"
25 function ForgotPasswordFlow({ onCancel }) {
26   const [step, setStep] = useState("email");
27   const [email, setEmail] = useState("");
28   const [otp, setOtp] = useState("");
29   const [resetToken, setResetToken] = useState("");
30   const [newPassword, setNewPassword] = useState("");
31   const [confirmPassword, setConfirmPassword] = useState("");
32   const [loading, setLoading] = useState(false);
33   const [resendSeconds, setResendSeconds] = useState(0);
34   const [showNewPass, setShowNewPass] = useState(false);
35   const [showConfirmPass, setShowConfirmPass] = useState(false);
36   const forgotPwRef = useRef(null);
37
38   useEffect(() => {
39     if (resendSeconds <= 0) return undefined;
40     const timer = window.setInterval(() => {
41       setResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
42     }, 1000);
43     return () => window.clearInterval(timer);
44   }, [resendSeconds]);
45
46   const handleSendOtp = async (e) => {
47     e.preventDefault();
48     if (!email.trim()) { notify("Please enter your email address.", { type: "error", title: "Failed" }); return; }
49     setLoading(true);
50     try {
51       await forgotPassword(email.trim());
52       setStep("otp");
53       setResendSeconds(60);
54     } catch (err) {
55       notify(err.response?.data?.message || "Failed to send OTP. Try again.", { type: "error", title: "Failed" });
56     } finally {
57       setLoading(false);
58     }
59   };
60
61   const handleResendOtp = async () => {
62     if (resendSeconds > 0) return;
63     setLoading(true);
64     try {
65       await resendResetOtp(email.trim());
66       setResendSeconds(60);
67       notify("OTP resent successfully.", { type: "success", title: "Sent" });
68     } catch (err) {
69       notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
70     } finally {
71       setLoading(false);
72     }
73   };

```

```

74
75 const handleVerifyOtp = async (e) => {
76   e.preventDefault();
77   if (!otp.trim()) { notify("Please enter the OTP.", { type: "error", title: "Failed" }); return; }
78   setLoading(true);
79   try {
80     const data = await verifyResetOtp(email.trim(), otp.trim());
81     setResetToken(data.resetToken);
82     setStep("newpass");
83   } catch (err) {
84     notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
85   } finally {
86     setLoading(false);
87   }
88 };
89
90 const handleResetPassword = async (e) => {
91   e.preventDefault();
92   if (!isStrongPassword(newPassword)) { notify(PASSWORD_POLICY_MESSAGE, { type: "error", title: "Failed" }); return; }
93   if (newPassword !== confirmPassword) { notify("Passwords do not match.", { type: "error", title: "Failed" }); return; }
94   setLoading(true);
95   try {
96     await resetPassword(email.trim(), resetToken, newPassword);
97     notify("Password reset successfully! Please log in with your new password.", { type: "success" });
98     onCancel();
99   } catch (err) {
100     notify(err.response?.data?.message || "Failed to reset password.", { type: "error", title: "Failed" });
101   } finally {
102     setLoading(false);
103   }
104 };
105
106 return (
107   <div className="landing-card">
108     <h2 className="landing-title">
109       {step === "email" && "Forgot Password"}
110       {step === "otp" && "Enter OTP"}
111       {step === "newpass" && "New Password"}
112     </h2>
113     {step === "email" && (
114       <form onSubmit={handleSendOtp} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
115         <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
116           Enter your registered email and we'll send you an OTP.
117         </p>
118         <div className="landing-field-group fl-wrap">
119           <input type="email" className="fl-input landing-field" placeholder=" " value={email} onChange={(e) => setEmail(e.target.value)} />
120           <label className="fl-label fl-label--glass">Email address</label>
121         </div>
122         <button type="submit" className="landing-submit" disabled={loading}>
123           {loading ? "Sending..." : "Send OTP"}
124         </button>
125         <button type="button" className="landing-link-btn" onClick={onCancel}>< Back to Login</button>
126       </form>
127     )}
128     {step === "otp" && (
129       <form onSubmit={handleVerifyOtp} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
130         <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
131           A 6-digit OTP was sent to <strong>{email}</strong>. It expires in 10 minutes.
132         </p>
133         <div className="landing-field-group fl-wrap">
134           <input type="text" className="fl-input landing-field" placeholder=" " value={otp} onChange={(e) => setOtp(e.target.value)} />
135           <label className="fl-label fl-label--glass">OTP Code</label>
136         </div>
137         <button type="submit" className="landing-submit" disabled={loading}>
138           {loading ? "Verifying..." : "Verify OTP"}
139         </button>
140         <button type="button" className="landing-link-btn" onClick={handleResendOtp} disabled={loading || resendSeconds > 0}>
141           {resendSeconds > 0 ? `Resend OTP in ${resendSeconds}s` : "Resend OTP"}
142         </button>
143       </form>
144     )}
145     {step === "newpass" && (
146       <form onSubmit={handleResetPassword} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
147         <div ref={forgotPwRef} className="landing-field-group">
148           <div className="landing-password-wrap fl-wrap">
149             <input type={showNewPass ? "text" : "password"} className="fl-input landing-field" placeholder=" " value={newPassword} />

```

```

150         <label className="fl-label fl-label--glass fl-label--pw">New Password</label>
151         <button type="button" className="landing-password-toggle" onClick={() => setShowNewPass((v) => !v)} tabIndex={-1}
152             {showNewPass ? <Eye size={18} /> : <EyeOff size={18} />}
153         </button>
154     </div>
155     <PasswordChecklist focused={newPassword.length > 0 && !isStrongPassword(newPassword)} anchorRef={forgotPwRef} side="right" />
156 </div>
157 <div className="landing-field-group">
158     <div className="landing-password-wrap fl-wrap">
159         <input type={showConfirmPass ? "text" : "password"} className="fl-input landing-field" placeholder=" " value={password} />
160         <label className="fl-label fl-label--glass fl-label--pw">Confirm Password</label>
161         <button type="button" className="landing-password-toggle" onClick={() => setShowConfirmPass((v) => !v)} tabIndex={-1}
162             {showConfirmPass ? <Eye size={18} /> : <EyeOff size={18} />}
163         </button>
164     </div>
165 </div>
166 <button type="submit" className="landing-submit" disabled={loading}>
167     {loading ? "Resetting..." : "Reset Password"}
168 </button>
169 </form>
170 )}
171 </div>
172 );
173 }
174
175 const Landing = () => {
176     const [mode, setMode] = useState("login"); // "login" | "register" | "forgot"
177     const [form, setForm] = useState({ firstName: "", middleInitial: "", lastName: "", email: "", password: "", program: "" });
178
179     const PROGRAMS = [
180         "BS Accountancy",
181         "BS Architecture",
182         "BS Business Administration",
183         "BS Civil Engineering",
184         "BS Computer Science",
185         "BS Information Technology",
186         "BS Nursing",
187         "BS Psychology",
188         "BS Tourism Management",
189         "Bachelor of Secondary Education",
190     ];
191     const [loading, setLoading] = useState(false);
192     const [showPassword, setShowPassword] = useState(false);
193     const [passwordFocused, setPasswordFocused] = useState(false);
194     const [programOpen, setProgramOpen] = useState(false);
195     const programRef = useRef(null);
196
197     useEffect(() => {
198         const handler = (e) => {
199             if (programRef.current && !programRef.current.contains(e.target)) setProgramOpen(false);
200         };
201         document.addEventListener("mousedown", handler);
202         return () => document.removeEventListener("mousedown", handler);
203     }, []);
204     const [hasMounted, setHasMounted] = useState(false);
205     const [bgLoaded, setBgLoaded] = useState(false);
206     const { login } = useContext(AuthContext);
207     const navigate = useNavigate();
208
209     // Parallax state
210     const bgRef = useRef(null);
211     const passwordHintAnchorRef = useRef(null);
212     const handleMouseMove = useCallback((e) => {
213         if (!bgRef.current) return;
214         const { clientX, clientY } = e;
215         const { innerWidth, innerHeight } = window;
216         const x = ((clientX / innerWidth) - 0.5) * -20;
217         const y = ((clientY / innerHeight) - 0.5) * -20;
218         bgRef.current.style.transform = `translate3d(${x}px, ${y}px, 0) scale(1.05)`;
219     }, []);
220
221     // OTP step state (for context-aware login OTP)
222     const [otpStep, setOtpStep] = useState(false);
223     const [loginOtp, setLoginOtp] = useState("");
224     const [pendingEmail, setPendingEmail] = useState("");
225     const [loginResendSeconds, setLoginResendSeconds] = useState(0);

```



```
226
227 useEffect(() => {
228   if (loginResendSeconds <= 0) return undefined;
229   const timer = window.setInterval(() => {
230     setLoginResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
231   }, 1000);
232   return () => window.clearInterval(timer);
233 }, [loginResendSeconds]);
234
235 const onChange = (key) => (e) => setForm((p) => ({ ...p, [key]: e.target.value }));
236 const resetForm = () => {
237   setForm({ firstName: "", middleInitial: "", lastName: "", email: "", password: "", program: "" });
238   setOtpStep(false);
239   setLoginOtp("");
240   setPendingEmail("");
241   setLoginResendSeconds(0);
242 };
243
244 const handleSubmit = async (e) => {
245   e.preventDefault();
246   setLoading(true);
247   try {
248     if (mode === "login") {
249       const data = await loginService(form.email, form.password);
250       if (data.requireOtp) {
251         setPendingEmail(form.email);
252         setOtpStep(true);
253         setLoginResendSeconds(60);
254         setLoading(false);
255         return;
256       }
257       if (data.requireVerification) {
258         navigate(`/verify-email?email=${encodeURIComponent(data.email || form.email)}`);
259         return;
260       }
261       login(data.user);
262       navigate(data.user.role === "student" ? "/student" : "/admin/requests");
263       return;
264     }
265     if (!isLogin && !isStrongPassword(form.password)) {
266       setLoading(false);
267       return;
268     }
269     if (!form.program) {
270       notify("Please select your program/course.", { type: "warning" });
271       setLoading(false);
272       return;
273     }
274     const res = await axios.post(`${API_URL}/register`, {
275       firstName: form.firstName,
276       middleInitial: form.middleInitial,
277       lastName: form.lastName,
278       email: form.email,
279       password: form.password,
280       program: form.program,
281     });
282     if (res.data.requireVerification) {
283       navigate(`/verify-email?email=${encodeURIComponent(res.data.email)}`);
284       return;
285     }
286     notify(res.data.message || "Registered! Check your email to verify your account.", { type: "success", title: "Success" });
287     resetForm();
288     setMode("login");
289   } catch (err) {
290     if (mode === "login" && err.response?.data?.requireVerification) {
291       navigate(`/verify-email?email=${encodeURIComponent(err.response?.data?.email || form.email)}`);
292       return;
293     }
294     notify(
295       err.response?.data?.message ||
296       (mode === "login" ? "Invalid credentials" : "Registration failed"),
297       { type: "error", title: "Failed" }
298     );
299   } finally {
300     setLoading(false);
301   }
}
```

```
302 };
303
304 const handleVerifyLoginOtp = async (e) => {
305   e.preventDefault();
306   setLoading(true);
307   try {
308     const data = await verifyLoginOtp(pendingEmail, loginOtp.trim());
309     login(data.user);
310     navigate(data.user.role === "student" ? "/student" : "/admin/requests");
311   } catch (err) {
312     notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
313   } finally {
314     setLoading(false);
315   }
316 };
317
318 const handleResendLoginOtp = async () => {
319   if (!pendingEmail || loginResendSeconds > 0) return;
320   setLoading(true);
321   try {
322     const data = await resendLoginOtp(pendingEmail);
323     notify(data.message || "A new OTP has been sent to your email.", { type: "success", title: "Sent" });
324     setLoginResendSeconds(60);
325   } catch (err) {
326     notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
327   } finally {
328     setLoading(false);
329   }
330 };
331
332 const isLogin = mode === "login";
333
334 useEffect(() => {
335   setHasMounted(true);
336
337   const preload = new Image();
338   preload.src = "/RTU-Background.webp";
339   preload.onload = () => setBgLoaded(true);
340   preload.onerror = () => setBgLoaded(false);
341 }, []);
342
343 useEffect(() => {
344   // Lock page scroll while on the landing/auth route for app-like immersion.
345   const previousBodyOverflow = document.body.style.overflow;
346   const previousHtmlOverflow = document.documentElement.style.overflow;
347
348   document.body.style.overflow = "hidden";
349   document.documentElement.style.overflow = "hidden";
350
351   return () => {
352     document.body.style.overflow = previousBodyOverflow;
353     document.documentElement.style.overflow = previousHtmlOverflow;
354   };
355 }, []);
356
357 return (
358   <div className={`landing ${hasMounted ? "is-mounted" : ""}`} onMouseMove={handleMouseMove}>
359     <div
360       ref={bgRef}
361       className={`landing-parallax-bg ${bgLoaded ? "is-loaded" : ""}`}
362       style={{ backgroundImage: "url('/RTU-Background.webp')" }}
363     />
364     
375     <div className="landing-overlay" />
376     <div className="landing-wrapper">
377       /* ■■■ Brand Header (on blue background) ■■■ */
```

```

378     <div className="landing-brand">
379       
388       <div className="landing-brand-text">
389         <div className="landing-brand-title">Data Protection Office</div>
390         <div className="landing-brand-subtitle">Rizal Technological University</div>
391       </div>
392     </div>
393
394     {/ * ■■ White Form Panel ■■ */}
395     <div className={`landing-panel ${hasMounted ? "is-mounted" : ""}`}>
396       {/ * ■■ Form Area ■■ */}
397       <AnimatePresence mode="wait">
398         {/ * Login OTP step */}
399         {otpStep ? (
400           <motion.div
401             key="login-otp"
402             initial={{ opacity: 0, y: 16 }}
403             animate={{ opacity: 1, y: 0 }}
404             exit={{ opacity: 0, y: -16 }}
405             transition={{ duration: 0.2 }}
406             style={{ width: "100%", maxWidth: 380 }}
407           >
408             <form onSubmit={handleVerifyLoginOtp} className="landing-card">
409               <h2 className="landing-title">Verify Your Identity</h2>
410               <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
411                 A 6-digit OTP was sent to <strong>{pendingEmail}</strong>. Enter it below to continue.
412               </p>
413               <div className="landing-field-group fl-wrap">
414                 <input
415                   type="text"
416                   className="fl-input landing-field"
417                   placeholder=" "
418                   value={loginOtp}
419                   onChange={(e) => setLoginOtp(e.target.value)}
420                   maxLength={6}
421                   autoFocus
422                   required
423                 />
424                 <label className="fl-label fl-label--glass">OTP Code</label>
425               </div>
426               <button type="submit" className="landing-submit" disabled={loading}>
427                 {loading ? "Verifying..." : "Verify & Login"}
428               </button>
429               <button
430                 type="button"
431                 className="landing-link-btn"
432                 onClick={handleResendLoginOtp}
433                 disabled={loading || loginResendSeconds > 0}
434               >
435                 {loginResendSeconds > 0 ? `Resend OTP in ${loginResendSeconds}s` : "Resend OTP"}
436               </button>
437               <button type="button" className="landing-link-btn" onClick={resetForm}>
438                 ← Back to Login
439               </button>
440             </form>
441           </motion.div>
442         ) : mode === "forgot" ? (
443           <motion.div
444             key="forgot"
445             initial={{ opacity: 0, y: 16 }}
446             animate={{ opacity: 1, y: 0 }}
447             exit={{ opacity: 0, y: -16 }}
448             transition={{ duration: 0.2 }}
449             style={{ width: "100%", maxWidth: 380 }}
450           >
451             <ForgotPasswordFlow onCancel={() => { setMode("login"); resetForm(); }} />
452           </motion.div>
453         ) : (

```

```

454     <motion.div
455       key="auth"
456       initial={{ opacity: 0, y: 16 }}
457       animate={{ opacity: 1, y: 0 }}
458       exit={{ opacity: 0, y: -16 }}
459       transition={{ duration: 0.2 }}
460       style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 24, width: "100%" }}
461     >
462       <div className="landing-toggle">
463         <button
464           type="button"
465           className={`landing-toggle-btn ${isLogin ? "active" : ""}`}
466           onClick={() => { setMode("login"); resetForm(); }}
467         >
468           Login
469         </button>
470         <button
471           type="button"
472           className={`landing-toggle-btn ${!isLogin ? "active" : ""}`}
473           onClick={() => { setMode("register"); resetForm(); }}
474         >
475           Register
476         </button>
477         <div className={`landing-toggle-indicator ${isLogin ? "left" : "right"} ` />
478       </div>
479
480       <form onSubmit={handleSubmit} className={`landing-card ${!isLogin ? "landing-card--register" : ""}`}>
481         <h2 className="landing-title">{isLogin ? "Welcome back" : "Create an account"}</h2>
482
483         {!isLogin && (
484           <div className="landing-name-grid">
485             <div className="landing-field-group fl-wrap">
486               <input id="landing-firstname" type="text" placeholder=" " value={form.firstName} onChange={onChange("fi
487               <label className="fl-label fl-label--glass">First Name</label>
488             </div>
489             <div className="landing-field-group fl-wrap">
490               <input id="landing-mi" type="text" placeholder=" " value={form.middleInitial} onChange={onChange("middl
491               <label className="fl-label fl-label--glass">Middle Name</label>
492             </div>
493             <div className="landing-field-group fl-wrap">
494               <input id="landing-lastname" type="text" placeholder=" " value={form.lastName} onChange={onChange("last
495               <label className="fl-label fl-label--glass">Last Name</label>
496             </div>
497           </div>
498         )}
499
500         {!isLogin && (
501           <div className="landing-field-group fl-wrap" ref={programRef}>
502             <button
503               type="button"
504               className="landing-field landing-program-trigger"
505               data-open={programOpen}
506               onClick={() => setProgramOpen((p) => !p)}
507             >
508               <span className="landing-program-value">{form.program || ""}</span>
509             </button>
510             <label className="fl-label fl-label--glass landing-program-label" data-active={!!(form.program || program
511               Program / Course
512             </label>
513             {programOpen && (
514               <div className="landing-program-menu">
515                 {PROGRAMS.map((p) => (
516                   <button
517                     key={p}
518                     type="button"
519                     className={`filter-select__option${form.program === p ? " filter-select__option--active" : ""}`}
520                     onClick={() => { onChange("program")({ target: { value: p } }); setProgramOpen(false); }}
521                   >
522                     {p}
523                   </button>
524                 )})}
525               </div>
526             )}
527           </div>
528         )}
529       </div>

```

```
530     <div className="landing-field-group fl-wrap">
531       <input id="landing-email" type="email" placeholder=" " value={form.email} onChange={onChange("email")} required="" />
532       <label className="fl-label fl-label--glass">Email address</label>
533     </div>
534
535     <div ref={passwordHintAnchorRef} className="landing-field-group password-hint-anchor">
536       <div className="landing-password-wrap fl-wrap">
537         <input
538           id="landing-password"
539           type={showPassword ? "text" : "password"}
540           placeholder=" "
541           value={form.password}
542           onChange={onChange("password")}
543           onFocus={() => setPasswordFocused(true)}
544           onBlur={() => setPasswordFocused(false)}
545           required
546           className="fl-input landing-field"
547         />
548         <label className="fl-label fl-label--glass fl-label--pw">Password</label>
549         <button
550           type="button"
551           className="landing-password-toggle"
552           onClick={() => setShowPassword((v) => !v)}
553           tabIndex={-1}
554           aria-label={showPassword ? "Hide password" : "Show password"}
555         >
556           {showPassword ? <Eye size={18} /> : <EyeOff size={18} />}
557         </button>
558       </div>
559       <PasswordChecklist focused={!isLogin && !isStrongPassword(form.password) && (passwordFocused || form.password)} />
560     </div>
561
562     {isLogin && (
563       <button
564         type="button"
565         className="landing-link-btn"
566         style={{ textAlign: "right", marginTop: -6 }}
567         onClick={() => { setMode("forgot"); resetForm(); }}
568       >
569         Forgot password?
570       </button>
571     )}
572
573     <button type="submit" className="landing-submit" disabled={loading}>
574       {loading ? "Please wait..." : (isLogin ? "Log in" : "Create Account")}
575     </button>
576   </form>
577 </motion.div>
578 )}
579 </AnimatePresence>
580 </div>
581 </div>
582 </div>
583 );
584 };
585
586 export default Landing;
587
```

45. client/src/pages/ActivateAccountPage.jsx (160 lines)

```

1 import { useState } from "react";
2 import { useSearchParams, Link } from "react-router-dom";
3 import { CheckCircle, XCircle } from "lucide-react";
4 import { activateAccount } from "../services/authService";
5 import PasswordChecklist from "../components/PasswordChecklist";
6 import { isStrongPassword, PASSWORD_POLICY_MESSAGE } from "../utils/passwordPolicy";
7
8 export default function ActivateAccountPage() {
9   const [params] = useSearchParams();
10  const token = params.get("token");
11  const [password, setPassword] = useState("");
12  const [confirmPassword, setConfirmPassword] = useState("");
13  const [passwordAttempted, setPasswordAttempted] = useState(false);
14  const [loading, setLoading] = useState(false);
15  const [status, setStatus] = useState("form"); // "form" | "success" | "error"
16  const [message, setMessage] = useState("");
17
18  if (!token) {
19    return (
20      <div style={{
21        minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center",
22        background: "#f1f5f9", fontFamily: "'Inter', system-ui, sans-serif",
23      }}>
24        <div style={{
25          background: "white", borderRadius: 14, padding: "40px 36px",
26          maxWidth: 420, width: "90%", textAlign: "center",
27          boxShadow: "0 1px 3px rgba(15,23,42,0.06), 0 4px 12px rgba(15,23,42,0.06)",
28          border: "1px solid #e2e8f0",
29        }}>
30          <XCircle size={48} color="#dc2626" style={{ marginBottom: 12 }} />
31          <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Invalid Link</h2>
32          <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>Missing activation token.</p>
33          <Link to="/" style={{
34            display: "inline-block", padding: "10px 24px", borderRadius: 10,
35            background: "#0f2d6b", color: "white", textDecoration: "none",
36            fontWeight: 600, fontSize: 14,
37          }}>Go to Login</Link>
38        </div>
39      </div>
40    );
41  }
42
43  const handleSubmit = async (e) => {
44    e.preventDefault();
45    setMessage("");
46    if (!isStrongPassword(password)) { setPasswordAttempted(true); setMessage(PASSWORD_POLICY_MESSAGE); return; }
47    if (password !== confirmPassword) { setMessage("Passwords do not match."); return; }
48    setLoading(true);
49    try {
50      const data = await activateAccount(token, password);
51      setStatus("success");
52      setMessage(data.message || "Account activated!");
53    } catch (err) {
54      setStatus("error");
55      setMessage(err.response?.data?.message || "Activation failed.");
56    } finally {
57      setLoading(false);
58    }
59  };
60
61  const containerStyle = {
62    minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center",
63    background: "#f1f5f9", fontFamily: "'Inter', system-ui, sans-serif",
64  };
65
66  const cardStyle = {
67    background: "white", borderRadius: 14, padding: "40px 36px",
68    maxWidth: 420, width: "90%", textAlign: "center",
69    boxShadow: "0 1px 3px rgba(15,23,42,0.06), 0 4px 12px rgba(15,23,42,0.06)",
70    border: "1px solid #e2e8f0",
71  };
72
73  if (status === "success") {

```

```

74     return (
75         <div style={containerStyle}>
76             <div style={cardStyle}>
77                 <CheckCircle size={48} color="#16a34a" style={{ marginBottom: 12 }} />
78                 <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Account Activated!</h2>
79                 <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>{message}</p>
80                 <Link to="/" style={{
81                     display: "inline-block", padding: "10px 24px", borderRadius: 10,
82                     background: "#0f2d6b", color: "white", textDecoration: "none",
83                     fontWeight: 600, fontSize: 14,
84                 }}>Go to Login</Link>
85             </div>
86         </div>
87     );
88 }
89
90 if (status === "error") {
91     return (
92         <div style={containerStyle}>
93             <div style={cardStyle}>
94                 <XCircle size={48} color="#dc2626" style={{ marginBottom: 12 }} />
95                 <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Activation Failed</h2>
96                 <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>{message}</p>
97                 <Link to="/" style={{
98                     display: "inline-block", padding: "10px 24px", borderRadius: 10,
99                     background: "#0f2d6b", color: "white", textDecoration: "none",
100                     fontWeight: 600, fontSize: 14,
101                 }}>Go to Login</Link>
102             </div>
103         </div>
104     );
105 }
106
107 return (
108     <div style={containerStyle}>
109         <div style={{ ...cardStyle, textAlign: "left" }}>
110             <h2 style={{ margin: "0 0 4px", fontSize: 20, fontWeight: 700, color: "#0f172a", textAlign: "center" }}>
111                 Set Your Password
112             </h2>
113             <p style={{ margin: "0 0 20px", fontSize: 13, color: "#475569", textAlign: "center" }}>
114                 Choose a secure password to activate your account.
115             </p>
116             {message && (
117                 <div style={{
118                     marginBottom: 14, padding: "8px 12px", borderRadius: 10,
119                     background: "#fee2e2", color: "#991b1b", border: "1px solid #fca5a5", fontSize: 13,
120                 }}>{message}</div>
121             )}
122             <form onSubmit={handleSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
123                 <div>
124                     <label style={{ fontSize: 12.5, fontWeight: 600, color: "#475569", display: "block", marginBottom: 6 }}>New Password
125                     <input
126                         type="password" value={password} onChange={(e) => setPassword(e.target.value)}
127                         placeholder="*****" required minLength={8}
128                         style={{
129                             width: "100%", padding: "10px 14px", borderRadius: 10,
130                             border: "1px solid #cbd5e1", fontSize: 14, fontFamily: "inherit",
131                             color: "#0f172a", boxSizing: "border-box",
132                         }}
133                     />
134                     {passwordAttempted ? <PasswordChecklist password={password} compact /> : null}
135                 </div>
136                 <div>
137                     <label style={{ fontSize: 12.5, fontWeight: 600, color: "#475569", display: "block", marginBottom: 6 }}>Confirm Password
138                     <input
139                         type="password" value={confirmPassword} onChange={(e) => setConfirmPassword(e.target.value)}
140                         placeholder="*****" required
141                         style={{
142                             width: "100%", padding: "10px 14px", borderRadius: 10,
143                             border: "1px solid #cbd5e1", fontSize: 14, fontFamily: "inherit",
144                             color: "#0f172a", boxSizing: "border-box",
145                         }}
146                     />
147                 </div>
148                 <button type="submit" disabled={loading} style={{
149                     width: "100%", padding: "10px 16px", borderRadius: 10, border: "none",

```

```
150         background: "#0f2d6b", color: "#fff", fontSize: 14, fontWeight: 600,
151         cursor: loading ? "not-allowed" : "pointer", fontFamily: "inherit",
152         opacity: loading ? 0.65 : 1, marginTop: 4,
153     }}>
154     {loading ? "Activating..." : "Activate Account"}
155 </button>
156 </form>
157 </div>
158 </div>
159 );
160 }
```


46. client/src/pages/VerifyEmailPage.jsx (237 lines)

```

1 import { useContext, useEffect, useState } from "react";
2 import { useNavigate, useSearchParams, Link } from "react-router-dom";
3 import { CheckCircle, XCircle } from "lucide-react";
4 import { verifyEmail, verifyEmailByToken, resendVerificationOtp } from "../services/authService";
5 import { AuthContext } from "../context/AuthContext";
6 import { notify } from "../utils/inPageFeedback";
7 import "../components/Landing.css";
8 import "../components/FloatingLabel.css";
9
10 export default function VerifyEmailPage() {
11   const [params] = useSearchParams();
12   const navigate = useNavigate();
13   const { login } = useContext(AuthContext);
14   const token = params.get("token");
15   const emailParam = params.get("email") || "";
16   const [otp, setOtp] = useState("");
17   const [loading, setLoading] = useState(false);
18   const [resending, setResending] = useState(false);
19   const [status, setStatus] = useState(token ? "loading" : "form"); // "loading" | "form" | "success" | "error"
20   const [resendSeconds, setResendSeconds] = useState(token ? 0 : 60);
21
22   useEffect(() => {
23     if (resendSeconds <= 0) return undefined;
24     const timer = window.setInterval(() => {
25       setResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
26     }, 1000);
27     return () => window.clearInterval(timer);
28   }, [resendSeconds]);
29
30   useEffect(() => {
31     const previousBodyOverflow = document.body.style.overflow;
32     const previousHtmlOverflow = document.documentElement.style.overflow;
33
34     document.body.style.overflow = "hidden";
35     document.documentElement.style.overflow = "hidden";
36
37     return () => {
38       document.body.style.overflow = previousBodyOverflow;
39       document.documentElement.style.overflow = previousHtmlOverflow;
40     };
41   }, []);
42
43   const finalizeLogin = (data) => {
44     if (!data?.user) return;
45     login(data.user);
46     navigate(data.user.role === "student" ? "/student" : "/admin", { replace: true });
47   };
48
49   // Legacy token-based auto-verification
50   useEffect(() => {
51     if (!token) return;
52     verifyEmailByToken(token)
53       .then((data) => {
54         if (data?.token && data?.user) {
55           finalizeLogin(data);
56           return;
57         }
58         setStatus("success");
59         setMessage(data.message || "Email verified successfully!");
60       })
61       .catch((err) => {
62         setStatus("error");
63         setMessage(err.response?.data?.message || "Verification failed.");
64       });
65   // eslint-disable-next-line react-hooks/exhaustive-deps
66   }, [token]);
67
68   const handleVerify = async (e) => {
69     e.preventDefault();
70     if (!otp.trim()) return;
71     setLoading(true);
72     try {
73       const data = await verifyEmail(emailParam, otp.trim());

```

```
74     if (data?.token && data?.user) {
75         finalizeLogin(data);
76         return;
77     }
78     setStatus("success");
79 } catch (err) {
80     notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
81 } finally {
82     setLoading(false);
83 }
84 };
85
86 const handleResend = async () => {
87     if (resendSeconds > 0) return;
88     setResending(true);
89     try {
90         const data = await resendVerificationOtp(emailParam);
91         notify(data.message || "A new OTP has been sent to your email.", { type: "success", title: "Sent" });
92         setResendSeconds(60);
93     } catch (err) {
94         notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
95     } finally {
96         setResending(false);
97     }
98 };
99
100 if (status === "loading") {
101     return (
102         <div className="landing is-mounted">
103             <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
104             <div className="landing-overlay" />
105             <div className="landing-wrapper">
106                 <div className="landing-brand">
107                     
109                         <div className="landing-brand-title">Data Protection Office</div>
110                         <div className="landing-brand-subtitle">Rizal Technological University</div>
111                     </div>
112                 </div>
113                 <div className="landing-panel is-mounted">
114                     <div className="landing-card">
115                         <h2 className="landing-title">Verifying Your Email</h2>
116                         <p>Please wait while we verify your account.</p>
117                     </div>
118                 </div>
119             </div>
120         </div>
121     );
122 }
123
124 if (status === "success") {
125     return (
126         <div className="landing is-mounted">
127             <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
128             <div className="landing-overlay" />
129             <div className="landing-wrapper">
130                 <div className="landing-brand">
131                     
133                         <div className="landing-brand-title">Data Protection Office</div>
134                         <div className="landing-brand-subtitle">Rizal Technological University</div>
135                     </div>
136                 </div>
137                 <div className="landing-panel is-mounted">
138                     <div className="landing-card">
139                         <CheckCircle size={42} color="#16a34a" />
140                         <h2 className="landing-title">Email Verified</h2>
141                         <p>{message}</p>
142                         <Link to="/" className="landing-submit" style={{ textDecoration: "none", display: "inline-flex", alignItems: "c
143                             Go to Login
144                         </Link>
145                     </div>
146                 </div>
147             </div>
148         </div>
149     );
150 }
```

```

150 }
151
152 if (status === "error") {
153     return (
154         <div className="landing is-mounted">
155             <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')"}} />
156             <div className="landing-overlay" />
157             <div className="landing-wrapper">
158                 <div className="landing-brand">
159                     
160                     <div className="landing-brand-text">
161                         <div className="landing-brand-title">Data Protection Office</div>
162                         <div className="landing-brand-subtitle">Rizal Technological University</div>
163                     </div>
164                 </div>
165                 <div className="landing-panel is-mounted">
166                     <div className="landing-card">
167                         <XCircle size={42} color="#dc2626" />
168                         <h2 className="landing-title">Verification Failed</h2>
169                         <p>{message}</p>
170                         <Link to="/" className="landing-submit" style={{ textDecoration: "none", display: "inline-flex", alignItems: "center" }}>
171                             Back to Login
172                         </Link>
173                     </div>
174                 </div>
175             </div>
176         </div>
177     );
178 }
179
180 // OTP form
181 return (
182     <div className="landing is-mounted">
183         <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')"}} />
184         <div className="landing-overlay" />
185         <div className="landing-wrapper">
186             <div className="landing-brand">
187                 
188                 <div className="landing-brand-text">
189                     <div className="landing-brand-title">Data Protection Office</div>
190                     <div className="landing-brand-subtitle">Rizal Technological University</div>
191                 </div>
192             </div>
193
194             <div className="landing-panel is-mounted">
195                 <form onSubmit={handleVerify} className="landing-card">
196                     <h2 className="landing-title">Verify Your Email</h2>
197                     <p>
198                         A 6-digit OTP was sent to <strong>{emailParam}</strong>. Enter it below to activate your account.
199                     </p>
200
201                     <div className="landing-field-group fl-wrap">
202                         <input
203                             type="text"
204                             className="fl-input landing-field"
205                             value={otp}
206                             onChange={(e) => setOtp(e.target.value)}
207                             placeholder=" "
208                             required
209                             maxLength={6}
210                             autoFocus
211                             inputMode="numeric"
212                         />
213                         <label className="fl-label fl-label--glass">OTP Code</label>
214                     </div>
215
216                     <button type="submit" className="landing-submit" disabled={loading}>
217                         {loading ? "Verifying..." : "Verify Email"}
218                     </button>
219
220                     <button
221                         type="button"
222                         className="landing-link-btn"
223                         onClick={handleResend}
224                         disabled={resending || resendSeconds > 0}
225                     >

```

```
226         {resending ? "Resending..." : resendSeconds > 0 ? `Resend OTP in ${resendSeconds}s` : "Resend OTP"}
227     </button>
228
229     <Link to="/" className="landing-link-btn" style={{ textDecoration: "none" }}>
230         Back to Login
231     </Link>
232 </form>
233 </div>
234 </div>
235 </div>
236 );
237 }
```

47. client/src/pages/VerifyDocument.jsx (60 lines)

```

1 import { useEffect, useState } from "react";
2 import { useParams } from "react-router-dom";
3 import { CheckCircle, XCircle } from "lucide-react";
4 import { verifyRequestCode } from "../services/requestService";
5
6 export default function VerifyDocument() {
7   const { code } = useParams();
8   const [data, setData] = useState(null);
9   const [error, setError] = useState("");
10
11   useEffect(() => {
12     const load = async () => {
13       try {
14         const res = await verifyRequestCode(code);
15         setData(res);
16       } catch (err) {
17         setError(err.response?.data?.message || "Verification failed");
18       }
19     };
20     load();
21   }, [code]);
22
23   if (error) {
24     return (
25       <div className="verify-outer">
26         <div className="verify-card">
27           <span className="verify-result-icon"><XCircle size={48} color="#dc2626" /></span>
28           <h2 className="verify-result-error">Verification Failed</h2>
29           <p style={{ color: "var(--text-muted)", fontSize: "14px", margin: 0 }}>{error}</p>
30         </div>
31       </div>
32     );
33   }
34
35   if (!data) return null;
36
37   return (
38     <div className="verify-outer">
39       <div className="verify-card">
40         <span className="verify-result-icon">{data.valid ? <CheckCircle size={48} color="#16a34a" /> : <XCircle size={48} color="#dc2626" />}</span>
41         {data.valid ? (
42           <h2 className="verify-result-title">Document Verified</h2>
43         ) : (
44           <h2 className="verify-result-error">Invalid Document</h2>
45         )}
46       </div>
47       <div className="verify-meta-grid">
48         <span className="verify-meta-label">Type</span>
49         <span className="verify-meta-value">{data.type || "N/A"}</span>
50
51         <span className="verify-meta-label">Status</span>
52         <span className="verify-meta-value">{data.status || "N/A"}</span>
53
54         <span className="verify-meta-label">Issued</span>
55         <span className="verify-meta-value">{data.issuedAt || "N/A"}</span>
56       </div>
57     </div>
58   );
59 }
60 }

```

48. client/src/pages/public/RepSigningPage.jsx (370 lines)

```

1 import { useEffect, useRef, useState } from "react";
2 import { useParams } from "react-router-dom";
3 import { AnimatePresence, motion } from "framer-motion";
4 import { getBySigningToken, repSubmit, repReject } from "../../services/requestService";
5 import { uploadSignatureImage, uploadRepGovId, deleteStorageFile } from "../../services/firebaseStorageService";
6 import SignaturePad from "../../components/SignaturePad";
7 import { notify } from "../../utils/inPageFeedback";
8 import "../../components/RepSigningPage.css";
9
10 export default function RepSigningPage() {
11   const { token } = useParams();
12
13   const [reqData, setReqData] = useState(null);
14   const [loading, setLoading] = useState(true);
15   const [error, setError] = useState("");
16   const [done, setDone] = useState(""); // success message
17
18   const [repFirstName, setRepFirstName] = useState("");
19   const [repMiddleName, setRepMiddleName] = useState("");
20   const [repLastName, setRepLastName] = useState("");
21   const [govIdFile, setGovIdFile] = useState(null);
22   const [accepted, setAccepted] = useState(false);
23   const [submitting, setSubmitting] = useState(false);
24   const [showConfirm, setShowConfirm] = useState(null); // "submit" | "decline" | null
25
26   const sigPadRef = useRef(null);
27   const fileInputRef = useRef(null);
28
29   useEffect(() => {
30     (async () => {
31       try {
32         const r = await getBySigningToken(token);
33         setReqData(r);
34       } catch (err) {
35         setError(err.response?.data?.message || "Invalid or expired signing link.");
36       } finally {
37         setLoading(false);
38       }
39     })();
40   }, [token]);
41
42   const sanitizeNamePart = (value) => String(value || "").replace(/^[^A-Za-z\s'\.-]/g, "");
43
44   // Keep this hook before any conditional return to preserve hook order.
45   useEffect(() => {
46     if (!reqData) return;
47     const fd = reqData.formData || {};
48     const first = fd.repFirstName || "";
49     const middle = fd.repMiddleInitial || "";
50     const last = fd.repLastName || "";
51
52     if (first || middle || last) {
53       setRepFirstName(sanitizeNamePart(first));
54       setRepMiddleName(sanitizeNamePart(middle));
55       setRepLastName(sanitizeNamePart(last));
56     }
57   }, [reqData]);
58
59   if (loading) return (
60     <div className="rep-signing-outer">
61       <div className="rep-signing-state"><p>Loading...</p></div>
62     </div>
63   );
64
65   if (error) return (
66     <div className="rep-signing-outer">
67       <div className="rep-signing-state">
68         <h2>Error</h2>
69         <p>{error}</p>
70       </div>
71     </div>
72   );
73

```

```

74   if (done) return (
75     <div className="rep-signing-outer">
76       <div className="rep-signing-state">
77         <h2>Thank you</h2>
78         <p>{done}</p>
79       </div>
80     </div>
81   );
82
83   // Guard: token already used
84   if (reqData.signingTokenUsed) {
85     return (
86       <div className="rep-signing-outer">
87         <div className="rep-signing-state">
88           <h2>Link Already Used</h2>
89           <p>This signing link has already been used. Please contact the requestor if you need a new one.</p>
90         </div>
91       </div>
92     );
93   }
94
95   // Guard: wrong status
96   const signingStatuses = [
97     "agr_awaiting_rep_signature",
98     "agr_rep_revision_requested",
99   ];
100  if (!signingStatuses.includes(reqData.status)) {
101    return (
102      <div className="rep-signing-outer">
103        <div className="rep-signing-state">
104          <h2>Link Not Active</h2>
105          <p>This signing link is no longer active (status: {reqData.status}).</p>
106        </div>
107      </div>
108    );
109  }
110
111  const isRevision = reqData.status === "agr_rep_revision_requested";
112  const student = reqData.userId || {};
113  const fd = reqData.formData || {};
114  const storedMiddle = sanitizeNamePart(fd.repMiddleInitial || "").trim();
115  const storedMiddleInitial = storedMiddle ? `${storedMiddle[0].toUpperCase()}.` : "";
116  const requestedRepName = [fd.repFirstName, storedMiddleInitial, fd.repLastName]
117    .filter(Boolean)
118    .join(" ")
119    .replace(/\s+/g, " ")
120    .trim() || fd.repName || "-";
121  const normalizedMiddleName = sanitizeNamePart(repMiddleName).replace(/\./g, "").trim();
122  const representativeFullName = [
123    sanitizeNamePart(repFirstName).trim(),
124    normalizedMiddleName ? `${normalizedMiddleName[0].toUpperCase()}.` : "",
125    sanitizeNamePart(repLastName).trim(),
126  ]
127    .filter(Boolean)
128    .join(" ")
129    .replace(/\s+/g, " ")
130    .trim();
131
132  const handleReject = () => {
133    setShowConfirm("decline");
134  };
135
136  const handleConfirmReject = async () => {
137    setShowConfirm(null);
138    setSubmitting(true);
139    try {
140      await repReject(token);
141      setDone("You have declined the signing request. The requestor has been notified.");
142    } catch (err) {
143      notify(err.response?.data?.message || "Failed to reject. Please try again.", { type: "error" });
144    } finally {
145      setSubmitting(false);
146    }
147  };
148
149  const handleSubmit = (e) => {

```

```

150     e.preventDefault();
151
152     if (!sanitizeNamePart(repFirstName).trim()) { notify("Please enter your first name.", { type: "warning" }); return; }
153     if (!sanitizeNamePart(repLastName).trim()) { notify("Please enter your last name.", { type: "warning" }); return; }
154     if (!govIdFile) { notify("Please upload your government-issued ID.", { type: "warning" }); return; }
155     if (!sigPadRef.current || sigPadRef.current.isEmpty()) { notify("Please draw your signature.", { type: "warning" }); return; }
156     if (!accepted) { notify("You must check 'I accept' to proceed.", { type: "warning" }); return; }
157
158     setShowConfirm("submit");
159   };
160
161   const handleConfirmSubmit = async () => {
162     setShowConfirm(null);
163     setSubmitting(true);
164     try {
165       const authSigPath = reqData.authorizerSigPath || "";
166       const pathParts = authSigPath.split ? authSigPath.split("/") : [];
167       const requestFolder = pathParts[3] || `rep_${Date.now()}`;
168       const studentName = student.name || "unknown";
169
170       if (isRevision && reqData.repInfo?.govIdDoc?.path) {
171         await deleteStorageFile(reqData.repInfo.govIdDoc.path);
172       }
173
174       const govIdDoc = await uploadRepGovId(govIdFile, "agreement", studentName, requestFolder);
175
176       const sigDataUrl = sigPadRef.current.getDataUrl();
177       const { url: repSigUrl, path: repSigPath } = await uploadSignatureImage(
178         sigDataUrl,
179         "agreement",
180         studentName,
181         requestFolder,
182         "rep_sig.png"
183       );
184
185       await repSubmit(token, {
186         repName: representativeFullName,
187         repGovIdDoc: govIdDoc,
188         repSigUrl,
189         repSigPath,
190       });
191
192       setDone("Your submission has been received. Thank you for signing the agreement.");
193     } catch (err) {
194       notify(err.response?.data?.message || "Submission failed. Please try again.", { type: "error" });
195     } finally {
196       setSubmitting(false);
197     }
198   };
199
200   return (
201     <div className="rep-signing-outer">
202       <div className="rep-signing-card">
203         <h2 className="rep-signing-title">Agreement Signing Request</h2>
204
205         {isRevision && (
206           <div className="rep-signing-revision-banner">
207             <strong>Revision Requested</strong>
208             <p>{reqData.remarks || "Please review and resubmit your information."}</p>
209           </div>
210         )}
211
212         <p className="rep-signing-intro">
213           You have been invited to sign an authorization agreement as a representative.
214           Please review the details below and complete the form to sign.
215         </p>
216
217         {/* Letter-format summary */}
218         <div className="rep-signing-summary" style={{ padding: "20px 24px", gap: 0 }}>
219           {/* Date – left */}
220           <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
221             {reqData.createdAt ? new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "long", da
222           </div>
223
224           {/* Authorization paragraph – centered */}
225           <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0 24px"

```



```
226     I, <strong>{student.name || "-"}</strong>, hereby authorize{" "}
227     <strong>{requestedRepName}</strong> to act as my representative and sign on my behalf
228     in connection with this agreement, as governed by the Data Protection Office.
229 </p>
230
231 /* Signature block – centered */
232 <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
233   {reqData.authorizerSigUrl && (
234     <img
235       src={reqData.authorizerSigUrl}
236       alt="Authorizer signature"
237       style={{ maxHeight: 72, display: "block" }}
238       loading="lazy"
239       decoding="async"
240     />
241   )}
242   <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", paddingTop: 6, fontSi
243     {student.name || "-"}
244   </div>
245 </div>
246 </div>
247
248 <hr className="rep-signing-divider" />
249
250 /* Representative's form */
251 <form onSubmit={handleSubmit}>
252   <h3 className="rep-signing-section-title">Your Information</h3>
253
254   <div className="rep-signing-field">
255     <p className="rep-signing-name-note">Please ensure the name entered matches the name shown on your government-iss
256     <label className="rep-signing-label">Full Name *</label>
257     <div className="rep-signing-name-grid">
258       <div className="rep-signing-name-col">
259         <label className="rep-signing-name-sublabel">First Name</label>
260         <input
261           className="rep-signing-input"
262           value={repFirstName}
263           onChange={(e) => setRepFirstName(sanitizeNamePart(e.target.value))}
264           required
265         />
266       </div>
267       <div className="rep-signing-name-col">
268         <label className="rep-signing-name-sublabel">Middle Name</label>
269         <input
270           className="rep-signing-input"
271           value={repMiddleName}
272           onChange={(e) => setRepMiddleName(sanitizeNamePart(e.target.value))}
273         />
274       </div>
275       <div className="rep-signing-name-col">
276         <label className="rep-signing-name-sublabel">Last Name</label>
277         <input
278           className="rep-signing-input"
279           value={repLastName}
280           onChange={(e) => setRepLastName(sanitizeNamePart(e.target.value))}
281           required
282         />
283       </div>
284     </div>
285   </div>
286
287   <div className="rep-signing-field">
288     <label className="rep-signing-label">Government-Issued Valid ID *</label>
289     <div className="rep-signing-file-row">
290       <div className="rep-signing-file-info">
291         <span className="rep-signing-file-title">
292           {govIdFile ? govIdFile.name : "No file chosen"}
293         </span>
294         <span className="rep-signing-file-subtitle">PDF or image accepted</span>
295       </div>
296       <label className="rep-signing-file-action">
297         {govIdFile ? "Change File" : "Upload File"}
298       <input
299         type="file"
300         accept="application/pdf,image/*"
301         onChange={(e) => setGovIdFile(e.target.files?.[0] || null)}

```

```
302         ref={fileInputRef}
303     />
304     </label>
305 </div>
306 </div>
307
308 <div className="rep-signing-field">
309     <label className="rep-signing-label">Your E-Signature *</label>
310     <p className="rep-signing-sig-hint">Draw your signature in the box below.</p>
311     <SignaturePad ref={sigPadRef} height={160} />
312 </div>
313
314 <label className="rep-signing-accept">
315     <input
316         type="checkbox"
317         checked={accepted}
318         onChange={(e) => setAccepted(e.target.checked)}
319     />
320     <span>
321         I accept and acknowledge that by signing this form I am authorizing the above-named requestor
322         and agree to the terms of this agreement as issued by the Data Protection Office.
323     </span>
324 </label>
325
326 <div className="rep-signing-actions">
327     <button type="submit" className="rep-signing-btn-submit" disabled={submitting}>
328         {submitting ? "Submitting..." : "Submit & Sign"}
329     </button>
330     <button
331         type="button"
332         className="rep-signing-btn-decline"
333         onClick={handleReject}
334         disabled={submitting}
335     >
336         Decline
337     </button>
338 </div>
339 </form>
340 </div>
341
342 <AnimatePresence>
343     {showConfirm && (
344         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
345             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
346                 <h3 className="modal-title">
347                     {showConfirm === "submit" ? "Submit & Sign Agreement" : "Decline Signing Request"}
348                 </h3>
349                 <p className="modal-text">
350                     {showConfirm === "submit"
351                         ? "Please confirm that all your information and signature are accurate before submitting."
352                         : "Are you sure you want to decline? This action is final and the requestor will be notified."}
353                 </p>
354                 <div className="modal-actions">
355                     <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(null)} disabled={submitting}>Cancel
356                     <button
357                         className="ui-btn ui-btn--primary"
358                         onClick={showConfirm === "submit" ? handleConfirmSubmit : handleConfirmReject}
359                         disabled={submitting}
360                     >
361                         {showConfirm === "submit" ? "Submit" : "Decline"}
362                     </button>
363                 </div>
364             </motion.div>
365         </motion.div>
366     )}
367 </AnimatePresence>
368 </div>
369 );
370 }
```

49. client/src/pages/student/StudentDashboard.jsx (377 lines)

```

1 import { useEffect, useMemo, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { RefreshCw, Search, Filter, FilePlus } from "lucide-react";
4 import { getMyRequests } from "../../../services/requestService";
5 import FilterSelect from "../../../components/FilterSelect";
6 import { notify } from "../../../utils/inPageFeedback";
7
8 const STATUS_LABEL = {
9   nda_pending: "Reviewal",
10  nda_approved: "Approved",
11  stud_revision_requested: "Student Revisions",
12  agr_pending_1: "Initial Reviewal",
13  agr_awaiting_rep_signature: "Recipient Reviewal",
14  agr_pending_2: "Final Reviewal",
15  agr_approved: "Approved",
16  agr_declined: "Recipient Declined",
17  agr_rep_revision_requested: "Recipient Revisions",
18 };
19
20 const STATUS_FILTER_OPTIONS = [
21   { value: "all", label: "All Statuses" },
22   { value: "reviewal", label: "Reviewal" },
23   { value: "approved", label: "Approved" },
24   { value: "stud_revision", label: "Student Revisions" },
25   { value: "rep_revision", label: "Recipient Revisions" },
26   { value: "rep_declined", label: "Recipient Declined" },
27   { value: "awaiting_rep", label: "Recipient Reviewal" },
28 ];
29
30 const prettyStatus = (s) =>
31   STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
32
33 const statusClass = (s) => {
34   if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
35   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
36   if (s === "stud_revision_requested") return "status-pill status-pill--orange";
37   if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
38   if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
39   if (s === "agr_declined") return "status-pill status-pill--red";
40   return "status-pill";
41 };
42
43 const normalizeStatusForStudentFilter = (status) => {
44   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(status)) return "reviewal";
45   if (["nda_approved", "agr_approved"].includes(status)) return "approved";
46   if (status === "stud_revision_requested") return "stud_revision";
47   if (status === "agr_rep_revision_requested") return "rep_revision";
48   if (status === "agr_declined") return "rep_declined";
49   if (status === "agr_awaiting_rep_signature") return "awaiting_rep";
50   return "reviewal";
51 };
52
53 const StudentDashboard = () => {
54   const navigate = useNavigate();
55   const [requests, setRequests] = useState([]);
56   const [loading, setLoading] = useState(true);
57   const [refreshing, setRefreshing] = useState(false);
58
59   // Search state
60   const [searchTerm, setSearchTerm] = useState("");
61
62   // Filter state
63   const [typeFilter, setTypeFilter] = useState("all");
64   const [activeFilter, setActiveFilter] = useState("all");
65   const [dateMode, setDateMode] = useState("preset");
66   const [preset, setPreset] = useState("all");
67   const [singleDate, setSingleDate] = useState("");
68   const [startDate, setStartDate] = useState("");
69   const [endDate, setEndDate] = useState("");
70
71   const [isFilterOpen, setIsFilterOpen] = useState(() =>
72     typeof window === "undefined" ? true : window.innerWidth >= 768
73   );

```

```
74
75 const loadRequests = async ({ withToast = false } = {}) => {
76   if (withToast) setRefreshing(true);
77   try {
78     const data = await getMyRequests();
79     setRequests(data);
80   } catch (err) {
81     notify(err.response?.data?.message || "Failed to load requests", { type: "error" });
82   } finally {
83     setLoading(false);
84     setRefreshing(false);
85   }
86 };
87
88 useEffect(() => {
89   loadRequests();
90 }, []);
91
92 useEffect(() => {
93   const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
94   window.addEventListener("resize", onResize);
95   return () => window.removeEventListener("resize", onResize);
96 }, []);
97
98 const clearSearch = () => setSearchTerm("");
99
100 const resetAndRefresh = () => {
101   setSearchTerm("");
102   setTypeFilter("all");
103   setActiveFilter("all");
104   setDateMode("preset");
105   setPreset("all");
106   setSingleDate("");
107   setStartDate("");
108   setEndDate("");
109   loadRequests({ withToast: true });
110 };
111
112 const filtered = useMemo(() => {
113   let result = requests;
114
115   // Search filter
116   if (searchTerm) {
117     const term = searchTerm.toLowerCase();
118     result = result.filter((r) => {
119       const id = (r._id || "").slice(-7).toLowerCase();
120       const type = r.type === "nda"
121         ? `nda ${r.formData?.ndaTypeLabel || ""}`.toLowerCase()
122         : "agreement";
123       return id.includes(term) || type.includes(term);
124     });
125   }
126
127   // Type filter
128   if (typeFilter !== "all") {
129     if (typeFilter === "agreement") {
130       result = result.filter((r) => r.type === "agreement");
131     } else if (typeFilter === "nda_orgactivities") {
132       result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
133     } else if (typeFilter === "nda_research") {
134       result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
135     }
136   }
137
138   // Status filter
139   if (activeFilter !== "all") {
140     result = result.filter((r) => normalizeStatusForStudentFilter(r.status) === activeFilter);
141   }
142
143   // Date filter
144   if (dateMode === "preset") {
145     const now = new Date();
146     if (preset === "today") {
147       const dayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
148       const dayEnd = new Date(dayStart.getTime() + 86400000 - 1);
149       result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
150     }
151   }
152 }
```

```

150     } else if (preset === "thisWeek") {
151         const day = now.getDay();
152         const monday = new Date(now);
153         monday.setDate(now.getDate() - ((day + 6) % 7));
154         monday.setHours(0, 0, 0, 0);
155         result = result.filter((r) => new Date(r.createdAt) >= monday);
156     } else if (preset === "thisMonth") {
157         const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
158         result = result.filter((r) => new Date(r.createdAt) >= monthStart);
159     } else if (preset === "thisYear") {
160         const yearStart = new Date(now.getFullYear(), 0, 1);
161         result = result.filter((r) => new Date(r.createdAt) >= yearStart);
162     }
163 } else if (dateMode === "single" && singleDate) {
164     const dayStart = new Date(singleDate);
165     const dayEnd = new Date(singleDate + "T23:59:59");
166     result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
167 } else if (dateMode === "range") {
168     if (startDate) result = result.filter((r) => new Date(r.createdAt) >= new Date(startDate));
169     if (endDate) result = result.filter((r) => new Date(r.createdAt) <= new Date(endDate + "T23:59:59"));
170 }
171
172 return result;
173 }, [requests, searchTerm, typeFilter, activeFilter, dateMode, preset, singleDate, startDate, endDate]);
174
175 return (
176     <div className="dashboard-page">
177         <div className="page-shell__header">
178             <button
179                 type="button"
180                 className="req-toolbar__action-btn"
181                 onClick={() => navigate("/student/new-request")}
182             >
183                 <FilePlus size={14} />
184                 Create Request
185             </button>
186         </div>
187
188         {/ * Search + Filter toolbar * /}
189         <div className="req-toolbar">
190             <div className="req-toolbar__search-row">
191                 <div className="req-toolbar__search-box">
192                     <input
193                         type="text"
194                         className="req-toolbar__search-input"
195                         placeholder="Search by request ID or type..."
196                         value={searchTerm}
197                         onChange={(e) => setSearchTerm(e.target.value)}
198                     />
199                     {searchTerm ? (
200                         <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
201                             x
202                         </button>
203                     ) : null}
204                     <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
205                         <Search size={14} />
206                     </span>
207                 </div>
208                 <button
209                     type="button"
210                     className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
211                     onClick={() => setIsFilterOpen((prev) => !prev)}
212                     aria-expanded={isFilterOpen}
213                 >
214                     <Filter size={13} />
215                     Filters
216                 </button>
217             </div>
218
219             <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
220                 <div className="req-toolbar__filter-group">
221                     <label className="req-toolbar__filter-label">Type</label>
222                     <FilterSelect
223                         value={typeFilter}
224                         onChange={setTypeFilter}
225                         className="filter-select--type"

```

```

226         options=[
227             { value: "all", label: "All Types" },
228             { value: "nda_orgactivities", label: "NDA – Student Organization Activities" },
229             { value: "nda_research", label: "NDA – Conduct of Research" },
230             { value: "agreement", label: "Agreement" },
231         ]
232     />
233 </div>
234
235 <div className="req-toolbar__filter-group">
236     <label className="req-toolbar__filter-label">Status</label>
237     <FilterSelect
238         value={activeFilter}
239         onChange={setActiveFilter}
240         className="filter-select--status"
241         options={STATUS_FILTER_OPTIONS}
242     />
243 </div>
244
245 <div className="req-toolbar__filter-group">
246     <label className="req-toolbar__filter-label">Date</label>
247     <FilterSelect
248         value={dateMode}
249         onChange={setDateMode}
250         options=[
251             { value: "preset", label: "Preset" },
252             { value: "single", label: "Specific Date" },
253             { value: "range", label: "Date Range" },
254         ]
255         defaultValue="preset"
256     />
257 </div>
258
259 {dateMode === "preset" ? (
260     <div className="req-toolbar__filter-group">
261         <label className="req-toolbar__filter-label">Period</label>
262         <FilterSelect
263             value={preset}
264             onChange={setPreset}
265             options=[
266                 { value: "all", label: "All Time" },
267                 { value: "today", label: "Today" },
268                 { value: "thisWeek", label: "This Week" },
269                 { value: "thisMonth", label: "This Month" },
270                 { value: "thisYear", label: "This Year" },
271             ]
272         />
273     </div>
274 ) : null}
275
276 {dateMode === "single" ? (
277     <div className="req-toolbar__filter-group">
278         <label className="req-toolbar__filter-label">Date</label>
279         <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`
280             />
281     </div>
282 ) : null}
283
284 {dateMode === "range" ? (
285     <div className="req-toolbar__filter-group">
286         <label className="req-toolbar__filter-label">From</label>
287         <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`
288             />
289     </div>
290     <div className="req-toolbar__filter-group">
291         <label className="req-toolbar__filter-label">To</label>
292         <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`
293             />
294     </div>
295 ) : null}
296
297 <button
298     type="button"
299     className="req-toolbar__reset-btn"
300     onClick={resetAndRefresh}
301     disabled={refreshing}
302     title="Reset filters and refresh"

```

```

302         >
303         <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
304     </button>
305 </div>
306 </div>
307
308 <div className="dashboard-card">
309     <div className="table-scroll">
310         <table className="dashboard-table">
311             <thead>
312                 <tr className="dashboard-table-title-row">
313                     <th colspan={5}>
314                         <div className="dashboard-table-title-wrap">
315                             <span className="dashboard-table-title">My Requests</span>
316                         </div>
317                     </th>
318                 </tr>
319                 <tr>
320                     <th>Request Date</th>
321                     <th>Request ID</th>
322                     <th>Type</th>
323                     <th>Status</th>
324                     <th />
325                 </tr>
326             </thead>
327
328             <tbody>
329                 {loading ? (
330                     [...Array(4)].map((_, i) => (
331                         <tr key={`sk-${i}`}>
332                             <td><span className="skeleton-block skeleton-text" /></td>
333                             <td><span className="skeleton-block skeleton-text" /></td>
334                             <td><span className="skeleton-block skeleton-text" /></td>
335                             <td><span className="skeleton-block skeleton-pill" /></td>
336                             <td />
337                         </tr>
338                     ))
339                 ) : filtered.length === 0 ? (
340                     <tr>
341                         <td colspan={5}>
342                             <div className="dashboard-empty">
343                                 <p className="dashboard-empty-title">No requests found</p>
344                                 <p className="dashboard-empty-text">No requests match the current filters.</p>
345                             </div>
346                         </td>
347                     </tr>
348                 ) : (
349                     filtered.map((r) => (
350                         <tr key={r._id} onClick={() => navigate(`/student/requests/${r._id}`)} style={{ cursor: "pointer" }}>
351                             <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}><new Date(r.createdAt).toLocaleDateString()>
352                             <td><span style={{ color: "var(--text-secondary)" }}><{r._id?.slice(-7).toUpperCase()}</span></td>
353                             <td>
354                                 {r.type === "nda"
355                                 ? `NDA${r.formData?.ndaTypeLabel ? ` - ${r.formData.ndaTypeLabel}` : ""}`
356                                 : "Agreement"}
357                             </td>
358
359                             <td>
360                                 <span className={statusClass(r.status)}>
361                                     {prettyStatus(r.status)}
362                                 </span>
363                             </td>
364
365                             <td className="row-caret">></td>
366                         </tr>
367                     ))
368                 )}
369             </tbody>
370         </table>
371     </div>
372 </div>
373 </div>
374 );
375 };
376
377 export default StudentDashboard;

```


50. client/src/pages/student/StudentProfile.jsx (314 lines)

```

1 import { useContext, useRef, useState } from "react";
2 import { Eye, EyeOff } from "lucide-react";
3 import { motion, AnimatePresence } from "framer-motion";
4 import { AuthContext } from "../../../context/AuthContext";
5 import { updateProfile, requestPasswordChangeOtp } from "../../../services/authService";
6 import PasswordChecklist, { ErrorTooltip } from "../../../components/PasswordChecklist";
7 import { isStrongPassword } from "../../../utils/passwordPolicy";
8 import { notify } from "../../../utils/inPageFeedback";
9 import "../../../components/RequestForm.css";
10 import "../../../components/FloatingLabel.css";
11
12 const StudentProfile = () => {
13   const { user, updateUser } = useContext(AuthContext);
14   const [firstName, setFirstName] = useState(user?.firstName || "");
15   const [middleName, setMiddleName] = useState(user?.middleInitial || "");
16   const [lastName, setLastName] = useState(user?.lastName || "");
17   const [currentPassword, setCurrentPassword] = useState("");
18   const [newPassword, setNewPassword] = useState("");
19   const [confirmPassword, setConfirmPassword] = useState("");
20   const [loading, setLoading] = useState(false);
21   const [showCurrentPw, setShowCurrentPw] = useState(false);
22   const [showNewPw, setShowNewPw] = useState(false);
23   const [showConfirmPw, setShowConfirmPw] = useState(false);
24   const [confirmName, setConfirmName] = useState(false);
25   const [confirmPw, setConfirmPw] = useState(false);
26   const [otp, setOtp] = useState("");
27   const [otpCooldown, setOtpCooldown] = useState(0);
28
29   const [newPwFocused, setNewPwFocused] = useState(false);
30   const [confirmPwFocused, setConfirmPwFocused] = useState(false);
31
32   // Refs for tooltip anchoring
33   const newPwAnchorRef = useRef(null);
34   const confirmPwAnchorRef = useRef(null);
35   const cooldownRef = useRef(null);
36
37   const initials = user?.name
38     ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
39     : "?";
40
41   const startCooldown = (seconds) => {
42     setOtpCooldown(seconds);
43     clearInterval(cooldownRef.current);
44     cooldownRef.current = setInterval(() => {
45       setOtpCooldown((s) => {
46         if (s <= 1) { clearInterval(cooldownRef.current); return 0; }
47         return s - 1;
48       });
49     }, 1000);
50   };
51
52   const handleNameSubmit = (e) => {
53     e.preventDefault();
54     if (!firstName.trim() || !lastName.trim()) {
55       notify("Failed.", { type: "error", title: "Failed" });
56       return;
57     }
58     const hasChanges =
59       firstName.trim() !== (user?.firstName || "") ||
60       middleName.trim() !== (user?.middleInitial || "") ||
61       lastName.trim() !== (user?.lastName || "");
62     if (!hasChanges) {
63       notify("No changes to save.", { type: "warning", title: "Notice" });
64       return;
65     }
66     setConfirmName(true);
67   };
68
69   const handleConfirmNameUpdate = async () => {
70     setLoading(true);
71     try {
72       const data = await updateProfile({ firstName: firstName.trim(), middleInitial: middleName.trim(), lastName: lastName.trim() });
73       updateUser(data.user);

```

```
74     notify("Name updated successfully.", { type: "success", title: "Success" });
75     setConfirmName(false);
76   } catch {
77     notify("Failed.", { type: "error", title: "Failed" });
78     setConfirmName(false);
79   } finally {
80     setLoading(false);
81   }
82 };
83
84 const handlePasswordSubmit = async (e) => {
85   e.preventDefault();
86   if (!currentPassword || !isStrongPassword(newPassword) || newPassword !== confirmPassword) return;
87   setLoading(true);
88   try {
89     await requestPasswordChangeOtp();
90     setOtp("");
91     startCooldown(60);
92     setConfirmPw(true);
93   } catch (err) {
94     const data = err?.response?.data;
95     if (err?.response?.status === 429 && data?.retryAfterSeconds) {
96       startCooldown(data.retryAfterSeconds);
97       setConfirmPw(true);
98     } else {
99       notify(data?.message || "Failed to send OTP.", { type: "error", title: "Error" });
100     }
101   } finally {
102     setLoading(false);
103   }
104 };
105
106 const handleResendOtp = async () => {
107   if (otpCooldown > 0) return;
108   setLoading(true);
109   try {
110     await requestPasswordChangeOtp();
111     setOtp("");
112     startCooldown(60);
113     notify("A new OTP has been sent to your email.", { type: "info", title: "OTP Sent" });
114   } catch (err) {
115     const data = err?.response?.data;
116     if (err?.response?.status === 429 && data?.retryAfterSeconds) {
117       startCooldown(data.retryAfterSeconds);
118     }
119     notify(data?.message || "Failed to resend OTP.", { type: "error", title: "Error" });
120   } finally {
121     setLoading(false);
122   }
123 };
124
125 const handleConfirmPasswordUpdate = async () => {
126   if (!otp.trim()) { notify("Please enter the OTP sent to your email.", { type: "warning" }); return; }
127   setLoading(true);
128   try {
129     await updateProfile({ currentPassword, newPassword, otp: otp.trim() });
130     setCurrentPassword("");
131     setNewPassword("");
132     setConfirmPassword("");
133     setOtp("");
134     setConfirmPwFocused(false);
135     setConfirmPw(false);
136     notify("Password changed successfully.", { type: "success", title: "Success" });
137   } catch (err) {
138     notify(err?.response?.data?.message || "Failed to change password.", { type: "error", title: "Failed" });
139   } finally {
140     setLoading(false);
141   }
142 };
143
144 const pwToggleStyle = {
145   position: "absolute", right: 6, top: "50%", transform: "translateY(-50%)",
146   background: "none", border: "none", padding: 4, cursor: "pointer",
147   color: "var(--text-muted)", display: "flex", alignItems: "center",
148 };
149
```

```
150 return (
151   <div style={{ display: "flex", justifyContent: "center" }}>
152     <div style={{ maxWidth: 520, width: "100%" }}>
153       /* Profile Header */
154       <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
155         <div style={{ width: 72, height: 72, borderRadius: "50%", background: "var(--primary)", display: "flex", alignItems: "center", justify-content: "center" }}>
156           <p style={{ margin: "0 0 4px", fontSize: 18, fontWeight: 700, color: "var(--text-primary)" }}>{user?.name}</p>
157           <p style={{ margin: "0 0 8px", fontSize: 13, color: "var(--text-muted)" }}>{user?.email}</p>
158           <span style={{ padding: "3px 12px", borderRadius: 99, fontSize: 11, fontWeight: 700, background: "#f1f5f9", color: "var(--text-primary)" }}>{user?.role}</span>
159         </div>
160       </div>
161       /* Change Name */
162       <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
163         <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Name</h3>
164         <form onSubmit={handleNameSubmit} style={{ display: "flex", flexDirection: "column", gap: 10 }}>
165           <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
166             <div className="fl-wrap">
167               <input placeholder=" " value={firstName} onChange={(e) => setFirstName(e.target.value)} className="fl-input request-input" />
168               <label className="fl-label">First Name</label>
169             </div>
170             <div className="fl-wrap">
171               <input placeholder=" " value={middleName} onChange={(e) => setMiddleName(e.target.value)} className="fl-input request-input" />
172               <label className="fl-label">Middle Name</label>
173             </div>
174             <div className="fl-wrap">
175               <input placeholder=" " value={lastName} onChange={(e) => setLastName(e.target.value)} className="fl-input request-input" />
176               <label className="fl-label">Last Name</label>
177             </div>
178           </div>
179           <div style={{ display: "flex", justifyContent: "flex-end" }}>
180             <button type="submit" className="ui-btn ui-btn--primary" disabled={loading}>Save Changes</button>
181           </div>
182         </form>
183       </div>
184     </div>
185     /* Change Password */
186     <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
187       <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Password</h3>
188       <form onSubmit={handlePasswordSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
189         /* Current Password */
190         <div className="fl-wrap">
191           <input
192             type={showCurrentPw ? "text" : "password"}
193             placeholder=" "
194             value={currentPassword}
195             onChange={(e) => setCurrentPassword(e.target.value)}
196             className="fl-input request-input"
197             style={{ paddingRight: 42 }}
198           />
199           <label className="fl-label fl-label--pw">Current Password</label>
200           <button type="button" onClick={() => setShowCurrentPw((v) => !v)} tabIndex={-1} aria-label={showCurrentPw ? "Hide" : "Show"}>
201             {showCurrentPw ? <Eye size={18} /> : <EyeOff size={18} />}
202           </button>
203         </div>
204         /* New Password */
205         <div className="fl-wrap" ref={newPwAnchorRef}>
206           <input
207             type={showNewPw ? "text" : "password"}
208             placeholder=" "
209             value={newPassword}
210             onChange={(e) => setNewPassword(e.target.value)}
211             onFocus={() => setNewPwFocused(true)}
212             onBlur={() => setNewPwFocused(false)}
213             minLength={8}
214             className="fl-input request-input"
215             style={{ paddingRight: 42 }}
216           />
217           <label className="fl-label fl-label--pw">New Password</label>
218           <button type="button" onClick={() => setShowNewPw((v) => !v)} tabIndex={-1} aria-label={showNewPw ? "Hide" : "Show"}>
219             {showNewPw ? <Eye size={18} /> : <EyeOff size={18} />}
220           </button>
221           <PasswordChecklist focused={!isStrongPassword(newPassword) && (newPwFocused || newPassword.length > 0)} anchorRef={newPwAnchorRef}>
222             {showNewPw ? <Eye size={18} /> : <EyeOff size={18} />}
223           </PasswordChecklist>
224         </div>
225       </form>
226     </div>
227   </div>
228 )
```

```

226
227     /* Confirm Password */
228     <div className="fl-wrap" ref={confirmPwAnchorRef}>
229         <input
230             type={showConfirmPw ? "text" : "password"}
231             placeholder=" "
232             value={confirmPassword}
233             onChange={(e) => setConfirmPassword(e.target.value)}
234             onFocus={() => setConfirmPwFocused(true)}
235             onBlur={() => setConfirmPwFocused(false)}
236             className="fl-input request-input"
237             style={{ paddingRight: 42 }}
238         />
239         <label className="fl-label fl-label--pw">Confirm New Password</label>
240         <button type="button" onClick={() => setShowConfirmPw((v) => !v)} tabIndex={-1} aria-label={showConfirmPw ? "Hide" : "Show"}>
241             {showConfirmPw ? <Eye size={18} /> : <EyeOff size={18} />}
242         </button>
243         <ErrorTooltip
244             show={confirmPassword !== newPassword && (confirmPwFocused || confirmPassword.length > 0)}
245             message="Passwords do not match"
246             anchorRef={confirmPwAnchorRef}
247             side="right"
248             persistent
249         />
250     </div>
251
252     <button type="submit" className="ui-btn ui-btn--primary" disabled={loading} style={{ alignSelf: "flex-end" }}>
253         {loading ? "Sending OTP..." : "Save Changes"}
254     </button>
255 </form>
256 </div>
257 </div>
258
259 /* Confirm Name Change Modal */
260 <AnimatePresence>
261     {confirmName && (
262         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
263             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
264                 <h3 className="modal-title">Confirm Change Name</h3>
265                 <p className="modal-text">Are you sure you want to update your name?</p>
266                 <p className="modal-text"><strong>{firstName.trim()} {middleName.trim()} {lastName.trim()}</strong> ? middleName.trim() + " " : ""</p>
267                 <div className="modal-actions">
268                     <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmName(false)} disabled={loading}>Cancel</button>
269                     <button className="ui-btn ui-btn--primary" onClick={handleConfirmNameUpdate} disabled={loading}>{loading ? "Saving" : "Save"}</button>
270                 </div>
271             </motion.div>
272         </motion.div>
273     )}
274 </AnimatePresence>
275
276 /* Confirm Password Change Modal */
277 <AnimatePresence>
278     {confirmPw && (
279         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
280             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
281                 <h3 className="modal-title">Verify Your Identity</h3>
282                 <p className="modal-text">An OTP has been sent to <strong>{user?.email}</strong>. Enter it below to confirm your identity.</p>
283                 <div className="fl-wrap" style={{ marginBottom: 4 }}>
284                     <input
285                         placeholder=" "
286                         value={otp}
287                         onChange={(e) => setOtp(e.target.value)}
288                         className="fl-input request-input"
289                         maxLength={6}
290                         autoFocus
291                     />
292                     <label className="fl-label">One-Time Password</label>
293                 </div>
294                 <button
295                     type="button"
296                     onClick={handleResendOtp}
297                     disabled={otpCooldown > 0 || loading}
298                     style={{ fontSize: 12, color: otpCooldown > 0 ? "var(--text-muted)" : "var(--primary)", background: "none", border: "1px solid #ccc" }}
299                 >
300                     {otpCooldown > 0 ? `Resend OTP in ${otpCooldown}s` : "Resend OTP"}
301                 </button>

```

```
302         <div className="modal-actions">
303             <button className="ui-btn ui-btn--secondary" onClick={() => { setConfirmPw(false); setOtp(""); }} disabled={loading} >Cancel</button>
304             <button className="ui-btn ui-btn--primary" onClick={handleConfirmPasswordUpdate} disabled={loading} >Confirm</button>
305         </div>
306     </motion.div>
307 </motion.div>
308 )}
309 </AnimatePresence>
310 </div>
311 );
312 };
313
314 export default StudentProfile;
```

51. client/src/pages/student/StudentNewRequest.jsx (34 lines)

```
1 import { useNavigate } from "react-router-dom";
2 import { FileLock2, FileSignature } from "lucide-react";
3 import "../../components/TypeChooser.css";
4
5 export default function StudentNewRequest({
6   basePath = "/student/new-request",
7   title = "What type of request would you like to submit?",
8 }) {
9   const navigate = useNavigate();
10
11   return (
12     <div className="type-chooser-page">
13       <div className="type-chooser-grid">
14         <button
15           type="button"
16           className="type-chooser-card-btn"
17           onClick={() => navigate(`${basePath}/nda`)}
18         >
19           <FileLock2 size={64} strokeWidth={1.5} className="type-chooser-card-icon" />
20           <span className="type-chooser-card-label">Non-Disclosure Agreement</span>
21         </button>
22
23         <button
24           type="button"
25           className="type-chooser-card-btn"
26           onClick={() => navigate(`${basePath}/agreement`)}
27         >
28           <FileSignature size={64} strokeWidth={1.5} className="type-chooser-card-icon" />
29           <span className="type-chooser-card-label">Agreement</span>
30         </button>
31       </div>
32     </div>
33   );
34 }
```

52. client/src/pages/student/StudentAgreementRequest.jsx (323 lines)

```

1 import { useMemo, useRef, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { AnimatePresence, motion } from "framer-motion";
4 import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
5 import { createRequest } from "../../services/requestService";
6 import {
7   uploadRequirements,
8   uploadSignatureImage,
9   getDateRequestFolder,
10 } from "../../services/firebaseStorageService";
11 import SignaturePad from "../../components/SignaturePad";
12 import { notify } from "../../utils/inPageFeedback";
13 import "../../components/RequestForm.css";
14 import "../../components/FloatingLabel.css";
15
16 export default function StudentAgreementRequest({ proxyMode = false, fallbackPath = "/student/new-request/agreement" }) {
17   const navigate = useNavigate();
18   const cfg = useMemo(() => FIELDS_FILE_SLOTS_CONFIG.agreement, []);
19
20   const [formData, setFormData] = useState(() => ({}));
21   const [proxyRequestee, setProxyRequestee] = useState({
22     firstName: "",
23     middleInitial: "",
24     lastName: "",
25     email: "",
26   });
27   const [files, setFiles] = useState(() => Array(cfg.fileSlots.length).fill(null));
28   const [accepted, setAccepted] = useState(false);
29   const [submitting, setSubmitting] = useState(false);
30   const [showConfirm, setShowConfirm] = useState(false);
31   const sigPadRef = useRef(null);
32
33   const fileSlots = cfg.fileSlots;
34
35   const onChangeField = (name, value) =>
36     setFormData((p) => ({ ...p, [name]: value }));
37
38   const onChangeProxy = (name, value) =>
39     setProxyRequestee((prev) => ({ ...prev, [name]: value }));
40
41   const handleSubmit = async (e) => {
42     e.preventDefault();
43
44     const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
45     for (const f of cfg.fields) {
46       if (f.required && !String(formData[f.name] || "").trim()) {
47         notify(`${f.label} is required`, { type: "warning" });
48         return;
49       }
50       if (f.type === "email" && String(formData[f.name] || "").trim() && !emailRegex.test(String(formData[f.name]).trim())) {
51         notify(`${f.label} must be a valid email address`, { type: "warning" });
52         return;
53       }
54     }
55
56     if (proxyMode) {
57       const hasName = String(proxyRequestee.firstName || "").trim() && String(proxyRequestee.lastName || "").trim();
58       if (!hasName) {
59         notify("Requestor First Name and Last Name are required", { type: "warning" });
60         return;
61       }
62       const requiredNonNameFields = [
63         ["email", "Requestor Email"],
64       ];
65       for (const [key, label] of requiredNonNameFields) {
66         if (!String(proxyRequestee[key] || "").trim()) {
67           notify(`${label} is required`, { type: "warning" });
68           return;
69         }
70       }
71     }
72
73     for (let i = 0; i < fileSlots.length; i++) {

```

```

74     if (fileSlots[i].required && !files[i]) {
75         notify(`${fileSlots[i].label} is required`, { type: "warning" });
76         return;
77     }
78 }
79
80 const selectedFiles = files.filter(Boolean);
81 if (selectedFiles.length === 0) {
82     notify("Please upload at least 1 requirement file.", { type: "warning" });
83     return;
84 }
85
86 if (!proxyMode) {
87     if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
88         notify("Please draw your e-signature before submitting.", { type: "warning" });
89         return;
90     }
91     if (!accepted) {
92         notify("Please confirm the declaration before submitting.", { type: "warning" });
93         return;
94     }
95 }
96
97 setShowConfirm(true);
98 };
99
100 const handleConfirmSubmit = async () => {
101     setShowConfirm(false);
102     setSubmitting(true);
103     try {
104         const selectedFiles = files.filter(Boolean);
105         const user = JSON.parse(localStorage.getItem("user") || "null");
106         const proxyFullName = proxyMode
107             ? `${proxyRequestee.firstName}${proxyRequestee.middleInitial?.trim() ? " " + proxyRequestee.middleInitial.trim().charAt(0) : ""}`
108             : "";
109         const requestSubjectName = proxyMode
110             ? proxyFullName || user?.name || "Proxy Requestor"
111             : user?.name || "Unknown Student";
112         const requestFolder = getDateRequestFolder();
113
114         const uploaded = await uploadRequirements(selectedFiles, "agreement", requestSubjectName, requestFolder);
115
116         let uploadIndex = 0;
117         const predocs = files
118             .map((f, i) => {
119                 if (!f) return null;
120                 const meta = uploaded[uploadIndex++];
121                 return { ...meta, requirementLabel: fileSlots[i]?.label || `File ${i + 1}` };
122             })
123             .filter(Boolean);
124
125         let authorizerSigUrl = "";
126         let authorizerSigPath = "";
127         if (!proxyMode && !sigPadRef.current.isEmpty()) {
128             const sigDataUrl = sigPadRef.current.getDataUrl();
129             const sigResult = await uploadSignatureImage(sigDataUrl, "agreement", requestSubjectName, requestFolder, "authorizer_");
130             authorizerSigUrl = sigResult.url;
131             authorizerSigPath = sigResult.path;
132         }
133
134         await createRequest({
135             type: "agreement",
136             formData,
137             predocs,
138             authorizerSigUrl,
139             authorizerSigPath,
140             ...(proxyMode ? { proxyRequestee: { ...proxyRequestee, fullName: proxyFullName } } : {}),
141         });
142
143         notify(proxyMode ? "Proxy agreement request submitted." : "Agreement request submitted!", { type: "success", title: "Success" });
144         navigate(fallbackPath);
145     } catch (err) {
146         notify(err.response?.data?.message || "Failed to submit Agreement request", { type: "error" });
147     } finally {
148         setSubmitting(false);
149     }

```



```

150   };
151
152   return (
153     <div className="request-form-page">
154       <form onSubmit={handleSubmit} className="request-form-card">
155         <div className="request-form-header">
156           <button
157             type="button"
158             className="request-form-back"
159             onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
160           >
161             < Back
162           </button>
163           <h2 className="request-form-title">Request Form</h2>
164         </div />
165       </div>
166
167       {/* Section 1: Representative Data */}
168       <div className="request-section">
169         <div className="request-section-title">Representative Information</div>
170         {/* Name fields – single row */}
171         {(() => {
172           const nameKeys = ["repFirstName", "repMiddleInitial", "repLastName"];
173           const nameFields = cfg.fields.filter((f) => nameKeys.includes(f.name));
174           const otherFields = cfg.fields.filter((f) => !nameKeys.includes(f.name));
175           return (
176             <>
177               {nameFields.length > 0 && (
178                 <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 8 }}>
179                   {nameFields.map((f) => (
180                     <div key={f.name} className="fl-wrap">
181                       <input
182                         value={formData[f.name] || ""}
183                         onChange={(e) => onChangeField(f.name, e.target.value)}
184                         required={f.required}
185                         placeholder=" "
186                         className="fl-input request-input"
187                       />
188                       <label className="fl-label">{f.label}</label>
189                     </div>
190                   ))}
191                 </div>
192               )}
193               {otherFields.map((f) => (
194                 <div key={f.name} className="fl-wrap">
195                   {f.kind === "textarea" ? (
196                     <textarea
197                       value={formData[f.name] || ""}
198                       onChange={(e) => onChangeField(f.name, e.target.value)}
199                       rows={f.rows || 4}
200                       placeholder=" "
201                       className="fl-input request-textarea"
202                     />
203                   ) : (
204                     <input
205                       type={f.type || "text"}
206                       value={formData[f.name] || ""}
207                       onChange={(e) => onChangeField(f.name, e.target.value)}
208                       required={f.required}
209                       placeholder=" "
210                       className="fl-input request-input"
211                     />
212                   )}
213                   <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}}>{f.label}</label>
214                 </div>
215               ))}
216             </>
217           );
218         })()}
219
220         {proxyMode && (
221           <>
222             <div className="request-section-title" style={{ marginTop: 10 }}>Requestor Details (F2F Walk-in)</div>
223             <div style={{ display: "grid", gridTemplateColumns: "1fr 0.55fr 1fr", gap: 8 }}>
224               <div className="fl-wrap">
225                 <input className="fl-input request-input" placeholder=" " value={proxyRequestee.firstName} onChange={(e) =>

```

```

226         <label className="fl-label">First Name</label>
227     </div>
228     <div className="fl-wrap">
229         <input className="fl-input request-input" placeholder=" " value={proxyRequestee.middleInitial} onChange={(e) => {
230             <label className="fl-label">Middle Name</label>
231         }}
232     </div>
233     <div className="fl-wrap">
234         <input className="fl-input request-input" placeholder=" " value={proxyRequestee.lastName} onChange={(e) => {
235             <label className="fl-label">Last Name</label>
236         }}
237     </div>
238     <div className="fl-wrap">
239         <input className="fl-input request-input" type="email" placeholder=" " value={proxyRequestee.email} onChange={(e) => {
240             <label className="fl-label">Requestor Email</label>
241         }}
242     </div>
243 }}
244 </div>
245
246 /* Section 2: Requirements */
247 <div className="request-section">
248     <div className="request-section-title">Requirements (Agreement)</div>
249     {files.map((file, index) => (
250         <div key={index} className="request-file-row">
251             <div className="request-file-info">
252                 <div className="request-file-title">
253                     {fileSlots[index]?.label || `Attach file ${index + 1}`}
254                     {fileSlots[index]?.required ? " * " : ""}
255                 </div>
256                 <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
257             </div>
258             <div className="request-file-action">
259                 {file ? "Change File" : "Upload File"}
260                 <input
261                     type="file"
262                     accept="application/pdf,image/*"
263                     onChange={(e) => {
264                         const selected = e.target.files?.[0] || null;
265                         setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
266                     }}
267                 </div>
268             </div>
269         </div>
270     ))}
271 </div>
272
273 /* Section 3: E-Signature */
274 <div className="request-section">
275     {proxyMode ? (
276         <div className="info-banner info-banner--info" style={{ marginBottom: 0 }}>
277             <strong>F2F Walk-in (Proxy Request)</strong>
278             <p>Digital signatures are bypassed for face-to-face walk-ins. The document will be printed for a physical wet s</p>
279         </div>
280     ) : (
281         <div className="request-section-title">Your E-Signature *</div>
282         <p className="request-sig-hint">Draw your signature below. It will be embedded in the agreement document.</p>
283         <SignaturePad ref={sigPadRef} height={160} />
284     </div>
285     </div>
286 </div>
287
288 /* Confirmation clause */
289 {!proxyMode && (
290     <div className="request-accept">
291         <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
292         <span>
293             I confirm that all information and attachments I have provided are accurate and complete,
294             and I consent to the processing of my personal data by the Data Protection Office for the purpose of this request.
295         </span>
296     </div>
297 </div>
298 </div>
299
300 <div className="request-form-actions">
301     <button type="submit" className="request-form-submit" disabled={submitting}>

```

```
302         {submitting ? "Submitting..." : "Submit"}
303     </button>
304 </div>
305 </form>
306
307 <AnimatePresence>
308     {showConfirm && (
309         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
310             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
311                 <h3 className="modal-title">Submit Agreement Request</h3>
312                 <p className="modal-text">Please confirm that all information and attachments are accurate before submitting.</p>
313                 <div className="modal-actions">
314                     <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Cancel</button>
315                     <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ? "Submitting" : "Submit"}</button>
316                 </div>
317             </motion.div>
318         </motion.div>
319     )}
320 </AnimatePresence>
321 </div>
322 );
323 }
```

53. client/src/pages/student/StudentAgreementRequirements.jsx (70 lines)

```

1  import { useNavigate } from "react-router-dom";
2  import { FileSignature, Info } from "lucide-react";
3  import "../components/TypeChooser.css";
4
5  function LoiTooltip({ content }) {
6    return (
7      <span
8        style={{ position: "relative", display: "inline-flex", alignItems: "center", marginLeft: 6, verticalAlign: "middle" }}
9        onClick={(e) => e.stopPropagation()}
10     >
11       <span className="loi-tooltip-trigger">
12         <Info size={13} style={{ color: "var(--text-muted)", cursor: "pointer" }} />
13         <span className="loi-tooltip-box">
14           <strong style={{ display: "block", marginBottom: 4, fontSize: 11 }}>Authorization Letter should include:</strong>
15           {content.map((item, i) => (
16             <span key={i} style={{ display: "block", fontSize: 11, lineHeight: 1.5 }}>• {item}</span>
17           ))}
18         </span>
19       </span>
20     </span>
21   );
22 }
23
24 export default function StudentAgreementRequirements({
25   formPath = "/student/new-request/agreement/form",
26   fallbackPath = "/student",
27 }) {
28   const navigate = useNavigate();
29
30   return (
31     <div className="type-chooser-page">
32       <button
33         type="button"
34         className="type-chooser-back"
35         onClick={() => {
36           if (window.history.length > 1) navigate(-1);
37           else navigate(fallbackPath);
38         }}
39       >
40         < Back
41       </button>
42
43       <div className="type-chooser-grid">
44         <button
45           type="button"
46           className="type-chooser-card-btn type-chooser-card-btn--detailed"
47           onClick={() => navigate(formPath)}
48         >
49           <FileSignature size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
50           <span className="type-chooser-card-label">Agreement</span>
51           <span className="type-chooser-card-subtext">
52             For assigning a representative to act on your behalf in data-related agreements with the DPO.
53           </span>
54           <span className="type-chooser-req-title">Requestor Requirements:</span>
55           <ul className="type-chooser-req-list" aria-label="Requestor requirements">
56             <li>Notarized Authorization Letter<LoiTooltip content={["Scope of authority granted to the representative", "If lo
57             <li>Government Issued Valid ID</li>
58           </ul>
59           <span className="type-chooser-req-title">Representative Requirements:</span>
60           <ul className="type-chooser-req-list" aria-label="Representative requirements">
61             <li>Government Issued Valid ID</li>
62           </ul>
63           <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
64             Click to continue to the form →
65           </span>
66         </button>
67       </div>
68     </div>
69   );
70 }

```

54. client/src/pages/student/StudentNDATypeChooser.jsx (91 lines)

```

1  import { useNavigate } from "react-router-dom";
2  import { Building2, FlaskConical, Info } from "lucide-react";
3  import "../components/TypeChooser.css";
4
5  function LoiTooltip({ content }) {
6    return (
7      <span
8        style={{ position: "relative", display: "inline-flex", alignItems: "center", marginLeft: 6, verticalAlign: "middle" }}
9        onClick={(e) => e.stopPropagation()}
10     >
11       <span className="loi-tooltip-trigger">
12         <Info size={13} style={{ color: "var(--text-muted)", cursor: "pointer" }} />
13         <span className="loi-tooltip-box">
14           <strong style={{ display: "block", marginBottom: 4, fontSize: 11 }}>Letter of Intent should include:</strong>
15           {content.map((item, i) => (
16             <span key={i} style={{ display: "block", fontSize: 11, lineHeight: 1.5 }}>• {item}</span>
17           ))}
18         </span>
19       </span>
20     </span>
21   );
22 }
23
24 export default function StudentNDATypeChooser({
25   basePath = "/student/new-request/nda",
26   fallbackPath = "/student",
27 }) {
28   const navigate = useNavigate();
29
30   return (
31     <div className="type-chooser-page">
32       <button
33         type="button"
34         className="type-chooser-back"
35         onClick={() => {
36           if (window.history.length > 1) navigate(-1);
37           else navigate(fallbackPath);
38         }}
39       >
40         < Back
41       </button>
42
43       <div className="type-chooser-grid">
44         <button
45           type="button"
46           className="type-chooser-card-btn type-chooser-card-btn--detailed"
47           onClick={() => navigate(`${basePath}/orgactivities`)}
48         >
49           <Building2 size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
50           <span className="type-chooser-card-label">School Organization Activities/Projects</span>
51           <span className="type-chooser-card-subtext">
52             For student organization events or partner engagements involving confidential documents.
53           </span>
54           <span className="type-chooser-req-title">Requirements:</span>
55           <ul className="type-chooser-req-list" aria-label="School organization NDA requirements">
56             <li>Letter of Intent<LoiTooltip content={["Title of the Activity/Project", "Objectives of the Activity/Project", "
57               <li>School ID</li>
58               <li>Registration Form</li>
59               <li>Sample Forms or Collaterals (if applicable)</li>
60             </ul>
61           <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
62             Click to continue to the form →
63           </span>
64         </button>
65
66         <button
67           type="button"
68           className="type-chooser-card-btn type-chooser-card-btn--detailed"
69           onClick={() => navigate(`${basePath}/research`)}
70         >
71           <FlaskConical size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
72           <span className="type-chooser-card-label">Conduct of Research within the University</span>

```

```
74         <span className="type-chooser-card-subtext">
75             For thesis, capstone, or research involving sensitive data or unpublished findings.
76         </span>
77         <span className="type-chooser-req-title">Requirements:</span>
78         <ul className="type-chooser-req-list" aria-label="Research NDA requirements">
79             <li>Letter of Intent<LoiTooltip content=[["Research Title", "Introduction / Background of the Research", "Research
80             <li>Questionnaire with Data Privacy Consent Form</li>
81             <li>School ID</li>
82             <li>Registration Form</li>
83         </ul>
84         <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
85             Click to continue to the form →
86         </span>
87     </button>
88 </div>
89 </div>
90 );
91 }
```

55. client/src/pages/student/StudentNDARequest.jsx (284 lines)

```

1 import { useMemo, useRef, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { AnimatePresence, motion } from "framer-motion";
4 import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
5 import { createRequest } from "../../services/requestService";
6 import { uploadRequirements, uploadSignatureImage, getDateRequestFolder } from "../../services/firebaseStorageService";
7 import SignaturePad from "../../components/SignaturePad";
8 import { notify } from "../../utils/inPageFeedback";
9 import "../../components/RequestForm.css";
10 import "../../components/FloatingLabel.css";
11
12 export default function StudentNDARequest({ ndaType, proxyMode = false, fallbackPath = "/student" }) {
13   const navigate = useNavigate();
14   const cfg = useMemo(() => FIELDS_FILE_SLOTS_CONFIG.nda[ndaType], [ndaType]);
15
16   const [formData, setFormData] = useState(() => ({}));
17   const [proxyRequestee, setProxyRequestee] = useState({
18     firstName: "",
19     middleInitial: "",
20     lastName: "",
21     email: "",
22   });
23   const [files, setFiles] = useState(() => Array(cfg.fileSlots.length).fill(null));
24   const [accepted, setAccepted] = useState(false);
25   const [submitting, setSubmitting] = useState(false);
26   const [showConfirm, setShowConfirm] = useState(false);
27   const sigPadRef = useRef(null);
28
29   const onChangeField = (name, value) => {
30     setFormData((p) => ({ ...p, [name]: value }));
31   };
32
33   const onChangeProxy = (name, value) => {
34     setProxyRequestee((prev) => ({ ...prev, [name]: value }));
35   };
36
37   const handleSubmit = async (e) => {
38     e.preventDefault();
39
40     for (const f of cfg.fields) {
41       if (f.required && !String(formData[f.name] || "").trim()) {
42         notify(`${f.label} is required`, { type: "warning" });
43         return;
44       }
45     }
46
47     if (proxyMode) {
48       const hasName = String(proxyRequestee.firstName || "").trim() && String(proxyRequestee.lastName || "").trim();
49       if (!hasName) {
50         notify("Requestor First Name and Last Name are required", { type: "warning" });
51         return;
52       }
53       const requiredNonNameFields = [
54         "email", "Requestor Email",
55       ];
56       for (const [key, label] of requiredNonNameFields) {
57         if (!String(proxyRequestee[key] || "").trim()) {
58           notify(`${label} is required`, { type: "warning" });
59           return;
60         }
61       }
62     }
63
64     for (let i = 0; i < cfg.fileSlots.length; i++) {
65       if (cfg.fileSlots[i].required && !files[i]) {
66         notify(`${cfg.fileSlots[i].label} is required`, { type: "warning" });
67         return;
68       }
69     }
70
71     const selectedFiles = files.filter(Boolean);
72     if (selectedFiles.length === 0) {
73       notify("Please upload at least 1 file.", { type: "warning" });
74       return;
75     }
76
77     // ... (rest of the function logic)
78   };
79 }

```

```
74
75   if (!proxyMode) {
76     if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
77       notify("Please draw your e-signature before submitting.", { type: "warning" });
78       return;
79     }
80     if (!accepted) {
81       notify("Please confirm the declaration before submitting.", { type: "warning" });
82       return;
83     }
84   }
85
86   setShowConfirm(true);
87 };
88
89 const handleConfirmSubmit = async () => {
90   setShowConfirm(false);
91   setSubmitting(true);
92   try {
93     const selectedFiles = files.filter(Boolean);
94     const user = JSON.parse(localStorage.getItem("user") || "null");
95     const proxyFullName = proxyMode
96       ? `${proxyRequestee.firstName}${proxyRequestee.middleInitial?.trim() ? " " + proxyRequestee.middleInitial.trim().charAt(0) : ""}` : "";
97     const requestSubjectName = proxyMode
98       ? proxyFullName || user?.name || "Proxy Requestor"
99       : user?.name || "Unknown Student";
100     const requestFolder = getDateRequestFolder();
101
102     const uploaded = await uploadRequirements(selectedFiles, "nda", requestSubjectName, requestFolder);
103
104     let uploadIndex = 0;
105     const predocs = files
106       .map((f, i) => {
107         if (!f) return null;
108         const meta = uploaded[uploadIndex++];
109         return { ...meta, requirementLabel: cfg.fileSlots[i]?.label || `File ${i + 1}` };
110       })
111       .filter(Boolean);
112
113     let studentSigUrl = "";
114     let studentSigPath = "";
115     if (!proxyMode && !sigPadRef.current.isEmpty()) {
116       const sigDataUrl = sigPadRef.current.getDataUrl();
117       const sigResult = await uploadSignatureImage(sigDataUrl, "nda", requestSubjectName, requestFolder, "student_sig.png");
118       studentSigUrl = sigResult.url;
119       studentSigPath = sigResult.path;
120     }
121
122     await createRequest({
123       type: "nda",
124       formData: { ...formData, ndaType, ndaTypeLabel: cfg.label },
125       predocs,
126       studentSigUrl,
127       studentSigPath,
128       ...(proxyMode ? { proxyRequestee: { ...proxyRequestee, fullName: proxyFullName } } : {}),
129     });
130
131     notify(proxyMode ? "Proxy NDA request submitted." : "NDA request submitted!", { type: "success", title: "Submitted" });
132     navigate(fallbackPath);
133   } catch (err) {
134     notify(err.response?.data?.message || "Failed to submit request", { type: "error" });
135   } finally {
136     setSubmitting(false);
137   }
138 };
139
140 return (
141   <div className="request-form-page">
142     <form onSubmit={handleSubmit} className="request-form-card">
143       <div className="request-form-header">
144         <button
145           type="button"
146           className="request-form-back"
147           onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
148         >
```



```

150     < Back
151   </button>
152   <h2 className="request-form-title">Request Form</h2>
153   <div />
154 </div>
155
156 { /* Section 1: Requestee Data */ }
157 <div className="request-section">
158   <div className="request-section-title">Requestor Information</div>
159   {cfg.fields.map((f) => (
160     <div key={f.name} className="fl-wrap">
161       {f.kind === "textarea" ? (
162         <textarea
163           value={formData[f.name] || ""}
164           onChange={(e) => onChangeField(f.name, e.target.value)}
165           rows={f.rows || 4}
166           placeholder=" "
167           className="fl-input request-textarea"
168         />
169       ) : (
170         <input
171           value={formData[f.name] || ""}
172           onChange={(e) => onChangeField(f.name, e.target.value)}
173           required={f.required}
174           placeholder=" "
175           className="fl-input request-input"
176         />
177       )}
178       <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}`}>{f.label}</label>
179     </div>
180   )]}
181
182   {proxyMode && (
183     <div className="request-section-title" style={{ marginTop: 10 }}>Requestor Details (F2F Walk-in)</div>
184     <div style={{ display: "grid", gridTemplateColumns: "1fr 0.55fr 1fr", gap: 8 }}>
185       <div className="fl-wrap">
186         <input className="fl-input request-input" placeholder=" " value={proxyRequestee.firstName} onChange={(e) =>
187           onChangeField(f.name, e.target.value)} />
188         <label className="fl-label">First Name</label>
189       </div>
190       <div className="fl-wrap">
191         <input className="fl-input request-input" placeholder=" " value={proxyRequestee.middleInitial} onChange={(e) =>
192           onChangeField(f.name, e.target.value)} />
193         <label className="fl-label">Middle Name</label>
194       </div>
195       <div className="fl-wrap">
196         <input className="fl-input request-input" placeholder=" " value={proxyRequestee.lastName} onChange={(e) =>
197           onChangeField(f.name, e.target.value)} />
198         <label className="fl-label">Last Name</label>
199       </div>
200       <div className="fl-wrap">
201         <input className="fl-input request-input" type="email" placeholder=" " value={proxyRequestee.email} onChange=
202           onChangeField(f.name, e.target.value)} />
203         <label className="fl-label">Requestor Email</label>
204       </div>
205     </div>
206   )}
207
208 { /* Section 2: Requirements */ }
209 <div className="request-section">
210   <div className="request-section-title">Requirements (NDA)</div>
211   {files.map((file, index) => (
212     <div key={index} className="request-file-row">
213       <div className="request-file-info">
214         <div className="request-file-title">
215           {cfg.fileSlots[index]?.label || `Attach file ${index + 1}`}
216           {cfg.fileSlots[index]?.required ? " *" : ""}
217         </div>
218         <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
219       </div>
220       <label className="request-file-action">
221         {file ? "Change File" : "Upload File"}
222       </label>
223       <input
224         type="file"
225         accept="application/pdf,image/*"
226         onChange={(e) => {
227           const selected = e.target.files?.[0] || null;

```

```

226         setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
227     }}
228     />
229     </label>
230 </div>
231 )}}
232 </div>
233
234 {/ Section 3: E-Signature */}
235 <div className="request-section">
236     {proxyMode ? (
237         <div className="info-banner info-banner--info" style={{ marginBottom: 0 }}>
238             <strong>F2F Walk-in (Proxy Request)</strong>
239             <p>Digital signatures are bypassed for face-to-face walk-ins. The document will be printed for a physical wet s
240         </div>
241     ) : (
242         <div className="request-section-title">Your E-Signature *</div>
243         <p className="request-sig-hint">Draw your signature below. It will be embedded in the NDA document.</p>
244         <SignaturePad ref={sigPadRef} height={160} />
245     </div>
246     )}
247 </div>
248
249 {/ Confirmation clause */}
250 {!proxyMode && (
251     <label className="request-accept">
252         <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
253         <span>
254             I confirm that all information and attachments I have provided are accurate and complete,
255             and I consent to the processing of my personal data by the Data Protection Office for the purpose of this requ
256         </span>
257     </label>
258 )}
259
260 <div className="request-form-actions">
261     <button type="submit" className="request-form-submit" disabled={submitting}>
262         {submitting ? "Submitting..." : "Submit"}
263     </button>
264 </div>
265 </form>
266
267 <AnimatePresence>
268     {showConfirm && (
269         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
270             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
271                 <h3 className="modal-title">Submit NDA Request</h3>
272                 <p className="modal-text">Please confirm that all information and attachments are accurate before submitting.</p>
273                 <div className="modal-actions">
274                     <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Canc
275                     <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ?
276                 </div>
277             </motion.div>
278         </motion.div>
279     )}
280 </AnimatePresence>
281 </div>
282 );
283 }

```

56. client/src/pages/student/StudentRequestReview.jsx (291 lines)

```

1 import { useEffect, useMemo, useState } from "react";
2 import { useNavigate, useParams } from "react-router-dom";
3 import { AlertTriangle, Copy, Paperclip } from "lucide-react";
4 import RequestStepper from "../../components/RequestStepper";
5 import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
6 import { getRequestById } from "../../services/requestService";
7 import { notify } from "../../utils/inPageFeedback";
8 import "../../components/RequestForm.css";
9
10 const statusPillClass = (s) => {
11   if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
12   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
13   if (s === "stud_revision_requested") return "status-pill status-pill--orange";
14   if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
15   if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
16   if (s === "agr_declined") return "status-pill status-pill--red";
17   return "status-pill";
18 };
19
20 const prettyStatus = (s) => {
21   const map = {
22     nda_pending: "Reviewal",
23     nda_approved: "Approved",
24     stud_revision_requested: "Student Revisions",
25     agr_pending_1: "Initial Reviewal",
26     agr_awaiting_rep_signature: "Recipient Reviewal",
27     agr_pending_2: "Final Reviewal",
28     agr_approved: "Approved",
29     agr_declined: "Recipient Declined",
30     agr_rep_revision_requested: "Recipient Revisions",
31   };
32   return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
33 };
34
35 export default function StudentRequestReview() {
36   const { id } = useParams();
37   const navigate = useNavigate();
38
39   const [reqData, setReqData] = useState(null);
40
41   const load = async () => {
42     try {
43       const r = await getRequestById(id);
44
45       // Redirect to resubmit page if revision is requested from student
46       if (r.status === "stud_revision_requested") {
47         navigate(`/student/resubmit/${id}`, { replace: true });
48         return;
49       }
50
51       setReqData(r);
52     } catch (err) {
53       notify(err.response?.data?.message || "Failed to load request", { type: "error" });
54       navigate("/student");
55     }
56   };
57
58   useEffect(() => {
59     load();
60     // eslint-disable-next-line react-hooks/exhaustive-deps
61   }, [id, navigate]);
62
63   const cfg = useMemo(() => {
64     if (!reqData) return null;
65     if (reqData.type === "agreement") return FIELDS_FILE_SLOTS_CONFIG.agreement;
66     if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda[reqData.formData?.ndaType];
67     return null;
68   }, [reqData]);
69
70   if (!reqData) return null;
71
72   const title =
73     reqData.type === "agreement"

```

```

74     ? "Agreement Request"
75     : `NDA Request${reqData.formData?.ndaTypeLabel ? ` - ${reqData.formData.ndaTypeLabel}` : ""}`;
76
77     const isApproved = reqData.status === "nda_approved" || reqData.status === "agr_approved";
78     const isAgreement = reqData.type === "agreement";
79
80     return (
81       <div className="review-page">
82         <div className="review-card">
83           <div className="request-form-header">
84             <button
85               type="button"
86               className="request-form-back"
87               onClick={() => {
88                 if (window.history.length > 1) navigate(-1);
89                 else navigate("/student");
90               }}
91             >
92               < Back
93             </button>
94             <h2 className="request-form-title">{title}</h2>
95           </div>
96         </div>
97
98         <div className="review-header">
99           <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", gap: 12, flexWrap: "wrap", width: "100%" }}>
100             <span className={statusPillClass(reqData.status)}>{prettyStatus(reqData.status)}</span>
101             <span className="review-meta-row" style={{ marginLeft: "auto" }}>
102               <b>Request Date:</b> {new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short" })}
103             </span>
104           </div>
105         </div>
106
107         <div className="review-section review-section--stepper">
108           <RequestStepper status={reqData.status} type={reqData.type} />
109         </div>
110
111         {/* Form Data / Representative */}
112         <div className="review-section">
113           <h4 className="review-section-title">{isAgreement ? "Representative" : "Form Data"}</h4>
114           {cfg?.fields?.length ? (
115             isAgreement ? (
116               <>
117                 {/* Full Name (bundled) */}
118                 <div className="review-field">
119                   <span className="review-field-label">Full Name</span>
120                   <div className="review-info-box">
121                     {{{() => {
122                       const fd = reqData.formData || {};
123                       const mid = String(fd.repMiddleInitial || "").trim();
124                       const midInitial = mid ? `${mid[0].toUpperCase()}.` : "";
125                       const full = [fd.repFirstName, midInitial, fd.repLastName].filter(Boolean).join(" ").replace(/\s+/g, " ");
126                       return full || <span className="review-info-box--muted"></span>;
127                     }}}()
128                   </div>
129                 </div>
130               </div>
131             {/* Remaining fields */}
132             {cfg.fields
133               .filter((f) => ![ "repFirstName", "repMiddleInitial", "repLastName" ].includes(f.name))
134               .map((f) => (
135                 <div key={f.name} className="review-field">
136                   <span className="review-field-label">{f.label}</span>
137                   <div className="review-info-box">
138                     {String(reqData.formData?.[f.name] ?? "") || (
139                       <span className="review-info-box--muted"></span>
140                     )}
141                   </div>
142                 </div>
143               ))}
144             </div>
145           ) : (
146             cfg.fields.map((f) => (
147               <div key={f.name} className="review-field">
148                 <span className="review-field-label">{f.label}</span>
149                 <div className="review-info-box">

```

```

150         {String(reqData.formData?.[f.name] ?? "") || (
151             <span className="review-info-box--muted"></span>
152         )}
153     </div>
154 </div>
155 ))
156 )
157 ) : (
158     <div className="review-info-box">
159         <span className="review-info-box--muted">No config fields found for this request.</span>
160     </div>
161 )}
162 </div>
163
164 {/ * Attachments */}
165 <div className="review-section">
166     <h4 className="review-section-title">My Attachments</h4>
167     {(() => {
168         const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
169         return initialDocs.length ? (
170             <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
171                 {initialDocs.map((f, idx) => (
172                     <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
173                         <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File $
174                     </a>
175                 ))}
176             </div>
177         ) : (
178             <div className="review-info-box">
179                 <span className="review-info-box--muted">No files.</span>
180             </div>
181         );
182     })()}
183 </div>
184
185 {(() => {
186     const rounds = {};
187     (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
188         const r = f.resubmissionRound || 1;
189         if (!rounds[r]) rounds[r] = [];
190         rounds[r].push(f);
191     });
192     return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
193         <div key={round} className="review-section">
194             <h4 className="review-section-title">
195                 Resubmitted Attachments{Object.keys(rounds).length > 1 ? ` (Round ${round})` : ""}
196             </h4>
197             <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
198                 {rounds[round].map((f, idx) => (
199                     <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
200                         <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File $
201                     </a>
202                 ))}
203             </div>
204         </div>
205     ));
206 })()}
207
208 {/ * Agreement-specific status banners */}
209 {isAgreement && reqData.status === "agr_awaiting_rep_signature" && (
210     <div className="info-banner info-banner--info">
211         <strong>Awaiting Representative</strong>
212         <p>
213             The admin has approved your request and a signing link has been generated for
214             the representative.
215         </p>
216     </div>
217 )}
218
219 {/ * Signing link visibility (only while awaiting rep action) */}
220 {isAgreement && reqData.signingToken && (reqData.status === "agr_awaiting_rep_signature" || reqData.status === "agr_r
221     <div className="review-section">
222         <h4 className="review-section-title">Representative Signing Link</h4>
223         <div className="review-info-box" style={{ fontSize: 13 }}>
224             <div className="signing-link-row">
225                 <span className="signing-link-code">Representative signing link ready – click to copy</span>

```

```

226         <button
227             className="signing-link-copy-btn"
228             onClick={() => {
229                 navigator.clipboard.writeText(`${window.location.origin}/sign/${reqData.signingToken}`);
230                 notify("Copied!", { type: "success" });
231             }}
232             title="Copy link"
233         >
234             <Copy size={14} strokeWidth={2} />
235         </button>
236     </div>
237     {reqData.signingTokenExpiresAt && (
238         <div style={{ marginTop: 6, fontSize: 12, color: new Date(reqData.signingTokenExpiresAt) < new Date() ? "#dc2
239             {new Date(reqData.signingTokenExpiresAt) < new Date()
240                 ? <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><AlertTriangle size={14} strokeW
241                 : `Expires: ${new Date(reqData.signingTokenExpiresAt).toLocaleDateString("en-PH", { year: "numeric", mont
242             </div>
243         )}
244     </div>
245 </div>
246 )}
247
248 {isAgreement && reqData.status === "agr_pending_2" && (
249     <div className="info-banner info-banner--success">
250         <strong>Representative Signed</strong>
251         <p>
252             The representative has submitted their signature. Awaiting final admin approval.
253         </p>
254     </div>
255 )}
256
257 {isAgreement && reqData.status === "agr_declined" && (
258     <div className="info-banner info-banner--danger">
259         <strong>Recipient Declined</strong>
260         <p>The recipient declined to sign this agreement.</p>
261     </div>
262 )}
263
264 {isAgreement && reqData.status === "agr_rep_revision_requested" && (
265     <div className="info-banner info-banner--warning">
266         <strong>Recipient Revisions Requested</strong>
267         <p>
268             The admin has requested the recipient to revise and resubmit their information.
269         </p>
270     </div>
271 )}
272
273 {/* Approved document */}
274 {isApproved && (
275     <div className="review-section">
276         <h4 className="review-section-title">Approved Document</h4>
277         {reqData.postdocs?.url ? (
278             <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
279                 <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved Document
280             </a>
281         ) : (
282             <div className="review-info-box">
283                 <span className="review-info-box--muted">No approved document uploaded.</span>
284             </div>
285         )}
286     </div>
287 )}
288 </div>
289 </div>
290 );
291 }

```

57. client/src/pages/student/StudentResubmitRequest.jsx (340 lines)

```

1 import { useEffect, useMemo, useRef, useState } from "react";
2 import { useNavigate, useParams } from "react-router-dom";
3 import { AnimatePresence, motion } from "framer-motion";
4 import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
5 import { getRequestById, resubmitRequest } from "../../services/requestService";
6 import {
7   uploadRequirements,
8   uploadSignatureImage,
9   getDateRequestFolder,
10 } from "../../services/firebaseStorageService";
11 import SignaturePad from "../../components/SignaturePad";
12 import { notify } from "../../utils/inPageFeedback";
13 import "../../components/RequestForm.css";
14 import "../../components/FloatingLabel.css";
15
16 const getInitialFolderFromPath = (path = "") => {
17   const parts = String(path).split("/");
18   return parts[3] || "";
19 };
20
21 export default function StudentResubmitRequest() {
22   const { id } = useParams();
23   const navigate = useNavigate();
24
25   const [reqData, setReqData] = useState(null);
26   const [formData, setFormData] = useState({});
27   const [files, setFiles] = useState([]);
28   const [accepted, setAccepted] = useState(false);
29   const [submitting, setSubmitting] = useState(false);
30   const [showConfirm, setShowConfirm] = useState(false);
31   const sigPadRef = useRef(null);
32
33   useEffect(() => {
34     const load = async () => {
35       try {
36         const r = await getRequestById(id);
37         setReqData(r);
38         setFormData(r.formData || {});
39       } catch (err) {
40         notify(err.response?.data?.message || "Failed to load request", { type: "error" });
41         navigate("/student");
42       }
43     };
44     load();
45   }, [id, navigate]);
46
47   const cfg = useMemo(() => {
48     if (!reqData) return null;
49     if (reqData.type === "agreement") return FIELDS_FILE_SLOTS_CONFIG.agreement;
50     if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda[reqData.formData?.ndaType];
51     return null;
52   }, [reqData]);
53
54   useEffect(() => {
55     if (!cfg) return;
56     setFiles(Array(cfg.fileSlots.length).fill(null));
57   }, [cfg]);
58
59   const onChangeField = (name, value) => {
60     setFormData((p) => ({ ...p, [name]: value }));
61   };
62
63   const handleSubmit = async (e) => {
64     e.preventDefault();
65     if (!reqData || !cfg) return;
66
67     if (!["stud_revison_requested"].includes(reqData.status)) {
68       notify("Only revision requested requests can be resubmitted.", { type: "warning" });
69       return;
70     }
71
72     const selectedFiles = files.filter(Boolean);
73     if (selectedFiles.length === 0) {
74       notify("Please upload at least 1 revised file.", { type: "warning" });

```

```
74     return;
75   }
76
77   if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
78     notify("Please draw your e-signature before resubmitting.", { type: "warning" });
79     return;
80   }
81
82   if (!accepted) {
83     notify("Please confirm the declaration before submitting.", { type: "warning" });
84     return;
85   }
86
87   const basePath = reqData.predocs?.[0]?.path || "";
88   const initialFolder = getInitialFolderFromPath(basePath);
89   if (!initialFolder) {
90     notify("Missing original request folder. (No existing file path found)", { type: "error" });
91     return;
92   }
93
94   setShowConfirm(true);
95 };
96
97 const handleConfirmSubmit = async () => {
98   setShowConfirm(false);
99   setSubmitting(true);
100   try {
101     const selectedFiles = files.filter(Boolean);
102     const user = JSON.parse(localStorage.getItem("user") || "null");
103     const studentName = user?.name || "Unknown Student";
104     const initialFolder = getInitialFolderFromPath(reqData.predocs?.[0]?.path || "");
105     const resubFolder = `resub${getDateRequestFolder()}`;
106     const resubPath = `${initialFolder}/${resubFolder}`;
107
108     const uploaded = await uploadRequirements(selectedFiles, reqData.type, studentName, resubPath);
109
110     let uploadIndex = 0;
111     const predocs = files
112       .map((f, i) => {
113         if (!f) return null;
114         const meta = uploaded[uploadIndex++];
115         return { ...meta, requirementLabel: cfg.fileSlots[i]?.label || `File ${i + 1}` };
116       })
117       .filter(Boolean);
118
119     const payload = { formData, predocs };
120
121     if (reqData.type === "agreement") {
122       const sigDataUrl = sigPadRef.current.getDataUrl();
123       const { url: authorizerSigUrl, path: authorizerSigPath } = await uploadSignatureImage(
124         sigDataUrl, "agreement", studentName, resubPath, "authorizer_sig.png"
125       );
126       payload.authorizerSigUrl = authorizerSigUrl;
127       payload.authorizerSigPath = authorizerSigPath;
128     }
129
130     if (reqData.type === "nda") {
131       const sigDataUrl = sigPadRef.current.getDataUrl();
132       const { url: studentSigUrl, path: studentSigPath } = await uploadSignatureImage(
133         sigDataUrl, "nda", studentName, resubPath, "student_sig.png"
134       );
135       payload.studentSigUrl = studentSigUrl;
136       payload.studentSigPath = studentSigPath;
137     }
138
139     await resubmitRequest(id, payload);
140     notify("Your request has been resubmitted successfully.", { type: "success", title: "Resubmitted" });
141     navigate("/student");
142   } catch (err) {
143     notify(err.response?.data?.message || "Failed to resubmit. Please try again.", { type: "error" });
144   } finally {
145     setSubmitting(false);
146   }
147 };
148
149 if (!reqData) return null;
```



```

150
151 if (!["stud_revision_requested"].includes(reqData.status)) {
152   return (
153     <div className="review-page">
154       <div className="review-card">
155         <button
156           type="button"
157           className="review-back-btn"
158           onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
159         >
160           ← Back
161         </button>
162         <div className="review-info-box">
163           <span className="review-info-box--muted">This request is not marked as revision requested.</span>
164         </div>
165       </div>
166     </div>
167   );
168 }
169
170 const dataTitle = reqData.type === "agreement" ? "Revised Representative Data" : "Revised Requestor Data";
171
172 return (
173   <div className="request-form-page">
174     <form onSubmit={handleSubmit} className="request-form-card">
175       <div className="request-form-header">
176         <button
177           type="button"
178           className="request-form-back"
179           onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
180         >
181           < Back
182         </button>
183         <h2 className="request-form-title">Resubmit Request Form</h2>
184       </div>
185     </div>
186
187     <div className="request-section">
188       <div className="request-section-title">Remarks from Admin</div>
189       <div style={{
190         padding: "12px 14px",
191         background: "var(--s-warning-bg)",
192         border: "1px solid var(--s-warning-dot)",
193         borderRadius: "var(--radius-md)",
194         fontSize: "13px",
195         color: "var(--s-warning-text)",
196       }}>
197         {reqData.remarks || <span style={{ opacity: 0.7 }}>No remarks provided.</span>}
198       </div>
199     </div>
200
201     <div className="request-section">
202       <div className="request-section-title">{dataTitle}</div>
203       {(() => {
204         const nameKeys = ["repFirstName", "repMiddleInitial", "repLastName"];
205         const nameFields = cfg.fields.filter((f) => nameKeys.includes(f.name));
206         const otherFields = cfg.fields.filter((f) => !nameKeys.includes(f.name));
207         return (
208           <>
209             {nameFields.length > 0 && (
210               <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 8 }}>
211                 {nameFields.map((f) => (
212                   <div key={f.name} className="fl-wrap">
213                     <input
214                       value={formData[f.name] || ""}
215                       onChange={(e) => onChangeField(f.name, e.target.value)}
216                       placeholder=" "
217                       className="fl-input request-input"
218                     />
219                     <label className="fl-label">{f.label}</label>
220                   </div>
221                 ))}
222               </div>
223             )}
224           </div>
225         )}

```

```

226      {otherFields.map((f) => (
227        <div key={f.name} className="fl-wrap">
228          {f.kind === "textarea" ? (
229            <textarea
230              value={formData[f.name] || ""}
231              onChange={(e) => onChangeField(f.name, e.target.value)}
232              rows={f.rows || 4}
233              placeholder=" "
234              className="fl-input request-textarea"
235            />
236          ) : (
237            <input
238              value={formData[f.name] || ""}
239              onChange={(e) => onChangeField(f.name, e.target.value)}
240              placeholder=" "
241              className="fl-input request-input"
242            />
243          )}
244          <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}`}>{f.label}</label>
245        </div>
246      )]}
247    </>
248  );
249  })()}
250 </div>
251
252 /* Section 3: Previous Attachments */
253 <div className="request-section">
254   <div className="request-section-title">Previous Attachments</div>
255   {reqData.predocs?.length ? (
256     reqData.predocs.map((f, idx) => (
257       <div key={idx} className="request-file-row">
258         <div className="request-file-info">
259           <div className="request-file-title">{f.requirementLabel || f.origName || `File ${idx + 1}`}</div>
260         </div>
261         <a href={f.url} target="_blank" rel="noreferrer" className="request-file-action" style={{ textDecoration: "no
262           View
263         </a>
264       </div>
265     ))
266   ) : (
267     <div className="request-file-row">
268       <div className="request-file-info">
269         <div className="request-file-subtitle">No files submitted.</div>
270       </div>
271     </div>
272   )}
273 </div>
274
275 /* Section 4: Revised Requirements */
276 <div className="request-section">
277   <div className="request-section-title">Revised Requirements</div>
278   {files.map((file, index) => (
279     <div key={index} className="request-file-row">
280       <div className="request-file-info">
281         <div className="request-file-title">
282           {cfg.fileSlots[index]?.label || `Attach file ${index + 1}`}
283         </div>
284         <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
285       </div>
286       <label className="request-file-action">
287         {file ? "Change File" : "Upload File"}
288       <input
289         type="file"
290         accept="application/pdf,image/*"
291         onChange={(e) => {
292           const selected = e.target.files?.[0] || null;
293           setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
294         }}
295       </>
296     </label>
297   </div>
298   )})
299 </div>
300
301 /* Section 5: E-Signature */

```

```
302     <div className="request-section">
303       <div className="request-section-title">Your E-Signature *</div>
304       <p className="request-sig-hint">Please draw your signature again for this resubmission.</p>
305       <SignaturePad ref={sigPadRef} height={160} />
306     </div>
307
308     { /* Confirmation clause */ }
309     <label className="request-accept">
310       <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
311       <span>
312         I confirm that all revised information and attachments I have provided are accurate and complete,
313         and I consent to the processing of my personal data by the Data Protection Office for the purpose of this request
314       </span>
315     </label>
316
317     <div className="request-form-actions">
318       <button type="submit" className="request-form-submit" disabled={submitting}>
319         {submitting ? "Submitting..." : "Resubmit"}
320       </button>
321     </div>
322   </form>
323
324   <AnimatePresence>
325     {showConfirm && (
326       <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
327         <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
328           <h3 className="modal-title">Resubmit Request</h3>
329           <p className="modal-text">Please confirm that all revised information and attachments are accurate before resub
330           <div className="modal-actions">
331             <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Canc
332             <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ?
333           </div>
334         </motion.div>
335       </motion.div>
336     )}
337   </AnimatePresence>
338 </div>
339 );
340 }
```

58. client/src/pages/admin/AdminDashboard.jsx (471 lines)

```
1 import { useContext, useEffect, useMemo, useState } from "react";
2 import { Link } from "react-router-dom";
3 import { motion } from "framer-motion";
4 import {
5   PieChart, Pie, Cell, Tooltip, ResponsiveContainer,
6   BarChart, Bar, XAxis, YAxis, CartesianGrid, Legend,
7   LineChart, Line,
8 } from "recharts";
9 import {
10   FileText, Check, X, Archive,
11   ArrowRight, RefreshCw, MoreHorizontal,
12 } from "lucide-react";
13 import FilterSelect from "../../components/FilterSelect";
14 import { getAllRequests } from "../../services/requestService";
15 import { getRecentAuditLogs } from "../../services/auditService";
16 import { AuthContext } from "../../context/AuthContext";
17 import {
18   APPROVED_STATUSES,
19   PENDING_STATUSES,
20   REVISION_STATUSES,
21   CHART_LABELS,
22   CHART_COLORS,
23   normalizeStatusForChart,
24 } from "../../utils/requestStatusCharts";
25
26 const DONUT_COLORS = ["#059669", "#ea580c", "#7c3aed", "#ca8a04", "#2563eb", "#dc2626", "#64748b", "#3b82f6"];
27 const MONTH_NAMES = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
28
29 const ACTION_LABEL = {
30   login: "User logged in",
31   login_failed: "Failed login attempt",
32   login_otp_sent: "Login OTP sent",
33   account_created: "New account created",
34   account_activated: "Account activated",
35   email_verified: "Email verified",
36   request_created: "Request submitted",
37   request_approved: "Request approved",
38   request_revision_required: "Revision requested",
39   request_resubmitted: "Request resubmitted",
40   rep_signature_submitted: "Representative signed",
41   rep_signature_declined: "Representative declined",
42   signing_link_generated: "Signing link generated",
43   password_changed: "Password changed",
44   password_reset_requested: "Password reset requested",
45   user_deactivated: "User deactivated",
46   user_activated: "User activated",
47 };
48
49 const formatAction = (action) =>
50   ACTION_LABEL[action] ||
51   action.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase());
52
53 const cardVariants = {
54   hidden: { opacity: 0, y: 20 },
55   visible: (i) => ({ opacity: 1, y: 0, transition: { delay: i * 0.07, duration: 0.35 } }),
56 };
57
58 export default function AdminDashboard() {
59   const { user } = useContext(AuthContext);
60   const [requests, setRequests] = useState([]);
61   const [auditLogs, setAuditLogs] = useState([]);
62   const [error, setError] = useState("");
63   const [refreshing, setRefreshing] = useState(false);
64   const [isCompact, setIsCompact] = useState(() => {
65     if (typeof window === "undefined") return false;
66     return window.innerWidth < 768;
67   });
68
69   // Date filter for charts
70   const [dateMode, setDateMode] = useState("preset");
71   const [preset, setPreset] = useState("all");
72   const [singleDate, setSingleDate] = useState("");
73   const [startDate, setStartDate] = useState("");
```

```
74 const [endDate, setEndDate] = useState("");
75
76 useEffect(() => {
77   const onResize = () => setIsCompact(window.innerWidth < 768);
78   window.addEventListener("resize", onResize);
79   return () => window.removeEventListener("resize", onResize);
80 }, []);
81
82 const load = async ({ withToast = false } = {}) => {
83   if (withToast) setRefreshing(true);
84   const errors = [];
85
86   // Fetch independently so one failure doesn't block the other
87   try {
88     const reqData = await getAllRequests();
89     setRequests(reqData);
90   } catch (err) {
91     const msg = err.response?.data?.message || err.message || "Failed to load requests";
92     const status = err.response?.status;
93     errors.push(`Requests: ${msg}${status ? ` (HTTP ${status})` : ""}`);
94     console.error("Dashboard requests error:", err);
95   }
96
97   try {
98     if (user?.role === "admin") {
99       const auditData = await getRecentAuditLogs(6);
100      setAuditLogs(auditData);
101     } else {
102       setAuditLogs([]);
103     }
104   } catch (err) {
105     const msg = err.response?.data?.message || err.message || "Failed to load audit logs";
106     const status = err.response?.status;
107     errors.push(`Audit logs: ${msg}${status ? ` (HTTP ${status})` : ""}`);
108     console.error("Dashboard audit error:", err);
109   }
110
111   if (errors.length) setError(errors.join(" | "));
112   setRefreshing(false);
113 };
114
115 useEffect(() => {
116   load();
117   // eslint-disable-next-line react-hooks/exhaustive-deps
118 }, []);
119
120 // Filtered requests for charts based on date filter
121 const chartFilteredRequests = useMemo(() => {
122   const now = new Date();
123   if (dateMode === "preset") {
124     if (preset === "all") return requests;
125     if (preset === "today") {
126       const s = new Date(now.getFullYear(), now.getMonth(), now.getDate());
127       const e = new Date(s.getTime() + 86400000 - 1);
128       return requests.filter((r) => { const d = new Date(r.createdAt); return d >= s && d <= e; });
129     }
130     if (preset === "thisWeek") {
131       const day = now.getDay();
132       const monday = new Date(now);
133       monday.setDate(now.getDate() - ((day + 6) % 7));
134       monday.setHours(0, 0, 0, 0);
135       return requests.filter((r) => new Date(r.createdAt) >= monday);
136     }
137     if (preset === "thisMonth") {
138       const s = new Date(now.getFullYear(), now.getMonth(), 1);
139       return requests.filter((r) => new Date(r.createdAt) >= s);
140     }
141     if (preset === "thisYear") {
142       const s = new Date(now.getFullYear(), 0, 1);
143       return requests.filter((r) => new Date(r.createdAt) >= s);
144     }
145   }
146   if (dateMode === "single" && singleDate) {
147     const s = new Date(singleDate);
148     const e = new Date(singleDate + "T23:59:59");
149     return requests.filter((r) => { const d = new Date(r.createdAt); return d >= s && d <= e; });
150   }
151 }
```

```

150     }
151     if (dateMode === "range") {
152         return requests.filter((r) => {
153             const d = new Date(r.createdAt);
154             if (startDate && d < new Date(startDate)) return false;
155             if (endDate && d > new Date(endDate + "T23:59:59")) return false;
156             return true;
157         });
158     }
159     return requests;
160 }, [requests, dateMode, preset, singleDate, startDate, endDate]);
161
162 // Derived chart data (same approach as Reports page)
163 const statusData = useMemo(() => {
164     const counts = {};
165     chartFilteredRequests.forEach((r) => {
166         if (!r.isArchived) {
167             const key = normalizeStatusForChart(r.status);
168             counts[key] = (counts[key] || 0) + 1;
169         }
170     });
171     return Object.entries(counts).map(([id, value]) => ({
172         id,
173         name: CHART_LABELS[id] || id,
174         value,
175     }));
176 }, [chartFilteredRequests]);
177
178 const typeData = useMemo(() => [
179     { name: "NDA", count: chartFilteredRequests.filter((r) => r.type === "nda").length },
180     { name: "Agreement", count: chartFilteredRequests.filter((r) => r.type === "agreement").length },
181 ], [chartFilteredRequests]);
182
183 // Monthly trend (last 6 months)
184 const trendData = useMemo(() => {
185     const now = new Date();
186     return Array.from({ length: 6 }, (_, i) => {
187         const d = new Date(now.getFullYear(), now.getMonth() - (5 - i), 1);
188         const monthStart = new Date(d.getFullYear(), d.getMonth(), 1);
189         const monthEnd = new Date(d.getFullYear(), d.getMonth() + 1, 0, 23, 59, 59);
190         const inMonth = requests.filter((r) => {
191             const c = new Date(r.createdAt);
192             return c >= monthStart && c <= monthEnd;
193         });
194         return {
195             month: MONTH_NAMES[d.getMonth()],
196             total: inMonth.length,
197             approved: inMonth.filter((r) => APPROVED_STATUSES.includes(r.status)).length,
198             pending: inMonth.filter((r) => PENDING_STATUSES.includes(r.status)).length,
199         };
200     });
201 }, [requests]);
202
203 const totalActive = requests.filter((r) => !r.isArchived).length;
204 const totalPending = requests.filter((r) => PENDING_STATUSES.includes(r.status)).length;
205 const totalApproved = requests.filter((r) => APPROVED_STATUSES.includes(r.status)).length;
206 const totalRevisions = requests.filter((r) => REVISION_STATUSES.includes(r.status)).length;
207 const totalArchived = requests.filter((r) => r.isArchived).length;
208
209 const summaryCards = [
210     { icon: FileText, label: "Total Requests", value: totalActive, color: "#3b82f6" },
211     { icon: MoreHorizontal, label: "Reviewal", value: totalPending, color: "#f59e0b" },
212     { icon: Check, label: "Approved", value: totalApproved, color: "#10b981" },
213     { icon: X, label: "Revisions", value: totalRevisions, color: "#ea580c" },
214     { icon: Archive, label: "Archived", value: totalArchived, color: "#64748b" },
215 ];
216
217 return (
218     <div className="dashboard-page">
219         {/* ■■ Error Banner ■■ */}
220         {error && <div className="admin-flash-banner admin-flash-banner--error">{error}</div>}
221         {/* ■■ Summary Cards ■■ */}
222         <div className="responsive-grid-5 dashboard-section-gap">
223             {summaryCards.map((card, i) => (
224                 <motion.div

```

```

226     key={card.label}
227     custom={i}
228     variants={cardVariants}
229     initial="hidden"
230     animate="visible"
231     style={{
232       background: "var(--surface)",
233       border: "1px solid var(--border)",
234       borderRadius: "var(--radius-lg)",
235       padding: "20px 24px",
236       boxShadow: "var(--shadow-sm)",
237       display: "flex",
238       alignItems: "center",
239       gap: 16,
240     }}
241   >
242     <div style={{
243       width: 44, height: 44, borderRadius: "var(--radius-md)",
244       background: `${card.color}18`, display: "flex", alignItems: "center", justifyContent: "center",
245       flexShrink: 0,
246     }}>
247       <card.icon size={20} color={card.color} strokeWidth={2} />
248     </div>
249     <div>
250       <div style={{ fontSize: 24, fontWeight: 700, color: "var(--text-primary)", lineHeight: 1.2 }}>
251         {card.value}
252       </div>
253       <div style={{ fontSize: 12, color: "var(--text-muted)", marginTop: 2 }}>{card.label}</div>
254     </div>
255   </motion.div>
256 )}}
257 </div>
258
259 /* ■■ Filtered Charts Container ■■ */
260 <motion.div
261   initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.28 }}
262   className="dashboard-card dashboard-section-gap"
263   style={{ padding: "16px 20px 20px" }}
264 >
265   {/* Toolbar */}
266   <div className="req-toolbar" style={{ marginBottom: 16, boxShadow: "none", border: "none", padding: 0, background: "t
267     <div className="req-toolbar__filters req-toolbar__filters--open" style={{ borderTop: "none", paddingTop: 0 }}>
268       <div className="req-toolbar__filter-group">
269         <label className="req-toolbar__filter-label">Date</label>
270         <FilterSelect
271           value={dateMode}
272           onChange={setDateMode}
273           options={[
274             { value: "preset", label: "Preset" },
275             { value: "single", label: "Specific Date" },
276             { value: "range", label: "Date Range" },
277           ]}
278           defaultValue="preset"
279         />
280       </div>
281
282       {dateMode === "preset" && (
283         <div className="req-toolbar__filter-group">
284           <label className="req-toolbar__filter-label">Period</label>
285           <FilterSelect
286             value={preset}
287             onChange={setPreset}
288             options={[
289               { value: "all", label: "All Time" },
290               { value: "today", label: "Today" },
291               { value: "thisWeek", label: "This Week" },
292               { value: "thisMonth", label: "This Month" },
293               { value: "thisYear", label: "This Year" },
294             ]}
295           />
296         </div>
297       )}
298
299       {dateMode === "single" && (
300         <div className="req-toolbar__filter-group">
301           <label className="req-toolbar__filter-label">Date</label>

```

```
302         <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}
303     </div>
304 }}
305
306 {dateMode === "range" && (
307     <>
308         <div className="req-toolbar__filter-group">
309             <label className="req-toolbar__filter-label">From</label>
310             <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}
311         </div>
312         <div className="req-toolbar__filter-group">
313             <label className="req-toolbar__filter-label">To</label>
314             <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}
315         </div>
316     </>
317 )}
318
319 <button
320     type="button"
321     className="req-toolbar__reset-btn"
322     onClick={() => {
323         setDateMode("preset");
324         setPreset("all");
325         setSingleDate("");
326         setStartDate("");
327         setEndDate("");
328         load({ withToast: true });
329     }}
330     disabled={refreshing}
331     title="Reset filters and refresh"
332     style={{ alignSelf: "flex-start" }}
333 >
334     <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
335 </button>
336 </div>
337 </div>
338
339 {/* Charts */}
340 <div className="responsive-grid-2">
341     {/* Donut Chart - Status Distribution */}
342     <div className="admin-chart-card dashboard-chart-card" style={{ boxShadow: "none", border: "1px solid var(--border)
343         <div className="dashboard-chart-title">Request Status Distribution</div>
344         <div className="dashboard-chart-subtitle">Current active requests by status</div>
345         {statusData.length > 0 ? (
346             <div className="dashboard-chart-canvas">
347                 <ResponsiveContainer width="100%" height="100%">
348                     <PieChart margin={{ top: 8, right: 12, bottom: isCompact ? 32 : 8, left: 12 }}>
349                         <Pie
350                             data={statusData}
351                             cx={isCompact ? "50%" : "36%"}
352                             cy={isCompact ? "45%" : "50%"}
353                             innerRadius={isCompact ? 45 : 55}
354                             outerRadius={isCompact ? 68 : 85}
355                             paddingAngle={3}
356                             dataKey="value"
357                         >
358                             {statusData.map((entry, index) => (
359                                 <Cell key={`cell-${index}`} fill={CHART_COLORS[entry.id] || DONUT_COLORS[index % DONUT_COLORS.length]
360                             ))}
361                         </Pie>
362                     <Tooltip formatter={(value, name) => [value, name]} />
363                     <Legend
364                         iconType="circle"
365                         iconSize={8}
366                         layout={isCompact ? "horizontal" : "vertical"}
367                         align={isCompact ? "center" : "right"}
368                         verticalAlign={isCompact ? "bottom" : "middle"}
369                         wrapperStyle={{ fontSize: 12, paddingLeft: isCompact ? 0 : 24 }}
370                         formatter={(value) => {
371                             const total = statusData.reduce((sum, d) => sum + d.value, 0);
372                             const entry = statusData.find((d) => d.name === value);
373                             const pct = entry && total > 0 ? ((entry.value / total) * 100).toFixed(1) : 0;
374                             return `${value} (${pct}%)`;
375                         }}
376                     </Legend>
377                 </PieChart>
```



```

378         </ResponsiveContainer>
379     </div>
380     ) : (
381         <div className="dashboard-chart-canvas dashboard-chart-empty">No data available</div>
382     )}
383 </div>
384
385 /* Bar Chart - Document Types */
386 <div className="admin-chart-card dashboard-chart-card" style={{ boxShadow: "none", border: "1px solid var(--border)" }}>
387     <div className="dashboard-chart-title">Document Types</div>
388     <div className="dashboard-chart-subtitle">NDA vs Agreement requests submitted</div>
389     <div className="dashboard-chart-canvas">
390         <ResponsiveContainer width="100%" height="100%">
391             <BarChart data={typeData} barSize={40}>
392                 <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
393                 <XAxis dataKey="name" tick={{ fontSize: 12, fill: "var(--text-secondary)" }} axisLine={false} tickLine={false} />
394                 <YAxis tick={{ fontSize: 12, fill: "var(--text-muted)" }} axisLine={false} tickLine={false} allowDecimals={false} />
395                 <Tooltip cursor={{ fill: "var(--primary-light)" }} />
396                 <Bar dataKey="count" name="Requests" fill="var(--primary)" radius={[6, 6, 0, 0]} />
397             </BarChart>
398         </ResponsiveContainer>
399     </div>
400 </div>
401 </div>
402 </motion.div>
403
404 /* Monthly Trend Line Chart */
405 <motion.div
406     initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.49 }}
407     className="admin-chart-card dashboard-chart-card dashboard-section-gap">
408 >
409     <div className="dashboard-chart-title">Monthly Trend</div>
410     <div className="dashboard-chart-subtitle">Request volume over the last 6 months</div>
411     <div className="dashboard-chart-canvas">
412         <ResponsiveContainer width="100%" height="100%">
413             <LineChart data={trendData}>
414                 <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
415                 <XAxis dataKey="month" tick={{ fontSize: 12, fill: "var(--text-secondary)" }} axisLine={false} tickLine={false} />
416                 <YAxis tick={{ fontSize: 12, fill: "var(--text-muted)" }} axisLine={false} tickLine={false} allowDecimals={false} />
417                 <Tooltip />
418                 <Legend wrapperStyle={{ fontSize: 12 }} />
419                 <Line type="monotone" dataKey="total" stroke="#3b82f6" strokeWidth={2} dot={{ r: 3 }} name="Total" />
420                 <Line type="monotone" dataKey="approved" stroke="#10b981" strokeWidth={2} dot={{ r: 3 }} name="Approved" />
421                 <Line type="monotone" dataKey="pending" stroke="#f59e0b" strokeWidth={2} dot={{ r: 3 }} name="Pending" />
422             </LineChart>
423         </ResponsiveContainer>
424     </div>
425 </motion.div>
426
427 /* ■■ Recent Audit Logs (admin-only) ■■ */
428 {user?.role === "admin" ? (
429     <motion.div
430         initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.56 }}
431         className="dashboard-audit-card">
432     >
433         <div className="dashboard-audit-header">
434             <div className="dashboard-audit-header-title">
435                 <span className="dashboard-audit-title-text">Most Recent Activity</span>
436             </div>
437             <Link to="/admin/audit" className="dashboard-view-all-link">
438                 View All <ArrowRight size={13} />
439             </Link>
440         </div>
441         <div className="dashboard-audit-list">
442             {auditLogs.length === 0 ? (
443                 <div className="dashboard-audit-empty">
444                     No activity yet
445                 </div>
446             ) : (
447                 auditLogs.map((log, idx) => (
448                     <div
449                         key={log._id}
450                         className={`dashboard-audit-item ${idx < auditLogs.length - 1 ? "dashboard-audit-item--bordered" : ""}`}
451                     >
452                         <span className="dashboard-audit-action">
453                             {formatAction(log.action)}

```

```
454         </span>
455         <span className="dashboard-audit-meta">
456             {log.userId?.name || "System"}
457             {" . "}
458             {new Date(log.createdAt).toLocaleString("en-US", { month: "short", day: "numeric", hour: "numeric", minute: "numeric" })}
459         </span>
460     </div>
461 ))
462 }}
463 </div>
464 </motion.div>
465 ) : null}
466
467 </div>
468 );
469 }
470
471
```

59. client/src/pages/admin/AdminProfile.jsx (314 lines)

```

1 import { useContext, useRef, useState } from "react";
2 import { Eye, EyeOff } from "lucide-react";
3 import { motion, AnimatePresence } from "framer-motion";
4 import { AuthContext } from "../../../context/AuthContext";
5 import { updateProfile, requestPasswordChangeOtp } from "../../../services/authService";
6 import PasswordChecklist, { ErrorTooltip } from "../../../components/PasswordChecklist";
7 import { isStrongPassword } from "../../../utils/passwordPolicy";
8 import { notify } from "../../../utils/inPageFeedback";
9 import "../../../components/RequestForm.css";
10 import "../../../components/FloatingLabel.css";
11
12 const AdminProfile = () => {
13   const { user, updateUser } = useContext(AuthContext);
14   const [firstName, setFirstName] = useState(user?.firstName || "");
15   const [middleName, setMiddleName] = useState(user?.middleInitial || "");
16   const [lastName, setLastName] = useState(user?.lastName || "");
17   const [currentPassword, setCurrentPassword] = useState("");
18   const [newPassword, setNewPassword] = useState("");
19   const [confirmPassword, setConfirmPassword] = useState("");
20   const [loading, setLoading] = useState(false);
21   const [showCurrentPw, setShowCurrentPw] = useState(false);
22   const [showNewPw, setShowNewPw] = useState(false);
23   const [showConfirmPw, setShowConfirmPw] = useState(false);
24   const [confirmName, setConfirmName] = useState(false);
25   const [confirmPw, setConfirmPw] = useState(false);
26   const [otp, setOtp] = useState("");
27   const [otpCooldown, setOtpCooldown] = useState(0);
28
29   const [newPwFocused, setNewPwFocused] = useState(false);
30   const [confirmPwFocused, setConfirmPwFocused] = useState(false);
31
32   // Refs for tooltip anchoring
33   const newPwAnchorRef = useRef(null);
34   const confirmPwAnchorRef = useRef(null);
35   const cooldownRef = useRef(null);
36
37   const initials = user?.name
38     ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
39     : "?";
40
41   const startCooldown = (seconds) => {
42     setOtpCooldown(seconds);
43     clearInterval(cooldownRef.current);
44     cooldownRef.current = setInterval(() => {
45       setOtpCooldown((s) => {
46         if (s <= 1) { clearInterval(cooldownRef.current); return 0; }
47         return s - 1;
48       });
49     }, 1000);
50   };
51
52   const handleNameSubmit = (e) => {
53     e.preventDefault();
54     if (!firstName.trim() || !lastName.trim()) {
55       notify("Failed.", { type: "error", title: "Failed" });
56       return;
57     }
58     const hasChanges =
59       firstName.trim() !== (user?.firstName || "") ||
60       middleName.trim() !== (user?.middleInitial || "") ||
61       lastName.trim() !== (user?.lastName || "");
62     if (!hasChanges) {
63       notify("No changes to save.", { type: "warning", title: "Notice" });
64       return;
65     }
66     setConfirmName(true);
67   };
68
69   const handleConfirmNameUpdate = async () => {
70     setLoading(true);
71     try {
72       const data = await updateProfile({ firstName: firstName.trim(), middleInitial: middleName.trim(), lastName: lastName.trim() });
73       updateUser(data.user);

```

```
74     notify("Name updated successfully.", { type: "success", title: "Success" });
75     setConfirmName(false);
76   } catch (err) {
77     notify("Failed.", { type: "error", title: "Failed" });
78     setConfirmName(false);
79   } finally {
80     setLoading(false);
81   }
82 };
83
84 const handlePasswordSubmit = async (e) => {
85   e.preventDefault();
86   if (!currentPassword || !isStrongPassword(newPassword) || newPassword !== confirmPassword) return;
87   setLoading(true);
88   try {
89     await requestPasswordChangeOtp();
90     setOtp("");
91     startCooldown(60);
92     setConfirmPw(true);
93   } catch (err) {
94     const data = err?.response?.data;
95     if (err?.response?.status === 429 && data?.retryAfterSeconds) {
96       startCooldown(data.retryAfterSeconds);
97       setConfirmPw(true);
98     } else {
99       notify(data?.message || "Failed to send OTP.", { type: "error", title: "Error" });
100     }
101   } finally {
102     setLoading(false);
103   }
104 };
105
106 const handleResendOtp = async () => {
107   if (otpCooldown > 0) return;
108   setLoading(true);
109   try {
110     await requestPasswordChangeOtp();
111     setOtp("");
112     startCooldown(60);
113     notify("A new OTP has been sent to your email.", { type: "info", title: "OTP Sent" });
114   } catch (err) {
115     const data = err?.response?.data;
116     if (err?.response?.status === 429 && data?.retryAfterSeconds) {
117       startCooldown(data.retryAfterSeconds);
118     }
119     notify(data?.message || "Failed to resend OTP.", { type: "error", title: "Error" });
120   } finally {
121     setLoading(false);
122   }
123 };
124
125 const handleConfirmPasswordUpdate = async () => {
126   if (!otp.trim()) { notify("Please enter the OTP sent to your email.", { type: "warning" }); return; }
127   setLoading(true);
128   try {
129     await updateProfile({ currentPassword, newPassword, otp: otp.trim() });
130     setCurrentPassword("");
131     setNewPassword("");
132     setConfirmPassword("");
133     setOtp("");
134     setConfirmPwFocused(false);
135     setConfirmPw(false);
136     notify("Password changed successfully.", { type: "success", title: "Success" });
137   } catch (err) {
138     notify(err?.response?.data?.message || "Failed to change password.", { type: "error", title: "Failed" });
139   } finally {
140     setLoading(false);
141   }
142 };
143
144 const pwToggleStyle = {
145   position: "absolute", right: 6, top: "50%", transform: "translateY(-50%)",
146   background: "none", border: "none", padding: 4, cursor: "pointer",
147   color: "var(--text-muted)", display: "flex", alignItems: "center",
148 };
149
```

```
150 return (
151   <div style={{ display: "flex", justifyContent: "center" }}>
152     <div style={{ maxWidth: 520, width: "100%" }}>
153
154       {/* Profile Header */}
155       <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
156         <div style={{ width: 72, height: 72, borderRadius: "50%", background: "var(--primary)", display: "flex", alignItems: "center", justify-content: "center" }}>
157           <p style={{ margin: "0 0 4px", fontSize: 18, fontWeight: 700, color: "var(--text-primary)" }}>{user?.name}</p>
158           <p style={{ margin: "0 0 8px", fontSize: 13, color: "var(--text-muted)" }}>{user?.email}</p>
159           <span style={{ padding: "3px 12px", borderRadius: 99, fontSize: 11, fontWeight: 700, background: "#e0e7ff", color: "var(--text-primary)" }}>Edit</span>
160         </div>
161
162       {/* Change Name */}
163       <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
164         <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Name</h3>
165         <form onSubmit={handleNameSubmit} style={{ display: "flex", flexDirection: "column", gap: 10 }}>
166           <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
167             <div className="fl-wrap">
168               <input placeholder=" " value={firstName} onChange={(e) => setFirstName(e.target.value)} className="fl-input request-input" />
169               <label className="fl-label">First Name</label>
170             </div>
171             <div className="fl-wrap">
172               <input placeholder=" " value={middleName} onChange={(e) => setMiddleName(e.target.value)} className="fl-input request-input" />
173               <label className="fl-label">Middle Name</label>
174             </div>
175             <div className="fl-wrap">
176               <input placeholder=" " value={lastName} onChange={(e) => setLastName(e.target.value)} className="fl-input request-input" />
177               <label className="fl-label">Last Name</label>
178             </div>
179           </div>
180           <div style={{ display: "flex", justifyContent: "flex-end" }}>
181             <button type="submit" className="ui-btn ui-btn--primary" disabled={loading}>Save Changes</button>
182           </div>
183         </form>
184       </div>
185
186       {/* Change Password */}
187       <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", padding: 10px }}>
188         <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Password</h3>
189         <form onSubmit={handlePasswordSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
190
191           {/* Current Password */}
192           <div className="fl-wrap">
193             <input
194               type={showCurrentPw ? "text" : "password"}
195               placeholder=" "
196               value={currentPassword}
197               onChange={(e) => setCurrentPassword(e.target.value)}
198               className="fl-input request-input"
199               style={{ paddingRight: 42 }}
200             />
201             <label className="fl-label fl-label--pw">Current Password</label>
202             <button type="button" onClick={() => setShowCurrentPw((v) => !v)} tabIndex={-1} aria-label={showCurrentPw ? "Hide" : "Show"}>
203               {showCurrentPw ? <Eye size={18} /> : <EyeOff size={18} />}
204             </button>
205           </div>
206
207           {/* New Password */}
208           <div className="fl-wrap" ref={newPwAnchorRef}>
209             <input
210               type={showNewPw ? "text" : "password"}
211               placeholder=" "
212               value={newPassword}
213               onChange={(e) => setNewPassword(e.target.value)}
214               onFocus={() => setNewPwFocused(true)}
215               onBlur={() => setNewPwFocused(false)}
216               minLength={8}
217               className="fl-input request-input"
218               style={{ paddingRight: 42 }}
219             />
220             <label className="fl-label fl-label--pw">New Password</label>
221             <button type="button" onClick={() => setShowNewPw((v) => !v)} tabIndex={-1} aria-label={showNewPw ? "Hide" : "Show"}>
222               {showNewPw ? <Eye size={18} /> : <EyeOff size={18} />}
223             </button>
224             <PasswordChecklist focused={!isStrongPassword(newPassword) && (newPwFocused || newPassword.length > 0)} anchorRef={newPwAnchorRef}>
225           </div>
```

```

226
227     /* Confirm Password */
228     <div className="fl-wrap" ref={confirmPwAnchorRef}>
229         <input
230             type={showConfirmPw ? "text" : "password"}
231             placeholder=" "
232             value={confirmPassword}
233             onChange={(e) => setConfirmPassword(e.target.value)}
234             onFocus={() => setConfirmPwFocused(true)}
235             onBlur={() => setConfirmPwFocused(false)}
236             className="fl-input request-input"
237             style={{ paddingRight: 42 }}
238         />
239         <label className="fl-label fl-label--pw">Confirm New Password</label>
240         <button type="button" onClick={() => setShowConfirmPw((v) => !v)} tabIndex={-1} aria-label={showConfirmPw ? "Hide" : "Show"}>
241             {showConfirmPw ? <Eye size={18} /> : <EyeOff size={18} />}
242         </button>
243         <ErrorTooltip
244             show={confirmPassword !== newPassword && (confirmPwFocused || confirmPassword.length > 0)}
245             message="Passwords do not match"
246             anchorRef={confirmPwAnchorRef}
247             side="right"
248             persistent
249         />
250     </div>
251
252     <button type="submit" className="ui-btn ui-btn--primary" disabled={loading} style={{ alignSelf: "flex-end" }}>
253         {loading ? "Sending OTP..." : "Save Changes"}
254     </button>
255 </form>
256 </div>
257 </div>
258
259 /* Confirm Name Change Modal */
260 <AnimatePresence>
261     {confirmName && (
262         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
263             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
264                 <h3 className="modal-title">Confirm Change Name</h3>
265                 <p className="modal-text">Are you sure you want to update your name?</p>
266                 <p className="modal-text"><strong>{firstName.trim()} {middleName.trim()} {lastName.trim()}</strong> ? middleName.trim() + " " : ""</p>
267                 <div className="modal-actions">
268                     <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmName(false)} disabled={loading}>Cancel</button>
269                     <button className="ui-btn ui-btn--primary" onClick={handleConfirmNameUpdate} disabled={loading}>{loading ? "Saving" : "Save"}</button>
270                 </div>
271             </motion.div>
272         </motion.div>
273     )}
274 </AnimatePresence>
275
276 /* Confirm Password Change Modal */
277 <AnimatePresence>
278     {confirmPw && (
279         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
280             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
281                 <h3 className="modal-title">Verify Your Identity</h3>
282                 <p className="modal-text">An OTP has been sent to <strong>{user?.email}</strong>. Enter it below to confirm your identity.</p>
283                 <div className="fl-wrap" style={{ marginBottom: 4 }}>
284                     <input
285                         placeholder=" "
286                         value={otp}
287                         onChange={(e) => setOtp(e.target.value)}
288                         className="fl-input request-input"
289                         maxLength={6}
290                         autoFocus
291                     />
292                     <label className="fl-label">One-Time Password</label>
293                 </div>
294                 <button
295                     type="button"
296                     onClick={handleResendOtp}
297                     disabled={otpCooldown > 0 || loading}
298                     style={{ fontSize: 12, color: otpCooldown > 0 ? "var(--text-muted)" : "var(--primary)", background: "none", border: "1px solid #ccc" }}
299                 >
300                     {otpCooldown > 0 ? `Resend OTP in ${otpCooldown}s` : "Resend OTP"}
301                 </button>

```

```
302         <div className="modal-actions">
303             <button className="ui-btn ui-btn--secondary" onClick={() => { setConfirmPw(false); setOtp(""); }} disabled={loading} >Cancel</button>
304             <button className="ui-btn ui-btn--primary" onClick={handleConfirmPasswordUpdate} disabled={loading} >Confirm</button>
305         </div>
306     </motion.div>
307 </motion.div>
308 )}
309 </AnimatePresence>
310 </div>
311 );
312 };
313
314 export default AdminProfile;
```

60. client/src/pages/admin/AdminUsers.jsx (474 lines)

```

1 import { useCallback, useEffect, useMemo, useRef, useState } from "react";
2 import { motion, AnimatePresence } from "framer-motion";
3 import { UserPlus, RefreshCw, Search, X, Eye, EyeOff } from "lucide-react";
4 import FilterSelect from "../../components/FilterSelect";
5 import {
6   getAllUsers,
7   adminCreateUser,
8   adminUpdateUser,
9 } from "../../services/authService";
10 import PasswordChecklist from "../../components/PasswordChecklist";
11 import "../../components/FloatingLabel.css";
12 import { isStrongPassword } from "../../utils/passwordPolicy";
13 import { notify } from "../../utils/inPageFeedback";
14
15 const ROLE_LABELS = { student: "Student", admin: "Admin", staff: "Staff" };
16
17 const ROLE_COLORS = {
18   admin: { bg: "#e0e7ff", color: "#3730a3" },
19   staff: { bg: "#d1fae5", color: "#065f46" },
20   student: { bg: "#f1f5f9", color: "#475569" },
21 };
22
23 export default function AdminUsers() {
24   const [users, setUsers] = useState([]);
25   const [loading, setLoading] = useState(true);
26   const [refreshing, setRefreshing] = useState(false);
27   const [search, setSearch] = useState("");
28   const [roleFilter, setRoleFilter] = useState("all");
29   const [statusFilter, setStatusFilter] = useState("all");
30   const [showCreate, setShowCreate] = useState(false);
31   const [creating, setCreating] = useState(false);
32   const [actionId, setActionId] = useState(null);
33   const [editingUser, setEditingUser] = useState(null);
34   const [editForm, setEditForm] = useState({ role: "student", isActive: true });
35
36   const [form, setForm] = useState({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" });
37   const [showCreatePw, setShowCreatePw] = useState(false);
38   const [createPwFocused, setCreatePwFocused] = useState(false);
39   const createPwAnchorRef = useRef(null);
40   const [confirmCreate, setConfirmCreate] = useState(false);
41   const [confirmEdit, setConfirmEdit] = useState(false);
42
43   const load = useCallback(async ({ withSpinner = false } = {}) => {
44     setLoading(true);
45     if (withSpinner) setRefreshing(true);
46     try {
47       const data = await getAllUsers();
48       setUsers(data);
49     } catch {
50       // silently fail on load
51     } finally {
52       setLoading(false);
53       setRefreshing(false);
54     }
55   }, []);
56
57   useEffect(() => { load(); }, [load]);
58
59   const handleCreate = (e) => {
60     e.preventDefault();
61     if (!form.firstName.trim() || !form.lastName.trim() || !form.email.trim()) return;
62     if (!isStrongPassword(form.password)) return;
63     setConfirmCreate(true);
64   };
65
66   const handleConfirmCreate = async () => {
67     setCreating(true);
68     try {
69       const mi = form.middleName.trim() ? form.middleName.trim().charAt(0).toUpperCase() + "." : "";
70       const composedName = `${form.firstName.trim()}${mi ? " " + mi : ""} ${form.lastName.trim()}`;
71       await adminCreateUser({ name: composedName, email: form.email, role: form.role, password: form.password });
72       notify("User created successfully.", { type: "success", title: "Success" });
73       setForm({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" });

```



```

74     setShowCreatePw(false);
75     setShowCreate(false);
76     setConfirmCreate(false);
77     await load();
78   } catch (err) {
79     notify(err.response?.data?.message || "Failed to create user.", { type: "error", title: "Failed" });
80     setConfirmCreate(false);
81   } finally {
82     setCreating(false);
83   }
84 };
85
86 const openEditModal = (user) => {
87   setEditingUser(user);
88   setEditForm({ role: user.role || "student", isActive: user.isActive !== false });
89 };
90
91 const handleSaveEdit = async () => {
92   if (!editingUser) return;
93   setActionId(editingUser._id);
94   try {
95     await adminUpdateUser(editingUser._id, { role: editForm.role, isActive: editForm.isActive });
96     notify("User updated successfully.", { type: "success", title: "Success" });
97     setEditingUser(null);
98     await load();
99   } catch (err) {
100     notify(err.response?.data?.message || "Failed to update user.", { type: "error", title: "Failed" });
101   } finally {
102     setActionId(null);
103   }
104 };
105
106 const resetAndRefresh = () => {
107   setSearch("");
108   setRoleFilter("all");
109   setStatusFilter("all");
110   load({ withSpinner: true });
111 };
112
113 const filtered = useMemo(() => {
114   const normalized = search.toLowerCase();
115   let result = users.filter(
116     (u) =>
117       u.name?.toLowerCase().includes(normalized) ||
118       u.email?.toLowerCase().includes(normalized)
119   );
120   if (roleFilter !== "all") result = result.filter((u) => u.role === roleFilter);
121   if (statusFilter === "active") result = result.filter((u) => u.isActive !== false);
122   if (statusFilter === "inactive") result = result.filter((u) => u.isActive === false);
123   return result;
124 }, [search, roleFilter, statusFilter, users]);
125
126 return (
127   <div className="page-shell">
128     <div className="page-shell__header">
129       <button
130         type="button"
131         onClick={() => { setForm({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" }) }}
132         className="req-toolbar__action-btn"
133       >
134         <UserPlus size={14} />
135         Create User
136       </button>
137     </div>
138
139     { /* ■■ Toolbar ■■ */ }
140     <div className="req-toolbar">
141       { /* Search row */ }
142       <div className="req-toolbar__search-row">
143         <div className="req-toolbar__search-box">
144           <input
145             type="text"
146             className="req-toolbar__search-input"
147             placeholder="Search by name or email..."
148             value={search}
149             onChange={(e) => setSearch(e.target.value)}

```

```

150     />
151     {search ? (
152       <button type="button" className="req-toolbar__search-clear" onClick={() => setSearch("")} aria-label="Clear sea
153       x
154       </button>
155     ) : null}
156     <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
157       <Search size={14} />
158     </span>
159   </div>
160 </div>
161
162   /* Filter row */
163   <div className="req-toolbar__filters req-toolbar__filters--open">
164     <div className="req-toolbar__filter-group">
165       <label className="req-toolbar__filter-label">Role</label>
166       <FilterSelect
167         value={roleFilter}
168         onChange={setRoleFilter}
169         options={[
170           { value: "all", label: "All Roles" },
171           { value: "student", label: "Student" },
172           { value: "staff", label: "Staff" },
173           { value: "admin", label: "Admin" },
174         ]}
175       />
176     </div>
177
178     <div className="req-toolbar__filter-group">
179       <label className="req-toolbar__filter-label">Status</label>
180       <FilterSelect
181         value={statusFilter}
182         onChange={setStatusFilter}
183         options={[
184           { value: "all", label: "All Statuses" },
185           { value: "active", label: "Active" },
186           { value: "inactive", label: "Inactive" },
187         ]}
188       />
189     </div>
190
191     <button
192       type="button"
193       className="req-toolbar__reset-btn"
194       onClick={resetAndRefresh}
195       disabled={refreshing}
196       title="Reset filters and refresh"
197     >
198       <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
199     </button>
200   </div>
201 </div>
202
203   /* Create User Modal */
204   <AnimatePresence>
205     {showCreate && (
206       <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}
207       onClick={(e) => { if (e.target === e.currentTarget) { setShowCreate(false); setShowCreatePw(false); setForm({ nam
208       >
209         <motion.div className="user-edit-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y:
210
211         /* Title row */
212         <div className="user-edit-modal__title-row">
213           <span className="user-edit-modal__title">Create User</span>
214           <button
215             type="button"
216             className="user-edit-modal__close user-edit-modal__close--danger"
217             onClick={() => { setShowCreate(false); setShowCreatePw(false); setForm({ name: "", email: "", role: "studen
218             aria-label="Close"
219           >
220             <X size={16} />
221           </button>
222         </div>
223
224         <div className="user-edit-modal__divider" />
225

```

```

226      { /* Form */
227      <form onSubmit={handleCreate}>
228        <div className="user-edit-modal__fields" style={{ gridTemplateColumns: "1fr" }}>
229          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
230            <div className="user-edit-modal__field fl-wrap">
231              <input className="fl-input ui-input" placeholder=" " value={form.firstName} onChange={(e) => setForm((p
232              <label className="fl-label">First Name</label>
233            </div>
234            <div className="user-edit-modal__field fl-wrap">
235              <input className="fl-input ui-input" placeholder=" " value={form.middleName} onChange={(e) => setForm((
236              <label className="fl-label">Middle Name</label>
237            </div>
238            <div className="user-edit-modal__field fl-wrap">
239              <input className="fl-input ui-input" placeholder=" " value={form.lastName} onChange={(e) => setForm((p
240              <label className="fl-label">Last Name</label>
241            </div>
242          </div>
243          <div className="user-edit-modal__field fl-wrap">
244            <input className="fl-input ui-input" type="email" placeholder=" " value={form.email} onChange={(e) => set
245            <label className="fl-label">Email</label>
246          </div>
247          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 14 }}>
248            <div className="user-edit-modal__field">
249              <select className="ui-input" value={form.role} onChange={(e) => setForm((p) => ({ ...p, role: e.target.
250              <option value="student">Student</option>
251              <option value="staff">Staff</option>
252              <option value="admin">Admin</option>
253            </select>
254          </div>
255          <div ref={createPwAnchorRef} className="user-edit-modal__field fl-wrap">
256            <input
257              className="fl-input ui-input"
258              type={showCreatePw ? "text" : "password"}
259              placeholder=" "
260              value={form.password}
261              onChange={(e) => setForm((p) => ({ ...p, password: e.target.value })))}
262              onFocus={() => setCreatePwFocused(true)}
263              onBlur={() => setCreatePwFocused(false)}
264              required
265              minLength={8}
266              style={{ paddingRight: 42 }}
267            />
268            <label className="fl-label fl-label--pw">Temporary Password</label>
269            <button type="button" onClick={() => setShowCreatePw((v) => !v)} tabIndex={-1} aria-label={showCreatePw
270              {showCreatePw ? <Eye size={18} /> : <EyeOff size={18} />}
271            </button>
272            <PasswordChecklist focused={!isStrongPassword(form.password) && (createPwFocused || form.password.length
273          </div>
274        </div>
275      </div>
276
277      { /* Footer */
278      <div className="user-edit-modal__footer" style={{ justifyContent: "flex-end" }}>
279        <button type="submit" className="ui-btn ui-btn--primary" disabled={creating}>
280          {creating ? "Creating..." : "Create User"}
281        </button>
282      </div>
283    </form>
284
285    </motion.div>
286  </motion.div>
287  )}
288 </AnimatePresence>
289
290 { /* Table */
291 <div className="dashboard-card">
292   <div className="table-scroll">
293     <table className="dashboard-table">
294       <thead>
295         <tr className="dashboard-table-title-row">
296           <th colspan={5}>
297             <div className="dashboard-table-title-wrap">
298               <span className="dashboard-table-title">Users</span>
299             </div>
300           </th>
301         </tr>

```

```

302     <tr>
303       {[ "Joined", "User", "Role", "Status"].map((h) => (
304         <th key={h}>{h}</th>
305       ))}
306     </tr>
307   </thead>
308   <tbody>
309     {loading ? (
310       [...Array(5)].map((_, i) => (
311         <tr key={`sk-${i}`}>
312           <td><span className="skeleton-block skeleton-text" /></td>
313           <td><span className="skeleton-block skeleton-text" /></td>
314           <td><span className="skeleton-block skeleton-pill" /></td>
315           <td><span className="skeleton-block skeleton-pill" /></td>
316           <td />
317         </tr>
318       ))
319     ) : filtered.length === 0 ? (
320       <tr>
321         <td colSpan={5}>
322           <div className="dashboard-empty">
323             <p className="dashboard-empty-title">No users found</p>
324             <p className="dashboard-empty-text">No accounts match the current filters.</p>
325           </div>
326         </td>
327       </tr>
328     ) : (
329       filtered.map((user, i) => {
330         const roleStyle = ROLE_COLORS[user.role] || ROLE_COLORS.student;
331         return (
332           <motion.tr
333             key={user._id}
334             initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.03 }}
335             className={user.isActive === false ? "admin-row-inactive" : ""}
336             onClick={() => openEditModal(user)}
337             style={{ cursor: "pointer" }}
338           >
339             <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
340               {user.createdAt ? new Date(user.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short"
341             </td>
342             <td>
343               <span style={{ fontWeight: 600 }}>{user.name}</span>
344               <span className="dashboard-subtext">{user.email}</span>
345             </td>
346             <td>
347               <span className="tag-pill" style={{ background: roleStyle.bg, color: roleStyle.color }}>
348                 {ROLE_LABELS[user.role] || user.role}
349               </span>
350             </td>
351             <td>
352               <span className="tag-pill" style={{
353                 background: user.isActive !== false ? "#d1fae5" : "#fee2e2",
354                 color: user.isActive !== false ? "#065f46" : "#991b1b",
355               }}>
356                 {user.isActive !== false ? "Active" : "Inactive"}
357               </span>
358             </td>
359             <td className="row-caret"></td>
360           </motion.tr>
361         );
362       })
363     )}
364   </tbody>
365 </table>
366 </div>
367 </div>
368
369
370 <AnimatePresence>
371   {editingUser && (() => {
372     const initials = (editingUser.name || "?").split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase();
373     const roleStyle = ROLE_COLORS[editingUser.role] || ROLE_COLORS.student;
374     return (
375       <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
376         <motion.div className="user-edit-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{
377

```

```

378      {/* Title row */}
379      <div className="user-edit-modal__title-row">
380        <span className="user-edit-modal__title">Edit User</span>
381        <button
382          type="button"
383          className="user-edit-modal__close user-edit-modal__close--danger"
384          onClick={() => setEditingUser(null)}
385          aria-label="Close"
386        >
387          <X size={16} />
388        </button>
389      </div>
390
391      {/* Profile header */}
392      <div className="user-edit-modal__header">
393        <div className="user-edit-modal__avatar" style={{ background: roleStyle.bg, color: roleStyle.color }}>
394          {initials}
395        </div>
396        <div className="user-edit-modal__identity">
397          <span className="user-edit-modal__name">{editingUser.name}</span>
398          <span className="user-edit-modal__email">{editingUser.email}</span>
399        </div>
400      </div>
401
402      {/* Divider */}
403      <div className="user-edit-modal__divider" />
404
405      {/* Editable fields */}
406      <div className="user-edit-modal__fields">
407        <div className="user-edit-modal__field">
408          <label className="user-edit-modal__label">Role</label>
409          <select className="ui-input" value={editForm.role} onChange={(e) => setEditForm((p) => ({ ...p, role: e.target.value })}>
410            <option value="student">Student</option>
411            <option value="staff">Staff</option>
412            <option value="admin">Admin</option>
413          </select>
414        </div>
415        <div className="user-edit-modal__field">
416          <label className="user-edit-modal__label">Status</label>
417          <select className="ui-input" value={editForm.isActive ? "active" : "inactive"} onChange={(e) => setEditForm((p) => ({ ...p, isActive: e.target.value === "active" })}>
418            <option value="active">Active</option>
419            <option value="inactive">Inactive</option>
420          </select>
421        </div>
422      </div>
423
424      {/* Footer */}
425      <div className="user-edit-modal__footer" style={{ justifyContent: "flex-end" }}>
426        <button className="ui-btn ui-btn--primary" onClick={() => setConfirmEdit(true)} disabled={actionId === editingUser._id ? "Saving..." : "Save Changes"}>
427          {actionId === editingUser._id ? "Saving..." : "Save Changes"}
428        </button>
429      </div>
430
431    </motion.div>
432  </motion.div>
433  );
434  }}())}
435 </AnimatePresence>
436
437 <AnimatePresence>
438   {confirmCreate && (
439     <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
440       <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
441         <h3 className="modal-title">Confirm Create User</h3>
442         <p className="modal-text">Are you sure you want to create this user?</p>
443         <p className="modal-text"><strong>{form.firstName.trim()} {form.middleName.trim() ? form.middleName.trim() + " " : ""}</strong></p>
444         <p className="modal-text"><strong>{form.email}</strong></p>
445         <div className="modal-actions">
446           <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmCreate(false)} disabled={creating}>Cancel</button>
447           <button className="ui-btn ui-btn--primary" onClick={handleConfirmCreate} disabled={creating}>{creating ? "Creating..." : "Create"}</button>
448         </div>
449       </motion.div>
450     </motion.div>
451   )}
452 </AnimatePresence>
453

```

```
454     <AnimatePresence>
455       {confirmEdit && editingUser && (
456         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
457           <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
458             <h3 className="modal-title">Confirm Save Changes</h3>
459             <p className="modal-text">Are you sure you want to save changes for this user?</p>
460             <p className="modal-text"><strong>{editingUser.name}</strong> &mdash; <strong>{ROLE_LABELS[editForm.role] || ed
461             <p className="modal-text"><strong>{editingUser.email}</strong></p>
462             <div className="modal-actions">
463               <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmEdit(false)} disabled={actionId === edi
464               <button className="ui-btn ui-btn--primary" onClick={() => { setConfirmEdit(false); handleSaveEdit(); }} disab
465             </div>
466           </motion.div>
467         </motion.div>
468       )}
469     </AnimatePresence>
470
471   </div>
472 );
473 }
474
```

61. client/src/pages/admin/AdminRequests.jsx (404 lines)

```

1 import { useEffect, useMemo, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { Filter, RefreshCw, Search } from "lucide-react";
4 import { getAllRequests } from "../../../services/requestService";
5 import FilterSelect from "../../../components/FilterSelect";
6
7 const statusClass = (s) => {
8   if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
9   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
10  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
11  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
12  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
13  if (s === "agr_declined") return "status-pill status-pill--red";
14  return "status-pill";
15 };
16
17 const prettyStatus = (s) => {
18   const map = {
19     nda_pending: "Reviewal",
20     nda_approved: "Approved",
21     stud_revision_requested: "Student Revisions",
22     agr_pending_1: "Initial Reviewal",
23     agr_awaiting_rep_signature: "Recipient Reviewal",
24     agr_pending_2: "Final Reviewal",
25     agr_approved: "Approved",
26     agr_declined: "Recipient Declined",
27     agr_rep_revision_requested: "Recipient Revisions",
28   };
29   return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
30 };
31
32 export default function AdminRequests() {
33   const navigate = useNavigate();
34   const [requests, setRequests] = useState([]);
35   const [loading, setLoading] = useState(true);
36   const [refreshing, setRefreshing] = useState(false);
37   const [error, setError] = useState("");
38   const [isFilterOpen, setIsFilterOpen] = useState(() => {
39     if (typeof window === "undefined") return true;
40     return window.innerWidth >= 768;
41   });
42
43   const [statusFilter, setStatusFilter] = useState("all");
44   const [typeFilter, setTypeFilter] = useState("all");
45   const [dateMode, setDateMode] = useState("preset");
46   const [preset, setPreset] = useState("all");
47   const [singleDate, setSingleDate] = useState("");
48   const [startDate, setStartDate] = useState("");
49   const [endDate, setEndDate] = useState("");
50   const [searchTerm, setSearchTerm] = useState("");
51
52   const buildFilterParams = () => {
53     const params = {};
54
55     if (statusFilter !== "all") {
56       params.status = statusFilter;
57     }
58
59     if (typeFilter !== "all") {
60       if (typeFilter === "agreement") {
61         params.type = "agreement";
62       } else if (typeFilter === "nda_orgactivities") {
63         params.type = "nda";
64         params.ndaType = "orgactivities";
65       } else if (typeFilter === "nda_research") {
66         params.type = "nda";
67         params.ndaType = "research";
68       }
69     }
70
71     if (dateMode === "preset") {
72       const now = new Date();

```

```
74     if (preset === "today") {
75         params.startDate = now.toISOString().slice(0, 10);
76         params.endDate = now.toISOString().slice(0, 10);
77     } else if (preset === "thisWeek") {
78         const day = now.getDay();
79         const monday = new Date(now);
80         monday.setDate(now.getDate() - ((day + 6) % 7));
81         params.startDate = monday.toISOString().slice(0, 10);
82     } else if (preset === "thisMonth") {
83         const start = new Date(now.getFullYear(), now.getMonth(), 1);
84         params.startDate = start.toISOString().slice(0, 10);
85     } else if (preset === "thisYear") {
86         const start = new Date(now.getFullYear(), 0, 1);
87         params.startDate = start.toISOString().slice(0, 10);
88     }
89 }
90
91 if (dateMode === "single" && singleDate) {
92     params.startDate = singleDate;
93     params.endDate = singleDate;
94 }
95
96 if (dateMode === "range") {
97     if (startDate) params.startDate = startDate;
98     if (endDate) params.endDate = endDate;
99 }
100
101 return params;
102 };
103
104 const loadRequests = async ({ withToast = false } = {}) => {
105     if (withToast) setRefreshing(true);
106     setError("");
107
108     try {
109         const data = await getAllRequests(buildFilterParams());
110         setRequests(data);
111     } catch (err) {
112         setError(err.response?.data?.message || "Failed to load requests");
113     } finally {
114         setLoading(false);
115         setRefreshing(false);
116     }
117 }
118
119
120 useEffect(() => {
121     loadRequests();
122     // eslint-disable-next-line react-hooks/exhaustive-deps
123 }, [statusFilter, typeFilter, dateMode, preset, singleDate, startDate, endDate]);
124
125
126 useEffect(() => {
127     const onResize = () => {
128         if (window.innerWidth >= 768) setIsFilterOpen(true);
129     };
130     window.addEventListener("resize", onResize);
131     return () => window.removeEventListener("resize", onResize);
132 }, []);
133
134 const filtered = useMemo(() => {
135     const APPROVED = ["nda_approved", "agr_approved"];
136     let result = requests;
137     if (searchTerm.trim()) {
138         const term = searchTerm.toLowerCase();
139         result = result.filter(r => {
140             const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
141             const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
142             const id = (r._id || "").slice(-7).toLowerCase();
143             return name.includes(term) || email.includes(term) || id.includes(term);
144         });
145     }
146     return [...result].sort((a, b) => {
147         const aApproved = APPROVED.includes(a.status) ? 1 : 0;
148         const bApproved = APPROVED.includes(b.status) ? 1 : 0;
149         return aApproved - bApproved;
150     });
151 });
```



```

150   }, [requests, searchTerm]);
151
152   const clearSearch = () => setSearchTerm("");
153
154   const resetAndRefresh = () => {
155     setSearchTerm("");
156     setTypeFilter("all");
157     setStatusFilter("all");
158     setDateMode("preset");
159     setPreset("all");
160     setSingleDate("");
161     setStartDate("");
162     setEndDate("");
163     loadRequests({ withToast: true });
164   };
165
166   return (
167     <div className="dashboard-page page-shell">
168
169       {error ? (
170         <div className="info-banner info-banner--danger" style={{ marginBottom: 12 }}>
171           <strong>{error}</strong>
172         </div>
173       ) : null}
174
175       {/ * ■■ Search + Filter toolbar ■■ */}
176       <div className="req-toolbar">
177         {/ * Search row */}
178         <div className="req-toolbar__search-row">
179           <div className="req-toolbar__search-box">
180             <input
181               type="text"
182               className="req-toolbar__search-input"
183               placeholder="Search by requestor, email, or request ID..."
184               value={searchTerm}
185               onChange={(e) => setSearchTerm(e.target.value)}
186             />
187             {searchTerm ? (
188               <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
189                 x
190               </button>
191             ) : null}
192             <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
193               <Search size={14} />
194             </span>
195           </div>
196           <button
197             type="button"
198             className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
199             onClick={() => setIsFilterOpen((prev) => !prev)}
200             aria-expanded={isFilterOpen}
201           >
202             <Filter size={13} />
203             Filters
204             {isFilterOpen ? null : <span className="req-toolbar__filter-badge" />}
205           </button>
206         </div>
207
208         {/ * Filter chips row */}
209         <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
210           <div className="req-toolbar__filter-group">
211             <label className="req-toolbar__filter-label">Type</label>
212             <FilterSelect
213               value={typeFilter}
214               onChange={setTypeFilter}
215               className="filter-select--type"
216               options={[
217                 { value: "all", label: "All Types" },
218                 { value: "nda_orgactivities", label: "NDA – Student Organization Activities" },
219                 { value: "nda_research", label: "NDA – Conduct of Research" },
220                 { value: "agreement", label: "Agreement" },
221               ]}
222             />
223           </div>
224
225           <div className="req-toolbar__filter-group">

```

```
226     <label className="req-toolbar__filter-label">Status</label>
227     <FilterSelect
228       value={statusFilter}
229       onChange={setStatusFilter}
230       className="filter-select--status"
231       options={[
232         { value: "all", label: "All Statuses" },
233         { value: "reviewal", label: "Reviewal" },
234         { value: "approved", label: "Approved" },
235         { value: "stud_revision", label: "Student Revisions" },
236         { value: "rep_revision", label: "Recipient Revisions" },
237         { value: "rep_declined", label: "Recipient Declined" },
238         { value: "awaiting_rep", label: "Recipient Reviewal" },
239       ]}
240     />
241   </div>
242
243   <div className="req-toolbar__filter-group">
244     <label className="req-toolbar__filter-label">Date</label>
245     <FilterSelect
246       value={dateMode}
247       onChange={setDateMode}
248       options={[
249         { value: "preset", label: "Preset" },
250         { value: "single", label: "Specific Date" },
251         { value: "range", label: "Date Range" },
252       ]}
253       defaultValue="preset"
254     />
255   </div>
256
257   {dateMode === "preset" ? (
258     <div className="req-toolbar__filter-group">
259       <label className="req-toolbar__filter-label">Period</label>
260       <FilterSelect
261         value={preset}
262         onChange={setPreset}
263         options={[
264           { value: "all", label: "All Time" },
265           { value: "today", label: "Today" },
266           { value: "thisWeek", label: "This Week" },
267           { value: "thisMonth", label: "This Month" },
268           { value: "thisYear", label: "This Year" },
269         ]}
270       />
271     </div>
272   ) : null}
273
274   {dateMode === "single" ? (
275     <div className="req-toolbar__filter-group">
276       <label className="req-toolbar__filter-label">Date</label>
277       <input
278         className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
279         type="date"
280         value={singleDate}
281         onChange={(e) => setSingleDate(e.target.value)}
282       />
283     </div>
284   ) : null}
285
286   {dateMode === "range" ? (
287     <>
288       <div className="req-toolbar__filter-group">
289         <label className="req-toolbar__filter-label">From</label>
290         <input
291           className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
292           type="date"
293           value={startDate}
294           onChange={(e) => setStartDate(e.target.value)}
295         />
296       </div>
297       <div className="req-toolbar__filter-group">
298         <label className="req-toolbar__filter-label">To</label>
299         <input
300           className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
301           type="date"
```

```

302         value={endDate}
303         onChange={(e) => setEndDate(e.target.value)}
304     />
305 </div>
306 </>
307 ) : null}
308
309 <button
310     type="button"
311     className="req-toolbar__reset-btn"
312     onClick={resetAndRefresh}
313     disabled={refreshing}
314     title="Reset filters and refresh"
315 >
316     <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
317 </button>
318 </div>
319 </div>
320
321 <div className="dashboard-card">
322     <div className="table-scroll">
323         <table className="dashboard-table">
324             <thead>
325                 <tr className="dashboard-table-title-row">
326                     <th colSpan={6}>
327                         <div className="dashboard-table-title-wrap">
328                             <span className="dashboard-table-title">Requests</span>
329                         </div>
330                     </th>
331                 </tr>
332                 <tr>
333                     <th>Request Date</th>
334                     <th>Requestor</th>
335                     <th>Request ID</th>
336                     <th>Type</th>
337                     <th>Status</th>
338                     <th />
339                 </tr>
340             </thead>
341
342             <tbody>
343                 {loading ? (
344                     [...Array(4)].map((_, i) => (
345                         <tr key={`sk-${i}`}>
346                             <td><span className="skeleton-block skeleton-text" /></td>
347                             <td><span className="skeleton-block skeleton-text" /></td>
348                             <td><span className="skeleton-block skeleton-text" /></td>
349                             <td><span className="skeleton-block skeleton-text" /></td>
350                             <td><span className="skeleton-block skeleton-pill" /></td>
351                             <td />
352                         </tr>
353                     ))
354                 ) : filtered.length === 0 ? (
355                     <tr>
356                         <td colSpan={6}>
357                             <div className="dashboard-empty">
358                                 <p className="dashboard-empty-title">No requests found</p>
359                                 <p className="dashboard-empty-text">No requests match the current filter.</p>
360                             </div>
361                         </td>
362                     </tr>
363                 ) : (
364                     filtered.map((r) => (
365                         <tr key={r._id} onClick={() => navigate(`/admin/requests/${r._id}`)} style={{ cursor: "pointer" }}>
366                             <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>{new Date(r.createdAt).toLocaleDateString()}
367
368                             <td>
369                                 <span style={{ fontWeight: 600 }}>
370                                     {r.proxyRequestee?.isProxy ? (r.proxyRequestee?.fullName || "Proxy Requestor") : (r.userId?.name || "Unknown")}
371                                 </span>
372                                 <span className="dashboard-subtext">
373                                     {r.proxyRequestee?.isProxy ? (r.proxyRequestee?.email || "") : (r.userId?.email || "")}
374                                 </span>
375                                 {r.proxyRequestee?.isProxy ? (
376                                     <span className="dashboard-subtext">Submitted by staff: {r.userId?.name || "Unknown"}</span>
377                                 ) : null}

```

```
378         </td>
379
380         <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
381
382         <td>
383             {r.type === "nda"
384               ? `NDA${r.formData?.ndaTypeLabel ? ` - ${r.formData.ndaTypeLabel}` : ""}`
385               : "Agreement"}
386         </td>
387
388         <td>
389             <span className={statusClass(r.status)}>
390                 {prettyStatus(r.status)}
391             </span>
392         </td>
393
394         <td className="row-caret"></td>
395     </tr>
396 ))
397 }}
398 </tbody>
399 </table>
400 </div>
401 </div>
402 </div>
403 );
404 }
```

62. client/src/pages/admin/AdminRequestReview.jsx (1054 lines)

```

1 import { useEffect, useRef, useState, useMemo, useContext } from "react";
2 import { useNavigate, useParams } from "react-router-dom";
3 import { AlertTriangle, Copy, Paperclip, RefreshCw } from "lucide-react";
4 import { AnimatePresence, motion } from "framer-motion";
5 import RequestStepper from "../../components/RequestStepper";
6 import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
7 import { AuthContext } from "../../context/AuthContext";
8
9 import {
10   getRequestById,
11   updateRequestStatus,
12   saveApprovedDocument,
13   generateSigningLink,
14   adminPhase3Action,
15   getSignatureImages,
16 } from "../../services/requestService";
17 import { confirmInPage, notify } from "../../utils/inPageFeedback";
18
19 import {
20   uploadApprovedForm,
21   uploadApprovedQrImage,
22   deleteStorageFile,
23   uploadSignatureImage,
24 } from "../../services/firebaseStorageService";
25 import { buildVerificationUrl, generateQrDataURL } from "../../services/qrService";
26 import SignaturePad from "../../components/SignaturePad";
27 import "../../components/RequestForm.css";
28
29 const statusPillClass = (s) => {
30   if ([ "nda_approved", "agr_approved" ].includes(s)) return "status-pill status-pill--green";
31   if ([ "nda_pending", "agr_pending_1", "agr_pending_2" ].includes(s)) return "status-pill status-pill--yellow";
32   if (s === "stud_revision_requested") return "status-pill status-pill--orange";
33   if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
34   if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
35   if (s === "agr_declined") return "status-pill status-pill--red";
36   return "status-pill";
37 };
38
39 const prettyStatus = (s) => {
40   const map = {
41     nda_pending:      "Reviewal",
42     nda_approved:    "Approved",
43     stud_revision_requested: "Student Revisions",
44     agr_pending_1:    "Initial Reviewal",
45     agr_awaiting_rep_signature: "Recipient Reviewal",
46     agr_pending_2:    "Final Reviewal",
47     agr_approved:     "Approved",
48     agr_declined:     "Recipient Declined",
49     agr_rep_revision_requested: "Recipient Revisions",
50   };
51   return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
52 };
53
54 const getRequestFolder = (predocs = []) => {
55   for (const r of predocs) {
56     if (r.path) {
57       const parts = String(r.path).split("/");
58       return parts[3] || "";
59     }
60   }
61   return "";
62 };
63
64 // ████████████████████████████████████████████████████████████████████████████████
65 // NDA review panel (flow: pending → approved | revision_requested)
66 // ████████████████████████████████████████████████████████████████████████████████
67 function NdaReviewPanel({ reqData, canProgress, onRequestUpdated }) {
68   const [ user ] = useContext(AuthContext);
69   const [ remarks, setRemarks ] = useState(reqData.remarks || "");
70   const [ activeAction, setActiveAction ] = useState(null); // "approve" | "revision" | null
71   const adminSigRef = useRef(null);
72
73   const cfg = useMemo(() => {

```

```
74     if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda?.[reqData.formData?.ndaType];
75     return null;
76 }, [reqData]);
77
78 const isPending = reqData?.status === "nda_pending";
79 const isRevision = reqData?.status === "stud_revision_requested";
80 const isApproved = reqData?.status === "nda_approved";
81 const [confirmAction, setConfirmAction] = useState(null); // "approve" | "revision" | null
82
83
84 const isProxy = reqData?.proxyRequestee?.isProxy;
85
86 const handleUpdate = async (status) => {
87   if (status === "nda_approved") {
88     if (!isProxy && (!adminSigRef.current || adminSigRef.current.isEmpty())) {
89       notify("Please draw your e-signature before approving.", { type: "warning" });
90       return;
91     }
92   }
93   if (status === "stud_revision_requested") {
94     if (!remarks.trim()) {
95       notify("Please enter remarks before requesting a revision.", { type: "warning" });
96       return;
97     }
98   }
99   setConfirmAction(null);
100  setActiveAction(status === "nda_approved" ? "approve" : "revision");
101  try {
102    if (status === "nda_approved") {
103      // For F2F proxy requests, skip digital signatures (wet signature will be applied on print)
104      if (isProxy) {
105        const updateRes = await updateRequestStatus(reqData._id, {
106          status,
107          remarks,
108        });
109        const updated = updateRes.request;
110        if (!updated?.serialNo) throw new Error("Serial number missing");
111        const verificationUrl = buildVerificationUrl(updated.serialNo);
112        const qrDataURL = await generateQrDataURL(verificationUrl);
113        const studentName = reqData.proxyRequestee?.fullName || "Proxy Requestor";
114        const requestFolder = getRequestFolder(reqData.predocs);
115        if (!requestFolder) throw new Error("Could not determine request folder");
116        await uploadApprovedQrImage(qrDataURL, reqData.type, studentName, requestFolder);
117        const docReq = {
118          ...updated,
119          userId: reqData.userId,
120          studentSigDataURL: null,
121          adminSigDataURL: null,
122          approverName: user?.name || "",
123          verificationUrl,
124          qrDataURL,
125        };
126        const { generateApprovedPDF } = await import("../config/documentTemplates");
127        const pdfBlob = await generateApprovedPDF(docReq);
128        const t = reqData.formData?.ndaType || "general";
129        const approvedFileName = `NDA_${t}_Approved_${Date.now()}.pdf`;
130        const uploaded = await uploadApprovedForm(
131          pdfBlob,
132          reqData.type,
133          studentName,
134          requestFolder,
135          approvedFileName
136        );
137        await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });
138        notify("Approved! The document is ready for printing and physical signature.", { type: "success", title: "Approved" });
139      } else {
140        const adminSigDataURL = adminSigRef.current.getDataUrl();
141
142        // Upload admin signature to Firebase
143        const studentName = reqData.userId?.name || "Unknown Student";
144        const requestFolder = getRequestFolder(reqData.predocs);
145
146        const { url: adminSigFirebaseUrl, path: adminSigFirebasePath } = await uploadSignatureImage(
147          adminSigDataURL,
148          "nda",
149          studentName,
```

```
150         requestFolder || "unknown",
151         "admin_sig.png"
152     );
153
154     const updateRes = await updateRequestStatus(reqData._id, {
155         status,
156         remarks,
157         adminSigUrl: adminSigFirebaseUrl,
158         adminSigPath: adminSigFirebasePath,
159     });
160     const updated = updateRes.request;
161
162     if (!updated?.serialNo) throw new Error("Serial number missing");
163
164     const verificationUrl = buildVerificationUrl(updated.serialNo);
165     const qrDataUrl = await generateQrDataUrl(verificationUrl);
166
167     if (!requestFolder) throw new Error("Could not determine request folder");
168
169     await uploadApprovedQrImage(qrDataUrl, reqData.type, studentName, requestFolder);
170
171     // Fetch signature images via proxy to avoid CORS
172     const sigImages = await getSignatureImages(reqData._id);
173
174     const docReq = {
175         ...updated,
176         userId: reqData.userId,
177         studentSigDataUrl: sigImages.studentSig,
178         adminSigDataUrl: sigImages.adminSig || adminSigDataUrl,
179         approverName: user?.name || "",
180         verificationUrl,
181         qrDataUrl,
182     };
183     const { generateApprovedPDF } = await import("../config/documentTemplates");
184     const pdfBlob = await generateApprovedPDF(docReq);
185     const t = reqData.formData?.ndaType || "general";
186     const approvedFileName = `NDA_${t}_Approved_${Date.now()}.pdf`;
187     const uploaded = await uploadApprovedForm(
188         pdfBlob,
189         reqData.type,
190         studentName,
191         requestFolder,
192         approvedFileName
193     );
194
195     await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });
196
197     notify("Approved and document generated!", { type: "success", title: "Approved" });
198 }
199 } else {
200     await updateRequestStatus(reqData._id, { status, remarks });
201     notify("Student revision requested.", { type: "success", title: "Revision Requested" });
202 }
203 await onRequestUpdated?.({ withToast: true });
204 } catch (err) {
205     notify(err.response?.data?.message || err.message || "Failed to update request", { type: "error" });
206 } finally {
207     setActiveAction(null);
208 }
209 };
210
211 return (
212     <>
213     <div className="review-section">
214         <h4 className="review-section-title">Form Data</h4>
215         {cfg?.fields?.length ? (
216             cfg.fields.map((f) => (
217                 <div key={f.name} className="review-field">
218                     <span className="review-field-label">{f.label}</span>
219                     <div className="review-info-box">
220                         {String(reqData.formData?.[f.name] ?? "") || (
221                             <span className="review-info-box--muted"></span>
222                         )}
223                     </div>
224                 </div>
225             ))
226         )
227     </div>
228 )
229 )
```

```

226     ) : (
227       <pre style={{
228         background: "var(--surface-2)",
229         padding: "10px",
230         borderRadius: "var(--radius-md)",
231         fontSize: "12px",
232         margin: 0,
233       }}>
234         {JSON.stringify(reqData.formData || {}, null, 2)}
235       </pre>
236     )}
237 </div>
238
239 <div className="review-section">
240   <h4 className="review-section-title">Attachments</h4>
241   {(() => {
242     const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
243     return initialDocs.length ? (
244       <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
245         {initialDocs.map((f, idx) => (
246           <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
247             <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${f.id}`}
248           </a>
249         ))}
250       </div>
251     ) : (
252       <div className="review-info-box">
253         <span className="review-info-box--muted">No files.</span>
254       </div>
255     );
256   })()}
257 </div>
258
259 {(() => {
260   const rounds = {};
261   (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
262     const r = f.resubmissionRound || 1;
263     if (!rounds[r]) rounds[r] = [];
264     rounds[r].push(f);
265   });
266   return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
267     <div key={round} className="review-section">
268       <h4 className="review-section-title">
269         Resubmitted Attachments{Object.keys(rounds).length > 1 ? ` (Round ${round})` : ""}
270       </h4>
271       <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
272         {rounds[round].map((f, idx) => (
273           <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
274             <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${f.id}`}
275           </a>
276         ))}
277       </div>
278     </div>
279   ));
280 })()}
281
282 {isRevision && (
283   <div className="review-section">
284     <h4 className="review-section-title">Remarks</h4>
285     <div className="review-info-box">
286       {reqData.remarks || <span className="review-info-box--muted">No remarks provided.</span>}
287     </div>
288   </div>
289 )}
290
291 {isApproved && (
292   <div className="review-section">
293     <h4 className="review-section-title">Approved Request Form</h4>
294     {reqData.postdocs?.url ? (
295       <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
296         <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved NDA Document
297       </a>
298     ) : (
299       <div className="review-info-box">
300         <span className="review-info-box--muted">No approved document uploaded.</span>
301       </div>

```



```
302     }}
303     {reqData.adminSignedAt && (
304         <span style={{ display: "block", marginTop: 6, fontSize: 11, color: "var(--text-muted)" }}>
305             Admin signed on {new Date(reqData.adminSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day: "numeric" })}
306         </span>
307     )}
308 </div>
309 }}
310
311 /* Student E-Signature */
312 {reqData.studentSigUrl && (
313     <div className="review-section">
314         <h4 className="review-section-title">Student E-Signature</h4>
315         <div className="review-sig-wrap">
316             <img src={reqData.studentSigUrl} alt="Student signature" className="review-sig-img" width="560" height="180" loading="lazy" />
317             <span className="review-sig-name">{reqData.userId?.name}</span>
318             {reqData.studentSignedAt && (
319                 <span style={{ fontSize: 11, color: "var(--text-muted)" }}>
320                     Signed on {new Date(reqData.studentSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day: "numeric" })}
321                 </span>
322             )}
323         </div>
324     </div>
325 )}
326
327 {canProgress && isPending && !isProxy && (
328     <div className="review-section">
329         <h4 className="review-section-title">Your E-Signature (Admin) *</h4>
330         <p style={{ fontSize: "13px", color: "var(--text-muted)", margin: "0 0 8px 0" }}>
331             Draw your signature below to sign off on the NDA approval.
332         </p>
333         <SignaturePad ref={adminSigRef} height={160} />
334     </div>
335 )}
336
337 {canProgress && isPending && isProxy && (
338     <div className="info-banner info-banner--info">
339         <strong>F2F Walk-in (Proxy)</strong>
340         <p>This is a face-to-face proxy request. Digital signatures are bypassed. The document will be printed for a physical signature.</p>
341     </div>
342 )}
343
344 {canProgress && isPending && (
345     <div className="review-section">
346         <h4 className="review-section-title">Remarks</h4>
347         <textarea
348             value={remarks}
349             onChange={(e) => setRemarks(e.target.value)}
350             rows={4}
351             className="review-textarea"
352         />
353     </div>
354 )}
355
356 {canProgress && isPending && (
357     <div className="review-actions">
358         <button
359             onClick={() => {
360                 if (!isProxy && (!adminSigRef.current || adminSigRef.current.isEmpty())) {
361                     notify("Please draw your e-signature before approving.", { type: "warning" });
362                 }
363                 return;
364             }}
365             setConfirmAction("approve");
366             disabled={!activeAction}
367             className="review-btn-primary"
368         >
369             {activeAction === "approve" ? "Generating..." : "Approve"}
370         </button>
371         <button
372             onClick={() => {
373                 if (!remarks.trim()) {
374                     notify("Please enter remarks before requesting a revision.", { type: "warning" });
375                 }
376                 return;
377             }}
378             setConfirmAction("revision");
```

```

378     }}
379     disabled={!activeAction}
380     className="review-btn-secondary"
381   >
382     {activeAction === "revision" ? "Submitting..." : "Request Student Revision"}
383   </button>
384 </div>
385 }}
386
387 <AnimatePresence>
388   {confirmAction && (
389     <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
390       <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
391         <h3 className="modal-title">
392           {confirmAction === "approve" ? "Approve NDA Request" : "Request Student Revision"}
393         </h3>
394         <p className="modal-text">
395           {confirmAction === "approve"
396             ? "This will finalize and generate the approved NDA document."
397             : "The student will be notified to revise and resubmit their request."}
398         </p>
399         <div className="modal-actions">
400           <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmAction(null)} disabled={!activeAction}>
401             <button className="ui-btn ui-btn--primary" onClick={() => handleUpdate(confirmAction === "approve" ? "nda_ap
402               {confirmAction === "approve" ? "Approve" : "Confirm"}
403             </button>
404           </div>
405         </motion.div>
406       </motion.div>
407     )}
408   </AnimatePresence>
409
410   {!canProgress && (
411     <div className="info-banner info-banner--info">
412       <strong>Read-only access</strong>
413       <p>Staff can view request details but only Admin can move, approve, or sign this request.</p>
414     </div>
415   )}
416 </>
417 );
418 }
419
420 // ████████████████████████████████████████████████████████████████████████████████████████████████████████████████
421 // Agreement review panel (multi-phase)
422 // ████████████████████████████████████████████████████████████████████████████████████████████████████████████████
423 function AgreementReviewPanel({ reqData, canProgress, onRequestUpdated }) {
424   const { user } = useContext(AuthContext);
425   const [remarks, setRemarks] = useState(reqData.remarks || "");
426   const [signingLink, setSigningLink] = useState(() => {
427     // Reconstruct signing link from persisted token (always show once generated)
428     if (reqData.signingToken) {
429       return `${window.location.origin}/sign/${reqData.signingToken}`;
430     }
431     return "";
432   });
433   const [signingLinkExpiry, setSigningLinkExpiry] = useState(() => reqData.signingTokenExpiresAt || null);
434   const [generating, setGenerating] = useState(false);
435   const [regenerating, setRegenerating] = useState(false);
436   const [phase1Revising, setPhase1Revising] = useState(false);
437   const [approving, setApproving] = useState(false);
438   const [phase3Revising, setPhase3Revising] = useState(false);
439   const [confirmAction, setConfirmAction] = useState(null); // "phase1_approve" | "phase1_revision" | "phase3_approve" | "ph
440   const adminSigRef = useRef(null);
441
442   const { status } = reqData;
443   const cfg = FIELDS_FILE_SLOTS_CONFIG.agreement;
444   const _fd = reqData.formData || {};
445   const _mid = String(_fd.repMiddleInitial || "").trim();
446   const _midInitial = _mid ? `${_mid[0].toUpperCase()}.` : "";
447   const requestedRepName = [_fd.repFirstName, _midInitial, _fd.repLastName].filter(Boolean).join(" ").replace(/\s+/g, " ").t
448
449   const handlePhase1Approve = async () => {
450     setConfirmAction(null);
451     setGenerating(true);
452     try {
453       const res = await generateSigningLink(reqData.id);

```

```
454     const link = `${window.location.origin}/sign/${res.signingToken}`;
455     setSigningLink(link);
456     setSigningLinkExpiry(res.signingTokenExpiresAt || null);
457     await onRequestUpdated?();
458     notify("Approved. Signing link sent to the representative's email.", { type: "success", title: "Approved" });
459   } catch (err) {
460     notify(err.response?.data?.message || err.message || "Failed to generate signing link", { type: "error" });
461   } finally {
462     setGenerating(false);
463   }
464 };
465
466 const handlePhase1Revision = async () => {
467   setConfirmAction(null);
468   setPhase1Revising(true);
469   try {
470     await updateRequestStatus(reqData._id, { status: "stud_revision_requested", remarks });
471     notify("Student revision requested.", { type: "success", title: "Revision Requested" });
472     await onRequestUpdated?({ withToast: true });
473   } catch (err) {
474     notify(err.response?.data?.message || "Failed", { type: "error" });
475   } finally {
476     setPhase1Revising(false);
477   }
478 };
479
480 const handleResendLink = async () => {
481   const confirmed = await confirmInPage({
482     title: "Resend Signing Link",
483     message: "This will generate a new signing link and send it to the representative's email. The old link will be invalid.",
484     confirmText: "Resend",
485     cancelText: "Cancel",
486     tone: "warning",
487   });
488   if (!confirmed) return;
489   setRegenerating(true);
490   try {
491     const res = await generateSigningLink(reqData._id);
492     const link = `${window.location.origin}/sign/${res.signingToken}`;
493     setSigningLink(link);
494     setSigningLinkExpiry(res.signingTokenExpiresAt || null);
495     await onRequestUpdated?();
496     notify("New signing link sent to the representative's email.", { type: "success" });
497   } catch (err) {
498     notify(err.response?.data?.message || err.message || "Failed to resend signing link", { type: "error" });
499   } finally {
500     setRegenerating(false);
501   }
502 };
503
504 const handlePhase3Approve = async () => {
505   setConfirmAction(null);
506   setApproving(true);
507   try {
508     const actionRes = await adminPhase3Action(reqData._id, { action: "approve" });
509     const updated = actionRes.request;
510
511     if (!updated?.serialNo) throw new Error("Serial number missing");
512
513     const verificationUrl = buildVerificationUrl(updated.serialNo);
514     const qrDataUrl = await generateQrDataUrl(verificationUrl);
515     const adminSigDataUrl = adminSigRef.current.getDataUrl();
516     const studentName = reqData.userId?.name || "Unknown Student";
517     const requestFolder = getRequestFolder(reqData.predocs);
518     if (!requestFolder) throw new Error("Could not determine request folder");
519
520     await uploadApprovedQrImage(qrDataUrl, "agreement", studentName, requestFolder);
521
522     const { authorizerSig, repSig } = await getSignatureImages(reqData._id);
523
524     const docReq = {
525       ...updated,
526       userId: reqData.userId,
527       repInfo: reqData.repInfo,
528       authorizerSigUrl: authorizerSig,
529       repSigUrl: repSig,
```

```

530     adminSigDataUrl,
531     approverName: user?.name || "",
532     verificationUrl,
533     qrDataUrl,
534   };
535
536   const { generateApprovedPDF } = await import("../config/documentTemplates");
537   const pdfBlob = await generateApprovedPDF(docReq);
538   const approvedFileName = `Agreement_Aproved_${Date.now()}.pdf`;
539   const uploaded = await uploadApprovedForm(
540     pdfBlob,
541     "agreement",
542     studentName,
543     requestFolder,
544     approvedFileName
545   );
546
547   await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });
548
549   await deleteStorageFile(reqData.authorizerSigPath);
550   await deleteStorageFile(reqData.repSigPath);
551
552   notify("Agreement fully approved and document generated!", { type: "success", title: "Approved" });
553   await onRequestUpdated?({ withToast: true });
554   setApproving(false);
555 } catch (err) {
556   notify(err.response?.data?.message || err.message || "Failed to approve", { type: "error" });
557 } finally {
558   setApproving(false);
559 }
560 };
561
562 const handlePhase3RepRevision = async () => {
563   setConfirmAction(null);
564   setPhase3Revising(true);
565   try {
566     const res = await adminPhase3Action(reqData._id, { action: "rep_revision_requested", remarks });
567     const newLink = `${window.location.origin}/sign/${res.signingToken || res.request.signingToken}`;
568     setSigningLink(newLink);
569     setSigningLinkExpiry(res.signingTokenExpiresAt || null);
570     await onRequestUpdated?({ withToast: true });
571     notify("Recipient revision requested. A new signing link has been sent to the representative's email.", { type: "success" });
572   } catch (err) {
573     notify(err.response?.data?.message || "Failed", { type: "error" });
574   } finally {
575     setPhase3Revising(false);
576   }
577 };
578
579 return (
580   <>
581     { /* Requestor Attachments */ }
582     <div className="review-section">
583       <h4 className="review-section-title">Requestor Attachments</h4>
584       {(() => {
585         const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
586         return initialDocs.length ? (
587           <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
588             {initialDocs.map((f, idx) => (
589               <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
590                 <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${f.id}`}
591               </a>
592             ))}
593           </div>
594         ) : (
595           <div className="review-info-box">
596             <span className="review-info-box--muted">No files.</span>
597           </div>
598         );
599       })()}
600     </div>
601
602     {(() => {
603       const rounds = {};
604       (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
605         const r = f.resubmissionRound || 1;

```

```

606         if (!rounds[r]) rounds[r] = [];
607         rounds[r].push(f);
608     });
609     return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
610         <div key={round} className="review-section">
611             <h4 className="review-section-title">
612                 Resubmitted Attachments{Object.keys(rounds).length > 1 ? `(Round ${round})` : ""}
613             </h4>
614             <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
615                 {rounds[round].map((f, idx) => (
616                     <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
617                         <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${f.name}`}
618                     </a>
619                 ))}
620             </div>
621         </div>
622     ));
623 })()
624
625 /* Representative */
626 <div className="review-section">
627     <h4 className="review-section-title">Representative</h4>
628     <div className="review-field">
629         <span className="review-field-label">Full Name</span>
630         <div className="review-info-box">
631             {reqData.repInfo?.name || (requestedRepName !== "-" ? requestedRepName : <span className="review-info-box--muted">
632                 </div>
633         </div>
634         {cfg.fields
635             .filter((f) => !["repFirstName", "repMiddleInitial", "repLastName"].includes(f.name))
636             .map((f) => (
637                 <div key={f.name} className="review-field">
638                     <span className="review-field-label">{f.label}</span>
639                     <div className="review-info-box">
640                         {String(reqData.formData?.[f.name] ?? "") || (
641                             <span className="review-info-box--muted"></span>
642                         )}
643                     </div>
644                 </div>
645             ))}
646     </div>
647
648 /* Representative Attachments (after rep signs) */
649 {reqData.repInfo?.govIdDoc?.url && (
650     <div className="review-section">
651         <h4 className="review-section-title">Representative Attachments</h4>
652         <a href={reqData.repInfo.govIdDoc.url} target="_blank" rel="noreferrer" className="review-file-link">
653             <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Government-Issued ID
654         </a>
655     </div>
656 )}
657
658 /* Authorization Letter */
659 {status !== "agr_approved" && (
660     <div className="review-section">
661         <div style={{ padding: "20px 24px", background: "var(--surface-2)", border: "1px solid var(--border)", borderRadius:
662             /* Requestee date – left */
663             <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
664                 {reqData.createdAt ? new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "long", d
665             </div>
666             /* Authorization paragraph – centered */
667             <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0 24px
668                 I, <strong>{reqData.userId?.name || "-"}</strong>, hereby authorize{" "}
669                 <strong>{requestedRepName}</strong> to act as my representative and sign on my behalf
670                 in connection with this agreement, as governed by the Data Protection Office.
671             </p>
672             /* Requestee signature block – centered */
673             <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
674                 {reqData.authorizerSigUrl && (
675                     <img src={reqData.authorizerSigUrl} alt="Authorizer signature" style={{ maxHeight: 72, display: "block" }} loa
676                 </div>
677             <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", padding: 6, fontS
678                 {reqData.userId?.name || "-"}
679             </div>
680             {reqData.studentSignedAt && (
681                 <span style={{ fontSize: 11, color: "var(--text-muted)" }}>

```

```

682         Signed on {new Date(reqData.studentSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day:
683         </span>
684     }}
685 </div>
686
687 /* Representative agreement section – hidden during rep revision, shown once rep re-submits */
688 {reqData.repSigUrl && status !== "agr_rep_revision_requested" && (
689     <=
690     <hr style={{ border: "none", borderTop: "1px solid var(--border)", margin: "24px 0" }} />
691     /* Rep agreement date – left */
692     <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
693         {reqData.updatedAt ? new Date(reqData.updatedAt).toLocaleDateString("en-US", { year: "numeric", month: "long
694     </div>
695     /* Rep agreement paragraph – centered */
696     <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0
697         I, <strong>{reqData.repInfo?.name || "-"}</strong>, hereby confirm my agreement to act as the authorized rep
698         of <strong>{reqData.userId?.name || "-"}</strong> and sign on their behalf,
699         as governed by the Data Protection Office.
700     </p>
701     /* Representative signature block – centered */
702     <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
703         <img src={reqData.repSigUrl} alt="Representative signature" style={{ maxHeight: 72, display: "block" }} load
704         <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", padding: 6, f
705             {reqData.repInfo?.name || "-"}
706         </div>
707         {reqData.repSignedAt && (
708             <span style={{ fontSize: 11, color: "var(--text-muted)" }}>
709                 Signed on {new Date(reqData.repSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day:
710             </span>
711         )}
712     </div>
713 </>
714 )}
715 </div>
716 </div>
717 )}
718
719 /* Remarks display */
720 {(status === "stud_revision_requested" || status === "agr_rep_revision_requested") &&
721     reqData.remarks && (
722         <div className="review-section">
723             <h4 className="review-section-title">
724                 {status === "agr_rep_revision_requested" ? "Remarks For Representative" : "Remarks"}
725             </h4>
726             <div className="review-info-box">{reqData.remarks}</div>
727         </div>
728     )}
729
730 /* Final approved document */
731 {status === "agr_approved" && (
732     <div className="review-section">
733         <h4 className="review-section-title">Approved Agreement</h4>
734         {reqData.postdocs?.url ? (
735             <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
736                 <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved Agreement Document
737             </a>
738         ) : (
739             <div className="review-info-box">
740                 <span className="review-info-box--muted">No document uploaded yet.</span>
741             </div>
742         )}
743     </div>
744 )}
745
746 /* Rep rejected */
747 {status === "agr_declined" && (
748     <div className="info-banner info-banner--danger">
749         <strong>Recipient Declined</strong>
750         <p>The recipient declined to sign. This request lifecycle has ended.</p>
751     </div>
752 )}
753
754 /* Generated signing link (visible only while awaiting rep action) */
755 {signingLink && (status === "agr_awaiting_rep_signature" || status === "agr_rep_revision_requested") && (
756     <div className="signing-link-box">
757         <p className="signing-link-title">Representative Signing Link</p>

```

```

758     <div className="signing-link-row">
759       <span className="signing-link-code">Representative signing link ready – click to copy</span>
760       <button
761         className="signing-link-copy-btn"
762         onClick={() => {
763           navigator.clipboard.writeText(signingLink);
764         }}
765         title="Copy link"
766       >
767         <Copy size={14} strokeWidth={2} />
768       </button>
769     </div>
770     {signingLinkExpiry && (
771       <div style={{ marginTop: 6, fontSize: 12, color: new Date(signingLinkExpiry) < new Date() ? "#dc2626" : "var(--t
772         {new Date(signingLinkExpiry) < new Date()
773           ? <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><AlertTriangle size={14} strokeWidt
774             : `Expires: ${new Date(signingLinkExpiry).toLocaleDateString("en-PH", { year: "numeric", month: "short", day
775         </div>
776     )}
777     {canProgress && (
778       <button
779         onClick={handleResendLink}
780         disabled={regenerating || !signingLinkExpiry || new Date(signingLinkExpiry) >= new Date()}
781         className="review-btn-secondary"
782         style={{ marginTop: 10, opacity: (!signingLinkExpiry || new Date(signingLinkExpiry) >= new Date()) ? 0.5 : 1 }
783         title={!signingLinkExpiry || new Date(signingLinkExpiry) >= new Date() ? "Link is still active – not yet exp
784       >
785         {regenerating ? "Sending..." : <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><RefreshCw
786       </button>
787     )}
788   </div>
789 )}
790
791   /* ■■ Phase 1 actions ■■ */
792   {canProgress && status === "agr_pending_1" && !signingLink && (
793     <div className="review-section">
794       <h4 className="review-section-title">Remarks</h4>
795       <textarea
796         value={remarks}
797         onChange={(e) => setRemarks(e.target.value)}
798         rows={3}
799         className="review-textarea"
800         placeholder=""
801       />
802     </div>
803     <div className="review-actions">
804       <button
805         onClick={() => setConfirmAction("phase1_approve")}
806         disabled={generating || phase1Revising}
807         className="review-btn-primary"
808       >
809         {generating ? "Generating link..." : "Approve & Generate Signing Link"}
810       </button>
811       <button
812         onClick={() => {
813           if (!remarks.trim()) {
814             notify("Please enter remarks before requesting a revision.", { type: "warning" });
815             return;
816           }
817           setConfirmAction("phase1_revision");
818         }}
819         disabled={generating || phase1Revising}
820         className="review-btn-secondary"
821       >
822         {phase1Revising ? "Submitting..." : "Request Student Revision"}
823       </button>
824     </div>
825   </div>
826 </>
827 )}
828
829   /* ■■ Phase 2: waiting on rep ■■ */
830   {status === "agr_awaiting_rep_signature" && !signingLink && (
831     <div className="info-banner info-banner--info">
832       <strong>Waiting for Representative</strong>
833     <p>

```



```

834         The signing link has been generated. Share it with the representative.
835         This page will update once they submit or decline.
836     </p>
837 </div>
838 }}
839
840 {/* ■■■ Phase 3 actions ■■■ */}
841 {canProgress && status === "agr_pending_2" && (
842     <
843         <div className="review-section">
844             <h4 className="review-section-title">Your E-Signature (Admin) *</h4>
845             <p style={{ fontSize: "13px", color: "var(--text-muted)", margin: "0 0 8px 0" }}>
846                 Draw your signature below to sign off on the final approval.
847             </p>
848             <SignaturePad ref={adminSigRef} height={160} />
849         </div>
850
851         <div className="review-section">
852             <h4 className="review-section-title">Remarks</h4>
853             <textarea
854                 value={remarks}
855                 onChange={(e) => setRemarks(e.target.value)}
856                 rows={3}
857                 className="review-textarea"
858                 placeholder=""
859             />
860         </div>
861
862         <div className="review-actions">
863             <button
864                 onClick={() => {
865                     if (!adminSigRef.current || adminSigRef.current.isEmpty()) {
866                         notify("Please draw your e-signature before approving.", { type: "warning" });
867                         return;
868                     }
869                     setConfirmAction("phase3_approve");
870                 }}
871                 disabled={approving || phase3Revising}
872                 className="review-btn-primary"
873             >
874                 {approving ? "Generating final document..." : "Final Approve & Sign"}
875             </button>
876             <button
877                 onClick={() => {
878                     if (!remarks.trim()) {
879                         notify("Please enter remarks explaining what the representative needs to revise.", { type: "warning" });
880                         return;
881                     }
882                     setConfirmAction("phase3_revision");
883                 }}
884                 disabled={approving || phase3Revising}
885                 className="review-btn-secondary"
886             >
887                 {phase3Revising ? "Submitting..." : "Request Representative Revision"}
888             </button>
889         </div>
890     </>
891 }}
892
893 {/* ■■■ Rep revision requested ■■■ */}
894 {status === "agr_rep_revision_requested" && !signingLink && (
895     <div className="info-banner info-banner--warning">
896         <strong>Recipient Revision Pending</strong>
897         <p>
898             A new signing link has been sent to the representative's email so they can resubmit.
899         </p>
900     </div>
901 )}
902
903 <AnimatePresence>
904     {confirmAction && (
905         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
906             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
907                 <h3 className="modal-title">
908                     {confirmAction === "phase1_approve" && "Approve & Send for Signing"}
909                     {confirmAction === "phase1_revision" && "Request Student Revision"}

```



```

910      {confirmAction === "phase3_approve" && "Final Approve Agreement"}
911      {confirmAction === "phase3_revision" && "Request Recipient Revision"}
912    </h3>
913    <p className="modal-text">
914      {confirmAction === "phase1_approve" && "This will approve the request and generate a signing link for the re
915      {confirmAction === "phase1_revision" && "The student will be notified to revise and resubmit their request."
916      {confirmAction === "phase3_approve" && "This will finalize the agreement and generate the approved document.
917      {confirmAction === "phase3_revision" && "A new signing link will be generated for the representative to resu
918    </p>
919    <div className="modal-actions">
920      <button
921        className="ui-btn ui-btn--secondary"
922        onClick={() => setConfirmAction(null)}
923        disabled={generating || phase1Revising || approving || phase3Revising}
924      >
925        Cancel
926      </button>
927      <button
928        className="ui-btn ui-btn--primary"
929        onClick={() => {
930          if (confirmAction === "phase1_approve") handlePhase1Approve();
931          else if (confirmAction === "phase1_revision") handlePhase1Revision();
932          else if (confirmAction === "phase3_approve") handlePhase3Approve();
933          else if (confirmAction === "phase3_revision") handlePhase3RepRevision();
934        }}
935        disabled={generating || phase1Revising || approving || phase3Revising}
936      >
937        {(confirmAction === "phase1_approve" || confirmAction === "phase3_approve") ? "Approve" : "Confirm"}
938      </button>
939    </div>
940  </motion.div>
941 </motion.div>
942 )}
943 </AnimatePresence>
944
945   {!canProgress && (
946     <div className="info-banner info-banner--info">
947       <strong>Read-only access</strong>
948       <p>Staff can view request details but only Admin can move, approve, or sign this request.</p>
949     </div>
950   )}
951 </>
952 );
953 }
954
955 // ████████████████████████████████████████████████████████████████████████████████
956 // Main component
957 // ████████████████████████████████████████████████████████████████████████████████
958 export default function AdminRequestReview() {
959   const { id } = useParams();
960   const navigate = useNavigate();
961   const { user } = useContext(AuthContext);
962
963   const [reqData, setReqData] = useState(null);
964   const loadRequest = async () => {
965     try {
966       const r = await getRequestById(id);
967       setReqData(r);
968     } catch (err) {
969       notify(err.response?.data?.message || "Failed to load request", { type: "error" });
970       navigate("/admin/requests");
971     }
972   };
973
974   useEffect(() => {
975     loadRequest();
976     // eslint-disable-next-line react-hooks/exhaustive-deps
977   }, [id, navigate]);
978
979   if (!reqData) return null;
980
981   const isAgreement = reqData.type === "agreement";
982   const canProgress = user?.role === "admin";
983
984   const requestTitle =
985     reqData.type === "agreement"

```

```

986     ? "Agreement Request"
987     : `NDA Request${reqData.formData?.ndaTypeLabel ? ` - ${reqData.formData.ndaTypeLabel}` : ""}`;
988
989     return (
990       <div className="review-page">
991         <div className="review-card">
992           <div className="request-form-header">
993             <button
994               className="request-form-back"
995               onClick={() => {
996                 if (window.history.length > 1) navigate(-1);
997                 else navigate("/admin");
998               }}
999             >
1000               < Back
1001             </button>
1002             <h2 className="request-form-title">{requestTitle}</h2>
1003           </div />
1004         </div>
1005
1006         <div className="review-header">
1007           <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", gap: 12, flexWrap: "wrap", w
1008             <span className={statusPillClass(reqData.status)}>{prettyStatus(reqData.status)}</span>
1009             <span className="review-meta-row" style={{ marginLeft: "auto" }}>
1010               <b>Request Date:</b> {new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short
1011             </span>
1012           </div>
1013         </div>
1014
1015         <div className="review-section">
1016           <RequestStepper status={reqData.status} type={reqData.type} />
1017         </div>
1018
1019         <div className="review-section">
1020           <h4 className="review-section-title">Requestor</h4>
1021           <div className="review-info-box">
1022             <div style={{ fontWeight: 600 }}>
1023               {reqData.proxyRequestee?.isProxy ? (reqData.proxyRequestee?.fullName || "Proxy Requestor") : (reqData.userId?.
1024             </div>
1025             <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
1026               {reqData.proxyRequestee?.isProxy ? (reqData.proxyRequestee?.email || "") : (reqData.userId?.email || "")}
1027             </div>
1028             {reqData.proxyRequestee?.isProxy ? (
1029               <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
1030                 Submitted by staff: {reqData.userId?.name || "Unknown"}
1031               </div>
1032             ) : null}
1033             {reqData.proxyRequestee?.isProxy && reqData.proxyRequestee?.idNumber ? (
1034               <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
1035                 ID: {reqData.proxyRequestee.idNumber}
1036               </div>
1037             ) : null}
1038             {reqData.proxyRequestee?.isProxy && reqData.proxyRequestee?.departmentOrOrganization ? (
1039               <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
1040                 Department/Org: {reqData.proxyRequestee.departmentOrOrganization}
1041               </div>
1042             ) : null}
1043           </div>
1044         </div>
1045
1046         {isAgreement ? (
1047           <AgreementReviewPanel reqData={reqData} canProgress={canProgress} onRequestUpdated={loadRequest} />
1048         ) : (
1049           <NdaReviewPanel reqData={reqData} canProgress={canProgress} onRequestUpdated={loadRequest} />
1050         )}
1051       </div>
1052     </div>
1053   );
1054 }

```

63. client/src/pages/admin/AdminReports.jsx (547 lines)

```

1 import { useEffect, useMemo, useState } from "react";
2 import { motion, AnimatePresence } from "framer-motion";
3 import { RefreshCw, Search, Filter, X } from "lucide-react";
4 import { getAllRequests } from "../../../services/requestService";
5 import FilterSelect from "../../../components/FilterSelect";
6 import { getAuditLogs } from "../../../services/auditService";
7
8 const STATUS_LABEL = {
9   nda_pending: "Reviewal",
10  nda_approved: "Approved",
11  stud_revision_requested: "Student Revisions",
12  agr_pending_1: "Initial Reviewal",
13  agr_awaiting_rep_signature: "Recipient Reviewal",
14  agr_pending_2: "Final Reviewal",
15  agr_approved: "Approved",
16  agr_declined: "Recipient Declined",
17  agr_rep_revision_requested: "Recipient Revisions",
18 };
19
20 const prettyStatus = (s) =>
21   STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
22
23 const statusPillClass = (s) => {
24   if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
25   if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
26   if (s === "stud_revision_requested") return "status-pill status-pill--orange";
27   if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
28   if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
29   if (s === "agr_declined") return "status-pill status-pill--red";
30   return "status-pill";
31 };
32
33 const STATUS_FILTER_OPTIONS = [
34   { value: "", label: "All Statuses" },
35   { value: "nda_pending,agr_pending_1,agr_pending_2", label: "Reviewal" },
36   { value: "nda_approved,agr_approved", label: "Approved" },
37   { value: "stud_revision_requested", label: "Student Revisions" },
38   { value: "agr_rep_revision_requested", label: "Recipient Revisions" },
39   { value: "agr_declined", label: "Recipient Declined" },
40   { value: "agr_awaiting_rep_signature", label: "Recipient Reviewal" },
41 ];
42
43
44
45 const TIMELINE_STEPS = {
46   request_created: { label: "Submitted", color: "#3b82f6" },
47   request_resubmitted: { label: "Re-submitted", color: "#3b82f6" },
48   signing_link_generated: { label: "Sent to Recipient", color: "#8b5cf6" },
49   rep_signature_submitted: { label: "Recipient Signed", color: "#10b981" },
50   rep_signature_declined: { label: "Recipient Declined", color: "#ef4444" },
51   request_approved: { label: "Approved", color: "#10b981" },
52   request_revision_required: (log) =>
53     log.details?.newStatus === "agr_rep_revision_requested"
54       ? { label: "Recipient Revision Requested", color: "#f59e0b" }
55       : { label: "Student Revision Requested", color: "#f59e0b" },
56 };
57
58 const resolveStep = (log) => {
59   const entry = TIMELINE_STEPS[log.action];
60   if (!entry) return null;
61   return typeof entry === "function" ? entry(log) : entry;
62 };
63
64 export default function AdminReports() {
65   const [requests, setRequests] = useState([]);
66   const [loading, setLoading] = useState(true);
67   const [refreshing, setRefreshing] = useState(false);
68   const [error, setError] = useState("");
69
70   const [timelineRequest, setTimelineRequest] = useState(null);
71   const [timelineLogs, setTimelineLogs] = useState([]);
72   const [timelineLoading, setTimelineLoading] = useState(false);
73

```

```
74 // Search state
75 const [searchTerm, setSearchTerm] = useState("");
76
77 // Filter state
78 const [typeFilter, setTypeFilter] = useState("all");
79 const [statusFilter, setStatusFilter] = useState("");
80 const [dateMode, setDateMode] = useState("preset");
81 const [preset, setPreset] = useState("all");
82 const [singleDate, setSingleDate] = useState("");
83 const [startDate, setStartDate] = useState("");
84 const [endDate, setEndDate] = useState("");
85
86 // Filter panel open/close
87 const [isFilterOpen, setIsFilterOpen] = useState(() =>
88   typeof window === "undefined" ? true : window.innerWidth >= 768
89 );
90
91 // Pagination
92 const [page, setPage] = useState(1);
93 const PAGE_SIZE = 15;
94
95 const buildDateParams = () => {
96   const params = {};
97   const now = new Date();
98   if (dateMode === "preset") {
99     if (preset === "today") {
100       params.startDate = now.toISOString().slice(0, 10);
101       params.endDate = now.toISOString().slice(0, 10);
102     } else if (preset === "thisWeek") {
103       const day = now.getDay();
104       const monday = new Date(now);
105       monday.setDate(now.getDate() - ((day + 6) % 7));
106       params.startDate = monday.toISOString().slice(0, 10);
107     } else if (preset === "thisMonth") {
108       params.startDate = new Date(now.getFullYear(), now.getMonth(), 1).toISOString().slice(0, 10);
109     } else if (preset === "thisYear") {
110       params.startDate = new Date(now.getFullYear(), 0, 1).toISOString().slice(0, 10);
111     }
112   } else if (dateMode === "single" && singleDate) {
113     params.startDate = singleDate;
114     params.endDate = singleDate;
115   } else if (dateMode === "range") {
116     if (startDate) params.startDate = startDate;
117     if (endDate) params.endDate = endDate;
118   }
119   return params;
120 };
121
122 const load = async ({ withToast = false } = {}) => {
123   setLoading(true);
124   if (withToast) setRefreshing(true);
125   setError("");
126   try {
127     const data = await getAllRequests(buildDateParams());
128     setRequests(Array.isArray(data) ? data : []);
129   } catch (err) {
130     const msg = err.response?.data?.message || err.message || "Failed to load data";
131     setError(msg);
132   } finally {
133     setLoading(false);
134     setRefreshing(false);
135   }
136 };
137
138 useEffect(() => { load(); }, []);
139 useEffect(() => { load(); setPage(1); }, [dateMode, preset, singleDate, startDate, endDate]); // eslint-disable-line react-
140
141 useEffect(() => {
142   const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
143   window.addEventListener("resize", onResize);
144   return () => window.removeEventListener("resize", onResize);
145 }, []);
146
147 const clearSearch = () => { setSearchTerm(""); setPage(1); };
148
149 const resetAndRefresh = () => {
```

```

150     setSearchTerm("");
151     setTypeFilter("all");
152     setStatusFilter("");
153     setDateMode("preset");
154     setPreset("all");
155     setSingleDate("");
156     setStartDate("");
157     setEndDate("");
158     setPage(1);
159     load({ withToast: true });
160 };
161
162 const filteredRequests = useMemo(() => {
163     let result = [...requests];
164
165     if (searchTerm) {
166         const term = searchTerm.toLowerCase();
167         result = result.filter((r) => {
168             const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
169             const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
170             const id = (r._id || "").slice(-7).toLowerCase();
171             return name.includes(term) || email.includes(term) || id.includes(term.toLowerCase());
172         });
173     }
174
175     if (statusFilter) {
176         const statuses = statusFilter.split(",");
177         result = result.filter((r) => statuses.includes(r.status));
178     }
179
180     if (typeFilter !== "all") {
181         if (typeFilter === "agreement") {
182             result = result.filter((r) => r.type === "agreement");
183         } else if (typeFilter === "nda_orgactivities") {
184             result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
185         } else if (typeFilter === "nda_research") {
186             result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
187         }
188     }
189
190     result.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
191
192     return result;
193 }, [requests, searchTerm, statusFilter, typeFilter]);
194
195 const totalPages = Math.max(1, Math.ceil(filteredRequests.length / PAGE_SIZE));
196 const paginatedRequests = filteredRequests.slice((page - 1) * PAGE_SIZE, page * PAGE_SIZE);
197
198 const openTimeline = async (r) => {
199     setTimelineRequest(r);
200     setTimelineLoading(true);
201     setTimelineLogs([]);
202     try {
203         const data = await getAuditLogs({ resourceId: r._id, limit: 50 });
204         const sorted = (data.logs || []).sort((a, b) => new Date(a.createdAt) - new Date(b.createdAt));
205         setTimelineLogs(sorted);
206     } catch { /* silent */ } finally {
207         setTimelineLoading(false);
208     }
209 };
210
211
212 return (
213     <div className="page-shell">
214         {error && (
215             <div className="info-banner info-banner--danger" style={{ marginBottom: 12 }}>{error}</div>
216         )}
217
218         {/* ■■■ Search + Filter toolbar ■■■ */}
219         <div className="req-toolbar">
220             {/* Search row */}
221             <div className="req-toolbar__search-row">
222                 <div className="req-toolbar__search-box">
223                     <input
224                         type="text"
225                         className="req-toolbar__search-input"

```

```
226     placeholder="Search by requestor, email, or request ID..."
227     value={searchTerm}
228     onChange={(e) => { setSearchTerm(e.target.value); setPage(1); }}
229   />
230   {searchTerm ? (
231     <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
232       x
233     </button>
234   ) : null}
235   <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
236     <Search size={14} />
237   </span>
238 </div>
239 <button
240   type="button"
241   className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
242   onClick={() => setIsFilterOpen((prev) => !prev)}
243   aria-expanded={isFilterOpen}
244 >
245   <Filter size={13} />
246   Filters
247 </button>
248 </div>
249
250 /* Filter chips row */
251 <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
252   <div className="req-toolbar__filter-group">
253     <label className="req-toolbar__filter-label">Type</label>
254     <FilterSelect
255       value={typeFilter}
256       onChange={(v) => { setTypeFilter(v); setPage(1); }}
257       className="filter-select--type"
258       options={[
259         { value: "all", label: "All Types" },
260         { value: "nda_orgactivities", label: "NDA – Student Organization Activities" },
261         { value: "nda_research", label: "NDA – Conduct of Research" },
262         { value: "agreement", label: "Agreement" },
263       ]}
264     />
265   </div>
266
267   <div className="req-toolbar__filter-group">
268     <label className="req-toolbar__filter-label">Status</label>
269     <FilterSelect
270       value={statusFilter}
271       onChange={(v) => { setStatusFilter(v); setPage(1); }}
272       className="filter-select--status"
273       options={STATUS_FILTER_OPTIONS}
274       defaultValue=""
275     />
276   </div>
277
278   <div className="req-toolbar__filter-group">
279     <label className="req-toolbar__filter-label">Date</label>
280     <FilterSelect
281       value={dateMode}
282       onChange={setDateMode}
283       options={[
284         { value: "preset", label: "Preset" },
285         { value: "single", label: "Specific Date" },
286         { value: "range", label: "Date Range" },
287       ]}
288       defaultValue="preset"
289     />
290   </div>
291
292   {dateMode === "preset" ? (
293     <div className="req-toolbar__filter-group">
294       <label className="req-toolbar__filter-label">Period</label>
295       <FilterSelect
296         value={preset}
297         onChange={setPreset}
298         options={[
299           { value: "all", label: "All Time" },
300           { value: "today", label: "Today" },
301           { value: "thisweek", label: "This Week" },
```

```

302         { value: "thisMonth", label: "This Month" },
303         { value: "thisYear", label: "This Year" },
304     ]}
305     />
306 </div>
307 ) : null}
308
309 {dateMode === "single" ? (
310   <div className="req-toolbar__filter-group">
311     <label className="req-toolbar__filter-label">Date</label>
312     <input
313       className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
314       type="date"
315       value={singleDate}
316       onChange={(e) => setSingleDate(e.target.value)}
317     />
318   </div>
319 ) : null}
320
321 {dateMode === "range" ? (
322   <div
323     <div className="req-toolbar__filter-group">
324       <label className="req-toolbar__filter-label">From</label>
325       <input
326         className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
327         type="date"
328         value={startDate}
329         onChange={(e) => setStartDate(e.target.value)}
330       />
331     </div>
332     <div className="req-toolbar__filter-group">
333       <label className="req-toolbar__filter-label">To</label>
334       <input
335         className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
336         type="date"
337         value={endDate}
338         onChange={(e) => setEndDate(e.target.value)}
339       />
340     </div>
341   </div>
342 ) : null}
343
344 <button
345   type="button"
346   className="req-toolbar__reset-btn"
347   onClick={resetAndRefresh}
348   disabled={refreshing}
349   title="Reset filters and refresh"
350 >
351   <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
352 </button>
353 </div>
354 </div>
355
356 {/* Table */}
357 <motion.div
358   initial={{ opacity: 0, y: 10 }}
359   animate={{ opacity: 1, y: 0 }}
360   transition={{ delay: 0.1 }}
361   className="dashboard-card"
362 >
363   <div className="table-scroll">
364     <table className="dashboard-table">
365       <thead>
366         <tr className="dashboard-table-title-row">
367           <th colspan={6}>
368             <div className="dashboard-table-title-wrap">
369               <span className="dashboard-table-title">Transactions</span>
370             </div>
371           </th>
372         </tr>
373         <tr>
374           <th>Last Updated</th>
375           <th>Requestor</th>
376           <th>Request ID</th>
377           <th>Type</th>

```

```

378         <th>Status</th>
379         <th />
380     </tr>
381 </thead>
382 <tbody>
383     {loading ? (
384         [...Array(4)].map((_, i) => (
385             <tr key={i}>
386                 <td><span className="skeleton-block skeleton-text" /></td>
387                 <td><span className="skeleton-block skeleton-text" /></td>
388                 <td><span className="skeleton-block skeleton-text" /></td>
389                 <td><span className="skeleton-block skeleton-text" /></td>
390                 <td><span className="skeleton-block skeleton-pill" /></td>
391             </td />
392         </tr>
393     )
394 ) : paginatedRequests.length === 0 ? (
395     <tr>
396         <td colspan={6}>
397             <div className="dashboard-empty">
398                 <p className="dashboard-empty-title">No transactions found</p>
399                 <p className="dashboard-empty-text">No records match the current filters.</p>
400             </div>
401         </td>
402     </tr>
403 ) : (
404     paginatedRequests.map((r) => {
405         const name = r.proxyRequestee?.isProxy
406           ? (`${r.proxyRequestee.firstName} || ""` + `${r.proxyRequestee.lastName} || ""`).trim() || r.proxyRequestee.fullName
407           : (r.userId?.name || "-");
408         const lastUpdated = r.updatedAt || r.createdAt;
409         const dateStr = lastUpdated
410           ? `${new Date(lastUpdated).toLocaleDateString("en-PH", { year: "numeric", month: "short", day: "numeric" })}`
411           : "-";
412         const typeLabel = r.type === "nda"
413           ? `NDA${r.formData?.ndaTypeLabel ? ` - ${r.formData.ndaTypeLabel}` : ""}`
414           : r.type === "agreement" ? "Agreement" : (r.type || "-");
415         return (
416             <tr key={r._id} onClick={() => openTimeline(r)} style={{ cursor: "pointer" }}>
417                 <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>{dateStr}</td>
418                 <td>
419                     <span style={{ fontWeight: 600 }}>{name}</span>
420                     {r.proxyRequestee?.isProxy && (
421                         <span className="dashboard-subtext">Proxy (F2F)</span>
422                     )}
423                 </td>
424                 <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
425                 <td>{typeLabel}</td>
426                 <td><span className={statusPillClass(r.status)}>{prettyStatus(r.status)}</span></td>
427                 <td className="row-caret"></td>
428             </tr>
429         );
430     })
431 )}
432 </tbody>
433 </table>
434 </div>
435
436 /* Pagination */
437 {totalPages > 1 && (
438     <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", padding: "10px 16px", border: "1px solid #ccc" }}>
439         <span style={{ fontSize: 12, color: "var(--text-muted)" }}>Page {page} of {totalPages}</span>
440     </div>
441     <div style={{ display: "flex", gap: 6 }}>
442         <button
443             className="ui-btn ui-btn--secondary ui-btn--compact"
444             onClick={() => setPage((p) => Math.max(1, p - 1))}
445             disabled={page === 1}
446         >
447             < Prev
448         </button>
449         <button
450             className="ui-btn ui-btn--secondary ui-btn--compact"
451             onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
452             disabled={page === totalPages}
453         >

```



```

454         >
455         Next >
456       </button>
457     </div>
458   </div>
459   })
460 </motion.div>
461
462 { /* ■■■ Request Timeline Modal ■■■ */}
463 <AnimatePresence>
464   {timelineRequest && (
465     <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
466     <motion.div className="req-timeline-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{
467
468       { /* Header */}
469       <div className="req-timeline-modal__header">
470         <div style={{ flex: 1, minWidth: 0 }}>
471           <div style={{ display: "flex", alignItems: "center", gap: 8, flexWrap: "wrap" }}>
472             <span className="req-timeline-modal__id">
473               #{timelineRequest._id?.slice(-7).toUpperCase()}
474             </span>
475             <span className={statusPillClass(timelineRequest.status)}>
476               {prettyStatus(timelineRequest.status)}
477             </span>
478           </div>
479           <div style={{ fontSize: 12, color: "var(--text-muted)", marginTop: 3 }}>
480             {timelineRequest.type === "nda"
481               ? `NDA${timelineRequest.formData?.ndaTypeLabel ? ` - ${timelineRequest.formData.ndaTypeLabel}` : ""}`
482               : "Agreement"}
483           </div>
484         </div>
485         <button
486           className="user-edit-modal__close user-edit-modal__close--danger"
487           onClick={() => setTimelineRequest(null)}
488           aria-label="Close"
489         >
490           <X size={16} />
491         </button>
492       </div>
493
494       { /* Body */}
495       <div className="req-timeline-modal__body">
496         {timelineLoading ? (
497           <div className="req-timeline-modal__loading">
498             {[...Array(4)].map((_, i) => (
499               <div key={i} className="req-timeline-modal__step">
500                 <div className="req-timeline-modal__dot-col">
501                   <span className="skeleton-block" style={{ width: 12, height: 12, borderRadius: "50%" }} />
502                   {i < 3 && <div className="req-timeline-modal__connector" />}
503                 </div>
504                 <div style={{ paddingBottom: 20 }}>
505                   <span className="skeleton-block skeleton-text" style={{ width: 140 }} />
506                 </div>
507               </div>
508             )]}
509           </div>
510         ) : timelineLogs.length === 0 ? (
511           <p style={{ color: "var(--text-muted)", fontSize: 13, textAlign: "center", padding: "24px 0" }}>
512             No timeline data available.
513           </p>
514         ) : (
515           timelineLogs.map((log, i) => {
516             const step = resolveStep(log);
517             if (!step) return null;
518             const isLast = i === timelineLogs.length - 1;
519             const dateStr = new Date(log.createdAt).toLocaleDateString("en-PH", { year: "numeric", month: "short", da
520             const timeStr = new Date(log.createdAt).toLocaleTimeString("en-PH", { hour: "2-digit", minute: "2-digit"
521             return (
522               <div key={log._id} className="req-timeline-modal__step">
523                 <div className="req-timeline-modal__dot-col">
524                   <div className="req-timeline-modal__dot" style={{ background: step.color }} />
525                   {!isLast && <div className="req-timeline-modal__connector" />}
526                 </div>
527                 <div className="req-timeline-modal__content" style={{ paddingBottom: isLast ? 4 : 20 }}>
528                   <div className="req-timeline-modal__label">{step.label}</div>
529                   <div className="req-timeline-modal__time">{dateStr} · {timeStr}</div>

```

```
530         {log.userId?.name && (  
531             <div className="req-timeline-modal__who">{log.userId.name}</div>  
532         )}  
533     </div>  
534 </div>  
535 );  
536 })  
537 )}  
538 </div>  
539  
540  
541     </motion.div>  
542 </motion.div>  
543 )}  
544 </AnimatePresence>  
545 </div>  
546 );  
547 }
```

64. client/src/pages/admin/AdminAuditLog.jsx (490 lines)

```

1 import { useCallback, useEffect, useMemo, useState } from "react";
2 import { motion } from "framer-motion";
3 import { ChevronLeft, ChevronRight, RefreshCw, Search } from "lucide-react";
4 import { getAuditLogs } from "../../services/auditService";
5 import FilterSelect from "../../components/FilterSelect";
6
7 const ACTION_COLORS = {
8   CREATE: { bg: "#d1fae5", color: "#065f46" },
9   UPDATE: { bg: "#d9e9f5", color: "#1e3a8a" },
10  DELETE: { bg: "#fee2e2", color: "#991b1b" },
11  LOGIN: { bg: "#ede9fe", color: "#4c1d95" },
12  LOGOUT: { bg: "#fff9c4", color: "#475569" },
13  ARCHIVE: { bg: "#f9f9f9", color: "#78350f" },
14  PASSWORD_RESET: { bg: "#fff9c4", color: "#9a3412" },
15  REGISTER: { bg: "#d1fae5", color: "#065f46" },
16  SIGNATURE: { bg: "#e0e7ff", color: "#3730a3" },
17  DECLINED: { bg: "#fee2e2", color: "#991b1b" },
18 };
19
20 const ACTION_LABELS = {
21   login: "Login",
22   login_failed: "Login Failed",
23   login_otp_sent: "Login OTP Sent",
24   account_created: "Account Created",
25   account_activated: "Account Activated",
26   email_verified: "Email Verified",
27   request_created: "Request Created",
28   request_approved: "Request Approved",
29   request_revision_required: "Revision Requested",
30   request_resubmitted: "Request Resubmitted",
31   password_changed: "Password Changed",
32   password_reset_requested: "Password Reset Requested",
33   password_reset_triggered_by_admin: "Password Reset (Admin)",
34   user_updated: "User Updated",
35   signing_link_generated: "Signing Link Generated",
36   rep_signature_submitted: "Representative Signature Submitted",
37   rep_signature_declined: "Representative Signature Declined",
38 };
39
40 const FIELD_LABELS = {
41   role: "role",
42   isActive: "active status",
43   email: "email address",
44   name: "name",
45   firstName: "first name",
46   lastName: "last name",
47   middleInitial: "middle name",
48 };
49
50 const prettyAction = (action) =>
51   ACTION_LABELS[action] || (action ? action.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase()) : "-");
52
53 const actionStyle = (action = "") => {
54   // Direct match
55   const upper = action.toUpperCase();
56   if (ACTION_COLORS[upper]) return ACTION_COLORS[upper];
57   // Category match
58   if (upper.includes("LOGIN") || upper.includes("OTP")) return ACTION_COLORS.LOGIN;
59   if (upper.includes("DECLINED") || upper.includes("FAILED") || upper.includes("DEACTIVATED")) return ACTION_COLORS.DECLINED;
60   if (upper.includes("CREATED") || upper.includes("REGISTER") || upper.includes("ACTIVATED") || upper.includes("VERIFIED")) return ACTION_COLORS.CREATE;
61   if (upper.includes("APPROVED") || upper.includes("UPDATED") || upper.includes("RESUBMIT")) return ACTION_COLORS.UPDATE;
62   if (upper.includes("PASSWORD") || upper.includes("RESET")) return ACTION_COLORS.PASSWORD_RESET;
63   if (upper.includes("SIGN")) return ACTION_COLORS.SIGNATURE;
64   if (upper.includes("ARCHIVE")) return ACTION_COLORS.ARCHIVE;
65   return { bg: "#fff9c4", color: "#475569" };
66 };
67
68 const formatActor = (user) => {
69   if (!user) return "System";
70   const role = user.role ? user.role.charAt(0).toUpperCase() + user.role.slice(1) : "User";
71   return `${role} ${user.name || user.email || "User"}`;
72 };
73

```

```

74 const LOGIN_FAIL_REASON = {
75   user_not_found: "account does not exist",
76   wrong_password: "incorrect password",
77   account_inactive: "account is deactivated",
78 };
79
80 const PASSWORD_METHOD_LABEL = {
81   profile: "via profile settings",
82   reset: "via password reset",
83 };
84
85 const prettyType = (type) =>
86   type === "nda" ? "NDA" : type === "agreement" ? "Agreement" : String(type || "document").toUpperCase();
87
88 const formatAuditDetails = (log) => {
89   const actor = formatActor(log.userId);
90   const details = log.details && typeof log.details === "object" ? log.details : {};
91
92   switch (log.action) {
93     case "login":
94       return details.method === "otp"
95         ? `${actor} signed in successfully via OTP verification.`
96         : `${actor} signed in successfully.`;
97     case "login_failed":
98       return `${actor} failed to sign in – ${LOGIN_FAIL_REASON[details.reason] || "invalid credentials"}`;
99     case "login_otp_sent":
100       return details.reason === "resend"
101         ? `${actor} requested a new login OTP.`
102         : `A login OTP was sent to ${actor}`;
103     case "verification_otp_sent":
104       return `A verification OTP was sent to ${actor} before account access.`;
105     case "account_created":
106       if (details.createdBy === "admin") {
107         return `${actor} created an account for ${details.targetEmail || "a user"} with role ${details.role || "student"}`;
108       }
109       return `${actor} registered a new account.`;
110     case "account_activated":
111       return `${actor} activated their account.`;
112     case "email_verified":
113       return `${actor} verified their email address.`;
114     case "request_created":
115       return details.isProxy
116         ? `${actor} submitted a ${prettyType(details.type)} request on behalf of a walk-in requestor.`
117         : `${actor} submitted a ${prettyType(details.type)} request.`;
118     case "request_resubmitted":
119       return `${actor} resubmitted a ${prettyType(details.type)} request.`;
120     case "request_approved":
121       return `${actor} approved a ${prettyType(details.type)} request.`;
122     case "request_revision_required":
123       return `${actor} requested revisions on a ${prettyType(details.type)} request.`;
124     case "signing_link_generated":
125       return `${actor} generated a representative signing link for an ${prettyType(details.type)} request.`;
126     case "rep_signature_submitted":
127       return `Representative ${details.repName || "user"} submitted their e-signature.`;
128     case "rep_signature_declined":
129       return `The representative declined the signing request.`;
130     case "password_changed":
131       return `${actor} changed their password ${PASSWORD_METHOD_LABEL[details.method] || ""}.`;
132     case "password_reset_requested":
133       return details.reason === "resend"
134         ? `${actor} requested a new password reset OTP.`
135         : `${actor} requested a password reset OTP.`;
136     case "password_reset_triggered_by_admin":
137       return `${actor} reset the password for ${details.targetEmail || "a user"}`;
138     case "user_updated": {
139       const fields = Array.isArray(details.fields)
140         ? details.fields.map(f => FIELD_LABELS[f] || f.replace(/_/g, " ").join(" "))
141         : "account details";
142       return `${actor} updated ${details.targetEmail ? `${details.targetEmail}'s` : "a user's"} ${fields}`;
143     }
144     default:
145       if (typeof log.details === "string" && log.details.trim()) return log.details;
146       return "No additional details were recorded.";
147   }
148 };
149

```

```
150 const PAGE_SIZE = 20;
151
152 export default function AdminAuditLog() {
153   const [logs, setLogs] = useState([]);
154   const [loading, setLoading] = useState(true);
155   const [refreshing, setRefreshing] = useState(false);
156   const [page, setPage] = useState(1);
157   const [totalPages, setTotalPages] = useState(1);
158   const [actionFilter, setActionFilter] = useState("");
159   const [searchQuery, setSearchQuery] = useState("");
160   const [error, setError] = useState("");
161   const [isPillsOpen, setIsPillsOpen] = useState(false);
162   const [dateMode, setDateMode] = useState("preset");
163   const [preset, setPreset] = useState("all");
164   const [singleDate, setSingleDate] = useState("");
165   const [startDate, setStartDate] = useState("");
166   const [endDate, setEndDate] = useState("");
167
168   const load = useCallback(async (p = page, action = actionFilter, withToast = false) => {
169     setLoading(true);
170     if (withToast) setRefreshing(true);
171     setError("");
172     try {
173       const res = await getAuditLogs({ page: p, limit: PAGE_SIZE, action: action || undefined });
174       setLogs(res.logs || res);
175       if (res.total) setTotalPages(Math.ceil(res.total / PAGE_SIZE));
176       else if (res.totalPages) setTotalPages(res.totalPages);
177     } catch (err) {
178       const msg = err.response?.data?.message || err.message || "Failed to load audit logs";
179       const status = err.response?.status;
180       setError(`${msg}${status ? ` (HTTP ${status})` : ""}`);
181       console.error("Audit log load error:", err);
182     } finally {
183       setLoading(false);
184       setRefreshing(false);
185     }
186   }, [actionFilter, page]);
187
188   useEffect(() => { load(page, actionFilter); }, [actionFilter, load, page]);
189
190   const handleActionFilter = (v) => {
191     setActionFilter(v);
192     setPage(1);
193   };
194
195   const HIDDEN_ACTIONS = new Set(["user_activated", "user_deactivated", "user_deleted"]);
196
197   const filtered = useMemo(() => {
198     let result = logs.filter((l) => !HIDDEN_ACTIONS.has(l.action));
199
200     if (dateMode === "preset" && preset !== "all") {
201       const now = new Date();
202       let from = null;
203       if (preset === "today") {
204         from = new Date(now.getFullYear(), now.getMonth(), now.getDate());
205       } else if (preset === "thisWeek") {
206         const day = now.getDay();
207         from = new Date(now); from.setDate(now.getDate() - ((day + 6) % 7)); from.setHours(0,0,0);
208       } else if (preset === "thisMonth") {
209         from = new Date(now.getFullYear(), now.getMonth(), 1);
210       } else if (preset === "thisYear") {
211         from = new Date(now.getFullYear(), 0, 1);
212       }
213       if (from) result = result.filter((l) => new Date(l.createdAt) >= from);
214     }
215
216     if (dateMode === "single" && singleDate) {
217       const d = new Date(singleDate);
218       const next = new Date(d); next.setDate(d.getDate() + 1);
219       result = result.filter((l) => { const t = new Date(l.createdAt); return t >= d && t < next; });
220     }
221
222     if (dateMode === "range") {
223       if (startDate) result = result.filter((l) => new Date(l.createdAt) >= new Date(startDate));
224       if (endDate) { const end = new Date(endDate); end.setDate(end.getDate() + 1); result = result.filter((l) => new Date(l.createdAt) < end); }
225     }
226
227     if (searchQuery.trim()) {
228       result = result.filter((l) => l.action.toLowerCase().includes(searchQuery.toLowerCase()));
229     }
230
231     return result;
232   }, [logs, dateMode, preset, singleDate, startDate, endDate, searchQuery]);
233
234   return { logs, loading, refreshing, error, isPillsOpen, dateMode, preset, singleDate, startDate, endDate, searchQuery, filtered };
235 }
```

```

226     const q = searchQuery.toLowerCase();
227     result = result.filter((l) => {
228         const name = (l.userId?.name || "").toLowerCase();
229         const email = (l.userId?.email || "").toLowerCase();
230         const resource = (l.resourceType || "").toLowerCase();
231         const resourceId = (l.resourceId ? String(l.resourceId).slice(-7).toLowerCase() : "");
232         return name.includes(q) || email.includes(q) || resource.includes(q) || resourceId.includes(q);
233     });
234 }
235
236 return result;
237 }, [logs, dateMode, preset, singleDate, startDate, endDate, searchQuery]);
238
239 const ACTION_TYPES = [
240     { label: "All Actions", value: "" },
241     { label: "Login", value: "login" },
242     { label: "Login Failed", value: "login_failed" },
243     { label: "Account Created", value: "account_created" },
244     { label: "Request Created", value: "request_created" },
245     { label: "Request Approved", value: "request_approved" },
246     { label: "Revision Requested", value: "request_revision_required" },
247     { label: "Request Resubmitted", value: "request_resubmitted" },
248     { label: "Password Changed", value: "password_changed" },
249     { label: "Password Reset", value: "password_reset_requested" },
250     { label: "User Updated", value: "user_updated" },
251     { label: "Email Verified", value: "email_verified" },
252     { label: "Signing Link Generated", value: "signing_link_generated" },
253     { label: "Representative Signature Submitted", value: "rep_signature_submitted" },
254     { label: "Representative Signature Declined", value: "rep_signature_declined" },
255 ];
256
257 return (
258     <div className="page-shell">
259         { /* Error Banner */ }
260         {error && (
261             <div className="info-banner info-banner--danger" style={{ marginBottom: 16 }}>
262                 <strong>{error}</strong>
263             </div>
264         )}
265
266         { /* Toolbar */ }
267         <div className="req-toolbar" style={{ borderTop: "none" }}>
268             { /* Search row */ }
269             <div className="req-toolbar__search-row">
270                 <div className="req-toolbar__search-box">
271                     <input
272                         type="text"
273                         className="req-toolbar__search-input"
274                         placeholder="Search by name or resource..."
275                         value={searchQuery}
276                         onChange={(e) => setSearchQuery(e.target.value)}
277                     />
278                     {searchQuery ? (
279                         <button type="button" className="req-toolbar__search-clear" onClick={() => setSearchQuery("")} aria-label="Clear"
280                             x
281                         </button>
282                     ) : null}
283                     <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
284                         <Search size={14} />
285                     </span>
286                 </div>
287             </div>
288
289             { /* Filters row: Action + date filters + refresh */ }
290             <div className="req-toolbar__filters">
291                 <div className="req-toolbar__filter-group">
292                     <label className="req-toolbar__filter-label">Action</label>
293                     <button
294                         type="button"
295                         className="req-toolbar__filter-btn"
296                         data-open={isPillsOpen}
297                         onClick={() => setIsPillsOpen((p) => !p)}
298                     >
299                         <span>{ACTION_TYPES.find((a) => a.value === actionFilter)?.label ?? "All Actions"}</span>
300                     </button>
301                 </div>

```

```

302
303     <div className="req-toolbar__filter-group">
304         <label className="req-toolbar__filter-label">Date</label>
305         <FilterSelect
306             value={dateMode}
307             onChange={setDateMode}
308             options={[
309                 { value: "preset", label: "Preset" },
310                 { value: "single", label: "Specific Date" },
311                 { value: "range", label: "Date Range" },
312             ]}
313             defaultValue="preset"
314         />
315     </div>
316
317     {dateMode === "preset" && (
318         <div className="req-toolbar__filter-group">
319             <label className="req-toolbar__filter-label">Period</label>
320             <FilterSelect
321                 value={preset}
322                 onChange={setPreset}
323                 options={[
324                     { value: "all", label: "All Time" },
325                     { value: "today", label: "Today" },
326                     { value: "thisWeek", label: "This Week" },
327                     { value: "thisMonth", label: "This Month" },
328                     { value: "thisYear", label: "This Year" },
329                 ]}
330             />
331         </div>
332     )}
333
334     {dateMode === "single" && (
335         <div className="req-toolbar__filter-group">
336             <label className="req-toolbar__filter-label">Date</label>
337             <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
338                 />
339         </div>
340     )}
341
342     {dateMode === "range" && (
343         <>
344             <div className="req-toolbar__filter-group">
345                 <label className="req-toolbar__filter-label">From</label>
346                 <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
347                     />
348             </div>
349             <div className="req-toolbar__filter-group">
350                 <label className="req-toolbar__filter-label">To</label>
351                 <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
352                     />
353             </div>
354         </>
355     )}
356
357     <button
358         type="button"
359         className="req-toolbar__reset-btn"
360         onClick={() => { load(page, actionFilter, true); setPreset("all"); setDateMode("preset"); setSingleDate(""); setS
361         disabled={refreshing}
362         title="Reset filters and refresh"
363     >
364         <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
365     </button>
366 </div>
367
368 /* Row 3: Action pills */
369 {isPillsOpen && (
370     <div className="req-toolbar__pills-row" style={{ borderTop: "1px solid var(--border)", paddingTop: "12px" }}>
371         {ACTION_TYPES.map((a) => (
372             <button
373                 key={a.value || "all"}
374                 type="button"
375                 onClick={() => handleActionFilter(a.value)}
376                 className={`req-toolbar__pill ${actionFilter === a.value ? "req-toolbar__pill--active" : ""}`}
377             >
378                 {a.label}
379             </button>
380         ))}
381     </div>
382 )}

```

```

378     </div>
379   })
380 </div>
381
382 { /* Table */
383 <div className="dashboard-card">
384   <div className="table-scroll">
385     <table className="dashboard-table">
386       <thead>
387         <tr className="dashboard-table-title-row">
388           <th colSpan={5}>
389             <div className="dashboard-table-title-wrap">
390               <span className="dashboard-table-title">Logs</span>
391             </div>
392           </th>
393         </tr>
394         <tr>
395           <th>Timestamp</th>
396           <th>Action</th>
397           <th>User</th>
398           <th>Resource</th>
399           <th>Details</th>
400         </tr>
401       </thead>
402       <tbody>
403         {loading ? (
404           [...Array(6)].map((_, i) => (
405             <tr key={`sk-${i}`}>
406               <td><span className="skeleton-block skeleton-text" /></td>
407               <td><span className="skeleton-block skeleton-pill" /></td>
408               <td><span className="skeleton-block skeleton-text" /></td>
409               <td><span className="skeleton-block skeleton-text" /></td>
410               <td><span className="skeleton-block skeleton-text" /></td>
411             </tr>
412           ))
413         ) : filtered.length === 0 ? (
414           <tr>
415             <td colSpan={5}>
416               <div className="dashboard-empty">
417                 <p className="dashboard-empty-title">No audit logs found</p>
418                 <p className="dashboard-empty-text">No logs match the current filters.</p>
419               </div>
420             </td>
421           </tr>
422         ) : (
423           filtered.map((log, i) => (
424             <motion.tr
425               key={log._id}
426               initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.015 }}
427             >
428               <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)", fontSize: 13 }}>
429                 {log.createdAt
430                   ? new Date(log.createdAt).toLocaleString("en-US", {
431                     month: "short", day: "numeric", year: "numeric",
432                     hour: "2-digit", minute: "2-digit",
433                   })
434                   : "-"}
435               </td>
436               <td style={{ fontWeight: 500 }}>{prettyAction(log.action)}</td>
437               <td>
438                 {log.userId ? (
439                   <>
440                     <div style={{ fontWeight: 600, fontSize: 13 }}>{log.userId.name || "-"}</div>
441                     <div style={{ fontSize: 11.5, color: "var(--text-muted)" }}>{log.userId.email || ""}</div>
442                   </>
443                 ) : (
444                   <span style={{ fontStyle: "italic", color: "var(--text-muted)", fontSize: 13 }}>System</span>
445                 )}
446               </td>
447               <td style={{ color: "var(--text-secondary)", fontSize: 13 }}>
448                 {log.resourceId
449                   ? <span>{String(log.resourceId).slice(-7).toUpperCase()}</span>
450                   : (log.resourceType ? log.resourceType.charAt(0).toUpperCase() + log.resourceType.slice(1) : "-")}
451               </td>
452               <td style={{ color: "var(--text-secondary)", fontSize: 13, maxWidth: 260 }}>
453                 <div style={{ overflow: "hidden", textOverflow: "ellipsis", whiteSpace: "nowrap" }} title={formatAuditID}

```



```
454             {formatAuditDetails(log)}
455         </div>
456     </td>
457 </motion.tr>
458 ))
459 }}
460 </tbody>
461 </table>
462 </div>
463 </div>
464
465 {/* Pagination */}
466 {totalPages > 1 && (
467   <div className="admin-pagination">
468     <button
469       onClick={() => setPage((p) => Math.max(1, p - 1))}
470       disabled={page <= 1}
471       className="admin-pagination-btn"
472     >
473       <ChevronLeft size={14} />
474     </button>
475     <span className="admin-pagination-text">
476       Page {page} of {totalPages}
477     </span>
478     <button
479       onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
480       disabled={page >= totalPages}
481       className="admin-pagination-btn"
482     >
483       <ChevronRight size={14} />
484     </button>
485   </div>
486   </div>
487   </div>
488   </div>
489   </div>
490 }
```

65. client/src/pages/admin/AdminArchives.jsx (459 lines)

```

1 import { useContext, useEffect, useState, useMemo } from "react";
2 import { AnimatePresence, motion } from "framer-motion";
3 import { Search, Filter, RefreshCw, ShieldCheck, Clock, Archive, AlertTriangle, Play } from "lucide-react";
4 import { getArchivedRequests, getRetentionStats, runArchiveNow } from "../../services/requestService";
5 import { AuthContext } from "../../../context/AuthContext";
6 import { notify } from "../../../utils/inPageFeedback";
7 import FilterSelect from "../../../components/FilterSelect";
8
9 const STATUS_LABEL = {
10   nda_approved: "Approved",
11   agr_approved: "Approved",
12 };
13
14 const prettyStatus = (s) =>
15   STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "-");
16
17 export default function AdminArchives() {
18   const { user } = useContext(AuthContext);
19   const isAdmin = user?.role === "admin";
20   const [requests, setRequests] = useState([]);
21   const [loading, setLoading] = useState(true);
22   const [refreshing, setRefreshing] = useState(false);
23   const [retentionStats, setRetentionStats] = useState(null);
24   const [archiving, setArchiving] = useState(false);
25   const [showArchiveConfirm, setShowArchiveConfirm] = useState(false);
26
27   const [searchInput, setSearchInput] = useState("");
28   const [searchTerm, setSearchTerm] = useState("");
29   const [typeFilter, setTypeFilter] = useState("all");
30   const [statusFilter, setStatusFilter] = useState("all");
31   const [dateMode, setDateMode] = useState("preset");
32   const [preset, setPreset] = useState("all");
33   const [singleDate, setSingleDate] = useState("");
34   const [startDate, setStartDate] = useState("");
35   const [endDate, setEndDate] = useState("");
36
37   const [isFilterOpen, setIsFilterOpen] = useState(() =>
38     typeof window === "undefined" ? true : window.innerWidth >= 768
39   );
40
41   const load = async ({ withToast = false } = {}) => {
42     setLoading(true);
43     if (withToast) setRefreshing(true);
44     try {
45       const [data, stats] = await Promise.all([
46         getArchivedRequests(),
47         isAdmin ? getRetentionStats() : Promise.resolve(null),
48       ]);
49       setRequests(data);
50       if (stats) setRetentionStats(stats);
51     } catch (err) {
52       console.error(err);
53     } finally {
54       setLoading(false);
55       setRefreshing(false);
56     }
57   };
58
59   useEffect(() => { load(); }, []);
60
61   const handleRunArchive = async () => {
62     setShowArchiveConfirm(false);
63     setArchiving(true);
64     try {
65       const result = await runArchiveNow();
66       notify(result.message, { type: "success", title: "Archive Complete" });
67       load();
68     } catch (err) {
69       notify(err.response?.data?.message || "Failed to run archive.", { type: "error" });
70     } finally {
71       setArchiving(false);
72     }
73   };

```

```
74
75 useEffect(() => {
76   const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
77   window.addEventListener("resize", onResize);
78   return () => window.removeEventListener("resize", onResize);
79 }, []);
80
81 const applySearch = () => setSearchTerm(searchInput.trim());
82 const clearSearch = () => { setSearchInput(""); setSearchTerm(""); };
83
84 const resetAndRefresh = () => {
85   setSearchInput("");
86   setSearchTerm("");
87   setTypeFilter("all");
88   setStatusFilter("all");
89   setDateMode("preset");
90   setPreset("all");
91   setSingleDate("");
92   setStartDate("");
93   setEndDate("");
94   load({ withToast: true });
95 };
96
97 const filtered = useMemo(() => {
98   let result = requests;
99
100   if (searchTerm) {
101     const term = searchTerm.toLowerCase();
102     result = result.filter((r) => {
103       const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
104       const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
105       const id = (r._id || "").slice(-7).toLowerCase();
106       return name.includes(term) || email.includes(term) || id.includes(term);
107     });
108   }
109
110   if (typeFilter !== "all") {
111     if (typeFilter === "agreement") {
112       result = result.filter((r) => r.type === "agreement");
113     } else if (typeFilter === "nda_orgactivities") {
114       result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
115     } else if (typeFilter === "nda_research") {
116       result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
117     }
118   }
119
120   if (statusFilter !== "all") {
121     const statusMap = {
122       reviewal: ["nda_pending", "agr_pending_1", "agr_pending_2"],
123       approved: ["nda_approved", "agr_approved"],
124       stud_revision: ["stud_revision_requested"],
125       rep_revision: ["agr_rep_revision_requested"],
126       rep_declined: ["agr_declined"],
127       awaiting_rep: ["agr_awaiting_rep_signature"],
128     };
129     const statuses = statusMap[statusFilter] || [];
130     result = result.filter((r) => statuses.includes(r.status));
131   }
132
133   if (dateMode === "preset") {
134     const now = new Date();
135     if (preset === "today") {
136       const dayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
137       const dayEnd = new Date(dayStart.getTime() + 86400000 - 1);
138       result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
139     } else if (preset === "thisWeek") {
140       const day = now.getDay();
141       const monday = new Date(now);
142       monday.setDate(now.getDate() - ((day + 6) % 7));
143       monday.setHours(0, 0, 0, 0);
144       result = result.filter((r) => new Date(r.createdAt) >= monday);
145     } else if (preset === "thisMonth") {
146       const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
147       result = result.filter((r) => new Date(r.createdAt) >= monthStart);
148     } else if (preset === "thisYear") {
149       const yearStart = new Date(now.getFullYear(), 0, 1);
```

```

150     result = result.filter((r) => new Date(r.createdAt) >= yearStart);
151   }
152   } else if (dateMode === "single" && singleDate) {
153     const dayStart = new Date(singleDate);
154     const dayEnd = new Date(singleDate + "T23:59:59");
155     result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
156   } else if (dateMode === "range") {
157     if (startDate) result = result.filter((r) => new Date(r.createdAt) >= new Date(startDate));
158     if (endDate) result = result.filter((r) => new Date(r.createdAt) <= new Date(endDate + "T23:59:59"));
159   }
160
161   return result;
162 }, [requests, searchTerm, typeFilter, statusFilter, dateMode, preset, singleDate, startDate, endDate]);
163
164 return (
165   <div className="page-shell">
166
167     /* ■■ Retention Policy Panel (admin only) ■■ */
168     {isAdmin && retentionStats && (
169       <div className="dashboard-card" style={{ marginBottom: 16, padding: "18px 20px" }}>
170         <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom: 14 }}>
171           <ShieldCheck size={18} strokeWidth={1.8} style={{ color: "var(--primary)" }} />
172           <h3 style={{ margin: 0, fontSize: 15, fontWeight: 700 }}>Retention Policy</h3>
173         </div>
174         <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(160px, 1fr))", gap: 12, marginBottom: 12 }}>
175           <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid var(--border)" }}>
176             <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, margin: 0 0 4px 0 }}>Retention Period</div>
177             <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.retentionYears} Years</div>
178           </div>
179           <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid var(--border)" }}>
180             <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, margin: 0 0 4px 0 }}>Active Requests</div>
181             <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.totalActive}</div>
182           </div>
183           <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid var(--border)" }}>
184             <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, margin: 0 0 4px 0 }}>Total Archived</div>
185             <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.totalArchived}</div>
186           </div>
187           <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid var(--border)" }}>
188             <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, margin: 0 0 4px 0 }}>Last Archived</div>
189             <div style={{ fontSize: 14, fontWeight: 600 }}>
190               {retentionStats.lastArchivedAt
191                 ? new Date(retentionStats.lastArchivedAt).toLocaleDateString("en-US", { year: "numeric", month: "short", day: "numeric" })
192                 : "Never"}
193             </div>
194           </div>
195         </div>
196
197         {retentionStats.approachingRetention > 0 && (
198           <div style={{ display: "flex", alignItems: "center", gap: 8, padding: "10px 12px", background: "var(--s-warning-bg)" }}>
199             <AlertTriangle size={14} />
200             <span><strong>{retentionStats.approachingRetention}</strong> request(s) approaching the {retentionStats.retentionYears} year retention period.</span>
201           </div>
202         )}
203
204         <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", flexWrap: "wrap", gap: 10 }}>
205           <p style={{ margin: 0, fontSize: 12, color: "var(--text-muted)", maxWidth: 480 }}>
206             Per institutional policy under RA 10173, requests older than {retentionStats.retentionYears} years are automatically eligible for archiving.
207             {retentionStats.eligibleForArchive > 0
208               ? ` ${retentionStats.eligibleForArchive} request(s) currently eligible.`
209               : " No requests currently eligible for archiving."}
210           </p>
211           <button
212             type="button"
213             className="ui-btn ui-btn--primary"
214             style={{ fontSize: 13, gap: 6, display: "inline-flex", alignItems: "center" }}
215             onClick={() => setShowArchiveConfirm(true)}
216             disabled={archiving || retentionStats.eligibleForArchive === 0}
217           >
218             <Play size={13} />
219             {archiving ? "Archiving..." : "Run Archive Now"}
220           </button>
221         </div>
222       </div>
223     )}
224
225     /* ■■ Toolbar ■■ */

```

```
226 <div className="req-toolbar">
227   <div className="req-toolbar__search-row">
228     <div className="req-toolbar__search-box">
229       <input
230         type="text"
231         className="req-toolbar__search-input"
232         placeholder="Search by requestor, email, or request ID..."
233         value={searchInput}
234         onChange={(e) => setSearchInput(e.target.value)}
235         onKeyDown={(e) => { if (e.key === "Enter") applySearch(); }}
236       />
237       {searchInput ? (
238         <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
239           x
240         </button>
241       ) : null}
242       <button type="button" className="req-toolbar__search-inline-btn" onClick={applySearch} aria-label="Search">
243         <Search size={14} />
244       </button>
245     </div>
246     <button
247       type="button"
248       className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
249       onClick={() => setIsFilterOpen((prev) => !prev)}
250       aria-expanded={isFilterOpen}
251     >
252       <Filter size={13} />
253       Filters
254     </button>
255   </div>
256
257   <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
258     <div className="req-toolbar__filter-group">
259       <label className="req-toolbar__filter-label">Type</label>
260       <FilterSelect
261         value={typeFilter}
262         onChange={setTypeFilter}
263         className="filter-select--type"
264         options={[
265           { value: "all", label: "All Types" },
266           { value: "nda_orgactivities", label: "NDA – Student Organization Activities" },
267           { value: "nda_research", label: "NDA – Conduct of Research" },
268           { value: "agreement", label: "Agreement" },
269         ]}
270       />
271     </div>
272
273     <div className="req-toolbar__filter-group">
274       <label className="req-toolbar__filter-label">Status</label>
275       <FilterSelect
276         value={statusFilter}
277         onChange={setStatusFilter}
278         className="filter-select--status"
279         options={[
280           { value: "all", label: "All Statuses" },
281           { value: "reviewal", label: "Reviewal" },
282           { value: "approved", label: "Approved" },
283           { value: "stud_revision", label: "Student Revisions" },
284           { value: "rep_revision", label: "Recipient Revisions" },
285           { value: "rep_declined", label: "Recipient Declined" },
286           { value: "awaiting_rep", label: "Recipient Reviewal" },
287         ]}
288       />
289     </div>
290
291     <div className="req-toolbar__filter-group">
292       <label className="req-toolbar__filter-label">Date</label>
293       <FilterSelect
294         value={dateMode}
295         onChange={setDateMode}
296         options={[
297           { value: "preset", label: "Preset" },
298           { value: "single", label: "Specific Date" },
299           { value: "range", label: "Date Range" },
300         ]}
301       defaultValue="preset"
```

```

302     />
303   </div>
304
305   {dateMode === "preset" ? (
306     <div className="req-toolbar__filter-group">
307       <label className="req-toolbar__filter-label">Period</label>
308       <FilterSelect
309         value={preset}
310         onChange={setPreset}
311         options={[
312           { value: "all", label: "All Time" },
313           { value: "today", label: "Today" },
314           { value: "thisWeek", label: "This Week" },
315           { value: "thisMonth", label: "This Month" },
316           { value: "thisYear", label: "This Year" },
317         ]}
318       />
319     </div>
320   ) : null}
321
322   {dateMode === "single" ? (
323     <div className="req-toolbar__filter-group">
324       <label className="req-toolbar__filter-label">Date</label>
325       <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
326       />
327     </div>
328   ) : null}
329
330   {dateMode === "range" ? (
331     <div className="req-toolbar__filter-group">
332       <label className="req-toolbar__filter-label">From</label>
333       <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
334       />
335       <div className="req-toolbar__filter-group">
336         <label className="req-toolbar__filter-label">To</label>
337         <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
338         />
339       </div>
340     </div>
341   ) : null}
342
343   <button
344     type="button"
345     className="req-toolbar__reset-btn"
346     onClick={resetAndRefresh}
347     disabled={refreshing}
348     title="Reset filters and refresh"
349   >
350     <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
351   </button>
352 </div>
353
354 {/* ■■ Table ■■ */}
355 <div className="dashboard-card">
356   <div className="table-scroll">
357     <table className="dashboard-table">
358       <thead>
359         <tr className="dashboard-table-title-row">
360           <th colspan={6}>
361             <div className="dashboard-table-title-wrap">
362               <span className="dashboard-table-title">Archives</span>
363             </div>
364           </th>
365         </tr>
366         <tr>
367           <th>Request Date</th>
368           <th>Archived Date</th>
369           <th>Requestor</th>
370           <th>Request ID</th>
371           <th>Type</th>
372           <th>Status</th>
373         </tr>
374       </thead>
375       <tbody>
376         {loading ? (
377           [...Array(4)].map((_, i) => (

```

```

378         <tr key={`sk-${i}`}>
379             <td><span className="skeleton-block skeleton-text" /></td>
380             <td><span className="skeleton-block skeleton-text" /></td>
381             <td><span className="skeleton-block skeleton-text" /></td>
382             <td><span className="skeleton-block skeleton-text" /></td>
383             <td><span className="skeleton-block skeleton-pill" /></td>
384             <td><span className="skeleton-block skeleton-pill" /></td>
385         </tr>
386     ))
387 ) : filtered.length === 0 ? (
388     <tr>
389         <td colSpan={6}>
390             <div className="dashboard-empty">
391                 <p className="dashboard-empty-title">No archived requests found</p>
392                 <p className="dashboard-empty-text">No archived requests match the current filters.</p>
393             </div>
394         </td>
395     </tr>
396 ) : (
397     filtered.map((r, i) => {
398         const name = r.proxyRequestee?.isProxy
399           ? (`${r.proxyRequestee.firstName} || ""` $r.proxyRequestee.lastName || "").trim() || r.proxyRequestee.fullName
400           : (r.userId?.name || "-");
401         const email = r.proxyRequestee?.isProxy
402           ? (r.proxyRequestee.email || "")
403           : (r.userId?.email || "");
404         const typeLabel = r.type === "nda"
405           ? `NDAS${r.formData?.ndaTypeLabel ? ` - ${r.formData.ndaTypeLabel}` : ""}`
406           : "Agreement";
407         return (
408             <motion.tr
409                 key={r._id}
410                 initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.02 }}
411             >
412                 <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
413                     {r.createdAt ? new Date(r.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short", day: "numeric" }) : ""}
414                 </td>
415                 <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
416                     {r.archivedAt ? new Date(r.archivedAt).toLocaleDateString("en-US", { year: "numeric", month: "short", day: "numeric" }) : ""}
417                 </td>
418                 <td>
419                     <span style={{ fontWeight: 600 }}>{name}</span>
420                     {email && <span className="dashboard-subtext">{email}</span>}
421                     {r.proxyRequestee?.isProxy && (
422                         <span className="dashboard-subtext">Proxy (F2F)</span>
423                     )}
424                 </td>
425                 <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
426                 <td>{typeLabel}</td>
427                 <td>
428                     <span className="tag-pill tag-pill--neutral">Archived</span>
429                 </td>
430             </motion.tr>
431         );
432     })
433 )}
434 </tbody>
435 </table>
436 </div>
437 </div>
438
439 /* ■■■ Archive Confirmation Modal ■■■ */
440 <AnimatePresence>
441     {showArchiveConfirm && (
442         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
443             <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, opacity: 0 }}>
444                 <h3 className="modal-title">Run Archive</h3>
445                 <p className="modal-text">
446                     This will archive <strong>{retentionStats?.eligibleForArchive || 0}</strong> request(s) that have exceeded
447                     the {retentionStats?.retentionYears || 5}-year retention period. This action cannot be undone from the UI.
448                 </p>
449                 <div className="modal-actions">
450                     <button className="ui-btn ui-btn--secondary" onClick={() => setShowArchiveConfirm(false)} disabled={archiving}>Cancel</button>
451                     <button className="ui-btn ui-btn--primary" onClick={handleRunArchive} disabled={archiving}>Archive</button>
452                 </div>
453             </motion.div>
454         )}
455 </AnimatePresence>

```

```
454         </motion.div>
455     )}
456 </AnimatePresence>
457 </div>
458 );
459 }
```